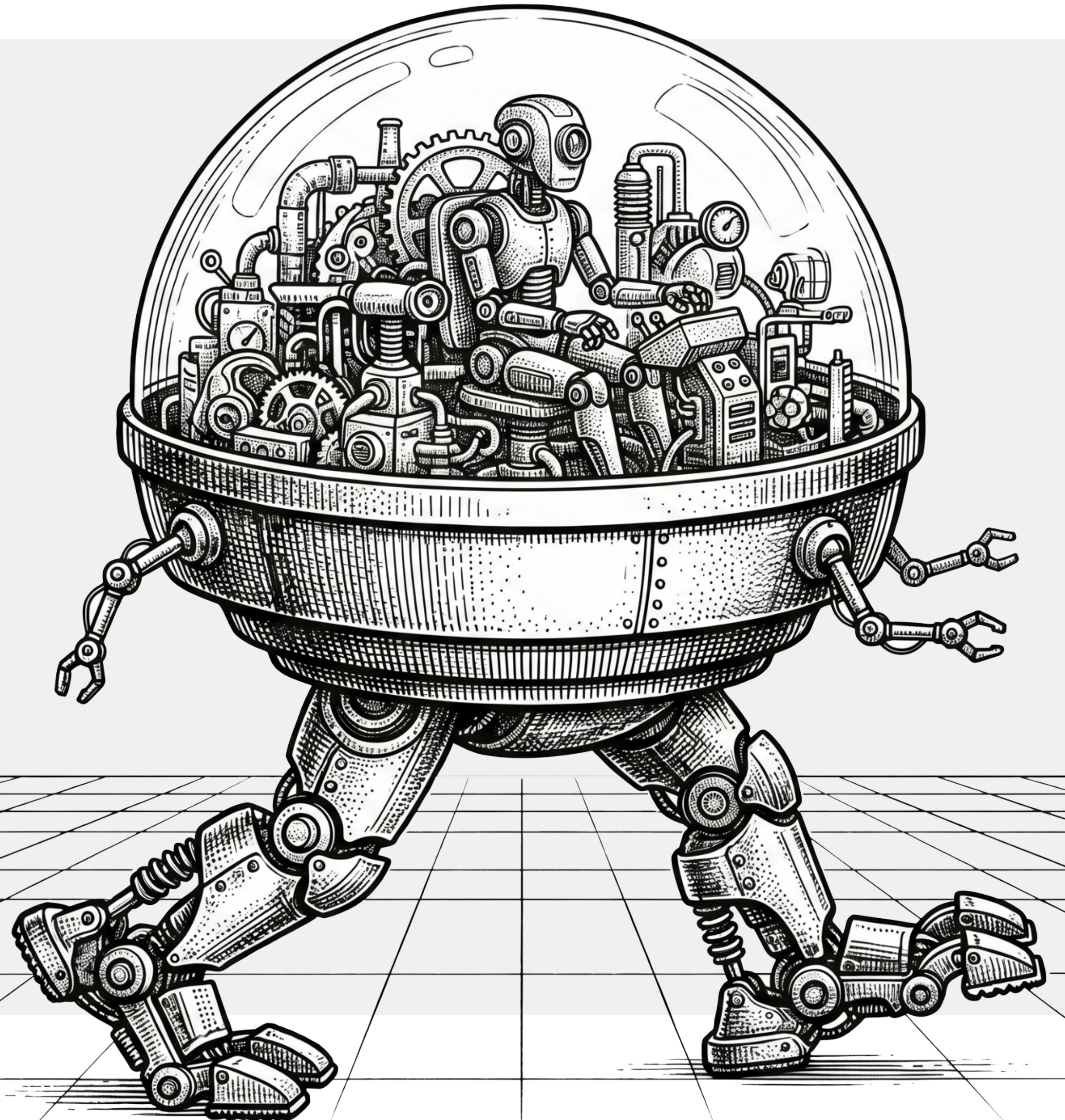


Introducción al Aprendizaje Automático

Felipe Ramírez Herrera



Ejercicio Práctico(10 min)

Adapte lo visto anteriormente para el conjunto de datos de ejemplo Wine.

```
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
from sklearn.datasets import load_wine
```

```
wine = load_wine()
```



Generalización

La generalización es la capacidad de un modelo para realizar predicciones precisas y comportarse adecuadamente frente a datos nuevos y nunca antes vistos (es decir, datos que no formaron parte de su proceso de entrenamiento).

Objetivo:

Cuando un algoritmo aprende, evaluamos su éxito utilizando dos métricas principales:

Error de entrenamiento (Training Error):

Mide los errores cometidos en los datos específicos utilizados para enseñar al modelo.

Error de generalización o prueba (Test

Error): Mide los errores cometidos en datos no vistos previamente.

Desafíos comunes:

Sobreajuste (Overfitting): Funciona a la perfección en el entrenamiento, pero falla con datos nuevos.

Subajuste (Underfitting): Tiene un rendimiento deficiente tanto en los datos de entrenamiento como en los de prueba.

El objetivo es encontrar el punto de equilibrio matemático donde la brecha entre el error de entrenamiento y el de prueba se minimice, maximizando así la precisión predictiva.

Regularización

Técnicas que penalizan los modelos excesivamente complejos, fomentando pesos de parámetros más simples.

Abandono (Dropout)

Una técnica en la que se “apagan” neuronas al azar durante el entrenamiento para evitar que la red neuronal dependa demasiado de rutas o detalles específicos.

Aumento de datos (Data Augmentation)

Agregar ligeras variaciones, ruido o transformaciones a los datos de entrenamiento, haciendo que el modelo sea más robusto ante anomalías del mundo real.

Parada temprana (Early Stopping)

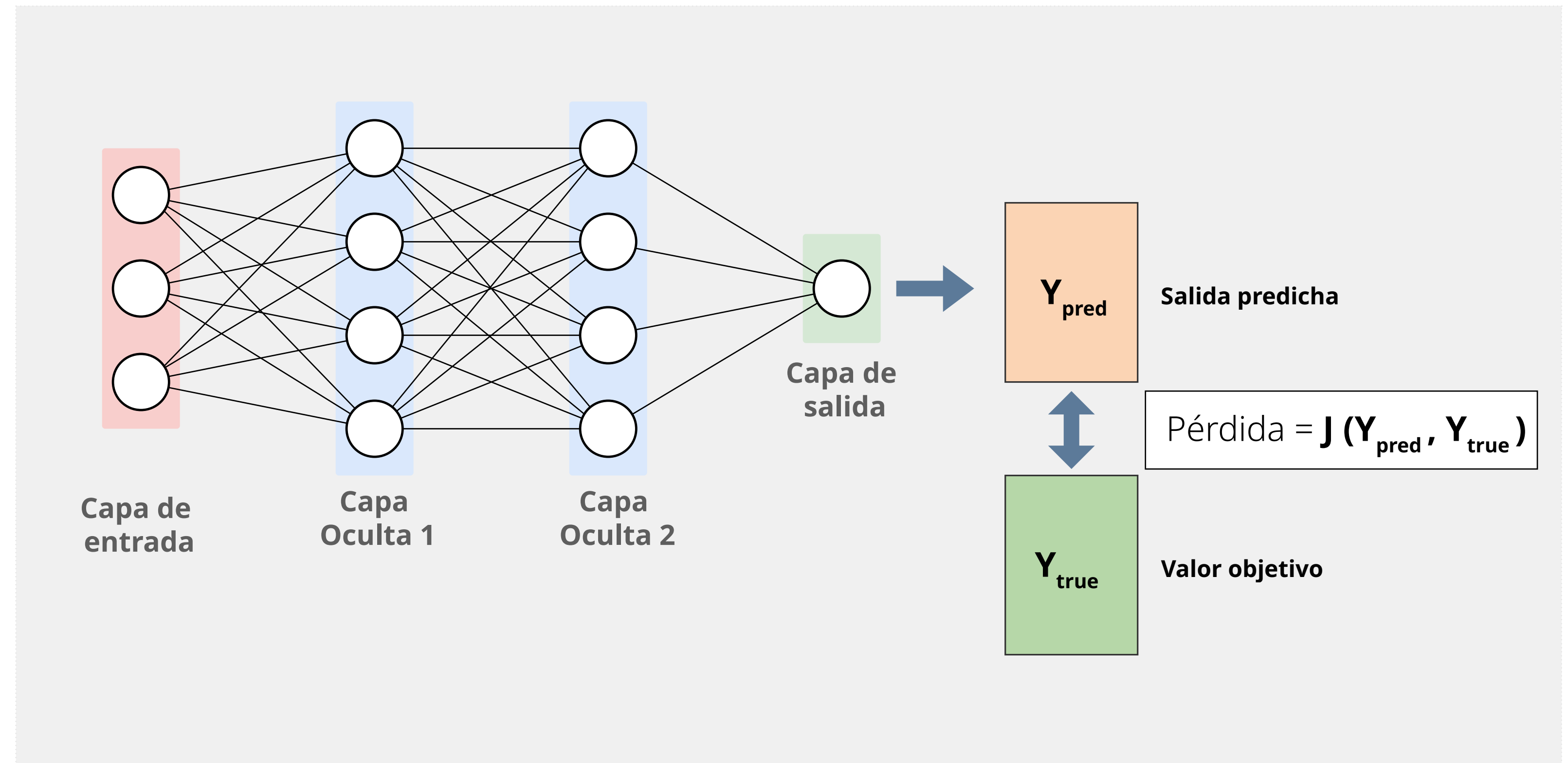
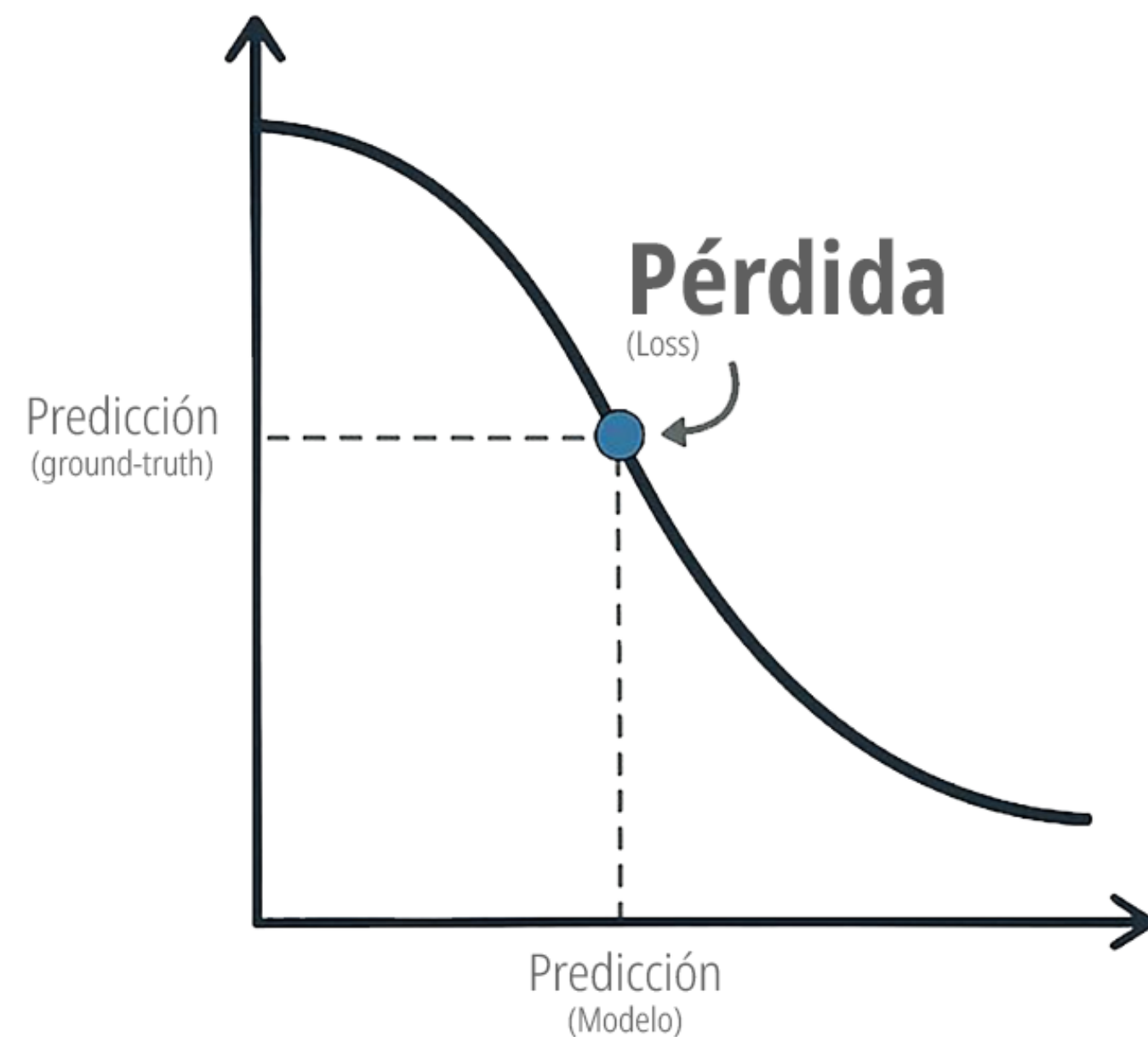
Detener el proceso de entrenamiento tan pronto como el error en un conjunto de datos de validación comienza a aumentar, evitando que el modelo se optimice en exceso para los datos de entrenamiento.

Validación cruzada (Cross-Validation)

Dividir los datos en múltiples subconjuntos para probar repetidamente cómo se comporta el modelo con entradas no vistas antes de su implementación final.

Función de pérdida (loss)

Una función de pérdida (loss function) es una fórmula matemática que cuantifica la diferencia entre la salida predicha por un modelo y el valor real o “verdad fundamental” (ground truth).

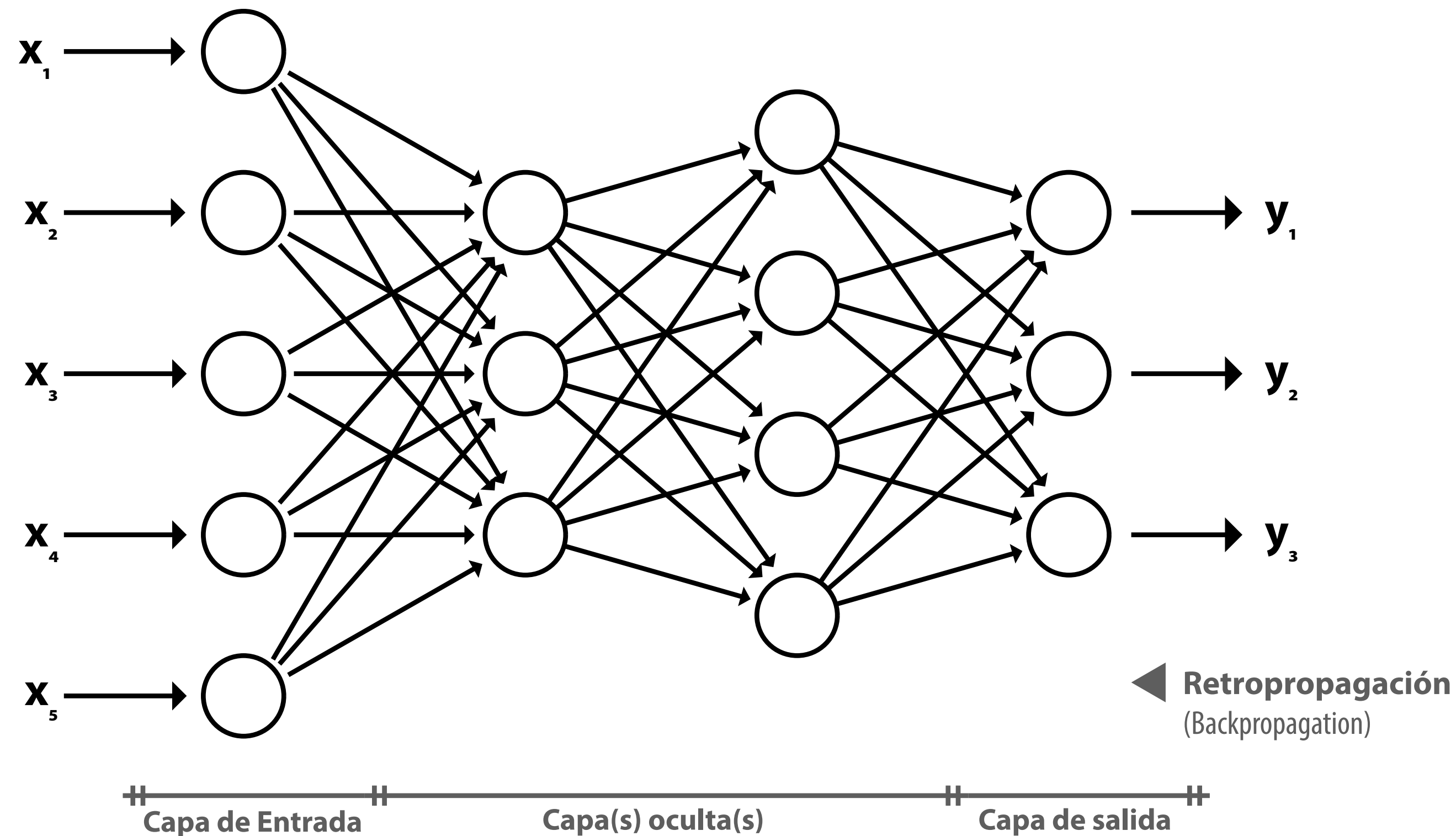


Sirve como un mecanismo de retroalimentación que le indica al modelo qué tan “erradas” son sus predicciones, siendo el objetivo final del entrenamiento **minimizar** este valor.

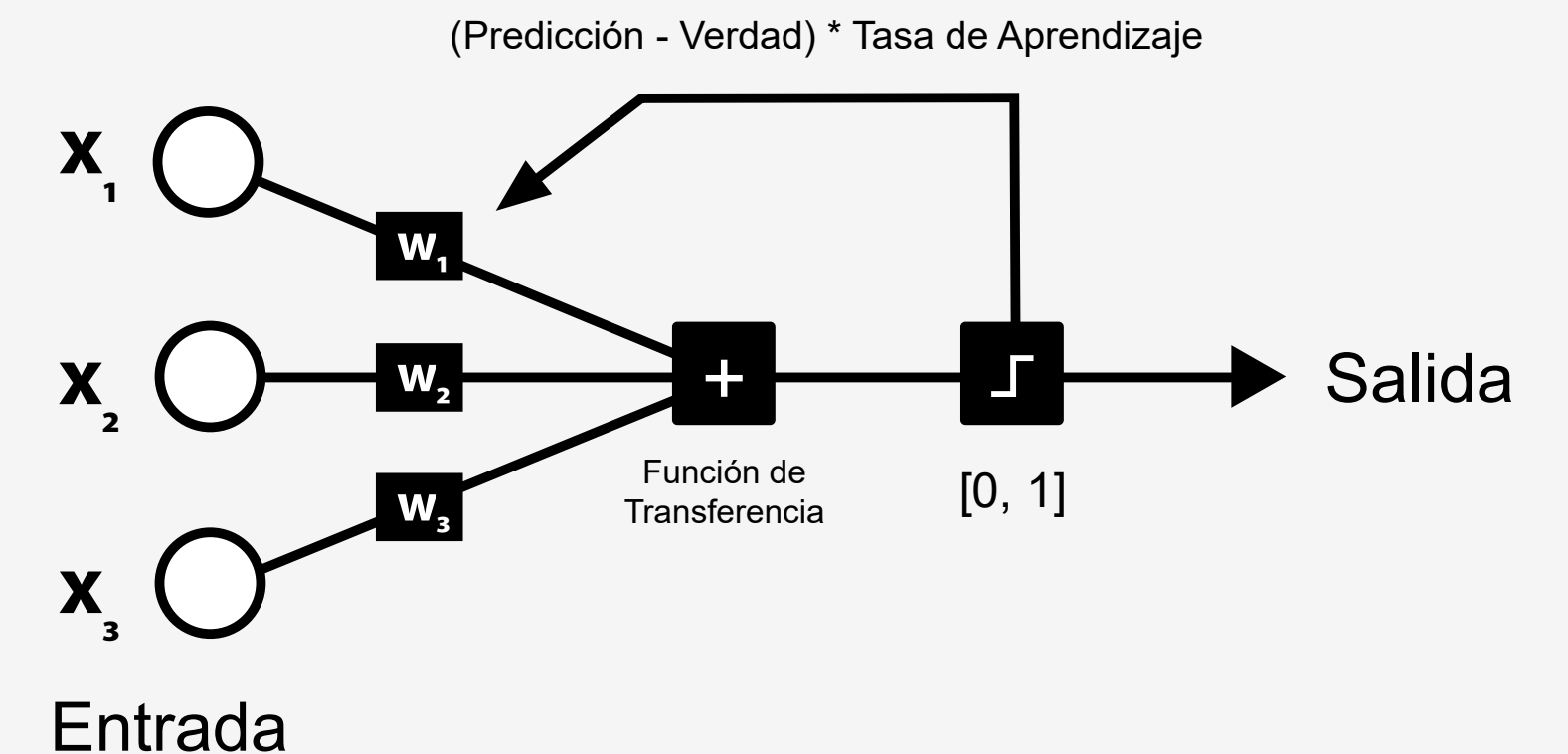
Capacidad/Complejidad del modelo: Redes neuronales

También conocidas como sistemas conexionistas, se trata de un conjunto de unidades, llamadas neuronas artificiales, conectadas entre sí para transmitirse señales.

Entrenamiento ►
(Training)



Neuronas artificiales



Nota: Las redes neuronales "superficiales" son un término que se usa para describir NN que generalmente tienen solo una capa oculta en comparación con NN profundas que tienen varias capas ocultas, a menudo de varios tipos.

Fuentes de error: Ruido, Sesgo y Varianza

En pocas palabras, es la prueba de fuego de cualquier algoritmo es lograr la generalización. Es la diferencia entre aprender un concepto y memorizar las respuestas de un examen.

A

Ruido (Noise): Es el error inherente a los datos en sí. Ocurre porque el mundo real es impredecible, porque hay variables que no estamos midiendo o porque los datos están mal etiquetados. Es una barrera insuperable; no importa qué tan bueno sea tu modelo, nunca podrá predecir el ruido aleatorio en datos nuevos (test data).

B

Sesgo (Bias): Es el error que proviene de la rigidez del modelo. Sucede cuando el modelo que elegimos es demasiado simple para capturar la complejidad real del problema. Por ejemplo, intentar usar una simple línea recta para predecir un comportamiento que en realidad tiene forma de onda. El modelo no es lo suficientemente flexible, por lo que comete errores sistemáticos (Subajuste o Underfitting).

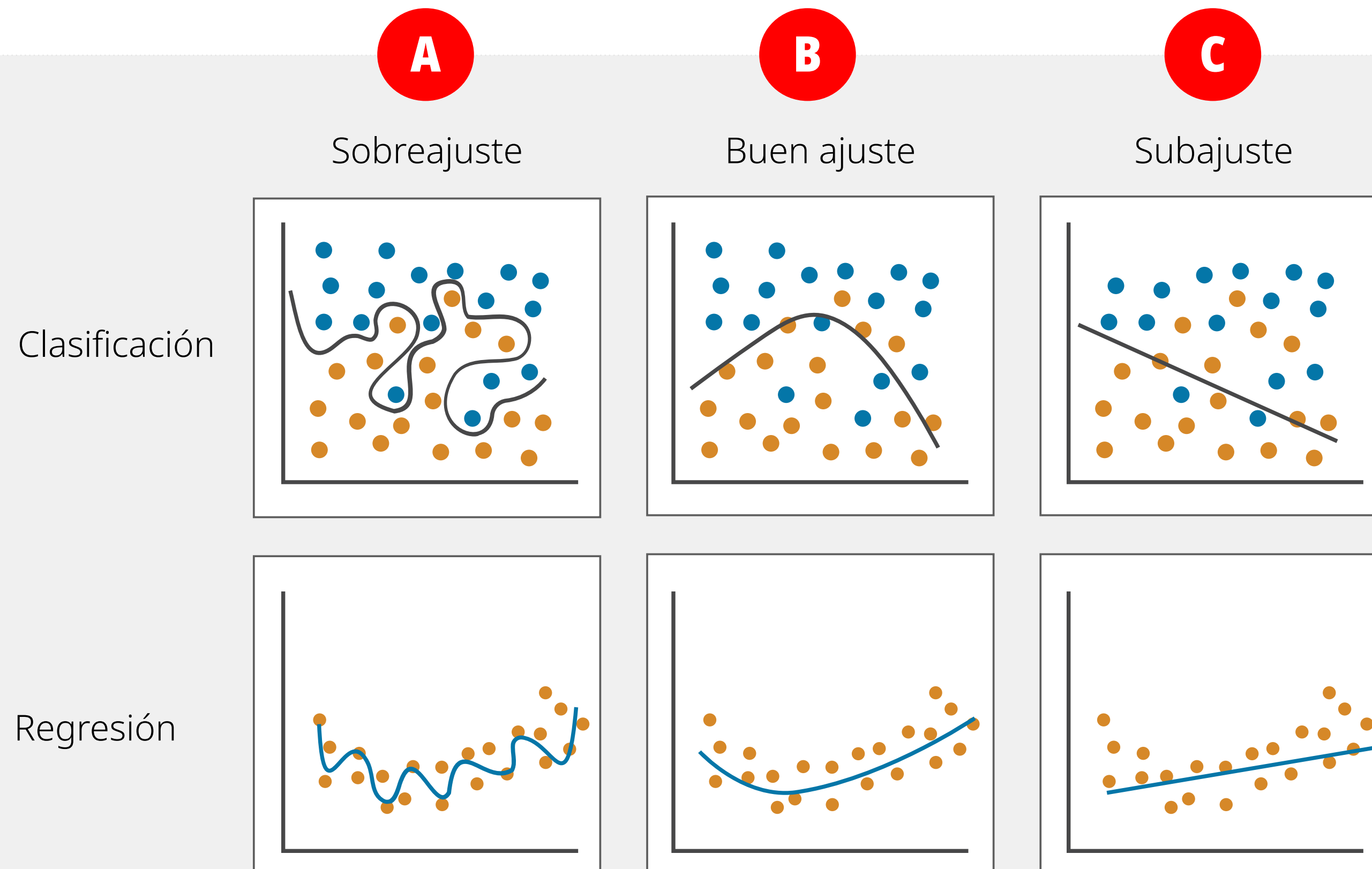
C

Varianza (Variance): Es el error que proviene de la hipersensibilidad del modelo. Sucede cuando tenemos datos de entrenamiento limitados y un modelo tan complejo que termina memorizando el ruido o las peculiaridades exactas de esos datos específicos, en lugar de aprender el patrón general. Si entrenaras el mismo modelo con un conjunto de datos ligeramente distinto, la predicción cambiaría drásticamente. (A esto también se le conoce como Sobreajuste u Overfitting).

La capacidad está directamente ligada a los parámetros. La capacidad del modelo se refiere al tamaño y la diversidad del conjunto de funciones matemáticas que el modelo es capaz de aprender y representar. En términos simples, es la "flexibilidad" o el "tamaño del cerebro" del algoritmo. Para entenderlo mejor, conectemos esto con los conceptos que acabamos de ver: 1) Capacidad Baja: Un modelo con baja capacidad solo puede entender relaciones muy simples y rígidas. Por ejemplo, una línea recta. Si le presentas un problema complejo (como el comportamiento de un mercado financiero), no tendrá la flexibilidad matemática para entenderlo. Esto resulta en Alto Sesgo (Subajuste). 2) Capacidad Alta: Un modelo con alta capacidad puede doblarse, torcerse y adaptarse a relaciones increíblemente complejas y no lineales. Sin embargo, si le das demasiada capacidad y no tienes suficientes datos, el modelo usará esa flexibilidad para memorizar el ruido y las anomalías de los datos de entrenamiento. Esto resulta en Alta Varianza (Sobreajuste). La regularización es una técnica matemática que utilizamos para evitar que un modelo sufra de sobreajuste (overfitting).

Sobreajuste, buen ajuste y subajuste

El subajuste (underfitting), el sobreajuste (overfitting) y el ajuste óptimo (best fit) describen qué tan bien un modelo de aprendizaje automático aprende los patrones de los datos de entrenamiento y los generaliza a datos nuevos y desconocidos. Lograr el ajuste óptimo requiere equilibrar la complejidad del modelo para evitar tanto el subaprendizaje rígido como la memorización caótica.

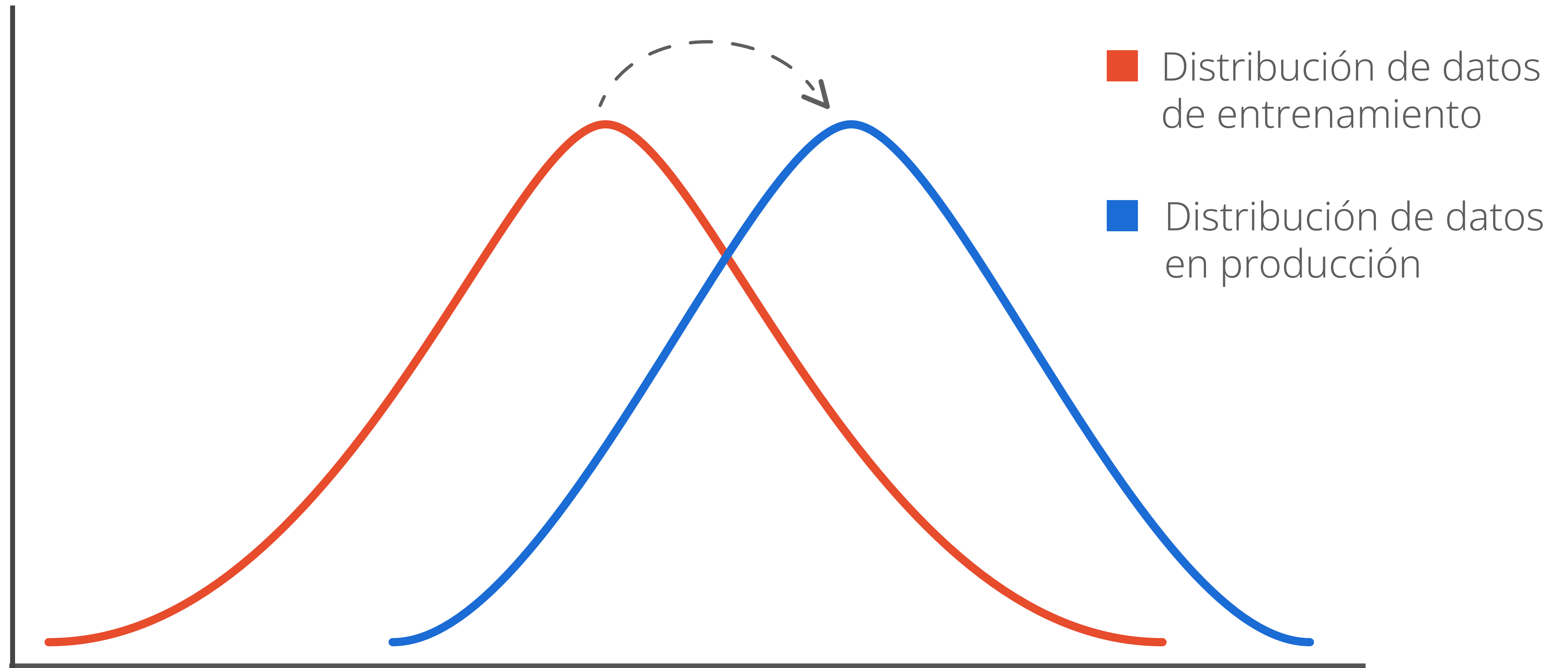


El **subajuste (underfitting)** ocurre cuando un modelo es demasiado simple y no logra capturar los patrones de los datos, generalmente por falta de capacidad algorítmica, escasez de variables o exceso de regularización. Se soluciona aumentando la complejidad del modelo, añadiendo características relevantes y reduciendo las restricciones de regularización.

El **sobreajuste (overfitting)** sucede cuando un modelo es tan complejo que memoriza los datos de entrenamiento (incluyendo el ruido) en lugar de aprender reglas generales, a menudo por usar muchos parámetros, pocos datos o entrenar en exceso. Para corregirlo, se deben usar más datos, simplificar la arquitectura del modelo, aplicar regularización o detener el entrenamiento a tiempo (early stopping).

El **ajuste óptimo (best fit)** es el equilibrio ideal en el que el modelo generaliza perfectamente ante datos nuevos, capturando la señal real y descartando el ruido. Se evidencia cuando los errores de entrenamiento y validación disminuyen a la par, estabilizándose en un punto mínimo con muy poca diferencia entre ambos.

Deriva (Data & Model Drifting)

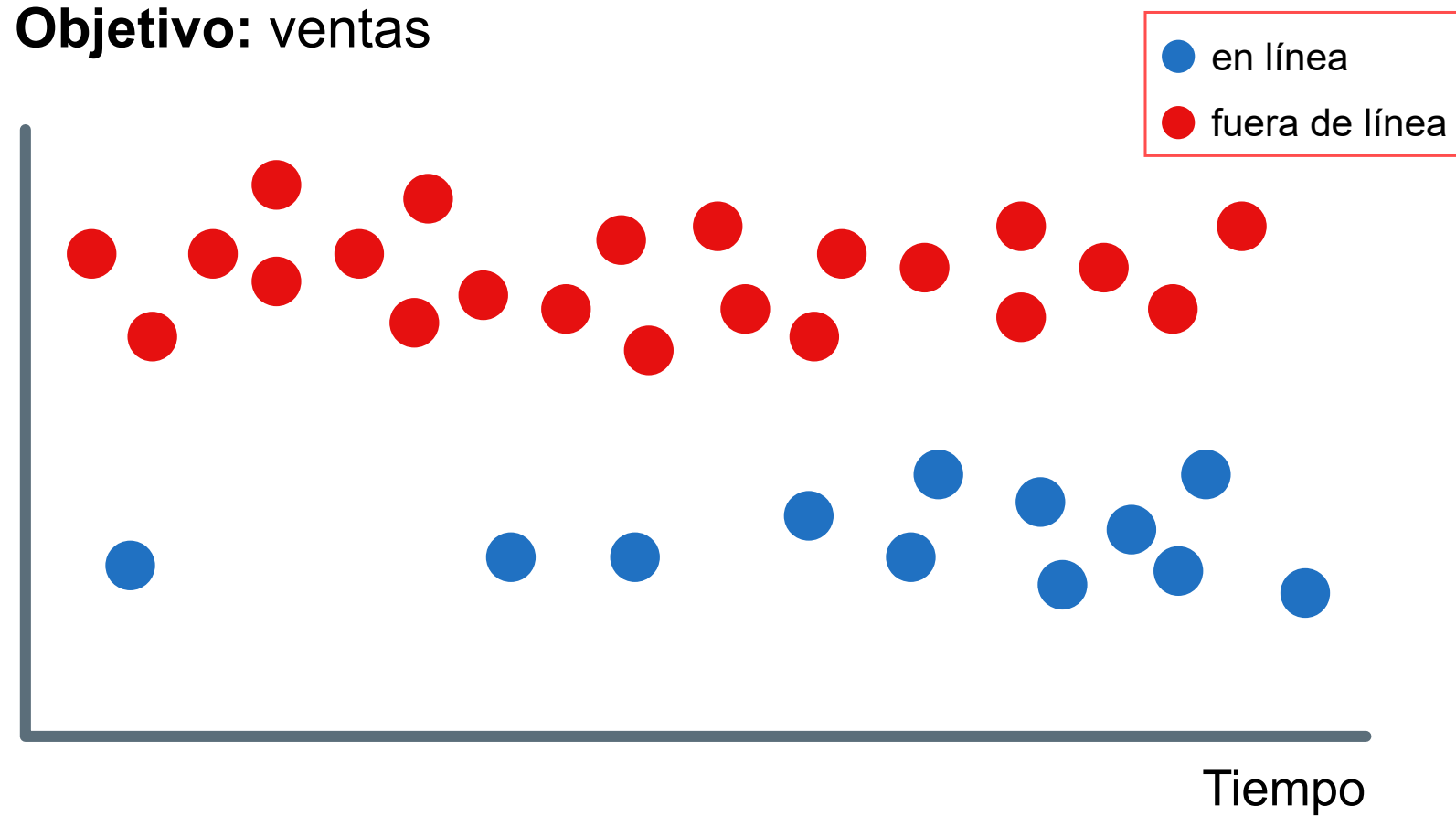


Deriva (Data & Model Drifting)

La deriva de datos (data drift) y la deriva de concepto (concept drift) suelen ocurrir simultáneamente y la primera puede ser síntoma de la segunda, aunque no siempre es así.

Deriva de datos

Objetivo: ventas

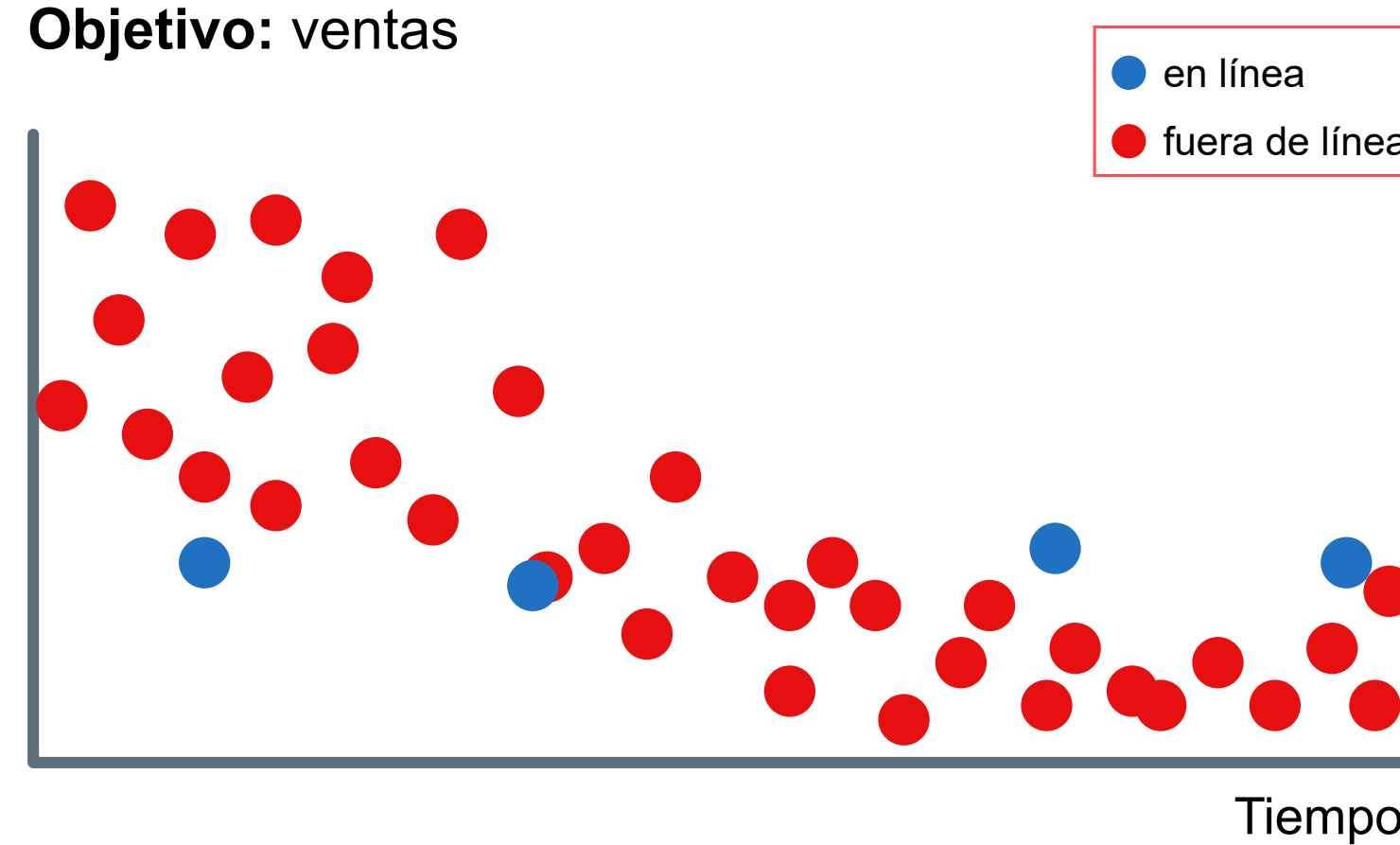


Distribución de características: canal de ventas



Deriva de concepto

Objetivo: ventas



Distribución de características: canal de ventas



La deriva de concepto implica un cambio fundamental en los patrones de comportamiento.

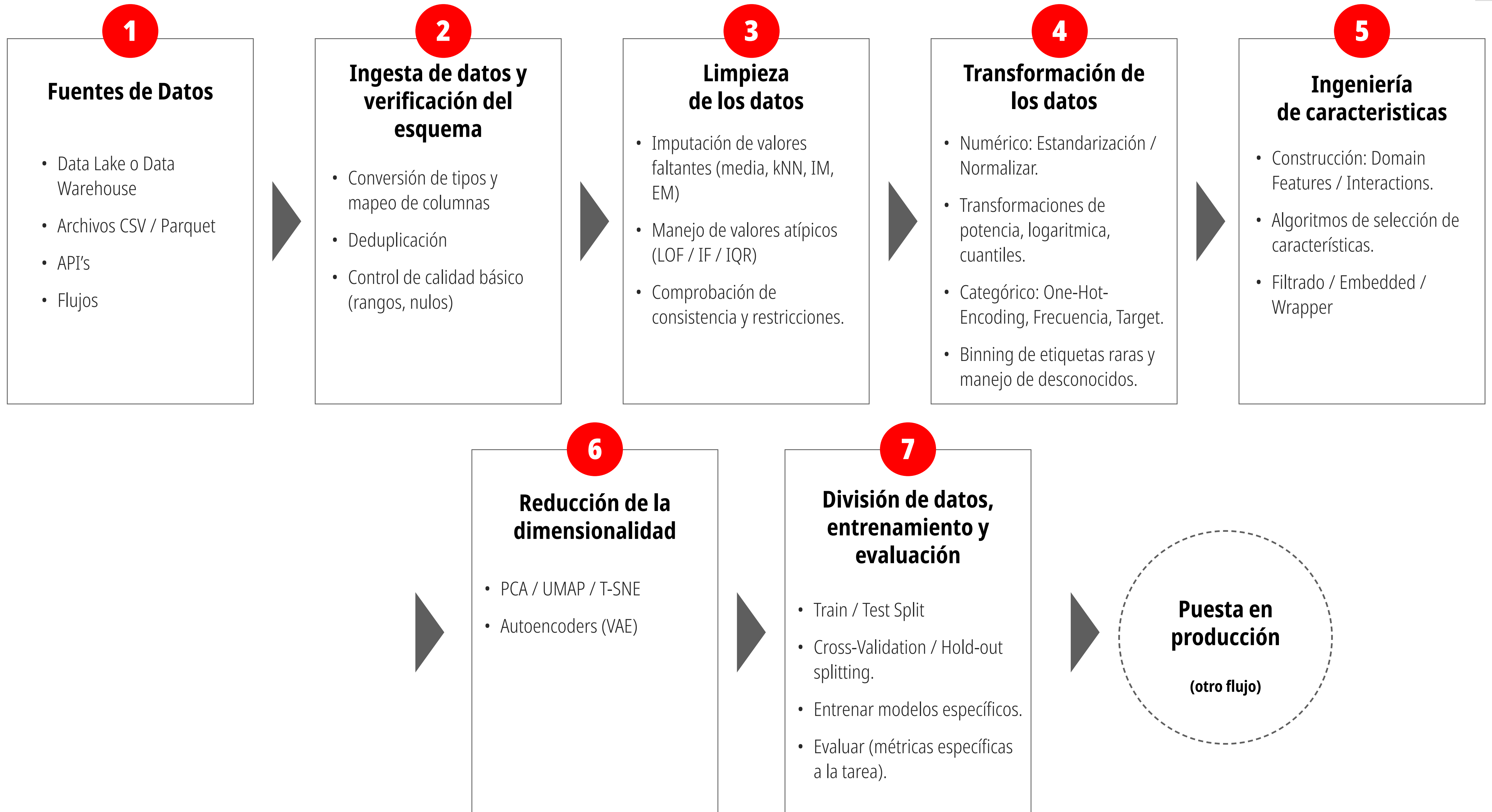
En estos casos de deriva de concepto, los modelos creados previamente quedan prácticamente obsoletos.

Deriva (Data & Model Drifting)

Cuando se habla de deriva de datos (data drift), normalmente nos referimos a las características (features) de entrada que se introducen en el modelo. En cambio, la deriva de predicción (prediction drift) es el cambio en la distribución de las salidas (outputs) del modelo.



La deriva de predicción (prediction drift) puede indicar muchos problemas, desde entradas de datos de baja calidad hasta la deriva de concepto (concept drift) en el proceso modelado. Al mismo tiempo, la deriva de predicción no siempre implica el deterioro del modelo. También puede ocurrir si el modelo se adapta bien al nuevo entorno. La diferencia: la deriva de datos se refiere a los cambios en los datos de entrada del modelo, mientras que la deriva de predicción se refiere a los cambios en las salidas del modelo.



Laboratorio 1: Flujo

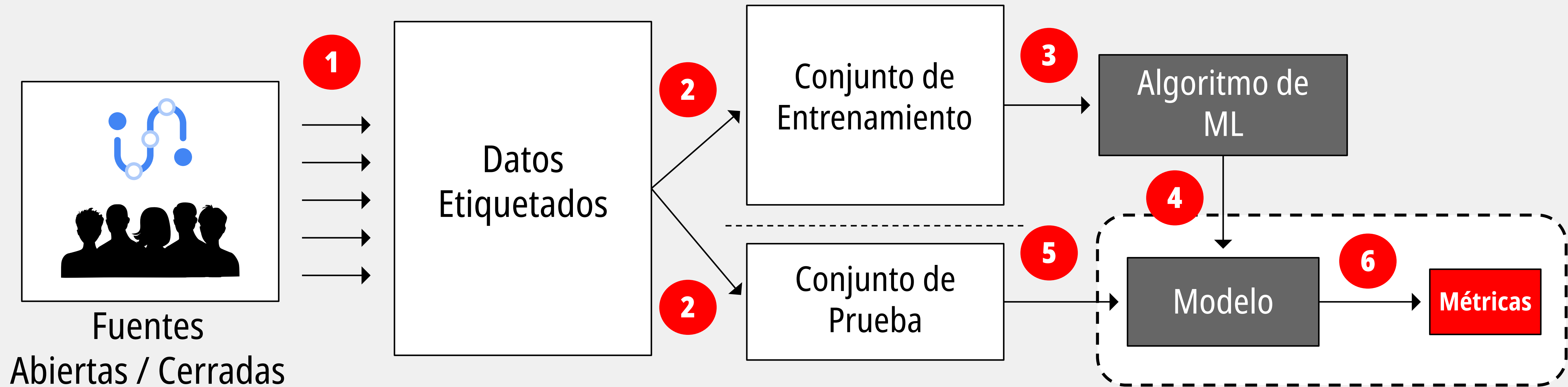
- Clasificación (Titanic)

Laboratorio 2: Generalización

- Regresión (Polinomio)
- Clasificación (Cáncer)



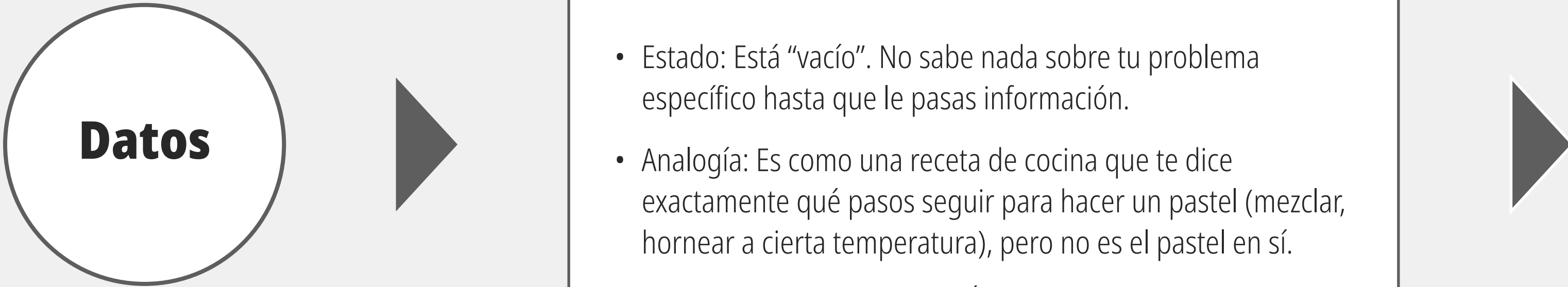
Concepto: Aprendizaje supervisado



En Machine Learning, un weak learner (aprendiz débil) es un modelo simple y ligeramente mejor que el azar (tasa de error $< 50\%$). Un strong learner (aprendiz fuerte) es un modelo complejo y altamente preciso. Los algoritmos de conjunto (ensemble methods) como Boosting combinan muchos weak learners secuencialmente para crear un modelo predictivo altamente robusto.

Desambiguación: Modelo vs Algoritmo

Es muy común confundir estos dos términos, ya que en las conversaciones del día a día suelen usarse como sinónimos. Sin embargo, en Machine Learning (Aprendizaje Automático), representan dos etapas completamente distintas del proceso.



Datos

Algoritmo

Es el conjunto de reglas matemáticas, procesos lógicos y cálculos estadísticos escritos en código. Su única función es aprender de los datos para encontrar patrones.

- Estado: Está “vacío”. No sabe nada sobre tu problema específico hasta que le pasas información.
- Analogía: Es como una receta de cocina que te dice exactamente qué pasos seguir para hacer un pastel (mezclar, hornear a cierta temperatura), pero no es el pastel en sí.
- Ejemplos: Regresión Lineal, Árboles de Decisión, Random Forest, K-Means, Redes Neuronales. Un algoritmo de Random Forest siempre funciona matemáticamente igual, sin importar si lo usas para predecir el clima o para detectar fraudes financieros.

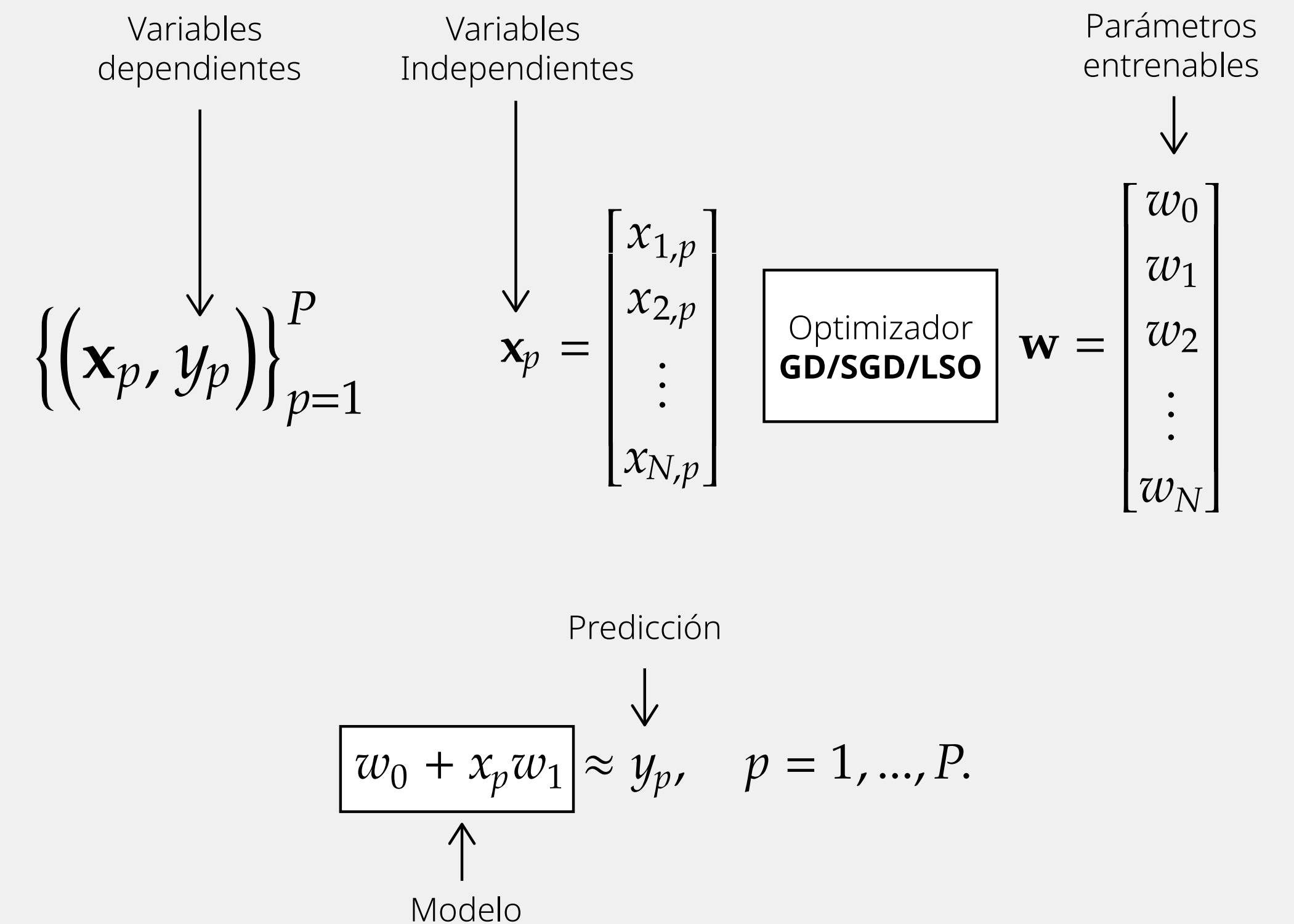
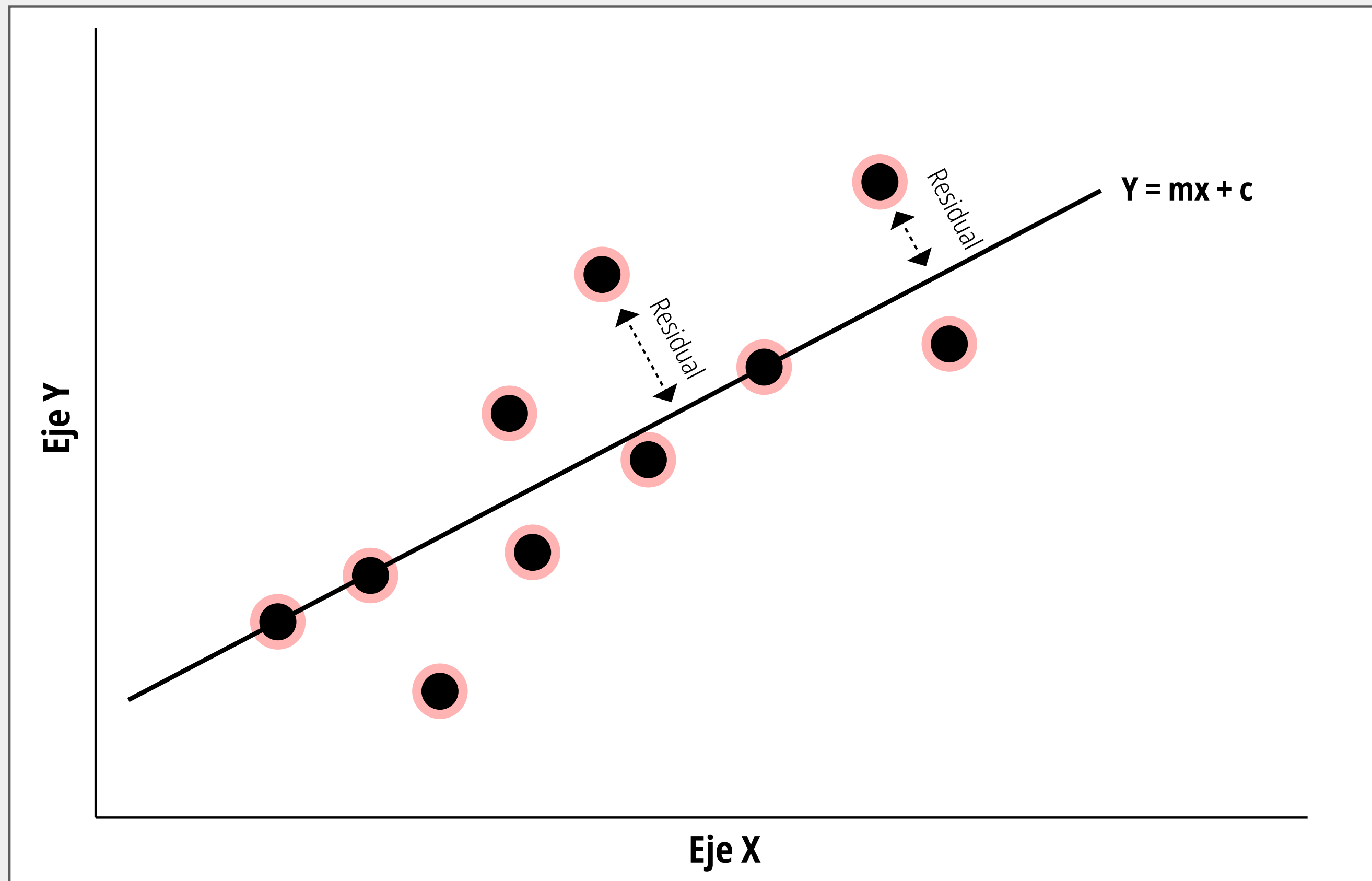
Modelo

Es el resultado (el artefacto) que obtienes después de que el algoritmo ha procesado tus datos de entrenamiento. Es la representación matemática de los patrones que el algoritmo logró extraer.

- Estado: Está “entrenado”. Contiene parámetros específicos (como pesos y sesgos en una red neuronal) que se ajustaron a tu información.
- Analogía: Es el pastel ya horneado y listo para comer. Tomaste la receta (algoritmo), le agregaste tus ingredientes específicos (datos), y el resultado es el modelo.
- Ejemplos: Un modelo de Regresión Lineal específicamente entrenado para predecir el precio de las casas en Costa Rica. Una vez que tienes el modelo, el algoritmo original ya hizo su trabajo de entrenamiento; ahora usas el modelo para hacer predicciones con datos nuevos.

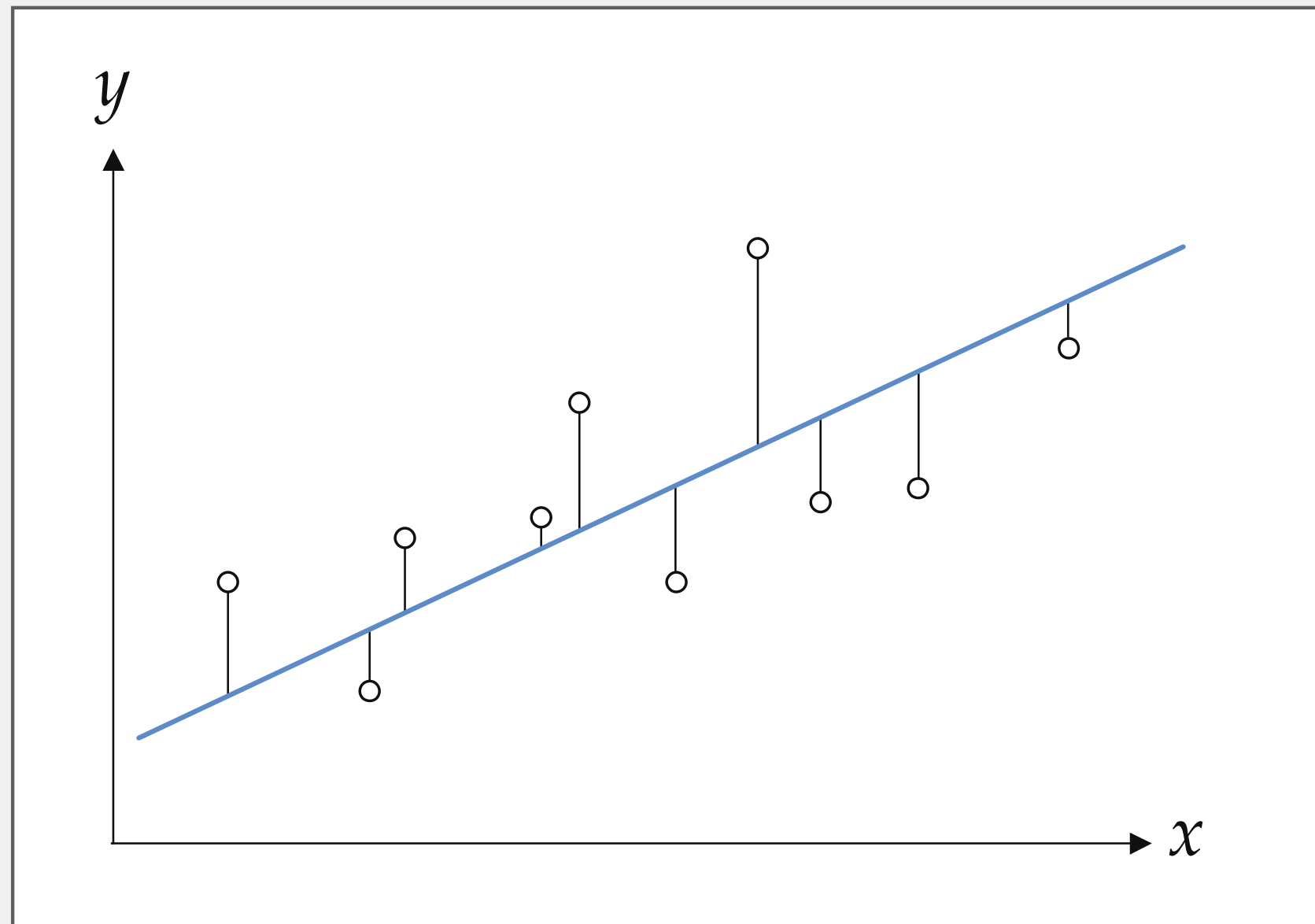
Tarea: Regresión Lineal (Forecasting)

El problema de la regresión lineal, o el ajuste de una línea representativa (o hiperplano en dimensiones superiores) a un conjunto de puntos de datos de entrada/salida.

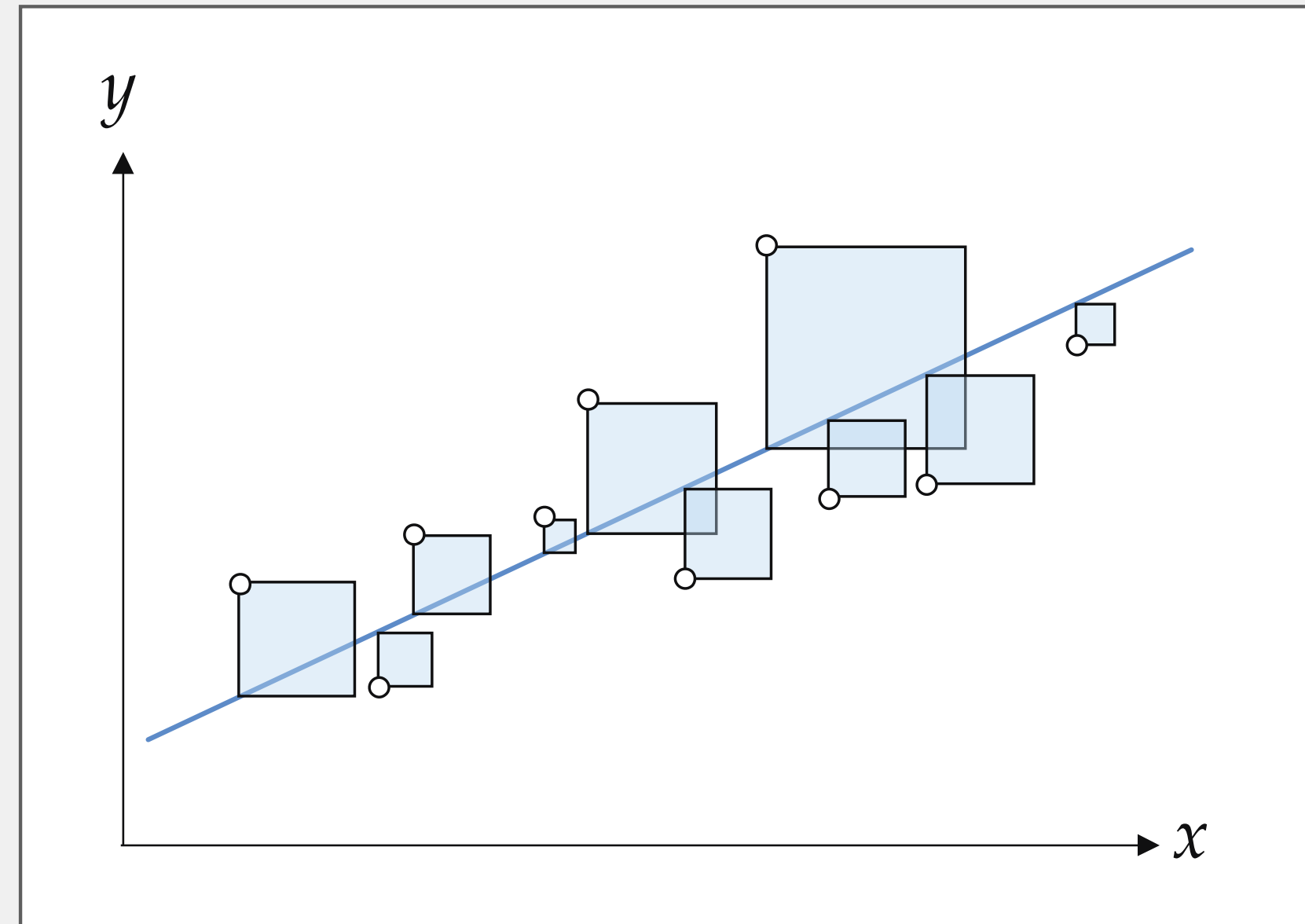


Un optimizador es un algoritmo encargado de ajustar los parámetros internos (pesos) de un modelo para minimizar el error en sus predicciones. Su objetivo principal es encontrar la configuración matemática ideal para que el modelo acierte lo más posible al comparar sus resultados con las etiquetas reales

Regresión Lineal: Mínimos Cuadrados



Un conjunto de datos bidimensional prototípico junto con una línea ajustada a los datos utilizando el marco de mínimos cuadrados (Least Squares), el cual tiene como objetivo recuperar el modelo lineal que minimiza la longitud total al cuadrado de las barras de error sólidas.



El error de mínimos cuadrados puede considerarse como el área total de los cuadrados azules, cuyos lados son las barras de error negras sólidas. La función de costo se denomina de mínimos cuadrados porque nos permite determinar un conjunto de parámetros cuya línea correspondiente minimiza la suma de estos errores al cuadrado.

Pérdida:

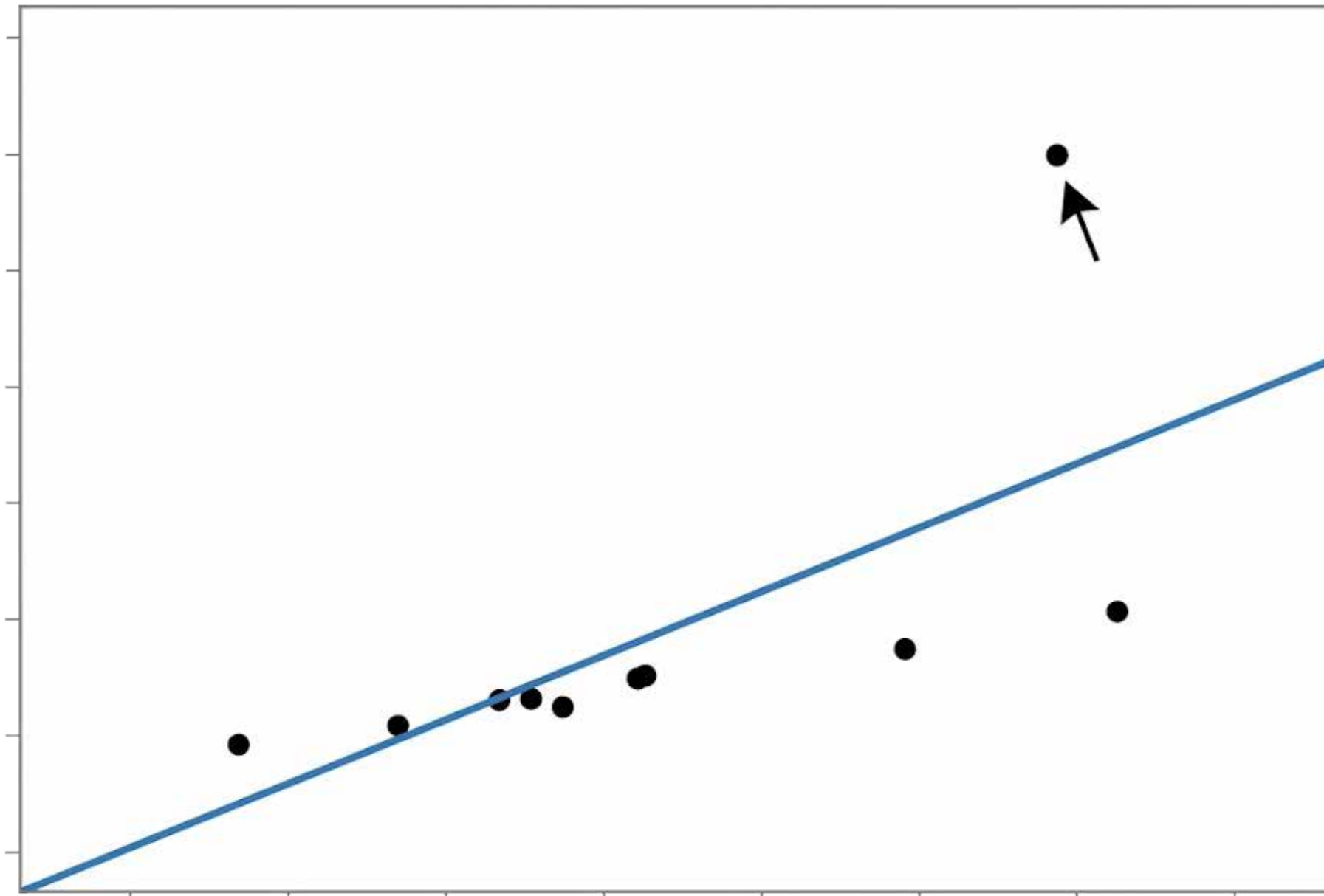
$$g(\mathbf{w}) = \frac{1}{P} \sum_{p=1}^P g_p(\mathbf{w}) = \frac{1}{P} \sum_{p=1}^P (\mathbf{x}_p^T \mathbf{w} - y_p)^2$$

$$\underset{\mathbf{w}}{\text{minimize}} \quad \frac{1}{P} \sum_{p=1}^P (\mathbf{x}_p^T \mathbf{w} - y_p)^2$$

- Convexidad: Si una función es convexa, tiene la forma de un tazón perfecto. Esto significa que solo tiene un punto más bajo (el mínimo global). No hay “valles falsos” (mínimos locales) donde un algoritmo pueda quedarse atascado.
- Infinitamente diferenciable: Significa que la curva es completamente suave. No tiene picos repentinos ni cortes abruptos.
- El resultado: Gracias a estas dos propiedades, matemáticamente puedes soltar cualquier algoritmo de optimización en esta función y, eventualmente, se deslizará hasta el fondo exacto para encontrar los pesos óptimos del modelo.

Regresión Lineal: Error absoluto

Una desventaja de usar el error cuadrático en el costo de Mínimos Cuadrados (como medida que luego minimizamos para obtener los parámetros óptimos de regresión lineal) es que elevar el error al cuadrado aumenta la importancia de los errores más grandes.



Al elevar los errores al cuadrado, esta función castiga de forma desproporcionada los errores grandes. Esto provoca que el modelo se desvíe intentando adaptarse a los valores atípicos (outliers), lo que genera un sobreajuste.

$$g_p(\mathbf{w}) = (\mathbf{x}_p^T \mathbf{w} - y_p)^2 \quad p = 1, \dots, P$$

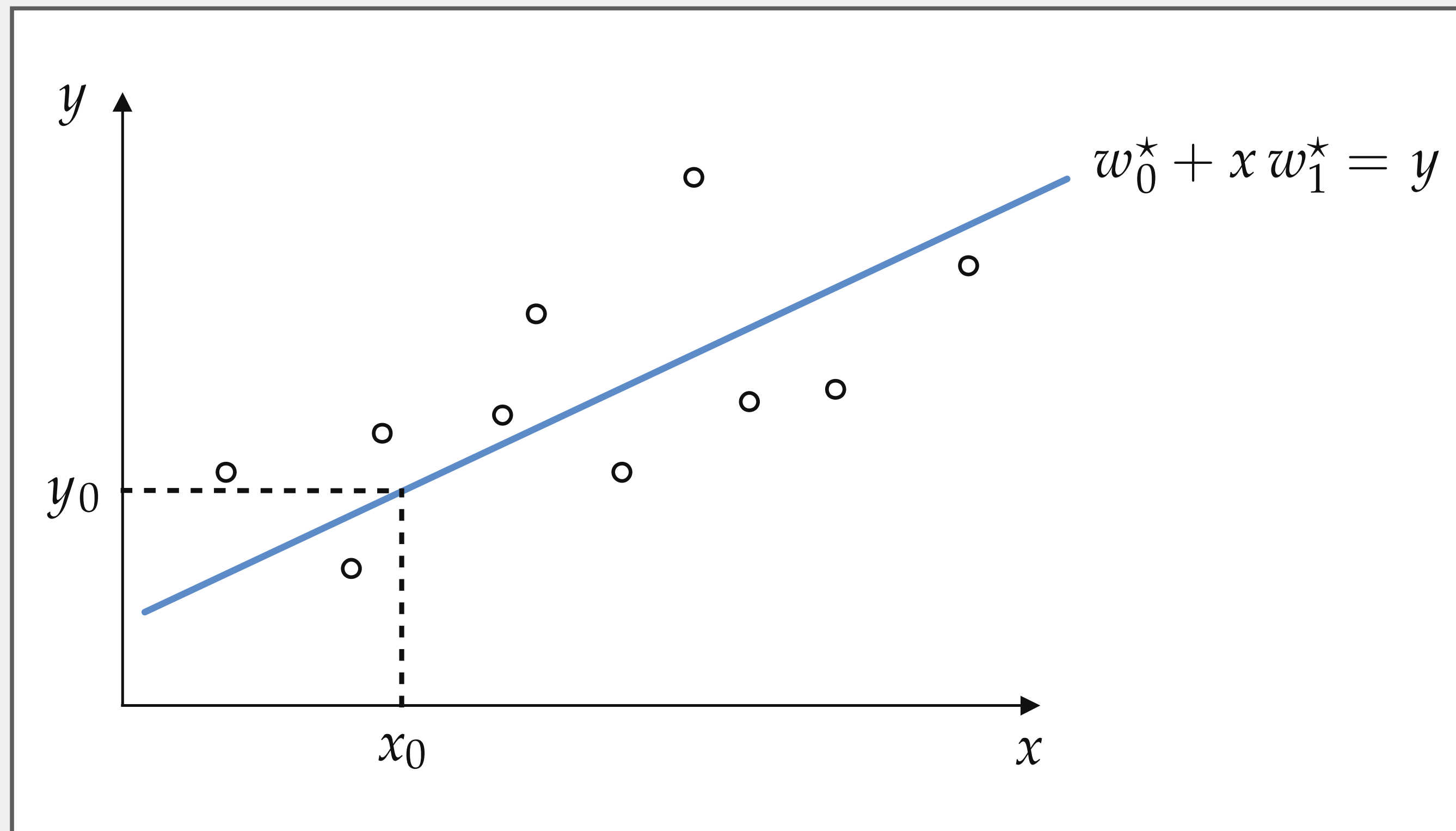
$$g_p(\mathbf{w}) = |\mathbf{x}_p^T \mathbf{w} - y_p| \quad p = 1, \dots, P.$$

$$g(\mathbf{w}) = \frac{1}{P} \sum_{p=1}^P g_p(\mathbf{w}) = \frac{1}{P} \sum_{p=1}^P |\mathbf{x}_p^T \mathbf{w} - y_p|.$$

Aunque la función de error absoluto es convexa, su segunda derivada es cero en casi todas partes. Esto nos impide utilizar métodos de optimización rápidos y avanzados (los de segundo orden), obligándonos a depender exclusivamente de algoritmos de orden cero o de primer orden.

Regresión Lineal: Evaluación

Una vez que hemos minimizado exitosamente una función de costo de regresión lineal, resulta sencillo determinar la calidad de nuestro modelo de regresión: simplemente evaluamos una función de costo utilizando nuestros pesos óptimos.



Para ello, introducimos los parámetros aprendidos del modelo junto con los datos en el costo de Mínimos Cuadrados, obteniendo el llamado Error Cuadrático Medio (o MSE, por sus siglas en inglés).

$$\text{MSE} = \frac{1}{P} \sum_{p=1}^P \left(\text{model}(\mathbf{x}_p, \mathbf{w}^*) - y_p \right)^2.$$

Para reducir el efecto de los valores atípicos (outliers) y otros valores grandes, a menudo se extrae la raíz cuadrada de este valor para usarla como métrica de calidad de regresión. A esto se le conoce como la Raíz del Error Cuadrático Medio (RMSE).

$$\text{MAD} = \frac{1}{P} \sum_{p=1}^P \left| \text{model}(\mathbf{x}_p, \mathbf{w}^*) - y_p \right|.$$

De igual manera, podemos emplear el costo de Mínimas Desviaciones Absolutas para determinar la calidad de nuestro modelo entrenado. Al introducir los parámetros aprendidos del modelo junto con los datos en este costo, se calculan las Desviaciones Absolutas Medias (o MAD, por sus siglas en inglés), que es precisamente lo que calcula esta función de costo.

Interludio: Meta-Learning

Después de desarrollar un único modelo, se podría pensar en cómo seguir mejorando su rendimiento. Un método que constantemente ha brindado una mejora en el rendimiento es utilizar un ensamble (o conjunto) de múltiples modelos en lugar de solo un modelo individual para realizar predicciones. Cada modelo en el ensamble se denomina aprendiz base (base learner).

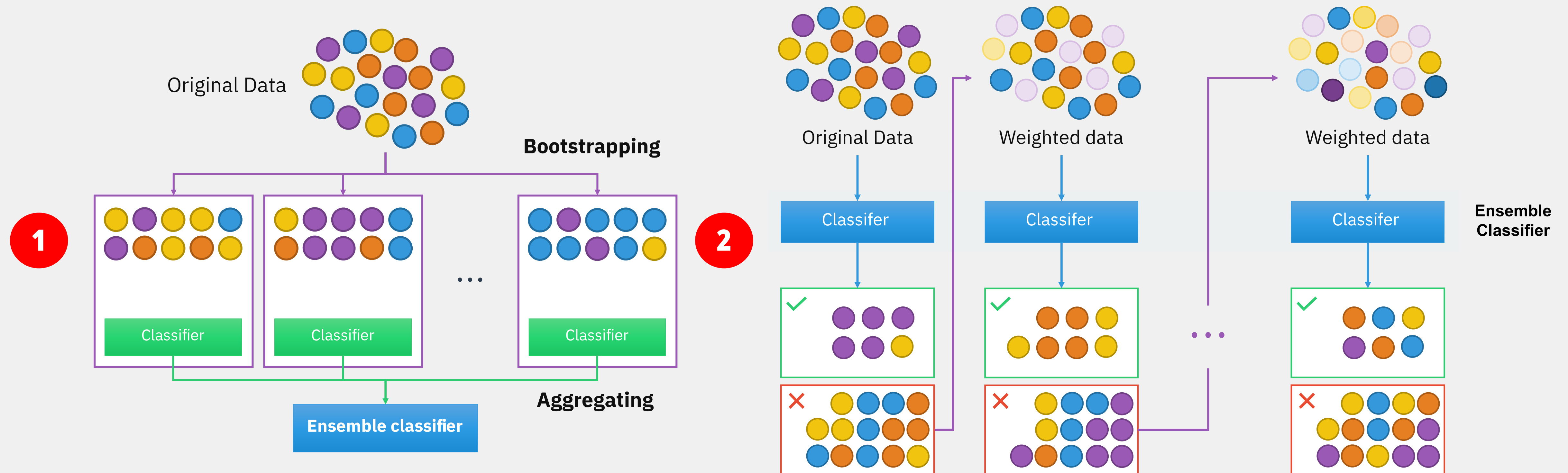
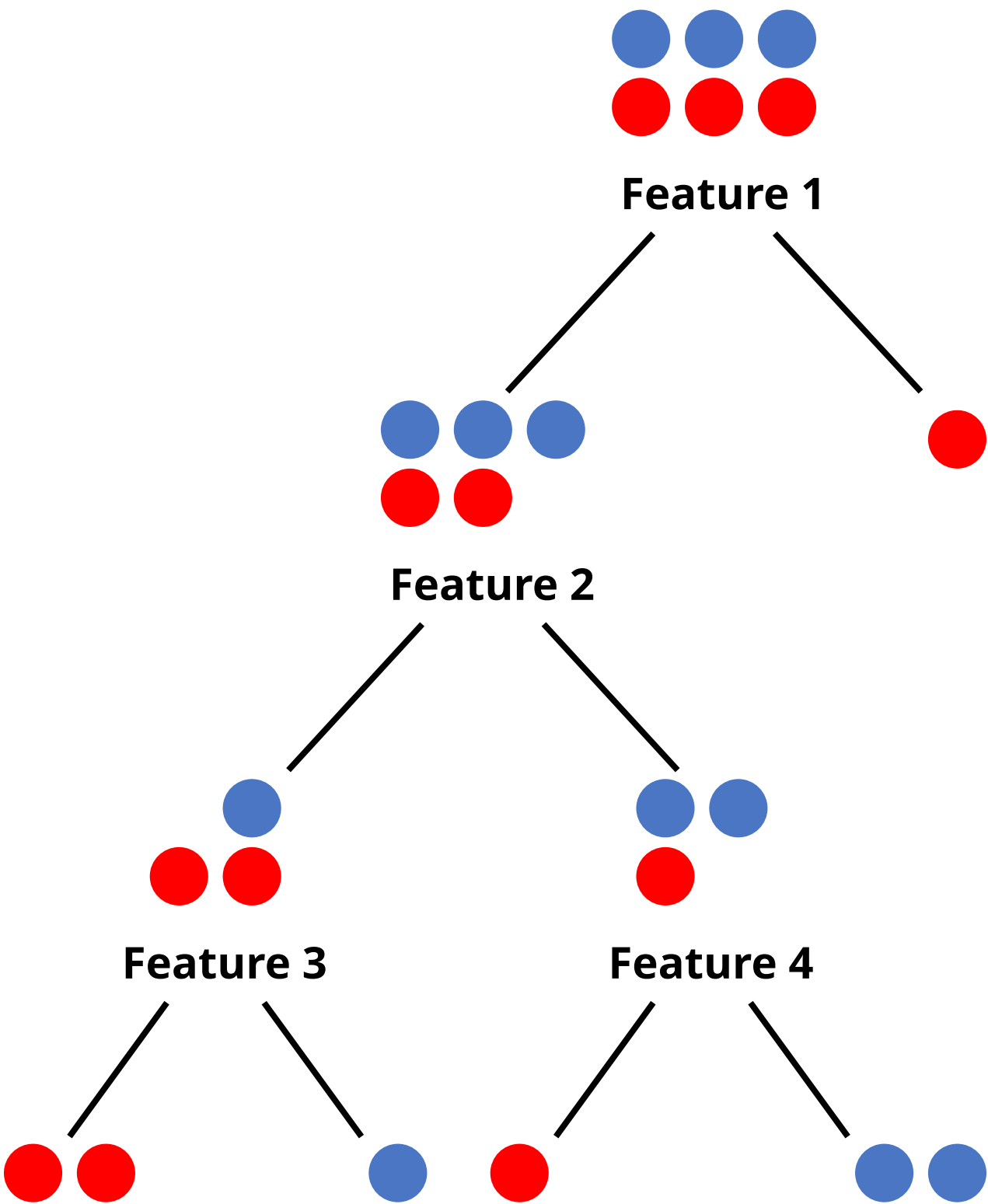


Figura: Bagging (1) y Boosting (2).

Los métodos de ensamble son menos preferidos en producción porque los ensambles son más complejos de desplegar y más difíciles de mantener. Sin embargo, siguen siendo comunes para tareas en las que una pequeña mejora en el rendimiento puede generar una enorme ganancia financiera, como predecir la tasa de clics (click-through rate) en los anuncios. Existen tres formas de crear un ensamble: bagging, boosting y stacking. Además de ayudar a mejorar el rendimiento, según varios artículos de revisión, los métodos de ensamble como boosting y bagging, junto con el remuestreo (resampling), han demostrado ser útiles con conjuntos de datos desbalanceados. En bagging se toman muestras con reemplazo para crear diferentes conjuntos de datos, llamados bootstraps. En boosting las muestras se vuelven a ponderar en función de qué tan bien las clasifica el primer clasificador; por ejemplo, a las muestras mal clasificadas se les asigna un peso mayor.

Regresión y Clasificación: CART

Las clases `DecisionTreeRegressor` y `DecisionTreeClassifier` implementan una versión optimizada del algoritmo CART (Classification and Regression Trees, o Árboles de Clasificación y Regresión).



A diferencia de otros algoritmos clásicos de árboles como ID3 o C4.5 (que pueden tener múltiples ramas por nodo), la implementación CART de scikit-learn construye estrictamente árboles binarios. Esto significa que cada nodo interno siempre evalúa una condición de verdadero/falso y se divide exactamente en dos ramas secundarias (por ejemplo, “¿La edad es mayor a 30?”).

Parámetro	Algoritmo	Descripción y Uso Estratégico
criterion	Clasificador	Mide la calidad del corte. Opciones: ‘gini’ (por defecto, más rápido), ‘entropy’ (ganancia de información) o ‘log_loss’.
criterion	Regresor	Opciones: ‘squared_error’ (MSE, por defecto), ‘absolute_error’ (MAE, más robusto a valores atípicos), ‘friedman_mse’ o ‘poisson’.
max_depth	Ambos	Profundidad máxima del árbol. Crítico para el control de memoria y sobreajuste. Si es None, los nodos se expanden hasta que las hojas sean puras, lo cual es computacionalmente costoso y garantiza sobreajuste en datasets complejos.
min_samples_split	Ambos	Mínimo de muestras necesarias para dividir un nodo interno (por defecto 2). Aumentar este valor (ej. 10 o 50) suaviza el modelo y evita crear reglas matemáticas para casos demasiado específicos.
min_samples_leaf	Ambos	Mínimo de muestras que deben quedar en un nodo hoja. Si una división deja una hoja con menos elementos, no se realiza. Excelente para filtrar ruido.
max_features	Ambos	Número de características a considerar al buscar el mejor corte. Opciones: ‘sqrt’, ‘log2’, o un número entero. Reduce drásticamente el tiempo de cómputo en conjuntos de datos anchos y añade aleatoriedad estructural.
class_weight	Clasificador	Fundamental para cuando las clases están fuertemente desbalanceadas . Pasarle ‘balanced’ ajusta automáticamente los pesos de las clases inversamente proporcionales a su frecuencia en los datos de entrada, forzando al modelo a prestar atención a la clase minoritaria.

Regresión y Clasificación: RandomForest (Classifier / Regressor)

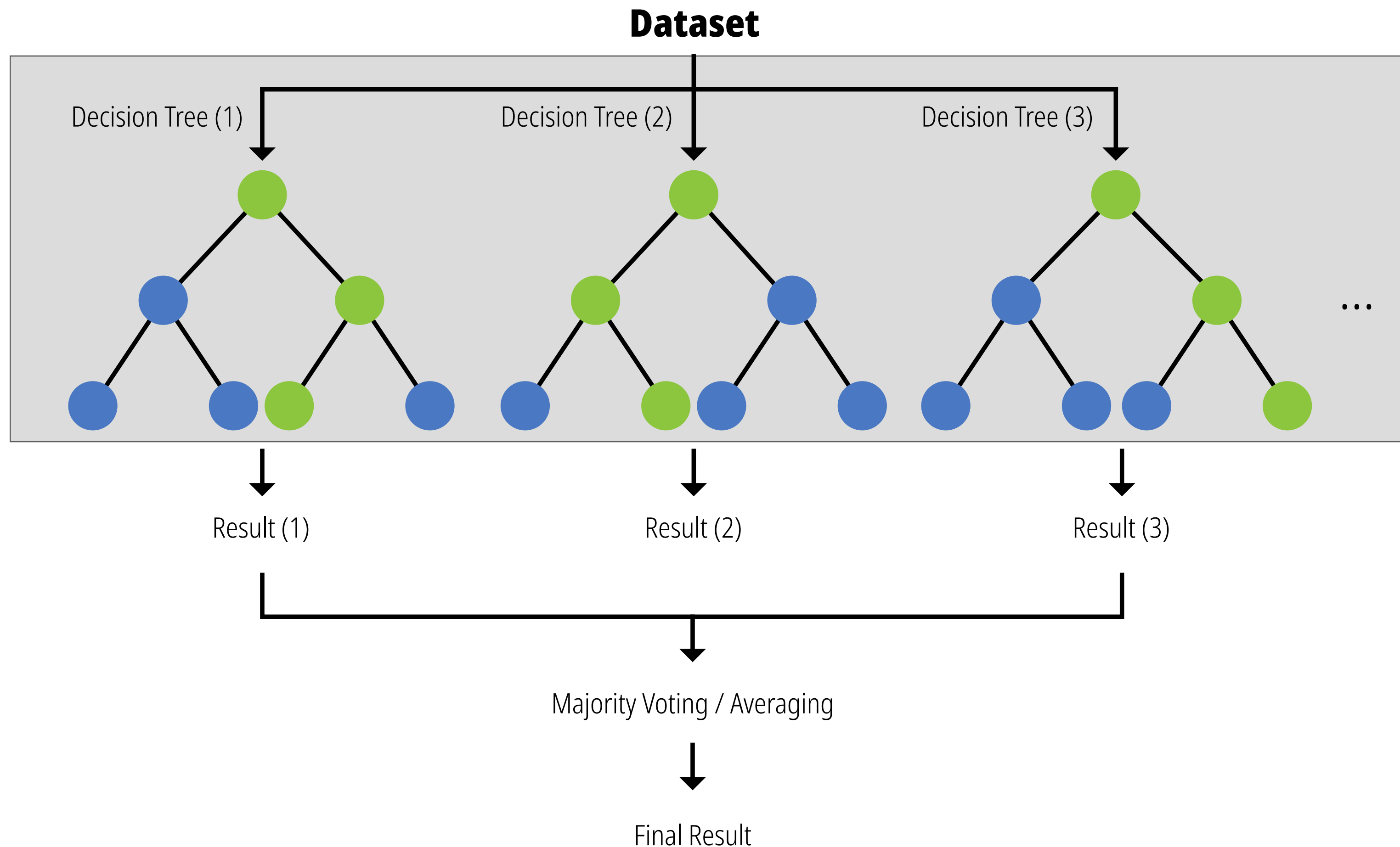
Parámetros del Bosque (ensamble)

Controlan cómo se construye el grupo de árboles en su totalidad.

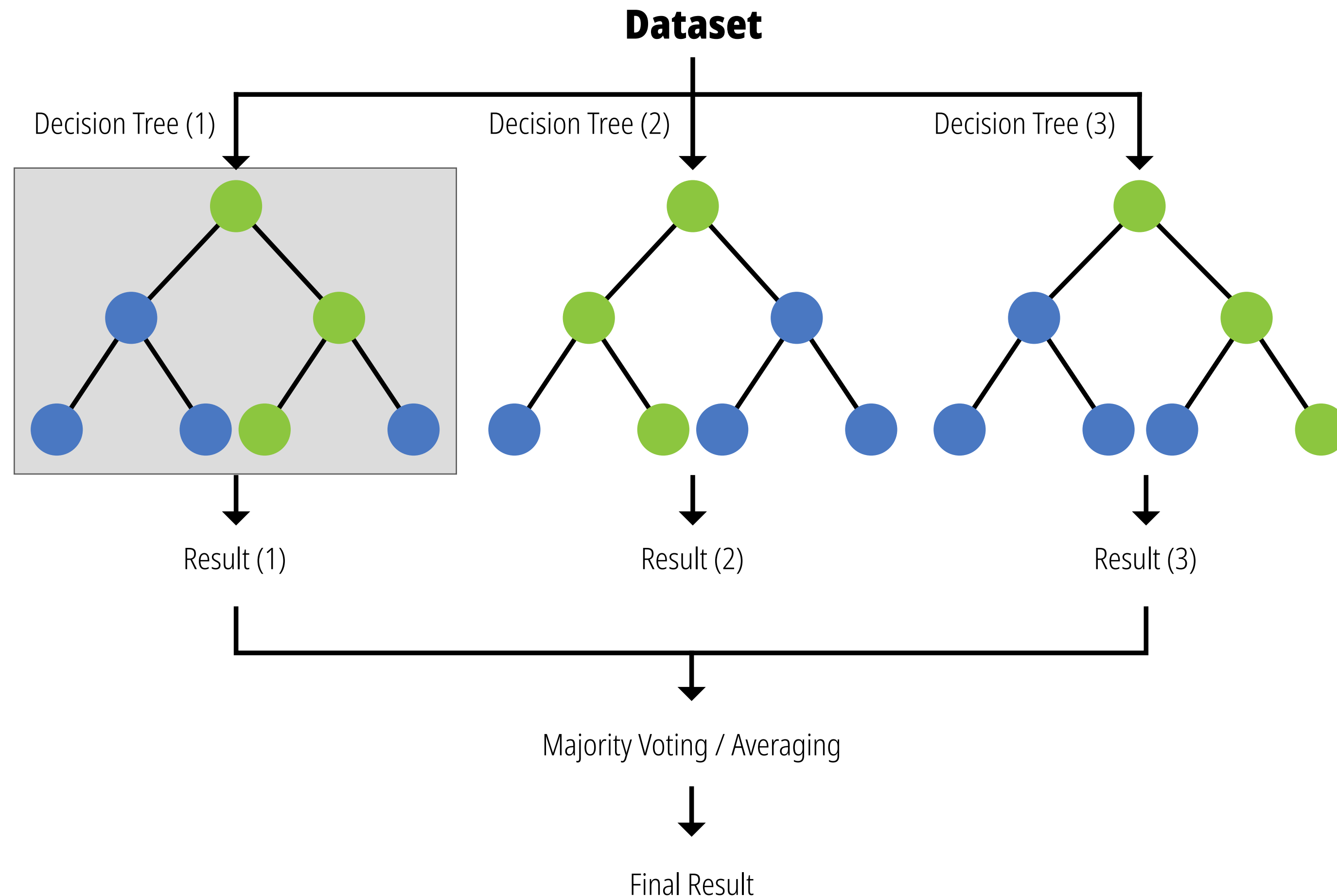
n_estimators (Por defecto: 100): Es la cantidad de árboles en el bosque. En general, más árboles significan un modelo más robusto y estable frente a variaciones en los datos. La ventaja del Random Forest es que aumentar este valor rara vez causa sobreajuste (overfitting), aunque sí incrementa el tiempo de entrenamiento y el uso de memoria.

bootstrap (Por defecto: True): Determina si los árboles se entrenan usando submuestras aleatorias de los datos originales con reemplazo. Si lo cambias a False, todo el conjunto de datos original se utilizará para construir cada árbol.

oob_score (Por defecto: False): Si usas bootstrap, siempre quedarán datos sin utilizar durante el entrenamiento de un árbol específico (conocidos como datos Out-of-Bag). Si activas este parámetro, el modelo usará esos datos sobrantes para autoevaluar su precisión sin necesidad de que dividas un conjunto de validación por separado.



Regresión y Clasificación: RandomForest (Classifier / Regressor)



Parámetros de los Árboles (Control de crecimiento)

Estos parámetros le dicen a cada árbol individual qué tanto puede crecer. Son tu principal herramienta para controlar el sobreajuste.

max_depth (Por defecto: None): Controla la profundidad máxima de cada árbol. Si lo dejas en None, los nodos se expandirán hasta que todas las hojas contengan una sola clase (lo que suele llevar a un sobreajuste masivo). Limitar este valor fuerza al modelo a encontrar patrones más generales.

...

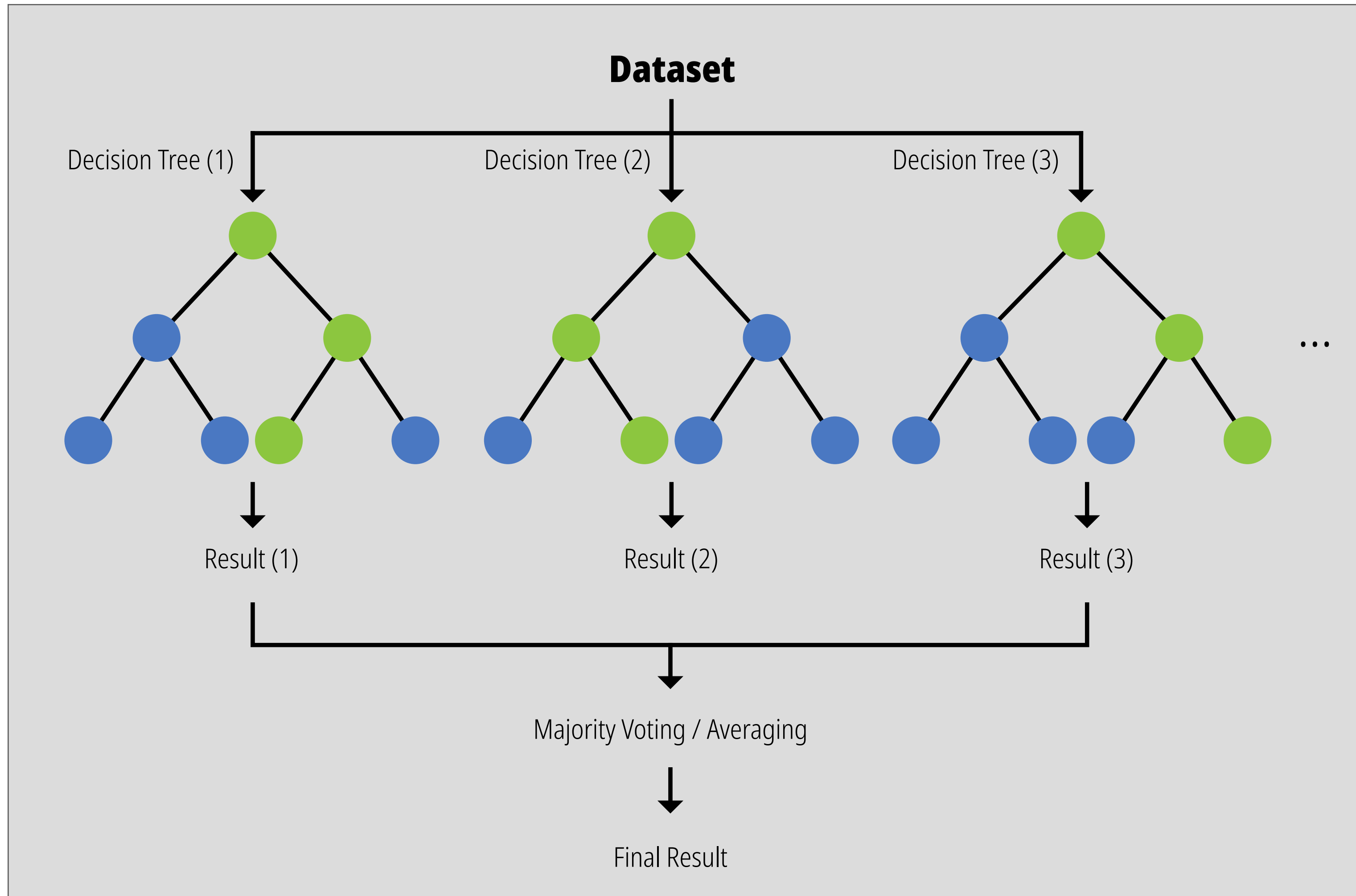
min_samples_split (Por defecto: 2): El número mínimo de muestras que un nodo interno debe tener para que se le permita dividirse en dos nuevas ramas.

min_samples_leaf (Por defecto: 1): El número mínimo de datos que debe quedar al final de una rama (en un nodo hoja). Aumentar este valor suaviza las predicciones del modelo y evita que el árbol cree reglas ultra-específicas para un solo dato anómalo.

max_features (Por defecto: 'sqrt'): El número máximo de características (columnas) que cada árbol puede evaluar al buscar la mejor forma de dividir los datos. Limitar esto es clave: garantiza que los árboles del bosque sean diferentes entre sí (decorrelacionados), lo que permite que el ensamble funcione correctamente.

Nota de implementación: Cuando configuras un parámetro del árbol (como `max_depth=5`) directamente dentro de la instancia del `RandomForestRegressor`, el bosque se encarga de inyectar esa regla internamente a los 100 árboles (o los que indiques en `n_estimators`) que genere durante el tiempo de ejecución.

Regresión y Clasificación: RandomForest (Classifier / Regressor)



Parámetros Computacionales y Prácticos

No afectan las matemáticas del modelo, pero sí la forma en que lo programas y ejecutas.

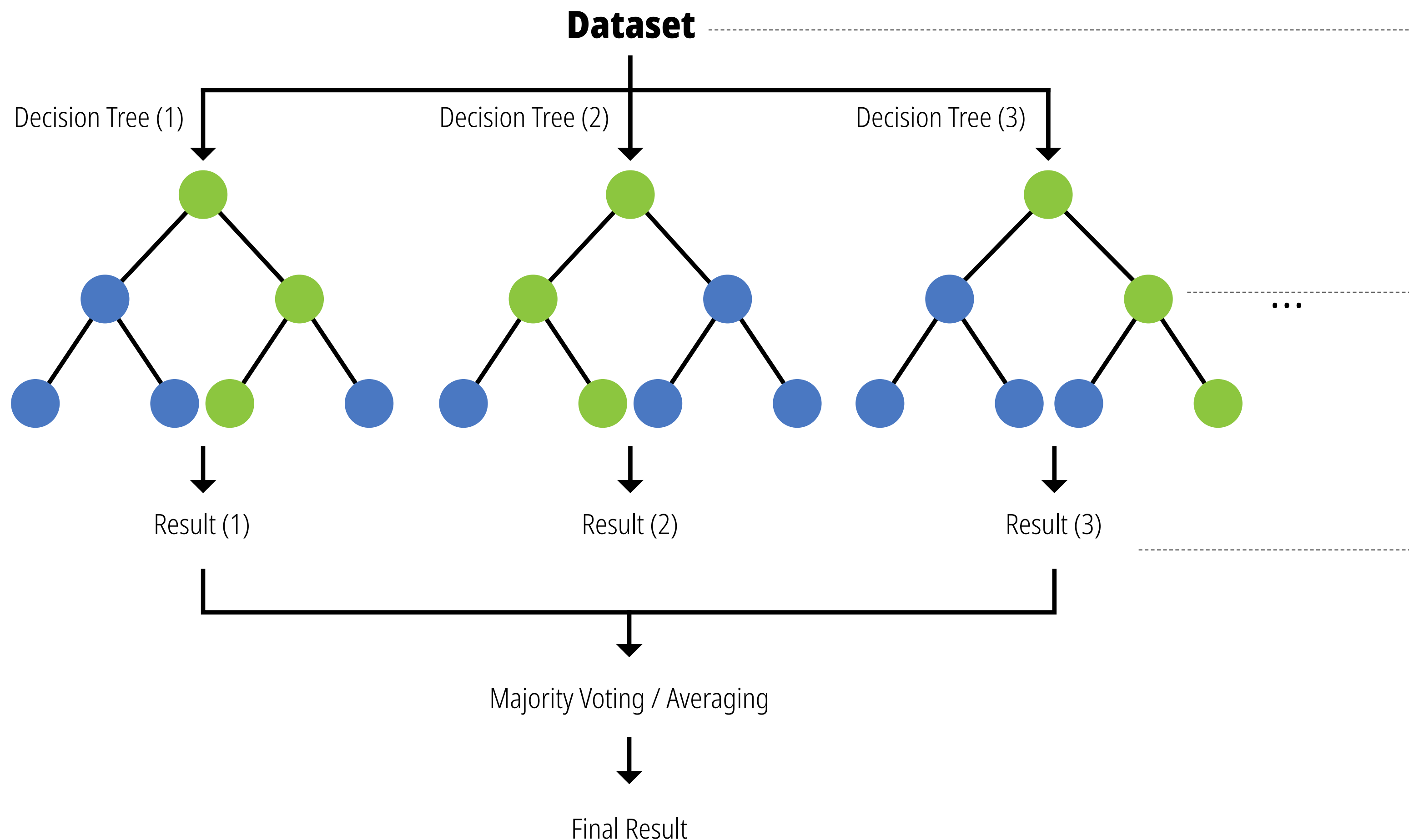
n_jobs (Por defecto: None): El número de núcleos de tu procesador que se usarán para entrenar el modelo en paralelo. Si le asignas -1, scikit-learn usará todos los núcleos disponibles en tu computadora, lo que acelera el entrenamiento drásticamente.

random_state (Por defecto: None): La semilla del generador de números aleatorios. Fijar un número (por ejemplo, 42 o 0) garantiza que si corres el código nuevamente, obtendrás exactamente los mismos resultados.

class_weight (Por defecto: None): Muy útil si tienes datos desbalanceados (por ejemplo, 99% de transacciones legítimas y 1% de fraudes). Puedes configurarlo como 'balanced' para que el algoritmo ajuste automáticamente los pesos y le dé más importancia a la clase minoritaria durante el entrenamiento.

Regresión y Clasificación: RandomForest (Classifier / Regressor)

Un árbol de Random Forest se ramifica aplicando divisiones binarias recursivas (haciendo preguntas de tipo "sí" o "no" sobre las variables). Para garantizar que cada árbol sea único, el algoritmo utiliza dos técnicas: selección aleatoria de datos y selección aleatoria de variables.



Muestreo aleatorio (Bagging): Para crear cada árbol, el algoritmo selecciona una muestra de los datos originales con reemplazo (los datos pueden repetirse). Esto significa que cada árbol se entrena con un subconjunto de datos ligeramente distinto.

1

Selección aleatoria de características: En cada nodo donde el árbol necesita ramificarse, el algoritmo no evalúa todas las variables disponibles para decidir el mejor corte. Elige aleatoriamente un grupo más pequeño (por ejemplo, la raíz cuadrada del total de variables) y solo busca la mejor división entre estas.

2

Criterio de división: Para saber exactamente dónde partir la rama, el algoritmo busca la pregunta que logre la mayor ganancia de información. Utiliza fórmulas matemáticas para medir qué tan puros quedan los grupos tras la división.

3

Regresión y Clasificación: RandomForest (Classifier / Regressor)

Las métricas matemáticas (Gini, Entropía o Varianza) no se aplican al bosque entero a la vez, sino en cada bifurcación individual de cada árbol mientras se están construyendo.

$$Gini = 1 - \sum_{i=1}^C p_i^2$$

$$Entropía = - \sum_{i=1}^C p_i \log_2(p_i)$$

$$Varianza = \frac{1}{N} \sum_{i=1}^N (y_i - \bar{y})^2$$

La pureza es la medida de la homogeneidad de clase en un nodo particular; por ejemplo, el porcentaje de casos A vs casos B.

Imagina que el algoritmo está a punto de crear una rama. Tiene un grupo de datos en un nodo. Primero, calcula la impureza (digamos, el Índice de Gini) de ese grupo completo. Si el grupo está mezclado (ej. 50% transacciones fraudulentas y 50% legítimas), la impureza será alta.

“Random” del Random Forest: En lugar de evaluar todas las variables disponibles (edad, monto, ubicación, etc.), el algoritmo selecciona un subconjunto aleatorio de variables. Ejemplo: Si evalúa la variable “Monto”, probará cortes como: ¿Monto < \$100?, ¿Monto < \$500?, ¿Monto < \$1000?

Por cada corte simulado, los datos se dividen temporalmente en dos grupos (Nodo Izquierdo y Nodo Derecho). El algoritmo aplica la fórmula matemática elegida (Gini o Entropía) a cada uno de estos nuevos grupos para ver qué tan “puros” quedaron.

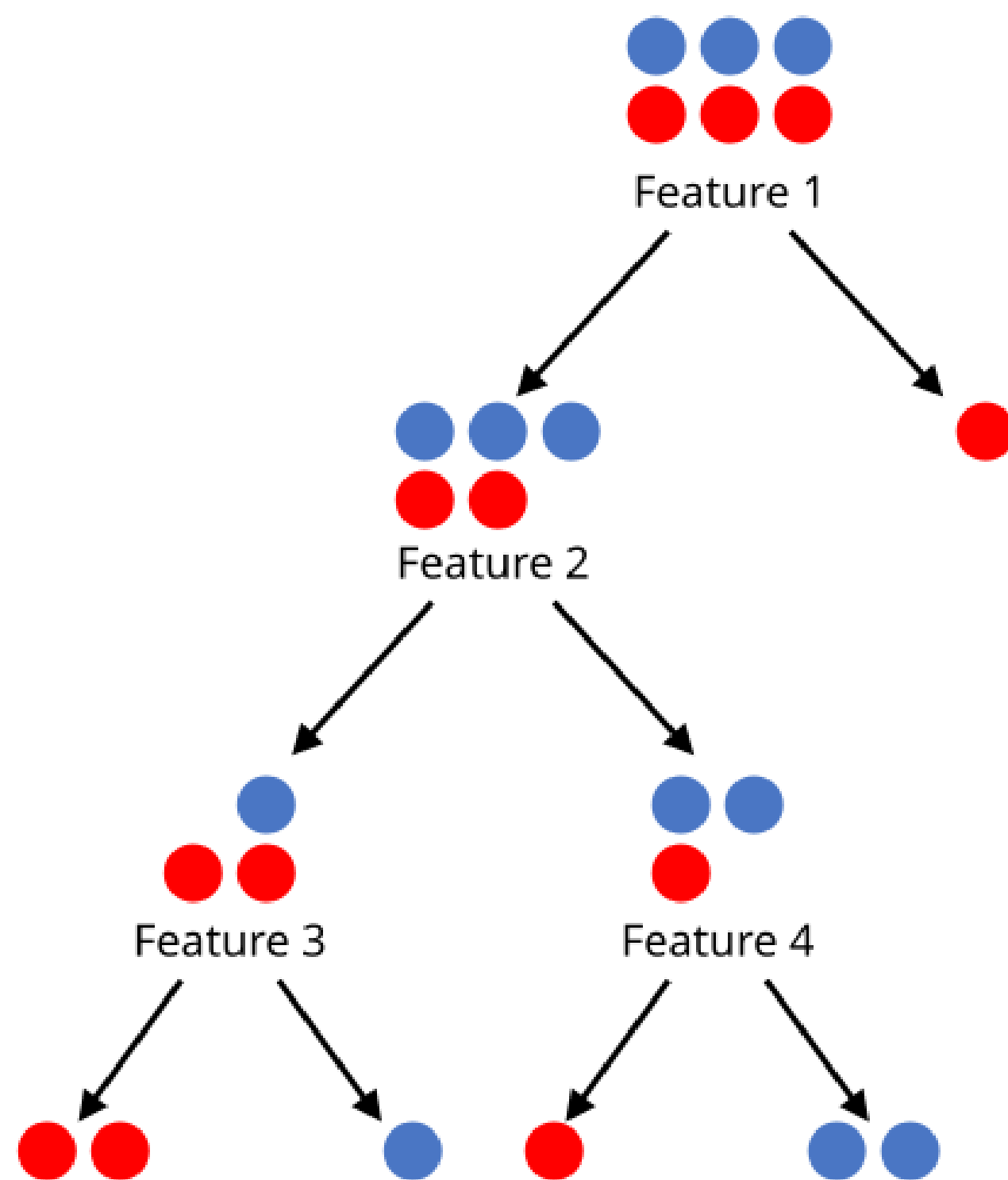
El algoritmo no se fija solo en un lado, sino en el resultado global del corte. Calcula el Gini Ponderado de ambos nodos hijos (dándole más peso al nodo que tenga más cantidad de datos). Luego, resta este resultado de la impureza que tenía el Nodo Padre originalmente. El resultado de esa resta es la **Ganancia de Información**.

Después de evaluar cientos de cortes posibles en ese subconjunto de variables, el algoritmo ejecuta la división real utilizando la variable y el punto de corte exacto que haya logrado la **mayor ganancia de información**.

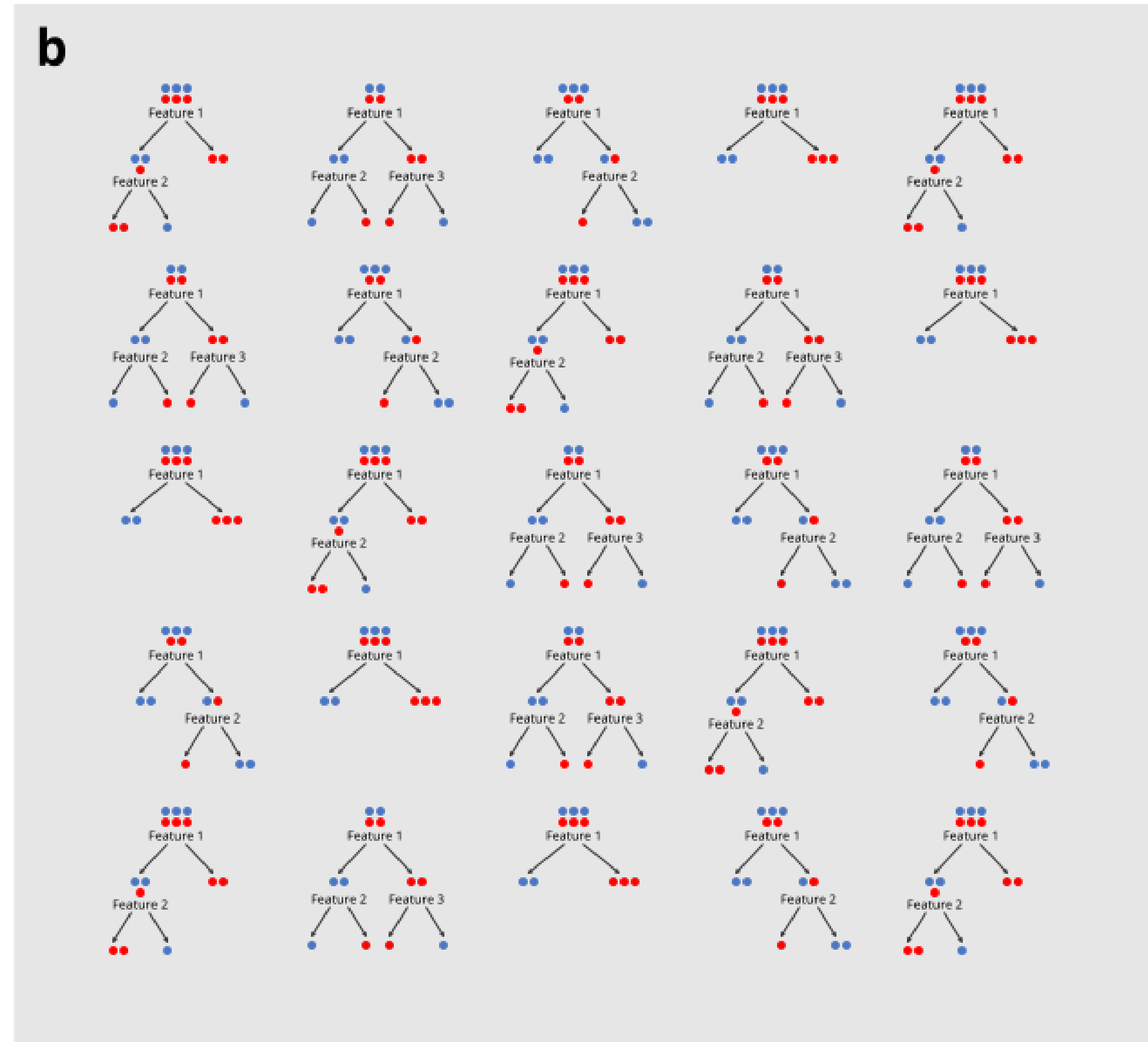
Regresión y Clasificación: CART vs RandomForest

RF (Random Forest o Bosque Aleatorio) es simplemente una colección de árboles CART (de ahí el nombre de “bosque”) que se construyen a partir de muestras bootstrap del conjunto de datos original. La clasificación se determina mediante una votación por mayoría entre los árboles del bosque.

a



b



La división cesa cuando se alcanza un umbral de pureza predeterminado; los nodos que no se dividen más se denominan nodos terminales. Los sujetos en los nodos terminales se contabilizan y, a continuación, una mayoría simple determina la clasificación del nodo.

Regresión y Clasificación: Otros tipos de árboles

Además del RandomForest, scikit-learn tiene un ecosistema muy robusto de algoritmos basados en árboles para problemas de regresión. Se dividen principalmente en tres categorías: el árbol base, y los ensambles (Bagging y Boosting)

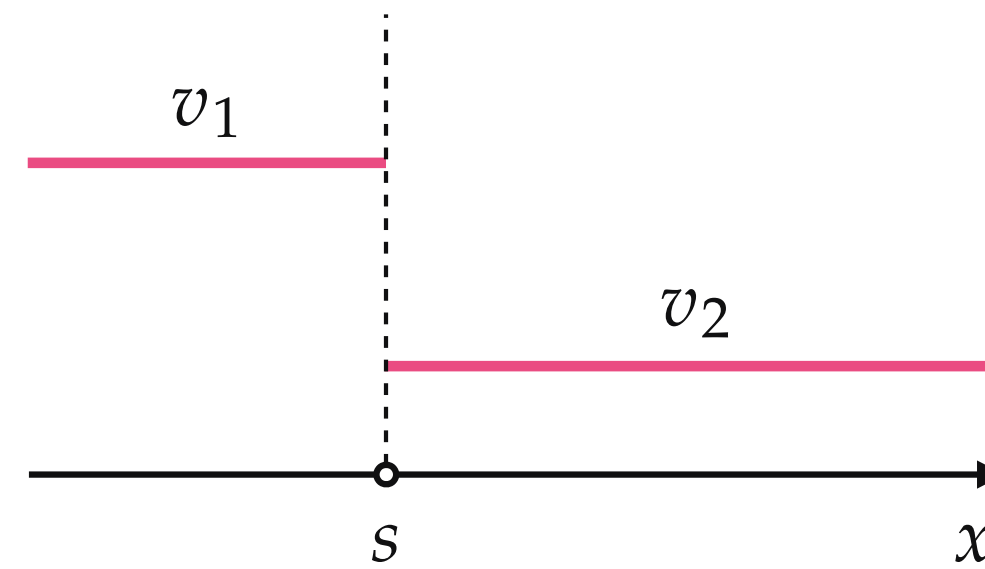
Modelo	Valor	Cuándo elegirlo
DecisionTreeRegressor: Es la contraparte directa del clasificador que acabamos de discutir. En lugar de usar la impureza de Gini o la Entropía, decide cómo dividir los datos buscando minimizar el error (generalmente el MSE o el MAE).	Es rápido e interpretable, pero muy propenso al sobreajuste si se usa de forma individual sin limitar su profundidad.	Cuando necesitas explicabilidad total del negocio y poder graficar el 100% de la lógica.
ExtraTreesRegressor: Es una variación directa del Random Forest. Mientras que el Random Forest busca el mejor punto de corte matemático para cada variable, <i>Extra Trees</i> genera puntos de corte al azar y se queda con el mejor de esos umbrales aleatorios.	Es más rápido de entrenar que un Random Forest y a menudo generaliza mejor, ya que introduce un nivel extra de aleatoriedad que ayuda a evitar la memorización de los datos.	Cuando tu Random Forest sigue sufriendo de sobreajuste o necesitas optimizar los tiempos de entrenamiento.
HistGradientBoostingRegressor: En lugar de ordenar y evaluar cada punto de datos exacto para hacer un corte, agrupa los valores continuos en contenedores o “histogramas”.	Es exponencialmente más rápido en conjuntos de datos grandes (más de 10,000 muestras) y, a diferencia de los demás modelos estándar de la librería, soporta valores nulos (NaN) nativamente en el código sin que el algoritmo se rompa	Cuando tienes un dataset mediano/grande, valores incompletos, y necesitas exprimir el máximo rendimiento predictivo posible.
GradientBoostingRegressor: El enfoque clásico. Entrena árboles secuencialmente para predecir los “residuos” (la diferencia entre el valor real y la predicción actual).	Tremendamente preciso y suele dar mejores resultados finales que el Random Forest.	Para datasets pequeños o medianos donde se busca superar la precisión de un Random Forest tradicional.

Regresión y Clasificación: Árboles

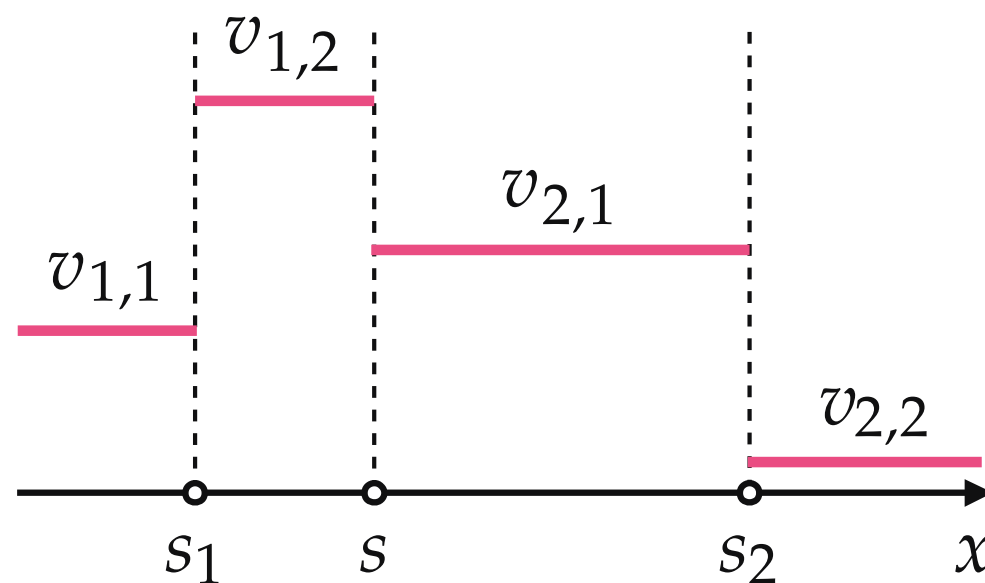
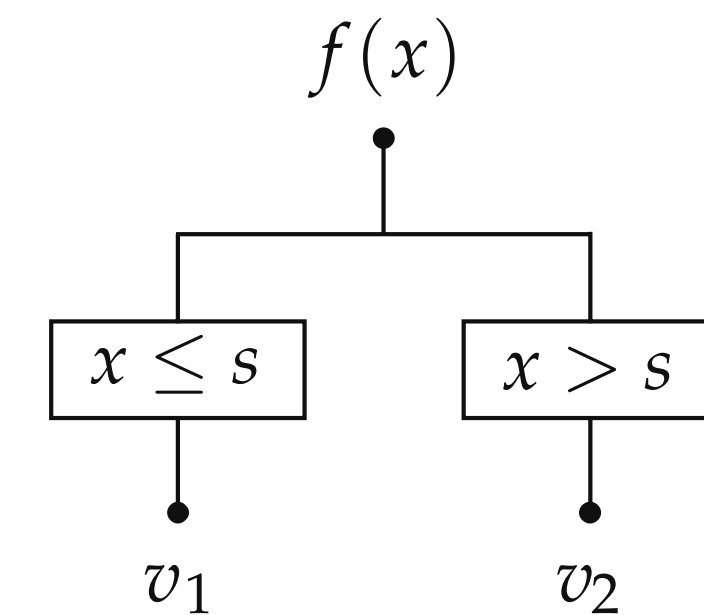
Los "tocones" (stumps) son el modelo base más simple para el aprendizaje basado en árboles. Se definen matemáticamente como una función de un solo paso o escalón que depende de tres parámetros: un punto de división (s) y dos valores de salida conocidos como "hojas" (v_1 y v_2).

1
$$f(x) = \begin{cases} v_1 & x \leq s \\ v_2 & x > s \end{cases}$$

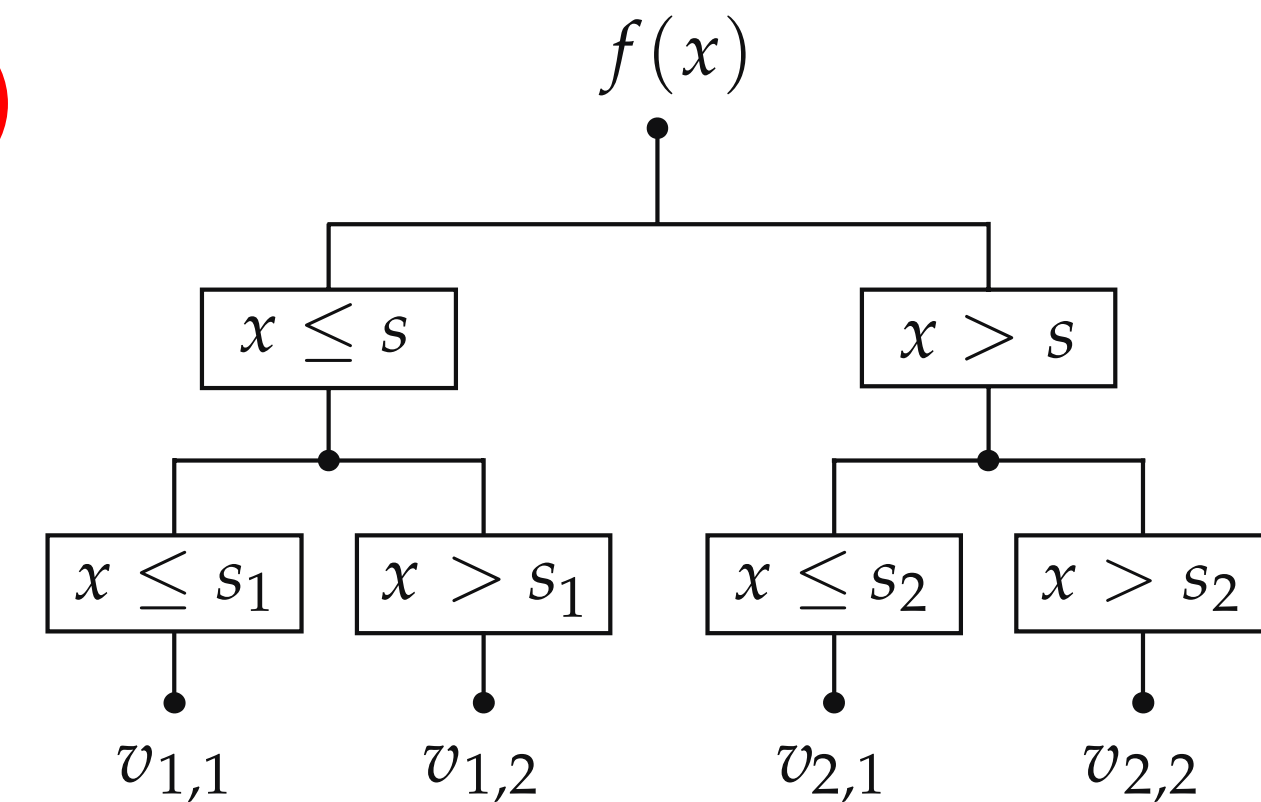
El aproximador universal basado en árboles más básico, el tocón, es una función escalonada simple.



2



3



Laboratorio 1: Regresión Lineal

- Regresión lineal al desnudo (+18)

Laboratorio 2: Regresión lineal y Random Forest

- California Housing
- EXTRA: Pesos y ONNX para producción.



Laboratorio 3: Árboles

- Clasificación y Regresión

Laboratorio 4: Tree Plot

- Decision Tree
- Random Forest



EDA

El Análisis Exploratorio de Datos busca entender la “materia prima” de tu proyecto antes de aplicar modelos más complejos.

Sus funciones principales incluyen:

Resumir características: Obtener una visión general de cómo se distribuyen los datos. Detectar anomalías: Identificar valores atípicos (outliers) o datos faltantes y corruptos.

Encontrar patrones: Revelar tendencias ocultas y correlaciones entre diferentes variables.

Formular hipótesis: Generar nuevas preguntas e ideas basadas en lo que los datos revelan.

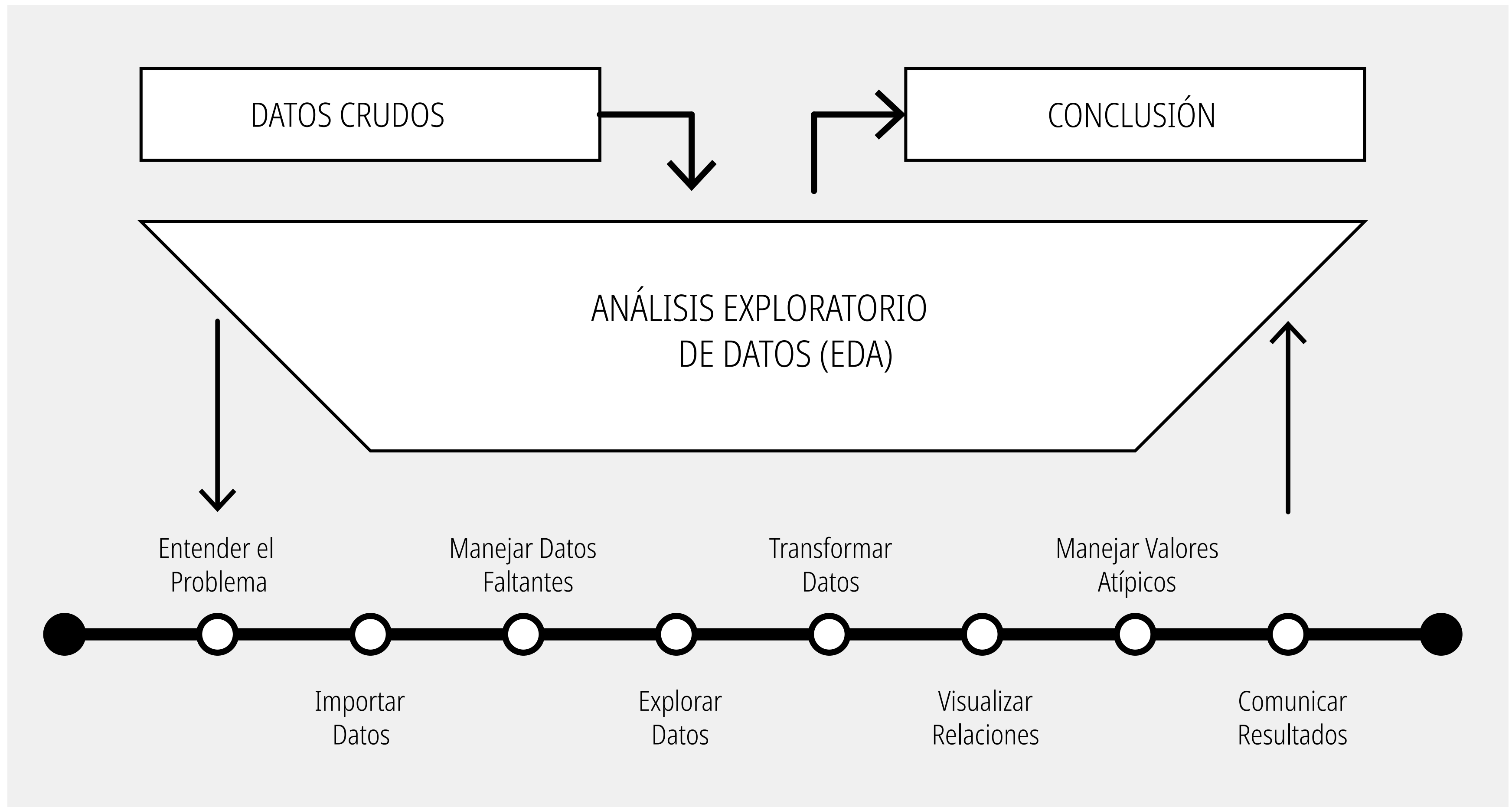
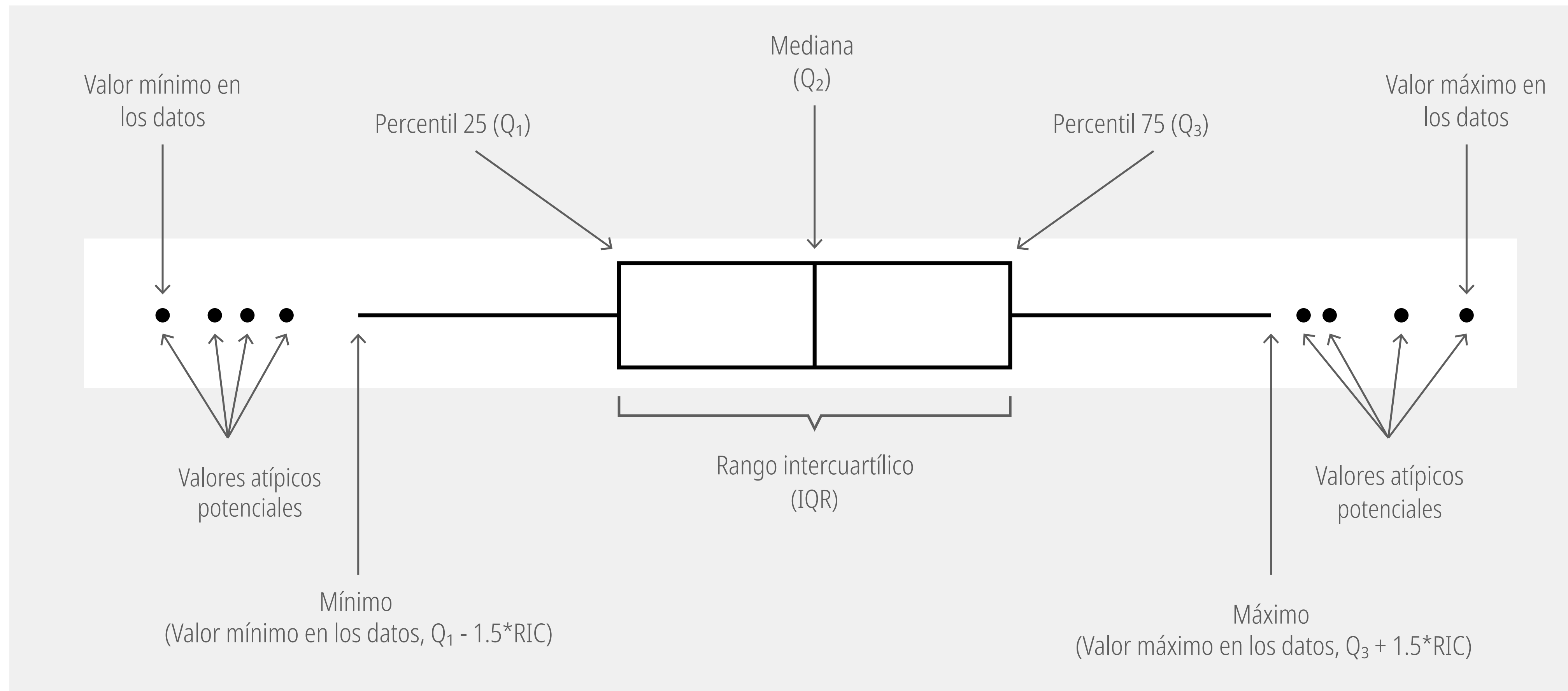


Diagrama de Box-Plot

La verdadera potencia del boxplot radica en su robustez frente a los valores extremos. Si utilizas un promedio matemático (la media) para evaluar el comportamiento de un sistema, un par de valores gigantescos pueden distorsionar todo el resultado. El boxplot, al basarse en la mediana y los cuartiles, no se deja engañar por esos extremos.

Un diagrama de caja (o boxplot) es una representación gráfica estandarizada que resume visualmente cómo se distribuye un conjunto de datos numéricos a través de sus cuartiles.

A diferencia de un histograma que muestra la frecuencia, el boxplot se centra en mostrar la dispersión, la tendencia central y, de manera crucial, aislar las anomalías.

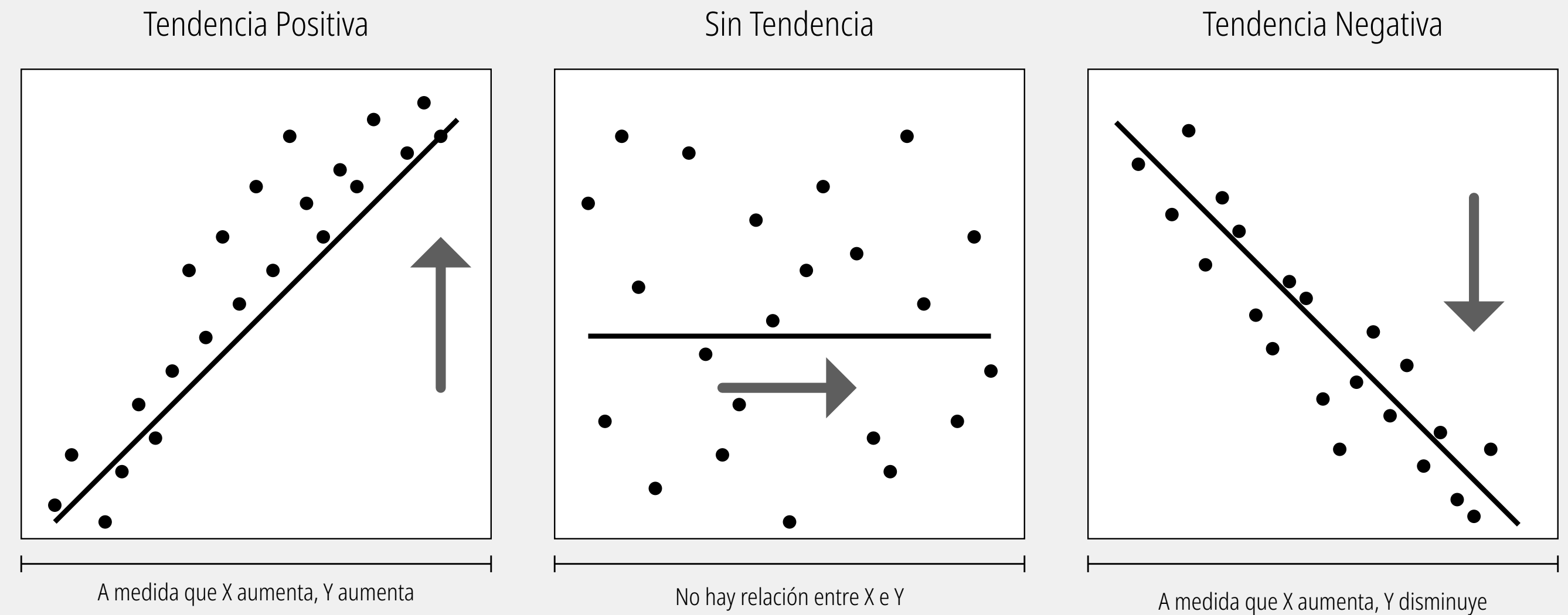


os Bigotes (Mínimo y Máximo calculados): Son las líneas que se extienden hacia afuera de la caja. No representan necesariamente el valor más pequeño o más grande de todos tus datos, sino el límite de lo que se considera un comportamiento "normal".

Matriz de correlación

Una matriz de correlación es una tabla que muestra los coeficientes entre muchos pares de variables. Es una herramienta clave para el análisis exploratorio de datos (EDA).

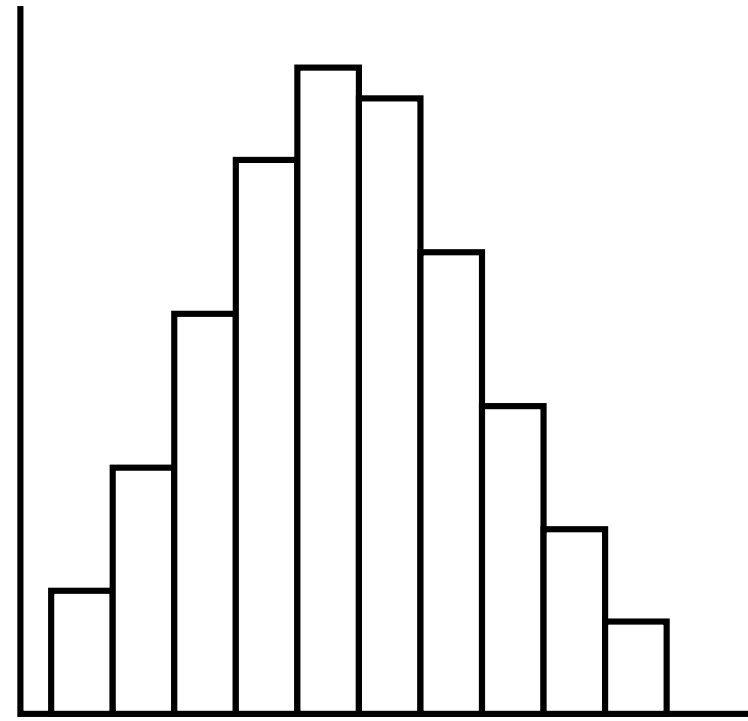
A	1.00								
B	0.51	1.00							
C	0.38	0.37	1.00						
D	0.57	0.22	0.09	1.00					
E	0.22	0.57	0.10	0.68	1.00				
F	0.11	0.11	0.46	0.59	0.58	1.00			
G	0.56	0.22	0.11	0.67	0.42	0.33	1.00		
H	0.23	0.58	0.12	0.43	0.66	0.34	0.67	1.00	
I	0.11	0.11	0.45	0.34	0.32	0.58	0.58	0.60	1.00
	A	B	C	D	E	F	G	H	I



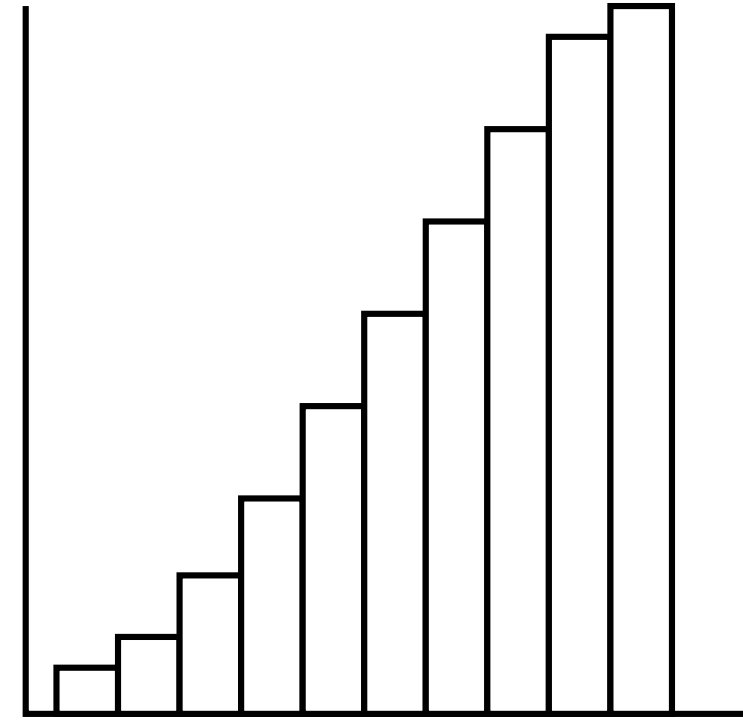
Interpretar un coeficiente o una matriz de correlación va mucho más allá de observar si un número es cercano a 1 o a -1. Para realizar un análisis riguroso y evitar conclusiones erróneas, una persona debe considerar los siguientes factores críticos:

- 1) Correlación no implica causalidad:** Este es el principio fundamental de la estadística. Que dos variables (X e Y) varíen de forma simultánea y predecible solo significa que comparten una tendencia, no que una sea la causa de la otra.
- 2) Limitación de la linealidad:** El coeficiente de correlación más utilizado (el de Pearson) solo mide relaciones lineales (en línea recta). Si la relación entre dos variables tiene forma de curva (por ejemplo, una "U" invertida o una curva exponencial), el coeficiente de Pearson puede ser cercano a cero, sugiriendo erróneamente que no hay relación cuando en realidad existe una conexión muy fuerte pero no lineal.
- 3) El impacto destructivo de los valores atípicos (Outliers):** Un solo dato extremo o inusual puede alterar por completo el coeficiente de correlación. Un outlier puede inflar artificialmente una correlación que en realidad es débil, o por el contrario, puede hacer que una correlación sólida parezca inexistente. Por esta razón, siempre se debe acompañar la matriz de correlación con un gráfico de dispersión (scatterplot) para visualizar la distribución real de los puntos.

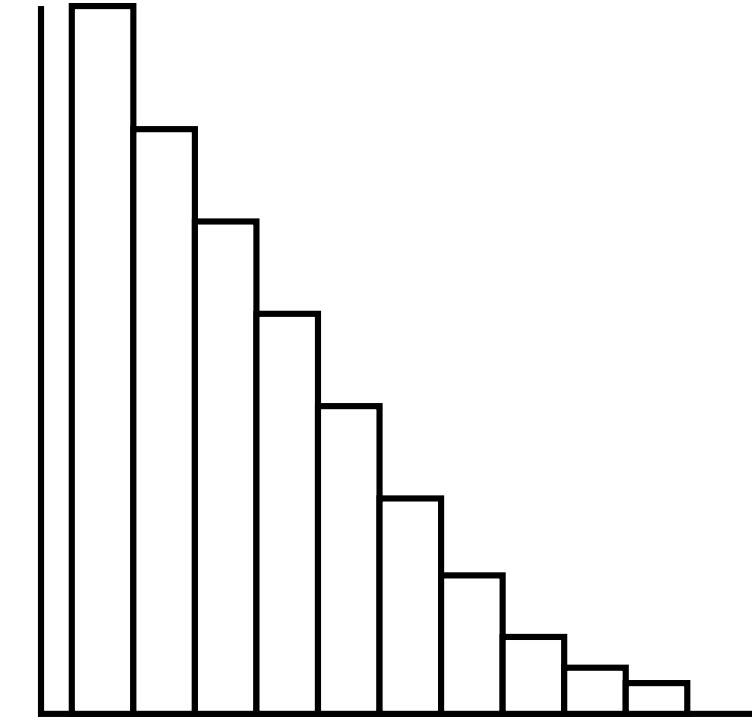
Histogramas: Tendencia y tipo de distribución



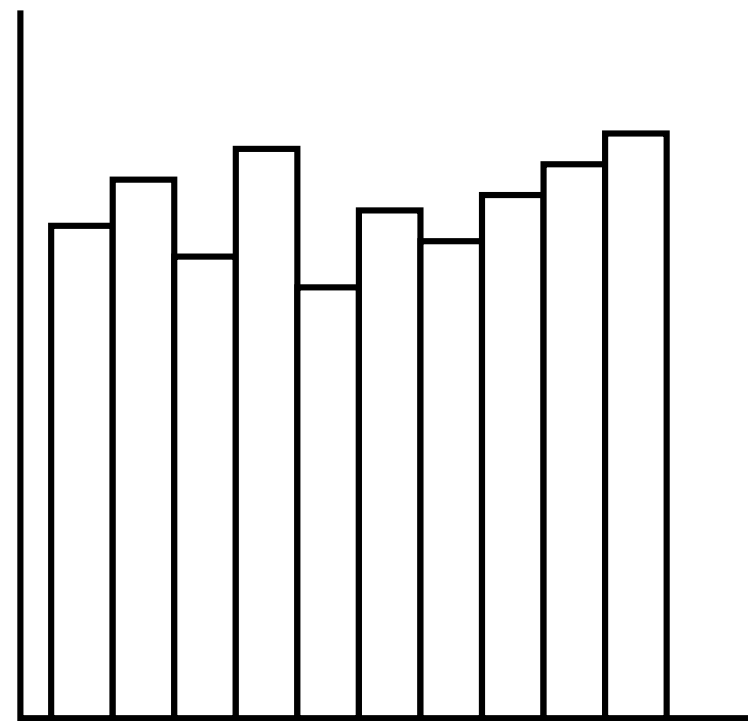
Distribución normal



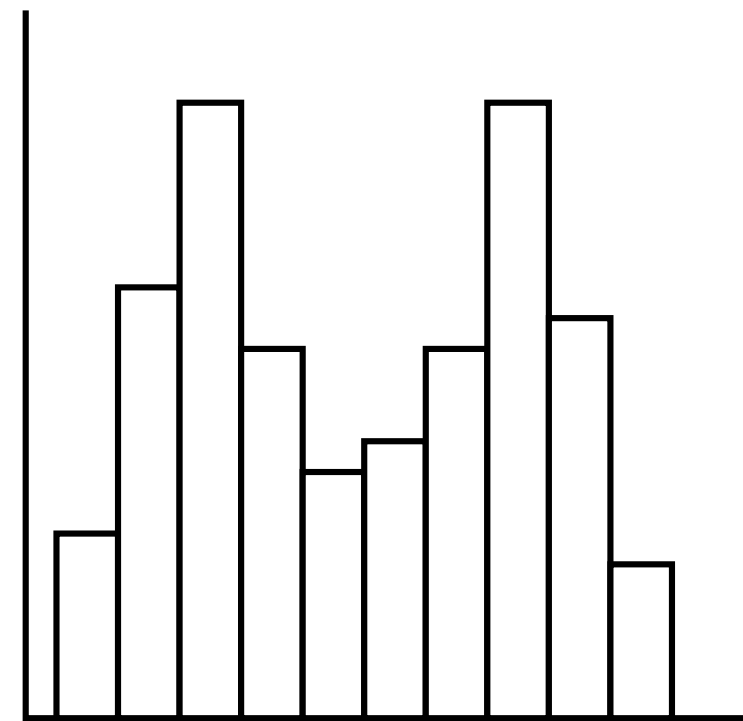
Distribución sesgada
a la izquierda



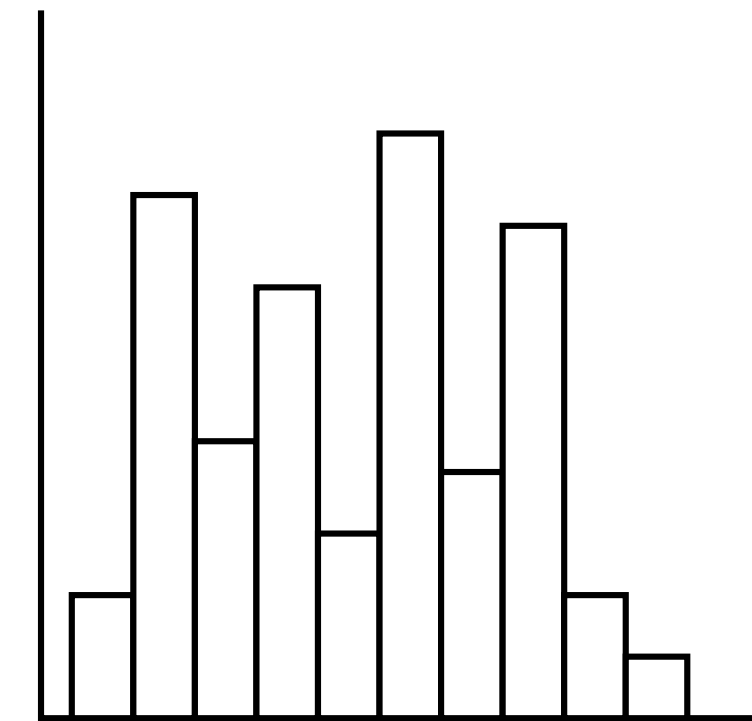
Distribución sesgada
a la derecha



Distribución uniforme



Distribución bimodal



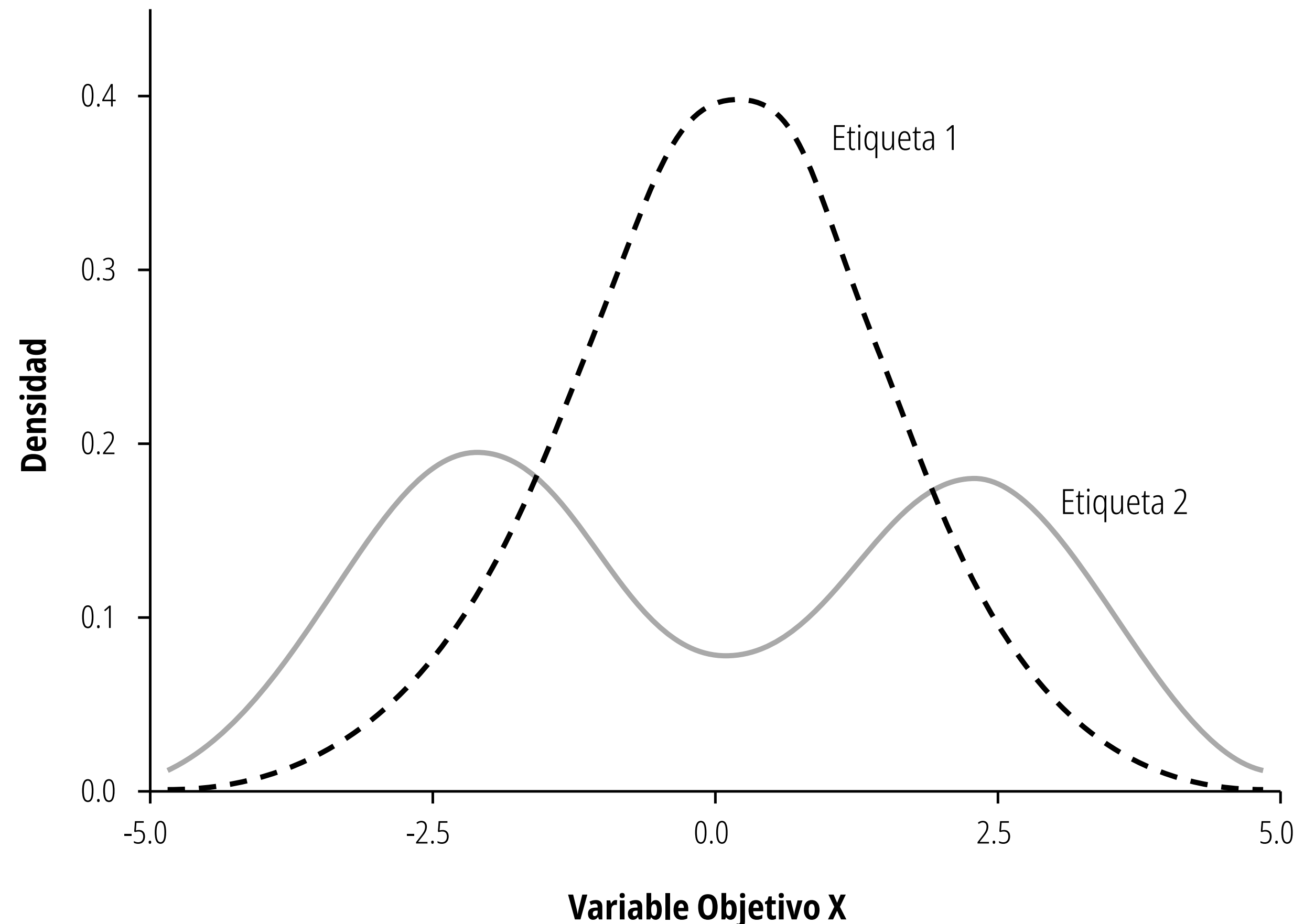
Distribución multimodal

KDE: Estimación de densidad por Kernel

En términos sencillos, es una versión suavizada de un histograma. En lugar de mostrar barras para contar cuántas observaciones caen en ciertos rangos, traza una curva continua que representa la probabilidad (o densidad) de encontrar datos en diferentes valores. El área total bajo cada curva siempre suma 1 (o el 100%).

El KDE funciona muy bien cuando queremos comparar el comportamiento de distintos grupos. Por ejemplo, graficar el KDE de etiqueta de la variable objetivo en el mismo eje muestra si hay alguna diferencia en el promedio o en la variación. Seaborn hace que sea súper fácil superponer gráficos KDE utilizando diferentes colores. Los gráficos KDE suelen ser menos confusos que los histogramas puestos lado a lado, y dan una mejor idea de cómo se diferencian los grupos.

Este diagrama es una representación visual excelente para evaluar qué tan comparables son dos grupos diferentes antes o después de aplicar una técnica estadística.

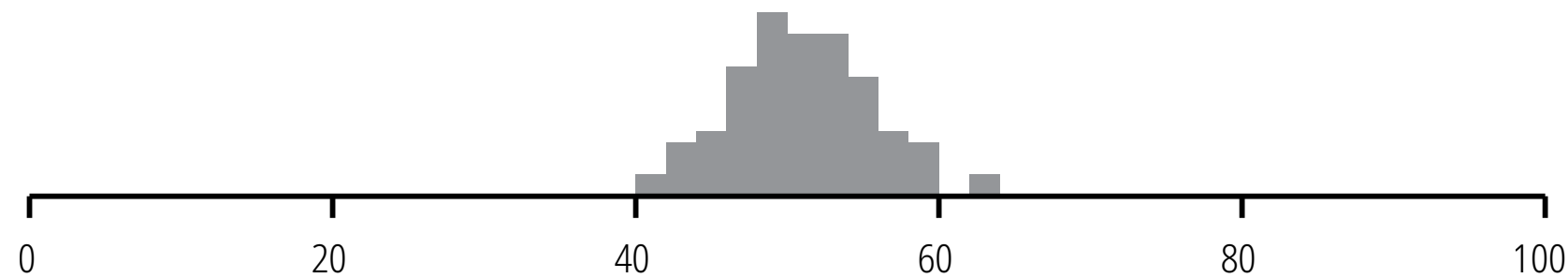


Distribuciones, Histogramas y Box-Plots

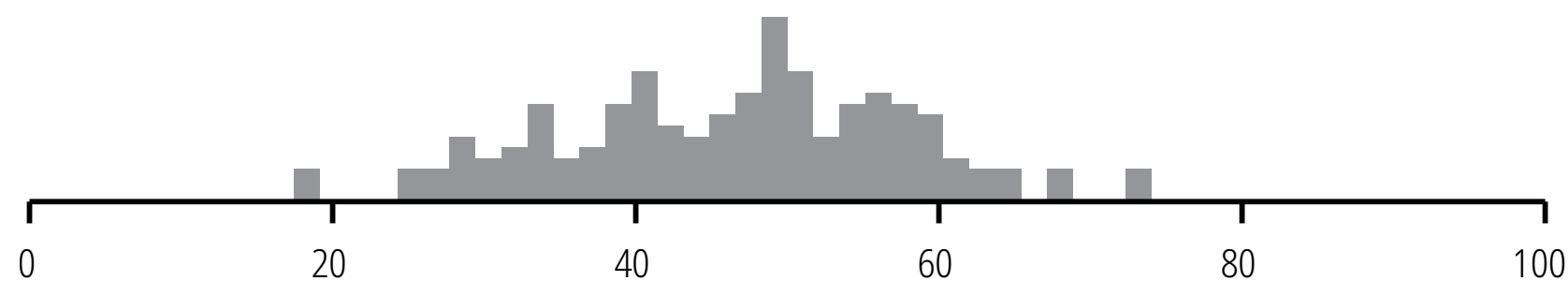
Uniforme



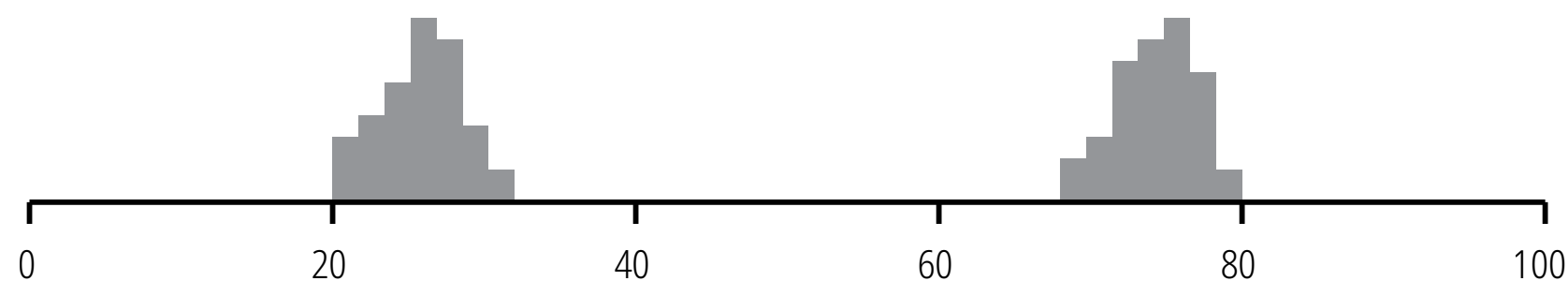
Unimodal



Unimodal 2



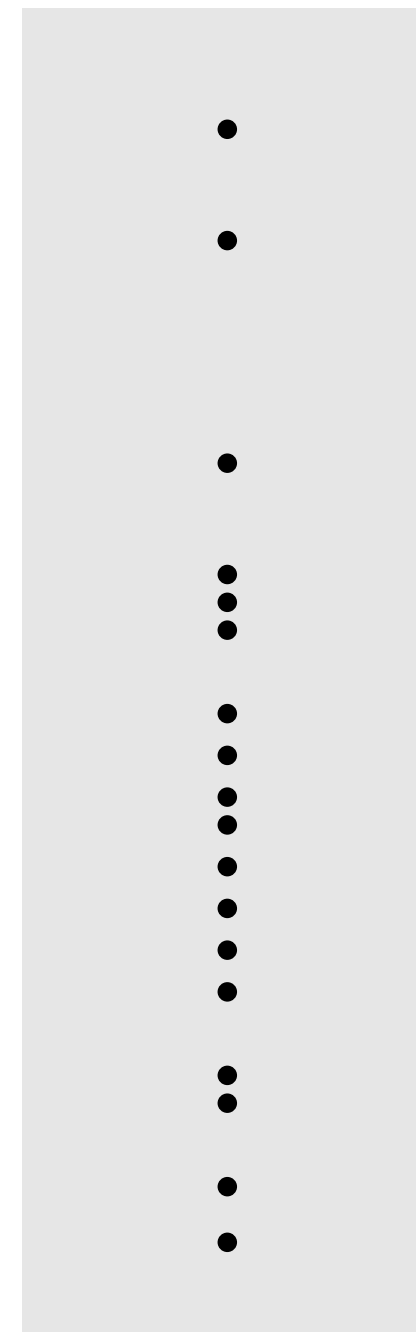
Bimodal



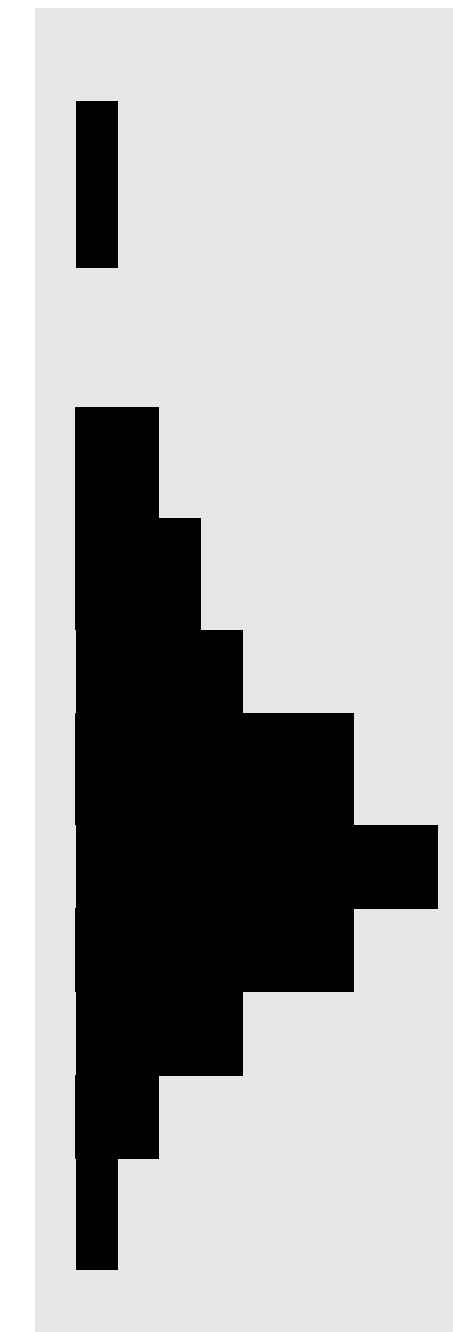
Valores de los datos

Mediante el uso de histogramas tradicionales, se ilustran comportamientos comunes en conjuntos de datos del 0 al 100: la distribución uniforme, caracterizada por una frecuencia constante y plana; las distribuciones unimodales, que concentran la mayor parte de sus observaciones en torno a un único pico central; y la distribución bimodal, la cual exhibe dos picos diferenciados que suelen delatar la presencia de dos subgrupos distintos mezclados en una misma muestra.

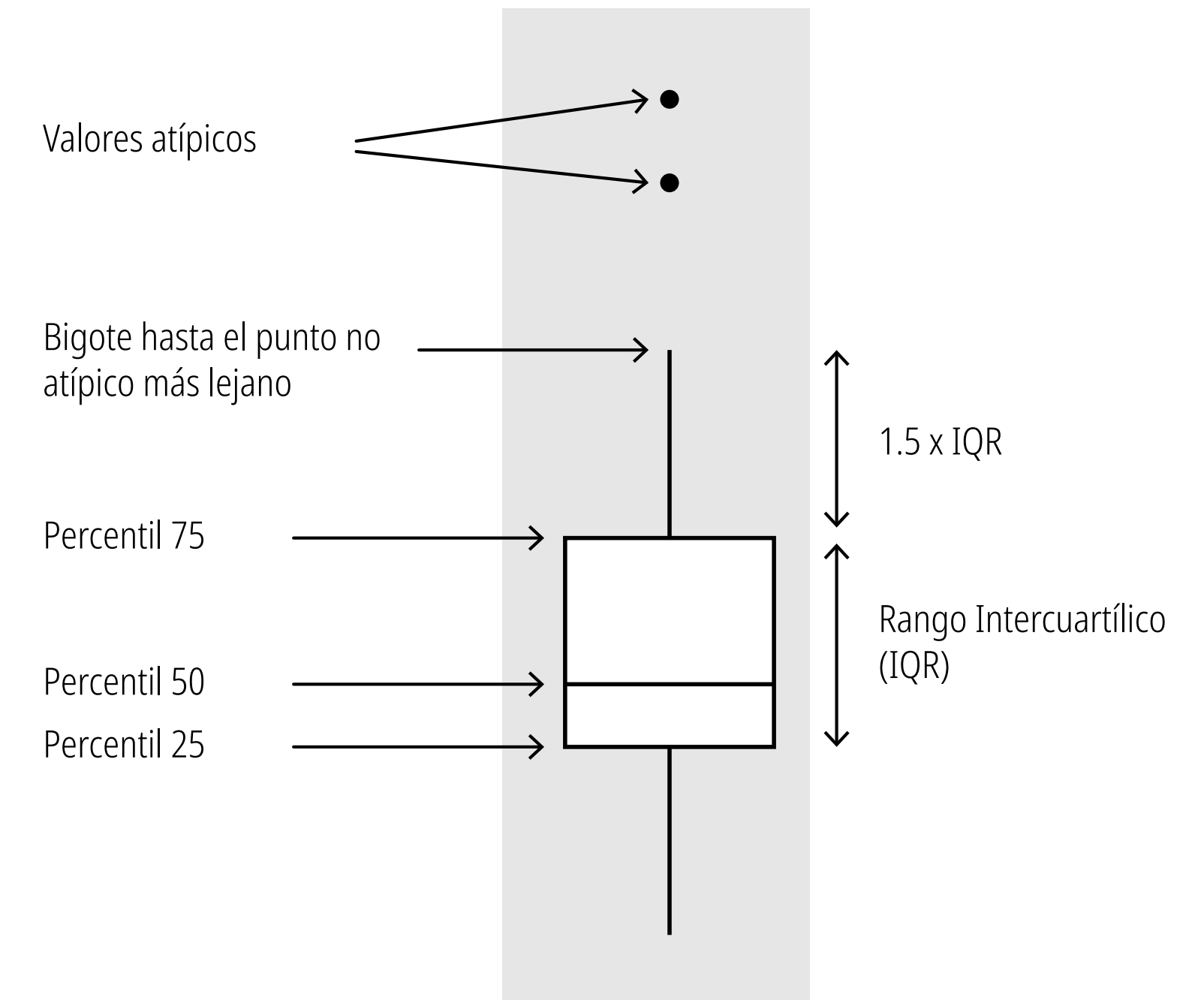
Los valores reales en una distribución



Cómo un histograma mostraría los valores (rotado)



Cómo un diagrama de caja mostraría los valores



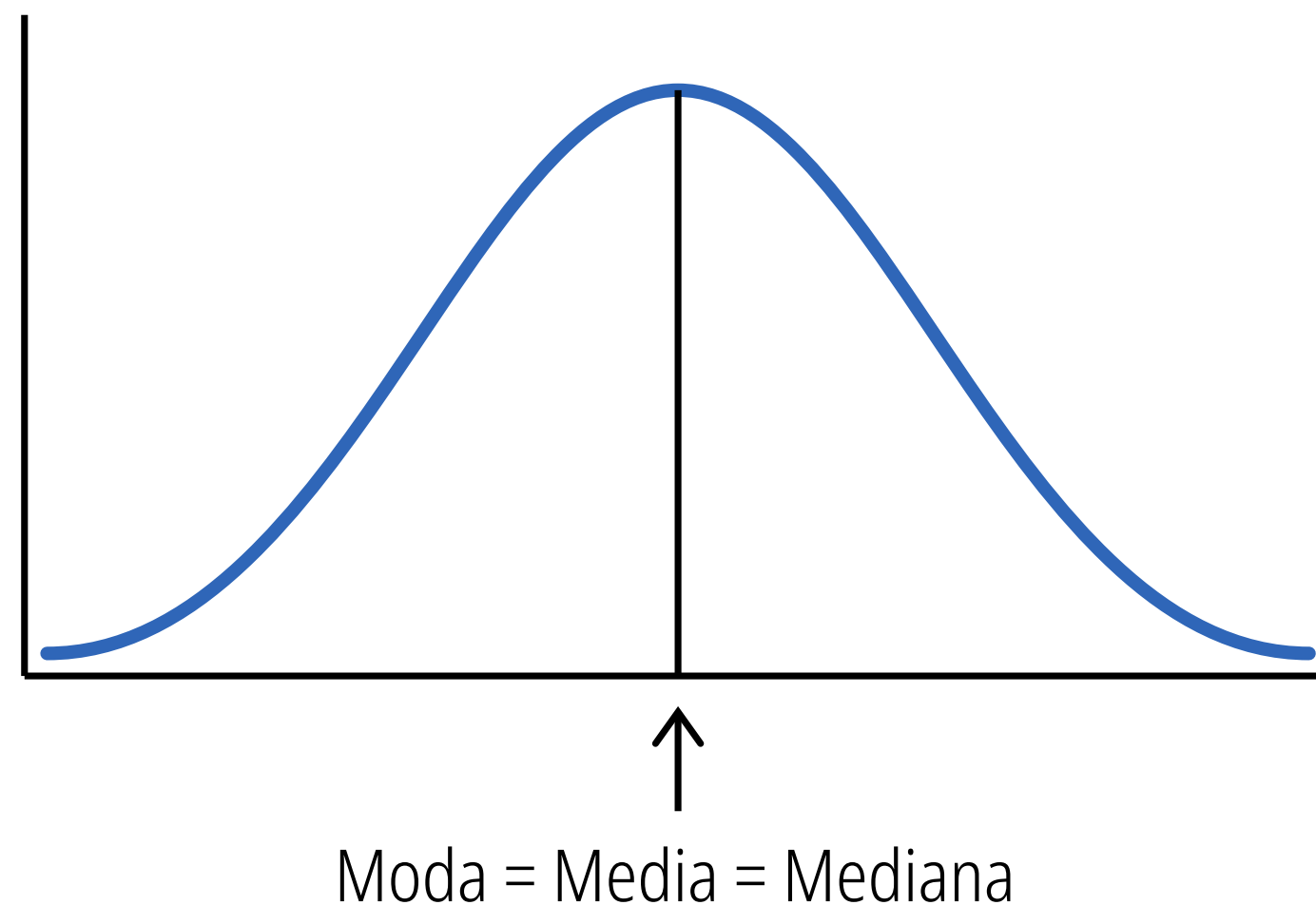
Se alinean verticalmente los valores reales de la distribución, evidenciando las zonas de mayor densidad y los elementos aislados; posteriormente, este comportamiento se traslada a un histograma rotado horizontalmente donde la longitud de las barras refleja fielmente dicha acumulación. Esta progresión demuestra de qué manera la dispersión y la concentración de las observaciones individuales determinan de forma directa la estructura final del gráfico de caja.

La caja central delimita el rango intercuartílico, que resguarda el 50% central de la información, marcando explícitamente los percentiles 25, 50 (la mediana) y 75. Desde los bordes de esta estructura se extienden los llamados bigotes, cuya longitud llega hasta el punto no atípico más lejano dentro de un límite de 1.5 veces el rango intercuartílico, dejando como puntos suspendidos en el exterior a los valores atípicos o outliers, que representan anomalías inusualmente distantes del resto del grupo.

Asimetría (Skewness)

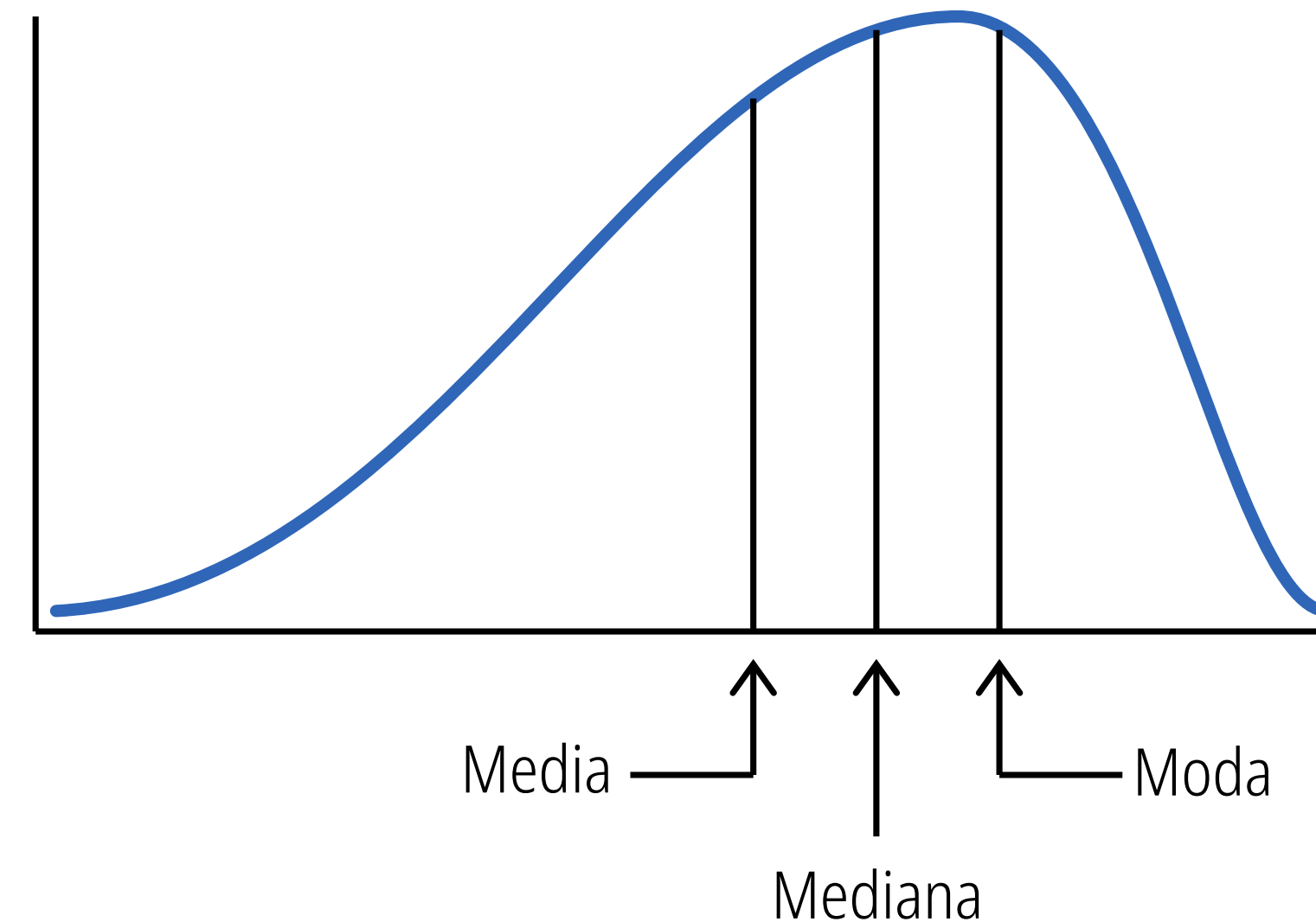
Conocer la asimetría de una variable es fundamental porque nos advierte cuándo el promedio tradicional (la media) es un indicador engañoso. Si un conjunto de datos es muy asimétrico, los valores extremos actúan como un imán y "arrastran" la media matemática hacia ellos, alejándola del punto donde realmente se concentra la mayoría de la información. Al detectar asimetría, sabemos de inmediato que debemos apoyarnos en otros indicadores, como la mediana o los percentiles, para comprender el comportamiento típico y real de ese modelo.

Simétrica



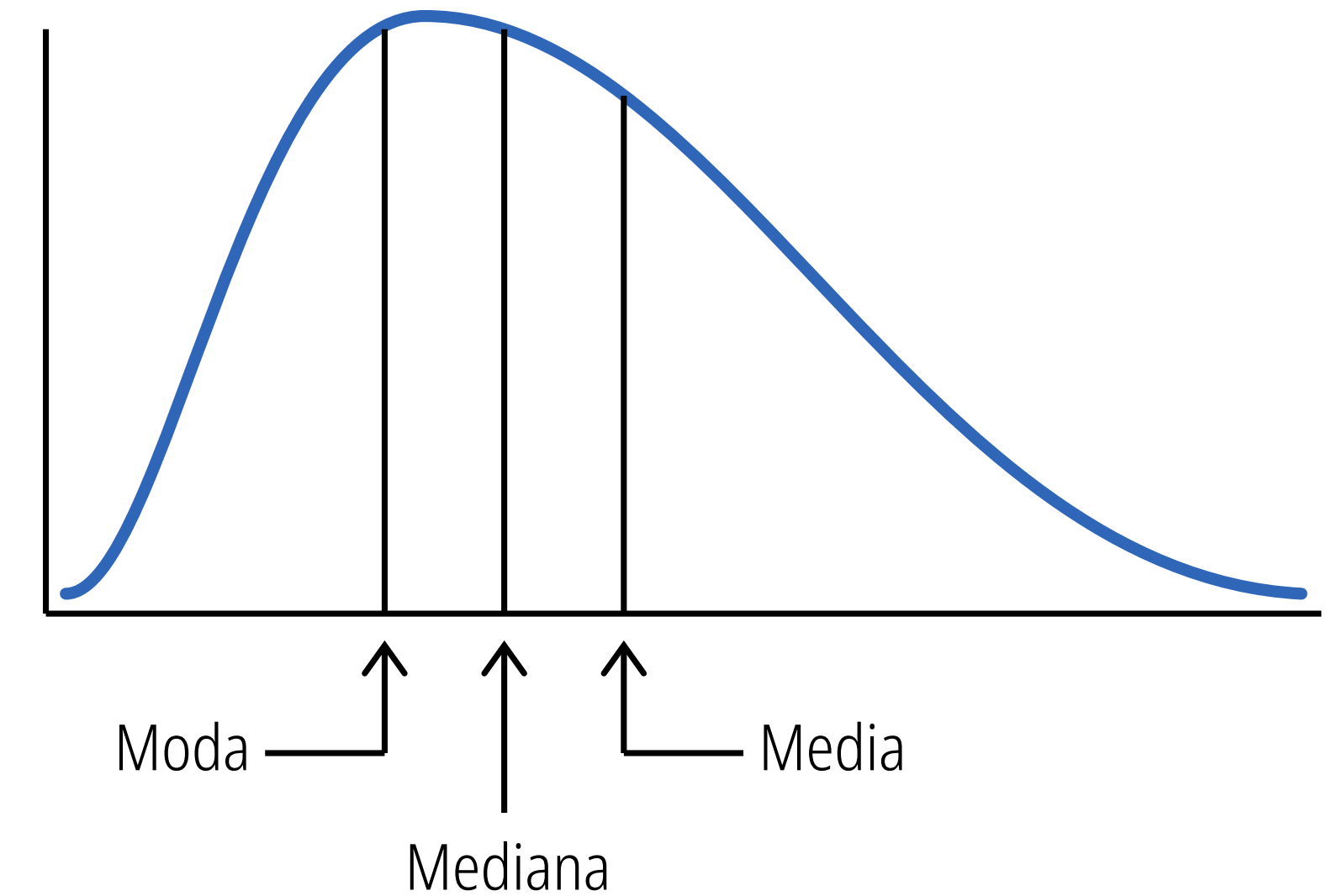
Es una campana perfecta, este balance establece una línea base teórica sumamente predecible, asumiendo que las desviaciones hacia arriba o hacia abajo del promedio tienen exactamente la misma probabilidad de ocurrir, lo cual simplifica enormemente la calibración inicial de cualquier sistema de análisis antes de enfrentarse a la realidad de los datos de producción.

Sesgada a la izquierda (negativamente)



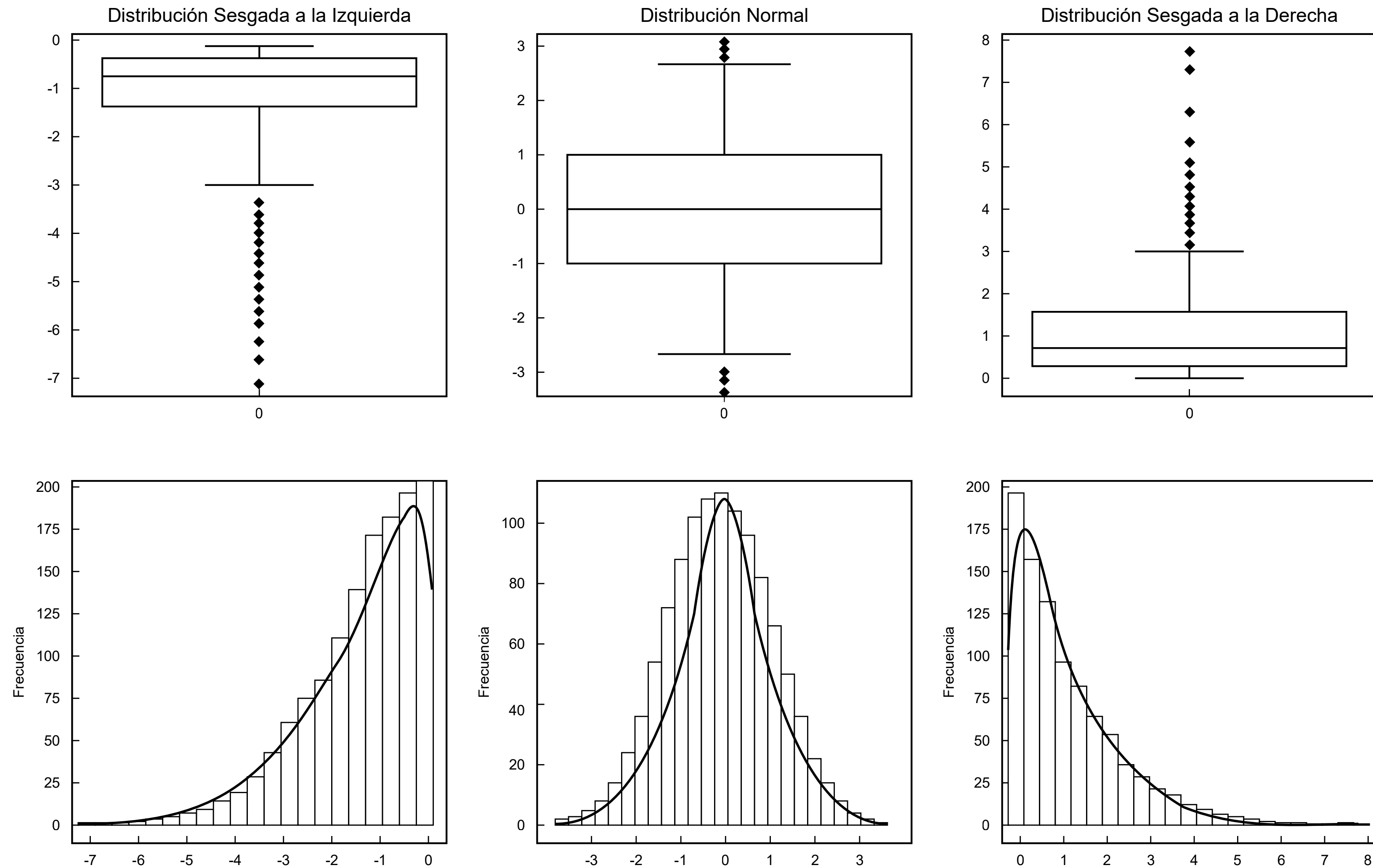
La asimetría negativa o sesgada a la izquierda invierte esta dinámica, presentando una concentración masiva de datos en los rangos superiores y dejando una cola que se arrastra hacia los valores más bajos, lo que empuja la media por debajo de la mediana.

Sesgada a la derecha (positivamente)



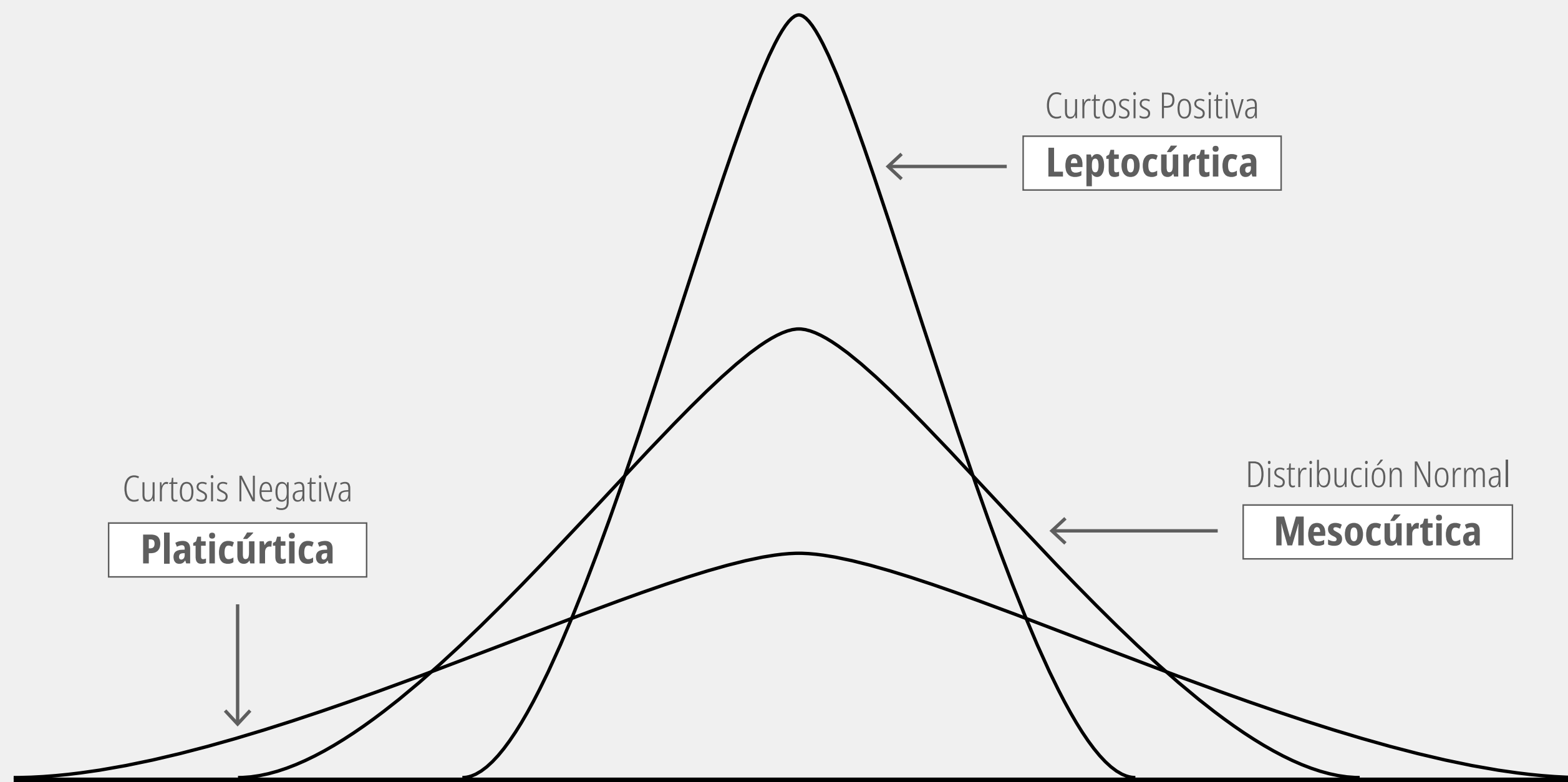
La asimetría positiva o sesgada a la derecha dibuja una curva donde la mayor concentración de datos se apila en los valores bajos (la moda), mientras que una cola larga y delgada se extiende hacia la derecha, arrastrando a la media matemática por encima de la mediana.

Asimetría: Box-Plots vs Histogramas



Curtosis

La curtosis es una medida estadística que describe cómo se distribuyen los datos alrededor de la media, enfocándose específicamente en el grosor de las "colas" de la distribución y, por ende, en la agudeza de su pico central.



Distribución mesocúrtica representa el estándar estadístico por excelencia, dibujando la clásica curva normal donde las observaciones se agrupan de manera simétrica y equilibrada en torno a la media. En este escenario, la probabilidad de encontrar valores extremos en las colas es moderada y matemáticamente predecible.

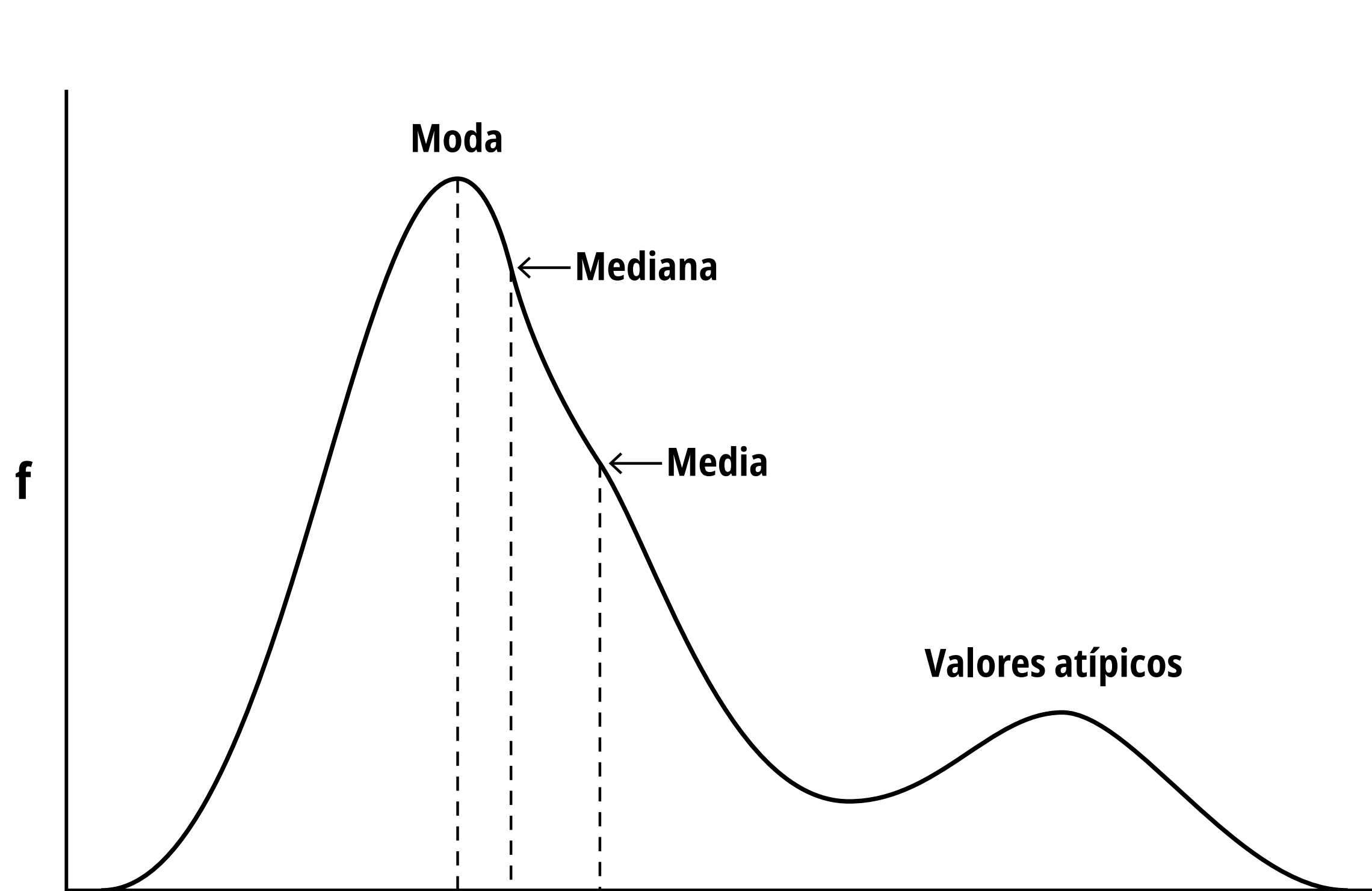
Distribución leptocúrtica se distingue por presentar un pico central mucho más agudo acompañado de colas densas o "pesadas", lo que transforma radicalmente el panorama analítico al evaluar riesgos o perfilar comportamientos. Esta estructura matemática indica que, aunque la enorme mayoría de los datos orbita muy cerca del promedio, las desviaciones extremas y anómalas ocurrirán con una frecuencia significativamente mayor a la esperada.

Distribución platicúrtica se manifiesta a través de una curva achatada y ancha, caracterizada por tener colas extremadamente delgadas que denotan una ausencia notable de elementos fuera de lo común. En esta configuración, la información tiende a distribuirse de forma más uniforme a lo largo del espectro en lugar de aglomerarse en el centro, creando un ecosistema estadístico altamente estable donde la aparición de eventos atípicos de alto impacto es improbable, lo que permite relajar la agresividad matemática y confiar en una varianza mucho más segura y controlada.

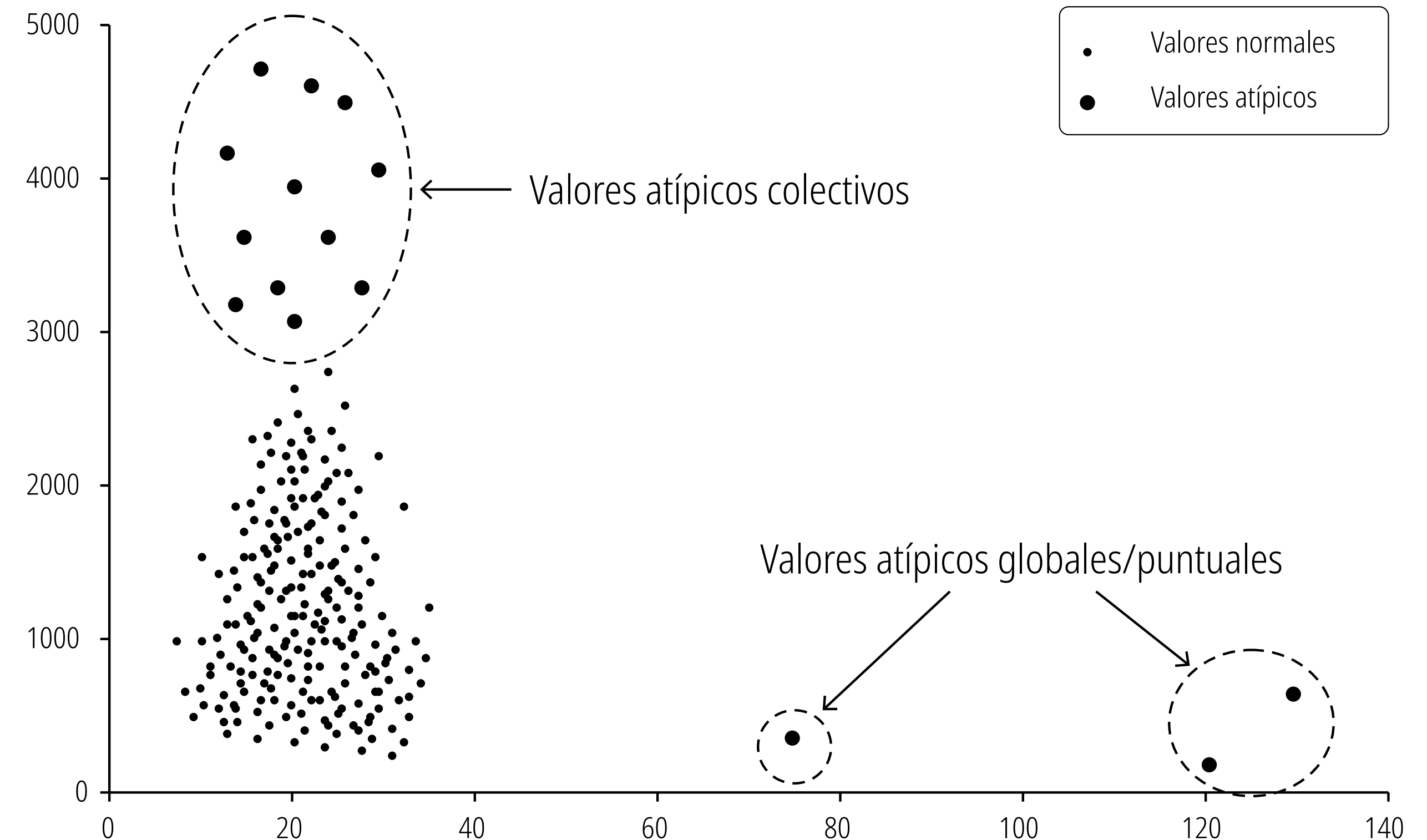
En términos prácticos, la curtosis nos dice qué tan probables son los valores extremos o atípicos (outliers) en un conjunto de datos en comparación con una distribución normal.

Valores atípicos / extremos

Un valor extremo o valor atípico (outlier) es una observación que se desvía drásticamente del patrón general de un conjunto de datos. En el análisis de datos, estos valores no son simples errores; con frecuencia representan las señales más importantes dentro del "ruido", indicando eventos extraordinarios, fraudes o fallos estructurales que los modelos analíticos deben aislar o comprender.

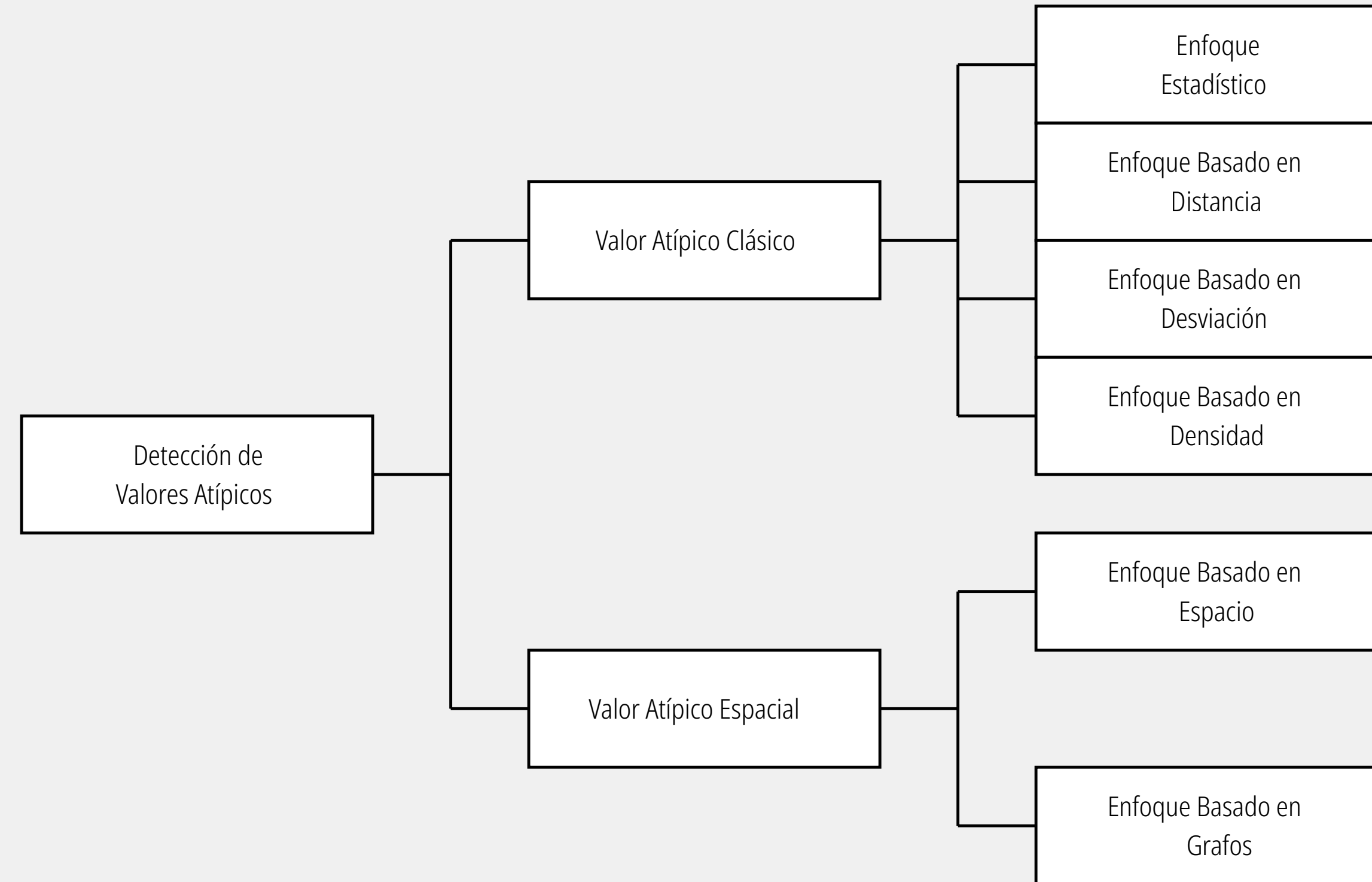


La media es frágil ante eventos extremos. Ese pequeño grupo de anomalías tiene la fuerza matemática suficiente para arrastrar la Media muy a la derecha, separándola drásticamente de la Mediana y la Moda. Si un modelo de perfilamiento de comportamiento dependiera de esa media matemática pura para establecer sus líneas base, generaría una cascada de falsos positivos al evaluar el comportamiento típico.



Se puede clasificar a los valores atípicos según su estructura, un concepto vital al diseñar sistemas que rastrean variables continuas y vectores de estado. **Valores normales (inliers):** Es la nube densa de puntos en la parte inferior izquierda. Representa el comportamiento base esperado y predecible del sistema. **Valores atípicos (outliers) globales/puntuales:** Son esos dos puntos aislados en la parte inferior derecha. Representan anomalías individuales y extremas. **Valores atípicos colectivos:** Esta es la anomalía más compleja de modelar. Observa el grupo encerrado en la parte superior izquierda. Aunque sus valores en el eje X son normales, en el eje Y están completamente desviados, y lo más importante: forman un grupo estructurado.

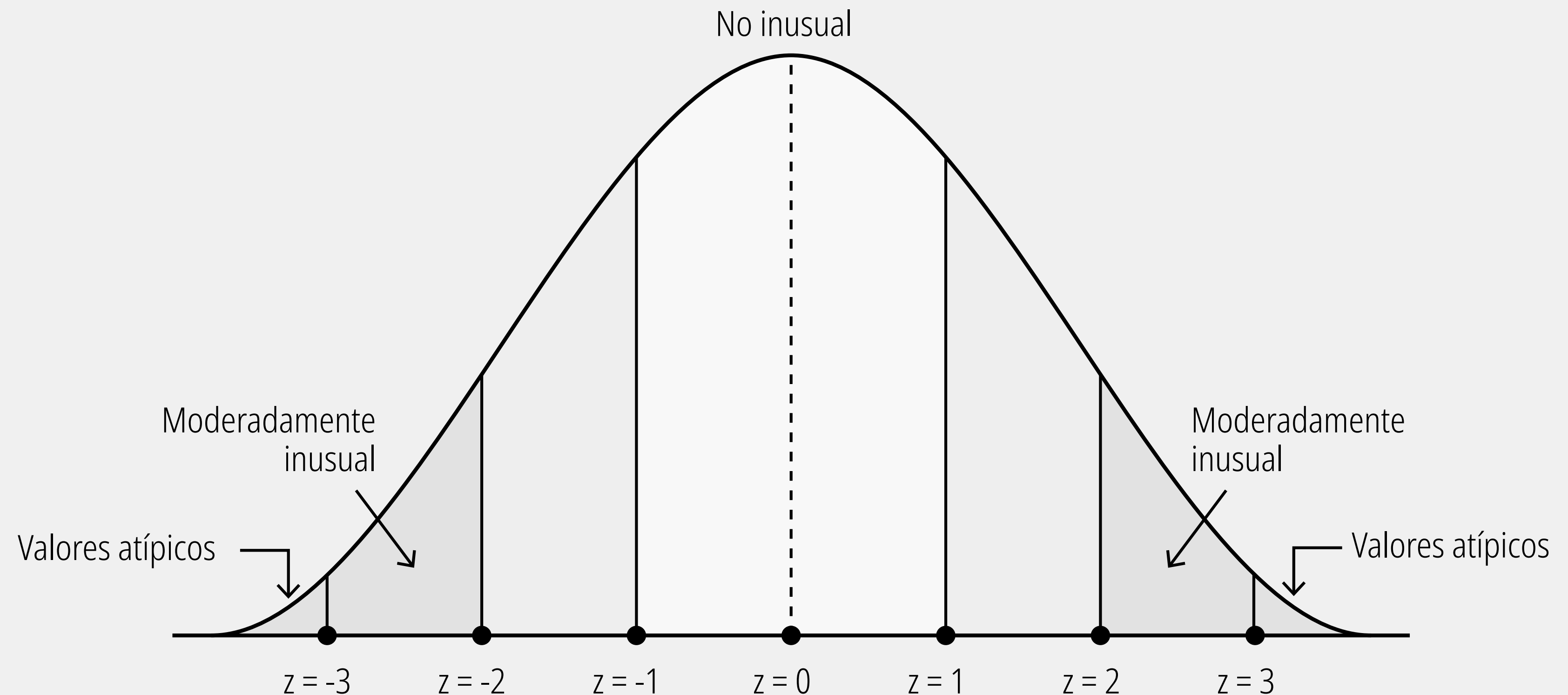
Abordaje de valores atípicos



Detección de valores atípicos con Z-Score

El Z-Score es una métrica estadística que indica a cuántas desviaciones estándar se encuentra un punto de datos específico respecto al promedio o media de todo el grupo.

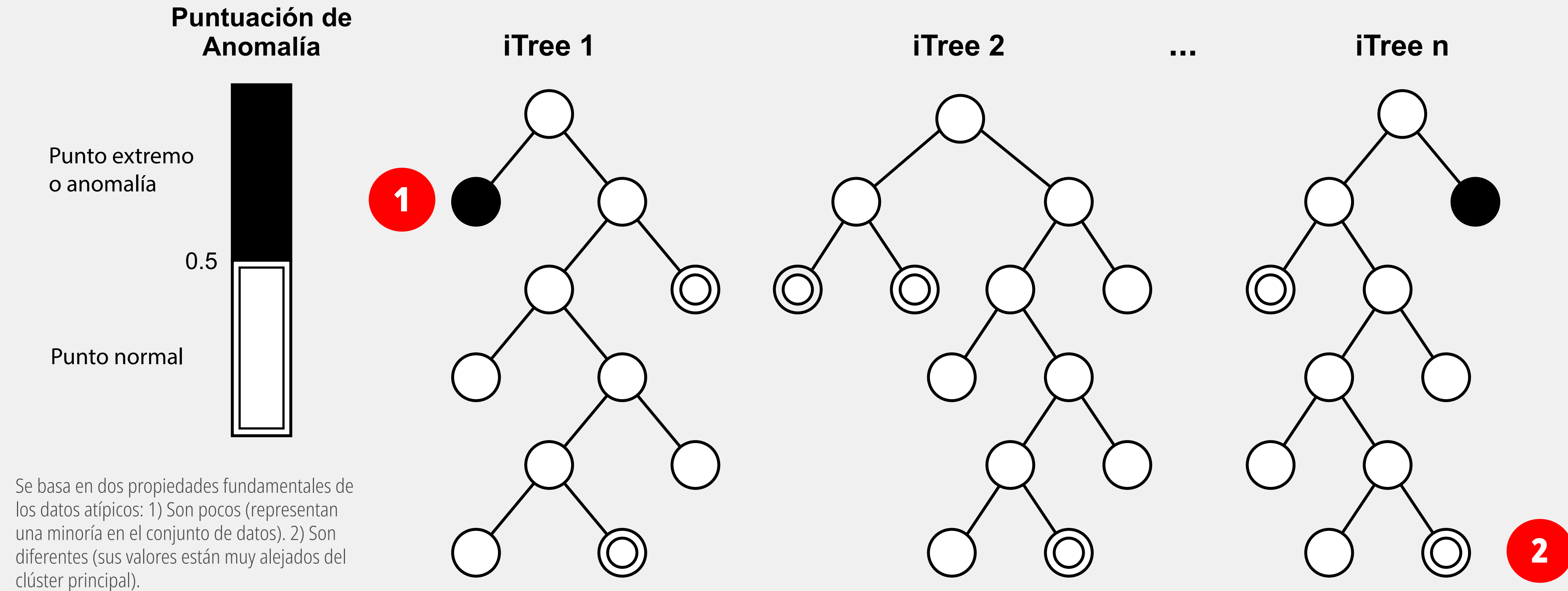
En la práctica, esta técnica permite establecer umbrales matemáticos claros (por ejemplo, "marcar cualquier registro donde $|z| > 3$ ") para automatizar la detección de anomalías en lugar de depender de reglas manuales.



Zona "Valores atípicos" (Más allá de $z = -3$ y $z = 3$): Las colas extremas de la gráfica. La probabilidad de que un dato legítimo caiga aquí es extremadamente baja (generalmente menos del 0.3%). En la ciencia de datos, cualquier valor en esta zona se considera un "outlier" o valor atípico, lo que levanta una alerta porque sugiere una anomalía severa, un error de medición o un evento extraordinario.

Isolation Forest

Isolation Forest (Bosque de Aislamiento) es un algoritmo de aprendizaje automático no supervisado, diseñado específicamente para la detección de anomalías.



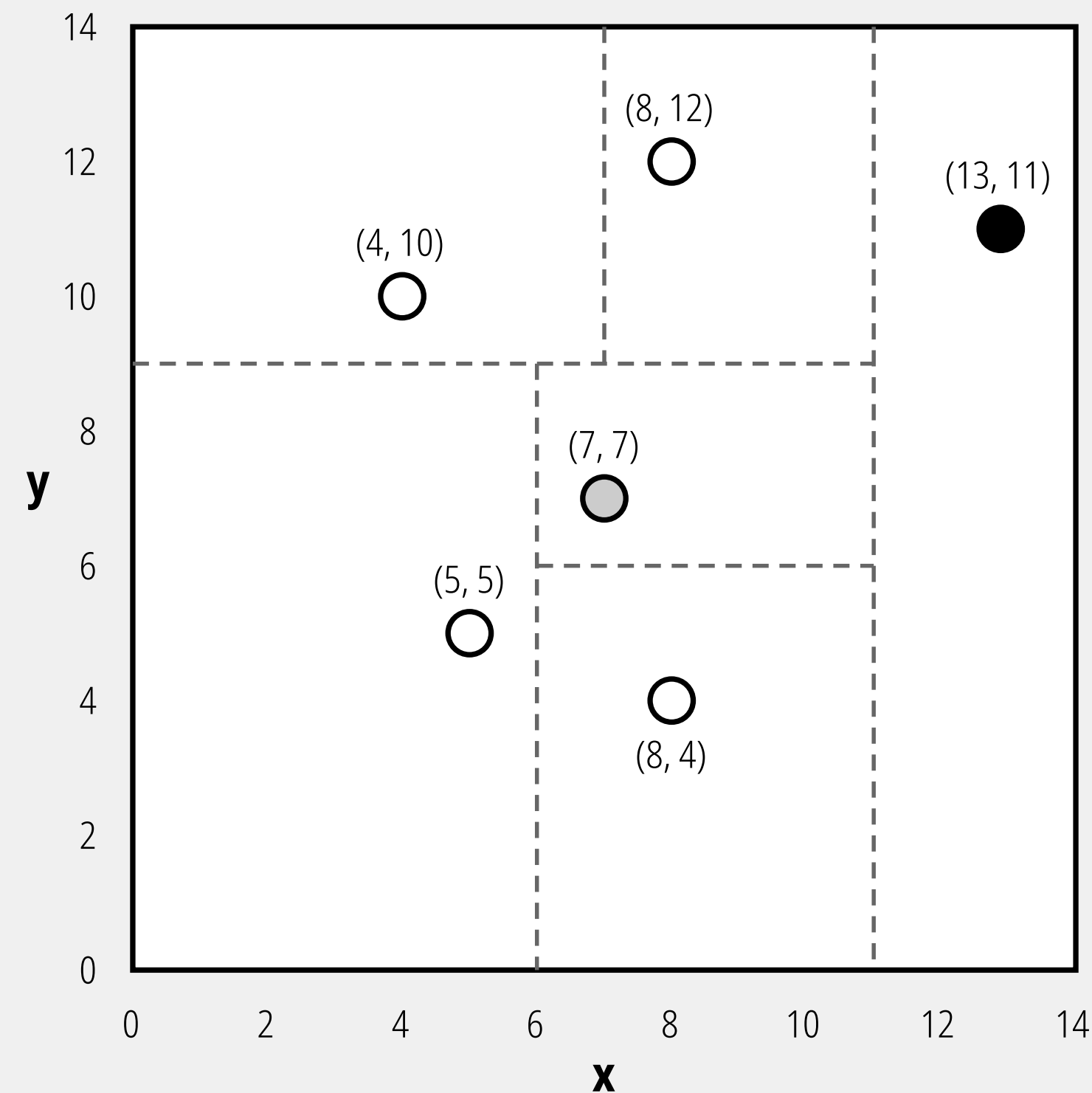
1 Anomalías son fáciles de aislar: Como están lejos del grupo principal y en zonas poco densas, un corte aleatorio tiene una alta probabilidad de separarlas del resto muy rápidamente. Requieren muy pocas particiones para quedar solas.

2 Datos normales son difíciles de aislar: Al estar agrupados densamente en el centro, se necesitan muchísimos cortes aleatorios minuciosos para lograr separar un punto normal de todos sus vecinos.

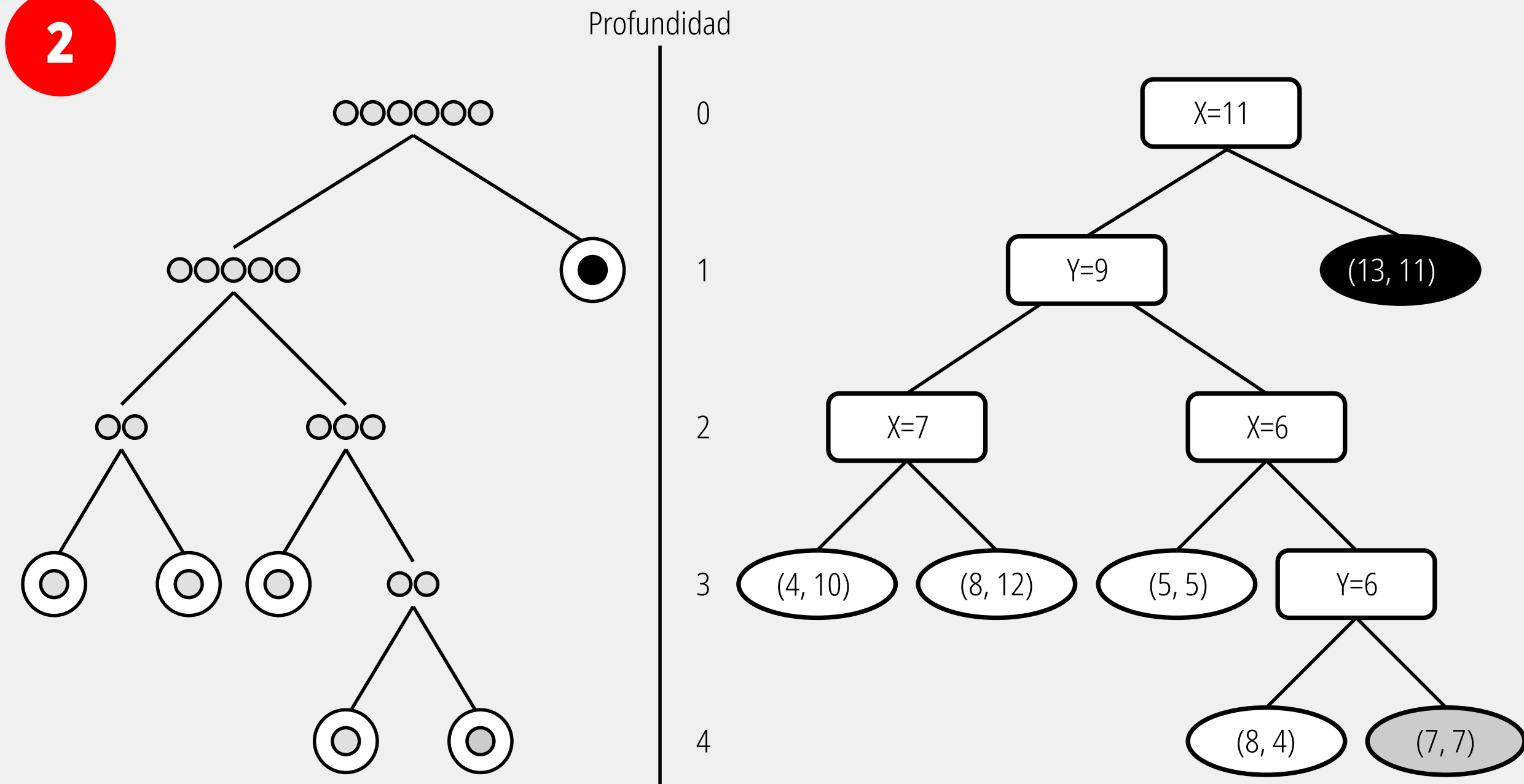
Isolation Forest

Paso a paso de cómo un único Árbol de Aislamiento (iTree) divide el espacio bidimensional para encontrar anomalías. El principio matemático central del modelo es que la anomalía queda aislada de forma casi instantánea, mientras que el valor normal requirió una disección exhaustiva del espacio para poder ser separado de su entorno.

1



2

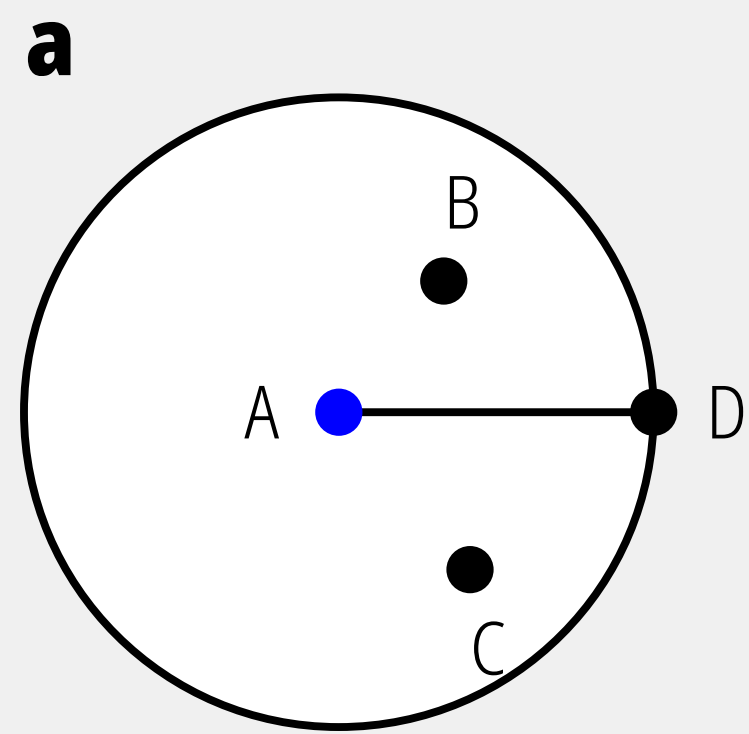


1 Las líneas discontinuas representan los cortes aleatorios, rectos y ortogonales, realizados por el algoritmo. Para separar los puntos que están más agrupados en el lado izquierdo, el algoritmo tuvo que seguir trazando líneas (primero horizontal en $Y=9$, luego vertical en $X=7$ y $X=6$, etc.), creando "cajas" cada vez más pequeñas. El punto gris (7, 7) queda aislado en su propio cuadrante solo después de múltiples divisiones.

2 Traduce esos cortes geométricos en un árbol de decisión lógico, revelando la métrica clave: la profundidad. El camino de la anomalía (Nodo Negro): En la raíz del árbol de la derecha, la primera regla es evaluar si el valor X es mayor o menor que 11. El punto (13, 11) es el único que cumple con la condición de estar a la derecha, por lo que se convierte inmediatamente en una "hoja" final del árbol. Se aísla rápidamente en una profundidad de 1. El camino del dato normal (Nodo Gris): Para que el algoritmo logre aislar el punto (7, 7), debe hacer un recorrido largo a través de las ramas evaluando múltiples condiciones para finalmente separarlo de su vecino más cercano, el punto (8, 4). Debido a esto, termina en la parte inferior del árbol con una profundidad de 4.

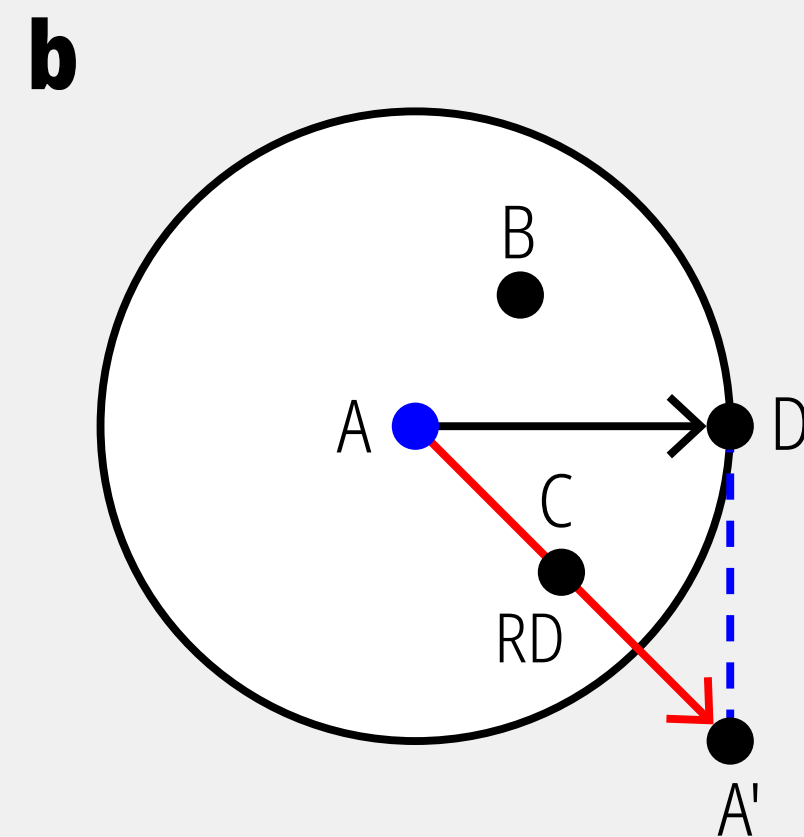
Local Outlier Factor (LOF)

Es un método de aprendizaje automático no supervisado utilizado para la detección de anomalías. En lugar de evaluar los datos a nivel global, LOF mide la desviación de la densidad local de un punto específico en comparación con sus vecinos más cercanos. En términos simples: si un dato se encuentra en una región mucho menos densa o más aislada que los puntos que lo rodean directamente, el algoritmo lo identifica como una anomalía o "outlier".

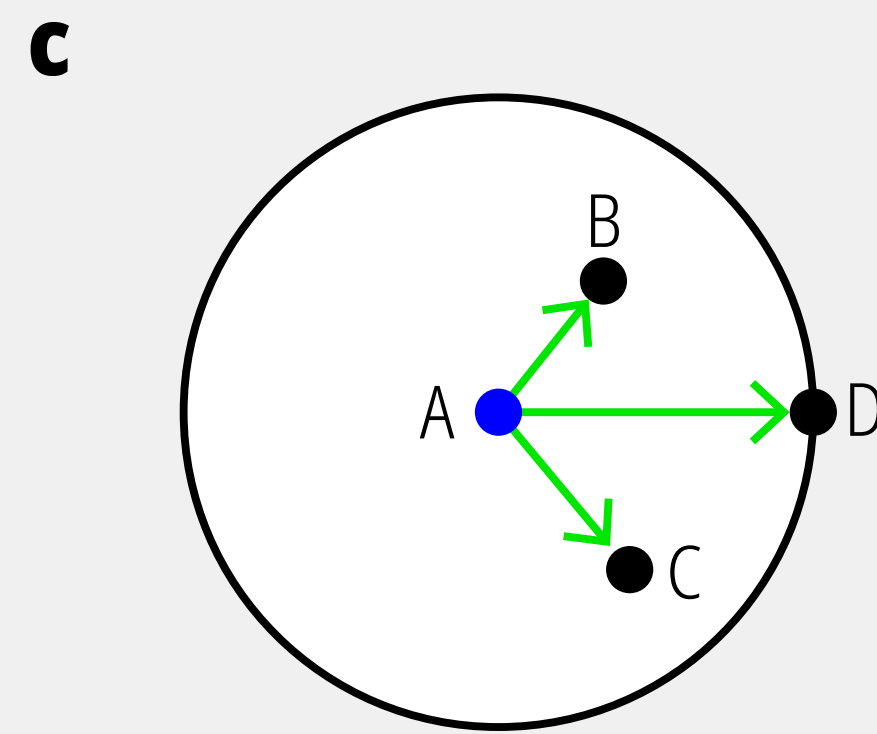


$K = 3$

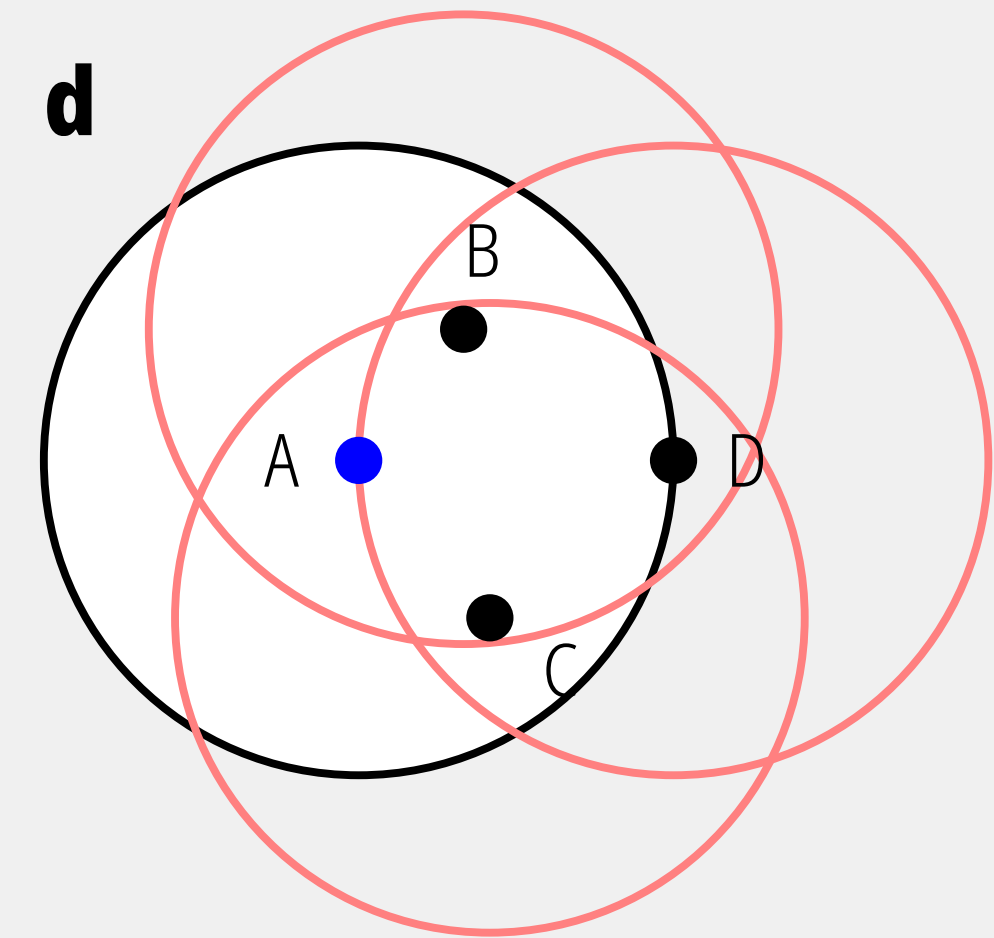
Determinar el número de K



RD = Distancia de Alcanzabilidad



Calcular la Distancia de Alcanzabilidad Local para cada punto de datos



Calcular LOF

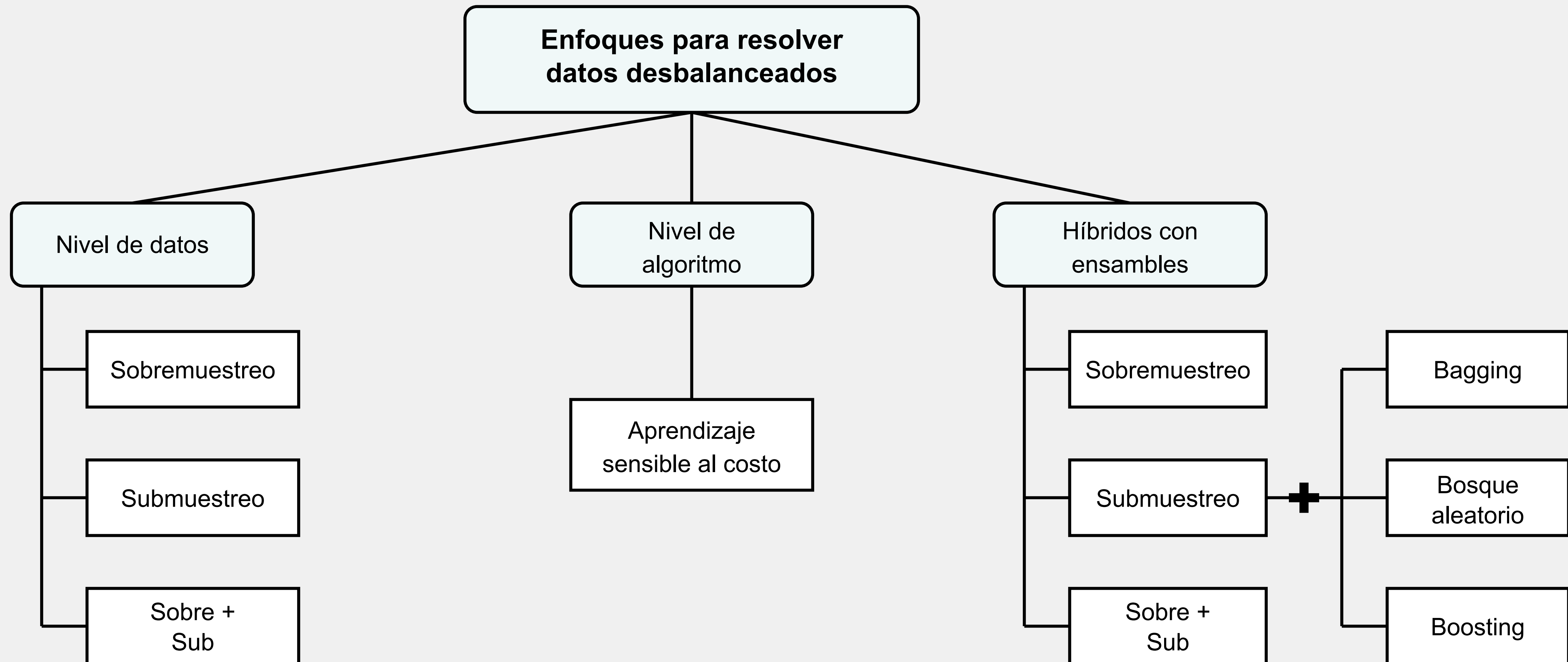
El primer paso consiste en definir un parámetro K , que representa la cantidad de vecinos más cercanos que se tomarán en cuenta para analizar un dato. En el primer panel de la imagen, se evalúa el punto A con un valor de $K = 3$. El círculo traza el radio necesario para abarcar exactamente a esos tres vecinos más cercanos (B, C y D), lo que define su vecindario local.

Calcula Reachability Distance (RD). Esta es una métrica de distancia ajustada que indica cuán "lejos" está un punto de otro, pero utilizando un límite inferior basado en la distancia del vecindario. Esto sirve para evitar fluctuaciones drásticas y suavizar el ruido estadístico. En la gráfica, se muestran los vectores que miden esta distancia hacia puntos dentro del límite (como D) y fuera de él (como A').

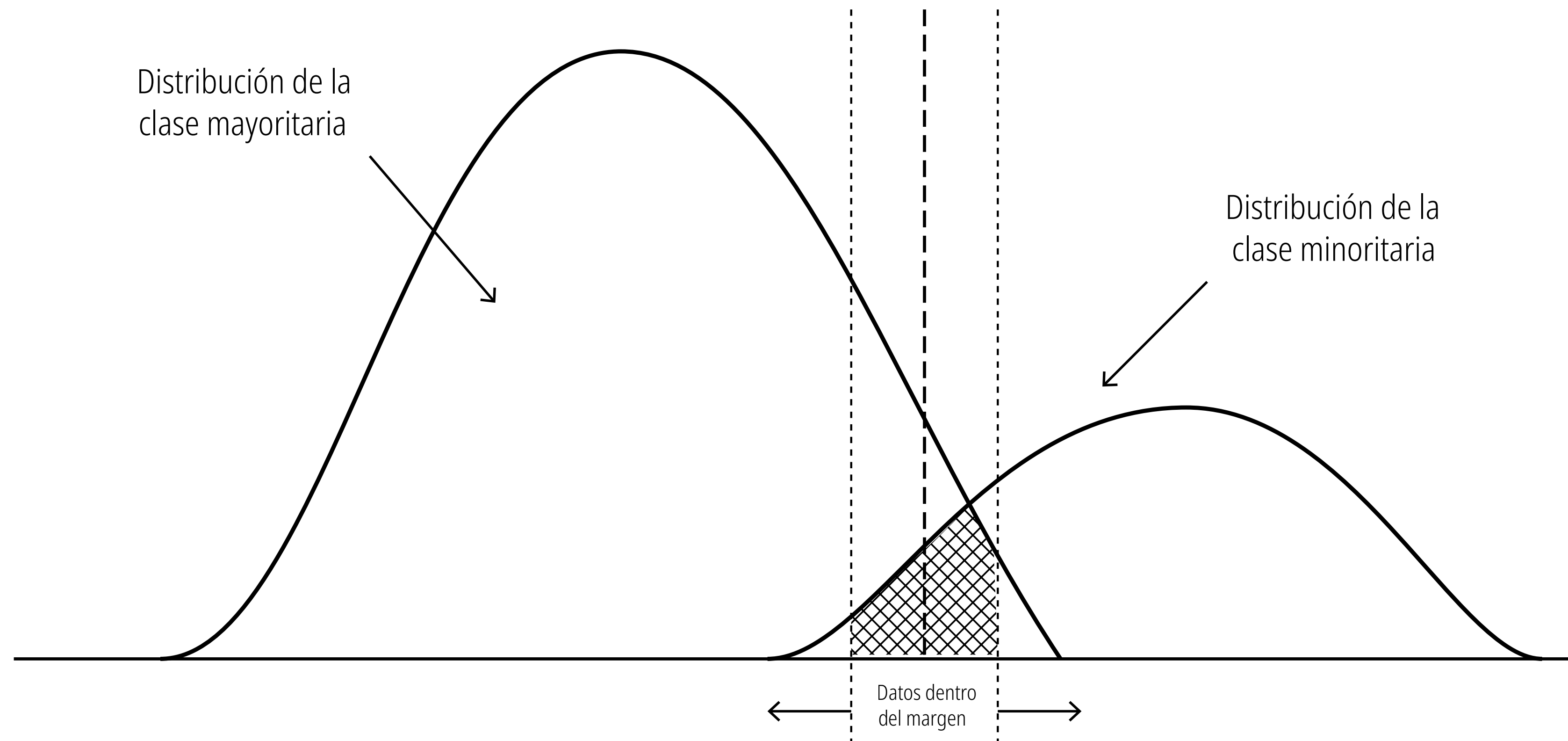
Calcula LRD para el punto A. Esto se logra promediando las distancias de alcanzabilidad calculadas en el paso anterior hacia todos sus vecinos (las flechas verdes hacia B, C y D) y tomando la inversa de ese promedio. Básicamente, este paso responde a la pregunta: ¿Qué tan apretados o dispersos están los datos alrededor del punto A?

El último paso compara la densidad del punto A con las densidades de sus vecinos (B, C y D), como lo sugieren los círculos superpuestos. El factor LOF es el ratio o proporción entre estas densidades: Si el valor LOF es cercano a 1, la densidad del punto es similar a la de sus vecinos (es un dato normal o "inlier"). Si el valor LOF es mayor a 1, significa que la densidad del punto es notablemente inferior a la de sus vecinos, lo que lo clasifica definitivamente como un valor atípico local.

Datos desbalanceados: Enfoques de tratamiento



Compromiso entre precisión y exhaustividad (Trade-off)



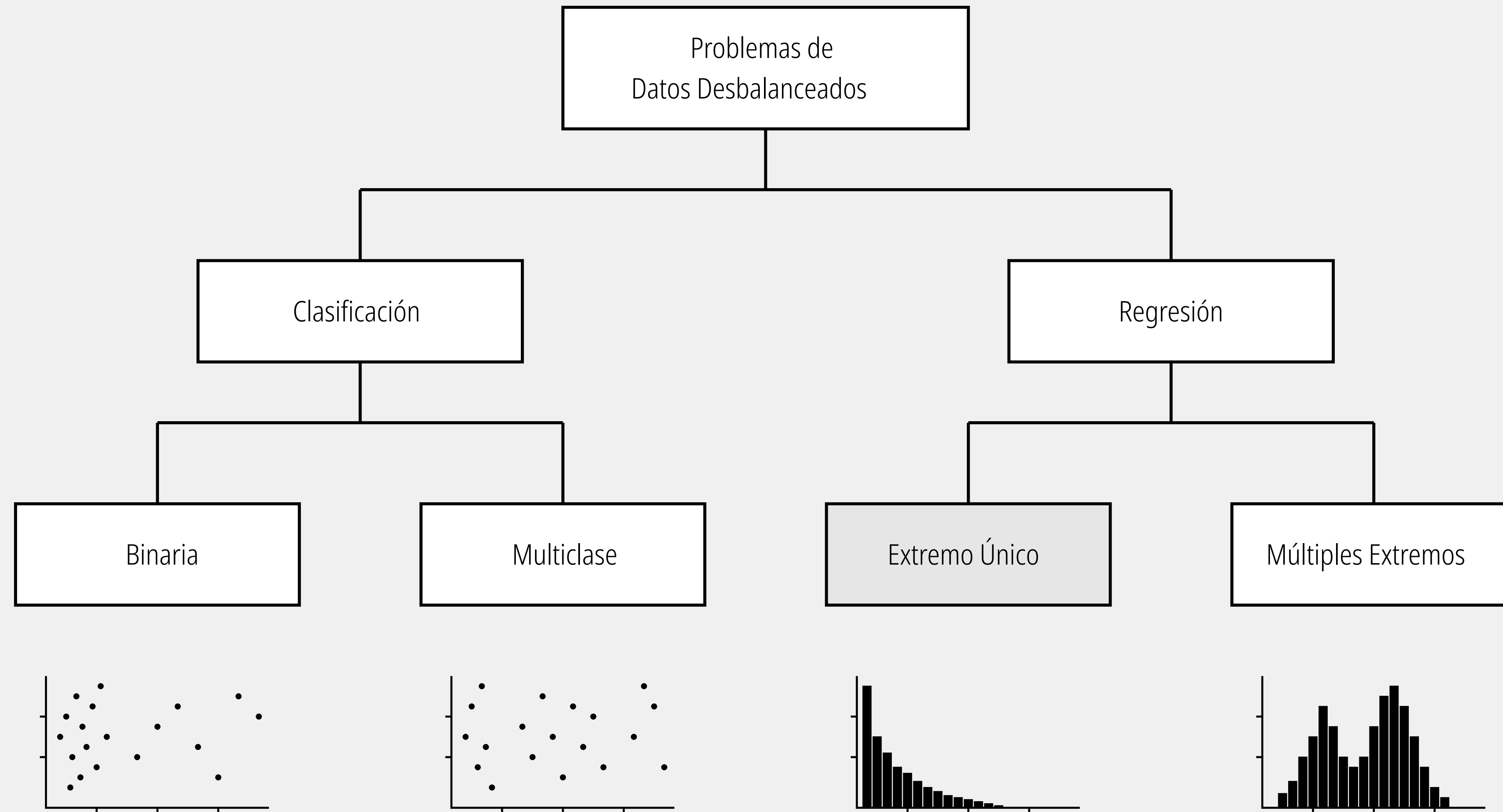
Zona de solapamiento: El espacio donde ambas curvas se cruzan indica que hay observaciones de ambas clases que comparten características (valores) muy similares. Es en esta intersección donde los modelos de clasificación cometen errores.

Frontera de decisión: Representa el umbral (o hiperplano) que el algoritmo utiliza para separar y clasificar las predicciones. Todo lo que cae a su izquierda se clasificaría como clase mayoritaria, y lo que cae a su derecha como clase minoritaria.

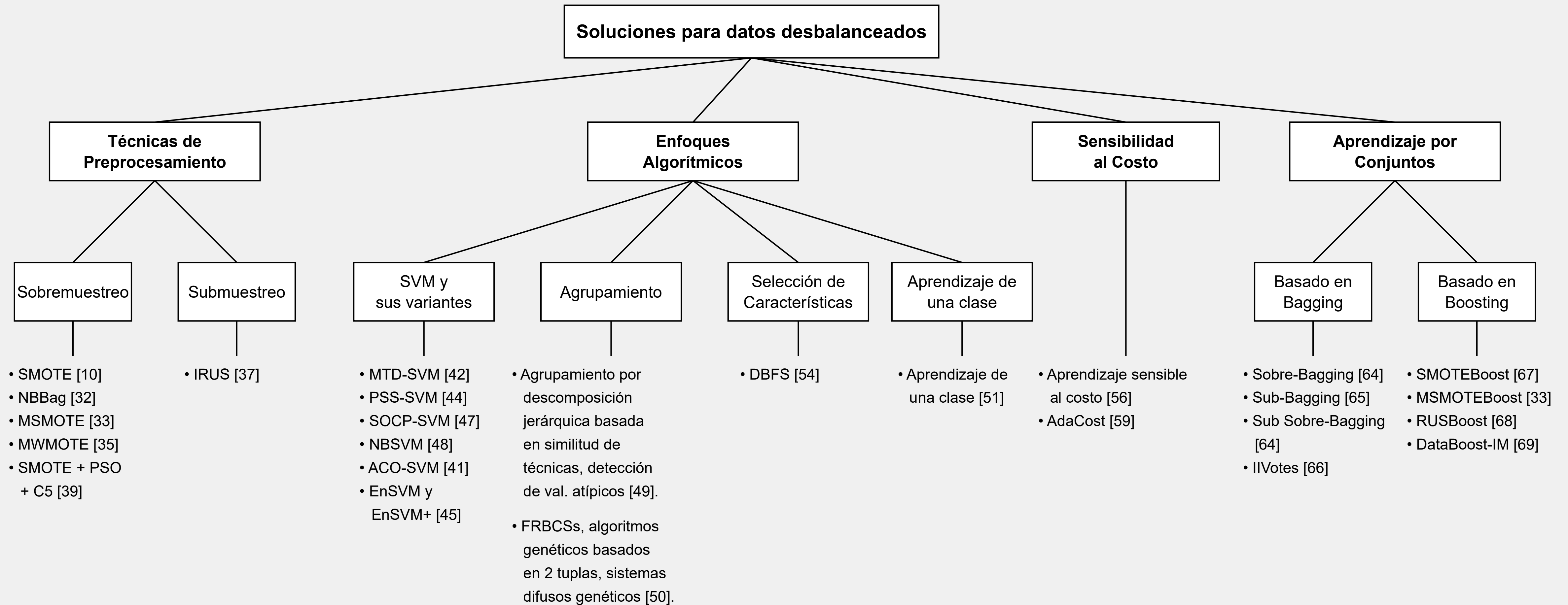
Datos dentro del margen: Representa el "margen" alrededor de la frontera de decisión. Este concepto es central en algoritmos como las Máquinas de Vectores de Soporte (SVM). El área sombreada contiene los datos (de ambas clases) que son más difíciles de clasificar porque están en la frontera o cruzando el límite hacia el territorio de la clase opuesta.

El gráfico captura visualmente el compromiso entre precisión y exhaustividad (Trade-off). Debido a que las clases se solapan: Falsos Positivos vs. Falsos Negativos: Cualquier dato de la clase mayoritaria que caiga a la derecha de la línea central será un falso positivo. Cualquier dato de la clase minoritaria que caiga a la izquierda será un falso negativo. Ajuste del Umbral: Si se mueve la frontera de decisión (la línea central) hacia la izquierda para capturar más de la clase minoritaria, inevitablemente se incluirá más área de la clase mayoritaria, aumentando los falsos positivos. Si se mueve a la derecha, ocurre lo contrario. Los "datos dentro del margen" son esencialmente la fuente de error del modelo y el área donde las técnicas de optimización deben enfocarse para mejorar la separación de las clases.

Datos desbalanceados: Problemas

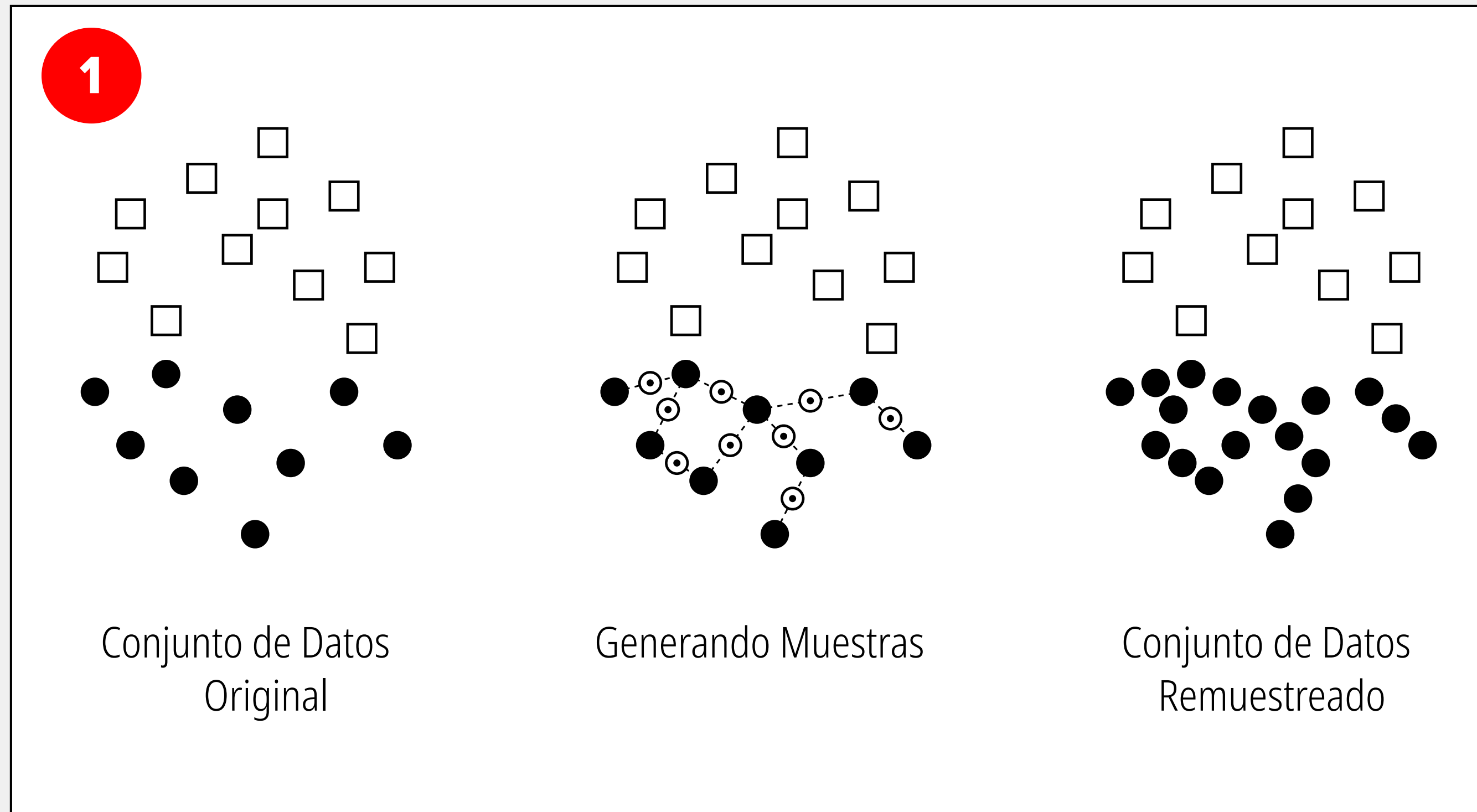


Datos desbalanceados: Métodos y técnicas algorítmicas



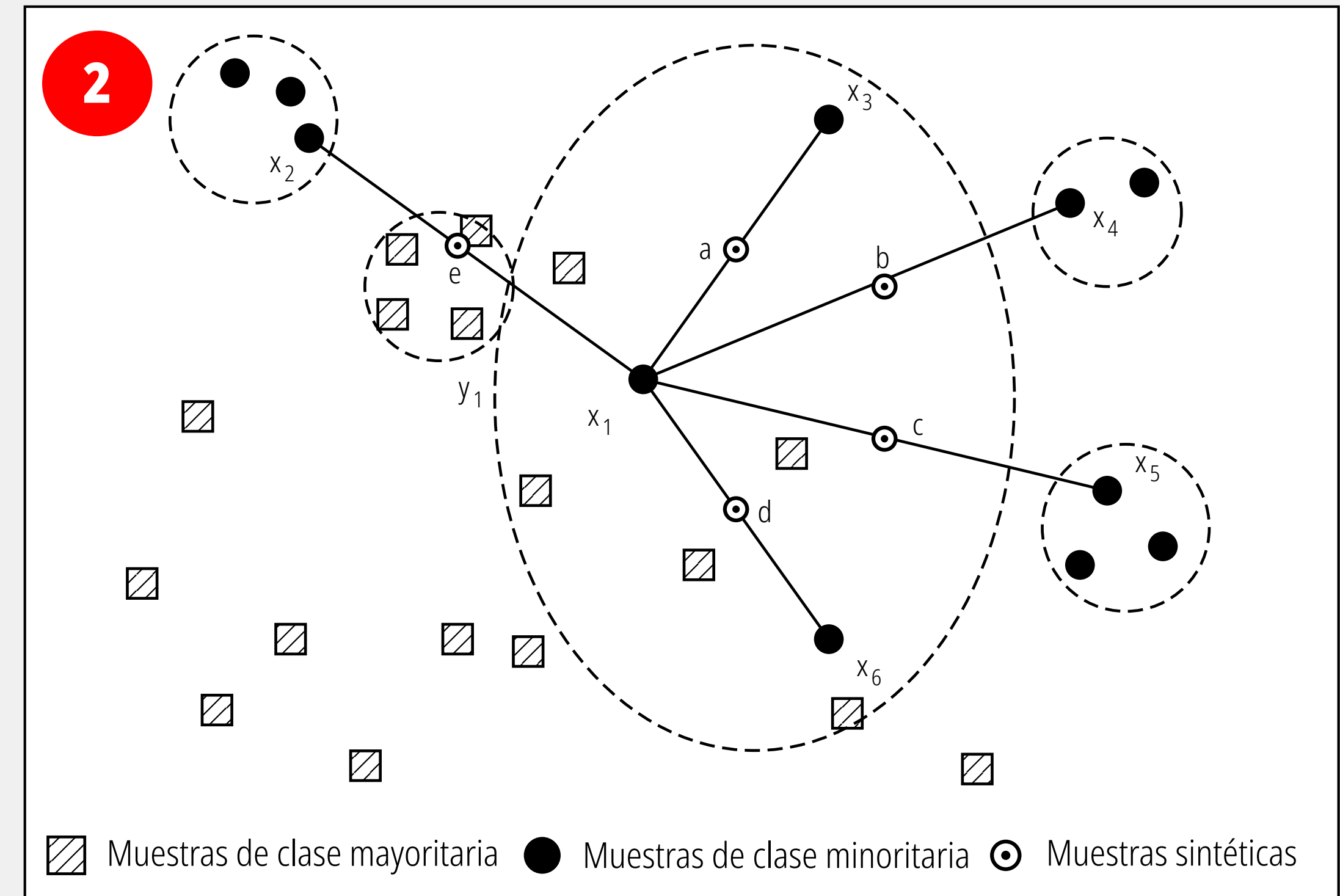
SMOTE: Synthetic Minority Oversampling Technique

Es un algoritmo clave en el preprocesamiento de datos de aprendizaje automático, utilizado específicamente para abordar el problema de los conjuntos de datos desequilibrados—un escenario muy común al entrenar modelos para la detección de anomalías o el monitoreo de transacciones.



Baja diversidad con SMOTE

En conjunto de datos original se observa un desequilibrio claro. La clase mayoritaria está representada por los cuadrados blancos y la clase minoritaria por los círculos negros. En lugar de simplemente duplicar los círculos negros existentes, SMOTE identifica las relaciones espaciales entre ellos. Las líneas punteadas muestran cómo el algoritmo conecta los puntos de la clase minoritaria cercanos entre sí. El resultado final es un conjunto de datos remuestreado, el algoritmo ha generado nuevos círculos negros a lo largo de las conexiones previamente establecidas, equilibrando la cantidad de datos entre ambas clases.

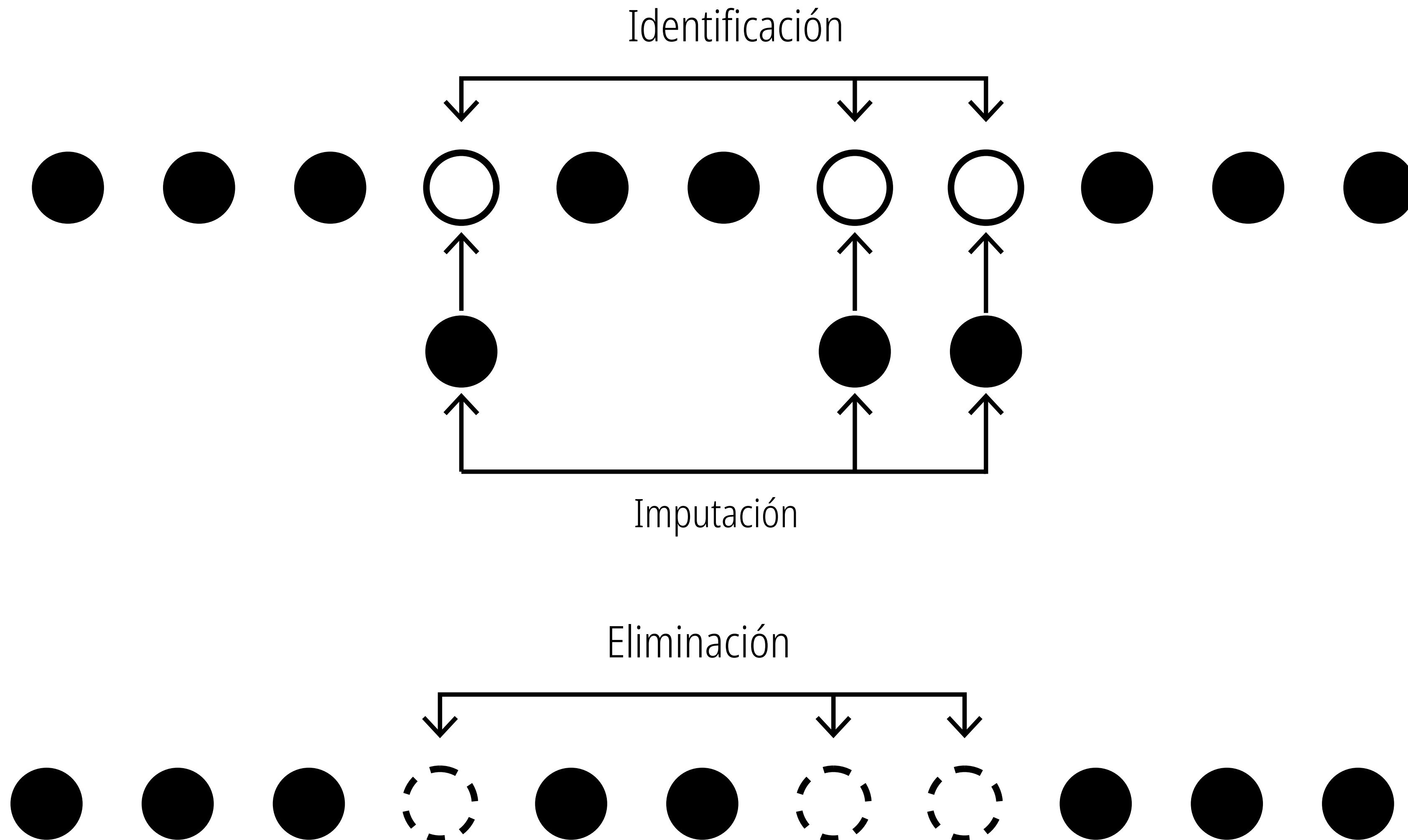


Interpolación con SMOTE

El algoritmo toma un punto específico de la clase minoritaria (marcado como x_1). Identifica cuáles son los k vecinos más cercanos a x_1 que también pertenecen a la clase minoritaria (en este caso, los puntos x_2 , x_3 , x_4 , x_5 y x_6). Interpolación: Se trazan vectores entre x_1 y cada uno de esos vecinos. Se generan las nuevas muestras sintéticas (a, b, c, d, e). Estos nuevos puntos no se colocan al azar en el espacio, sino que se ubican en posiciones aleatorias estrictamente a lo largo de las líneas que conectan x_1 con sus vecinos.

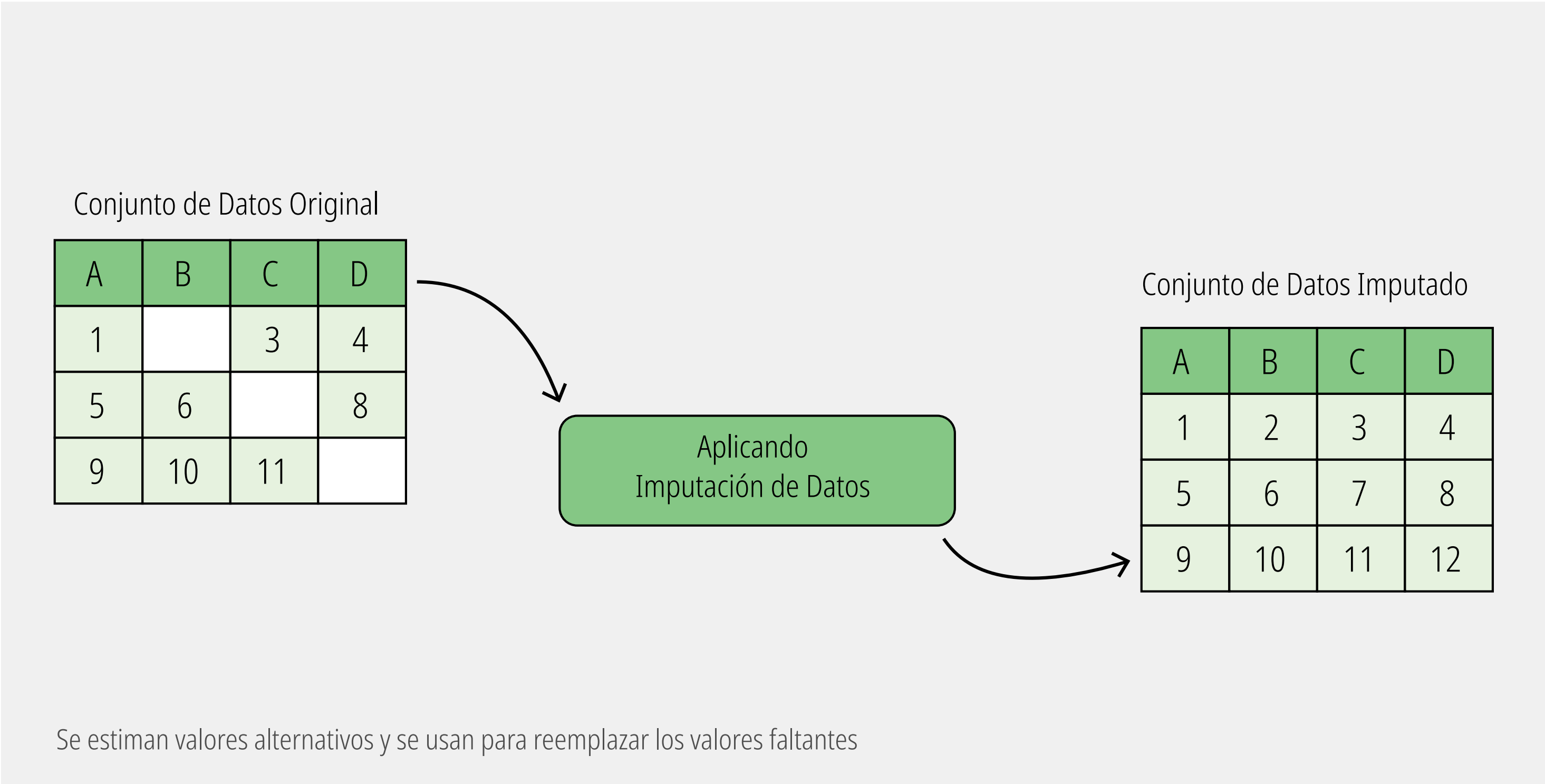
Valores faltantes

Nulos, ceros o espacios vacíos se consideran valores faltantes.



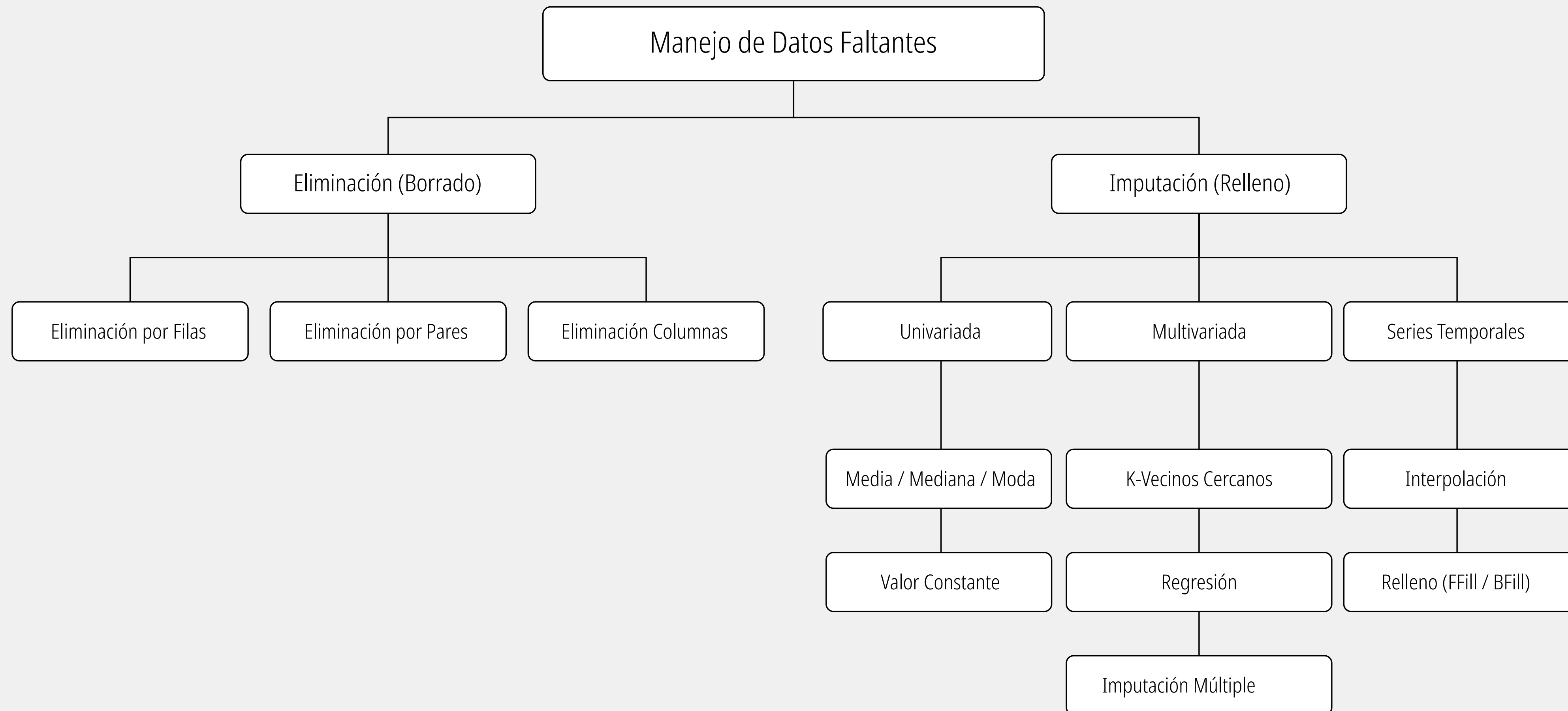
Imputación de valores faltantes

La imputación de valores faltantes es el proceso estadístico y de preparación de datos que consiste en reemplazar los datos nulos, vacíos o ausentes (a menudo representados como NaN, Null o NA en programación) por valores estimados o calculados.



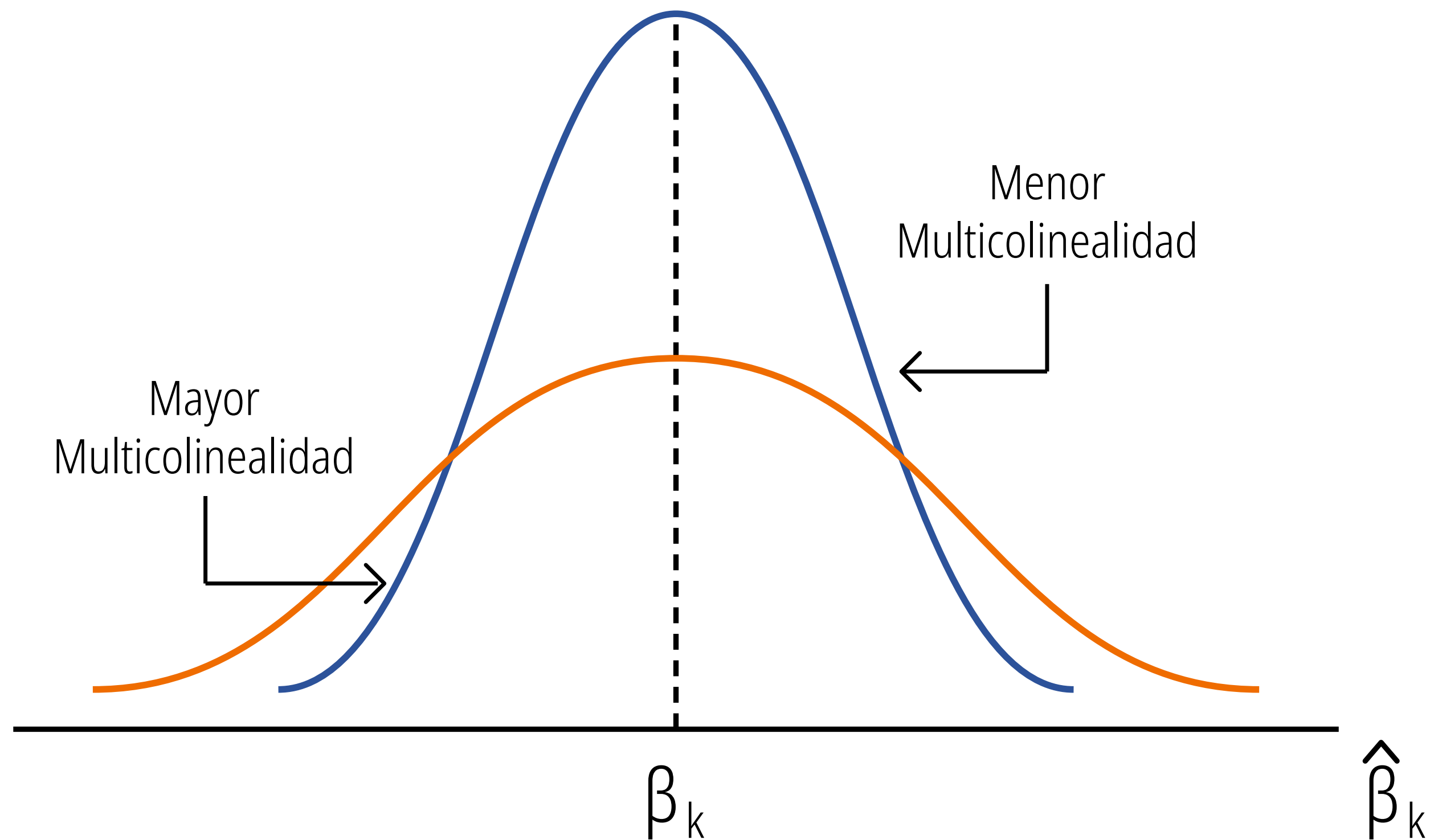
Estrategia	Ventaja Principal	Desventaja Principal
Media / Mediana	Rápido, sencillo y funciona bien si faltan pocos datos.	Reduce artificialmente la varianza de la distribución (los datos se agrupan en el centro).
Borrar Filas	Mantiene un conjunto de datos 100% puro y real.	Destruye información útil en otras columnas de la misma fila.
Predictivo (KNN/ Regresión)	Altamente preciso; mantiene las relaciones complejas entre variables.	Computacionalmente costoso y requiere más tiempo de desarrollo.

Imputación de valores faltantes

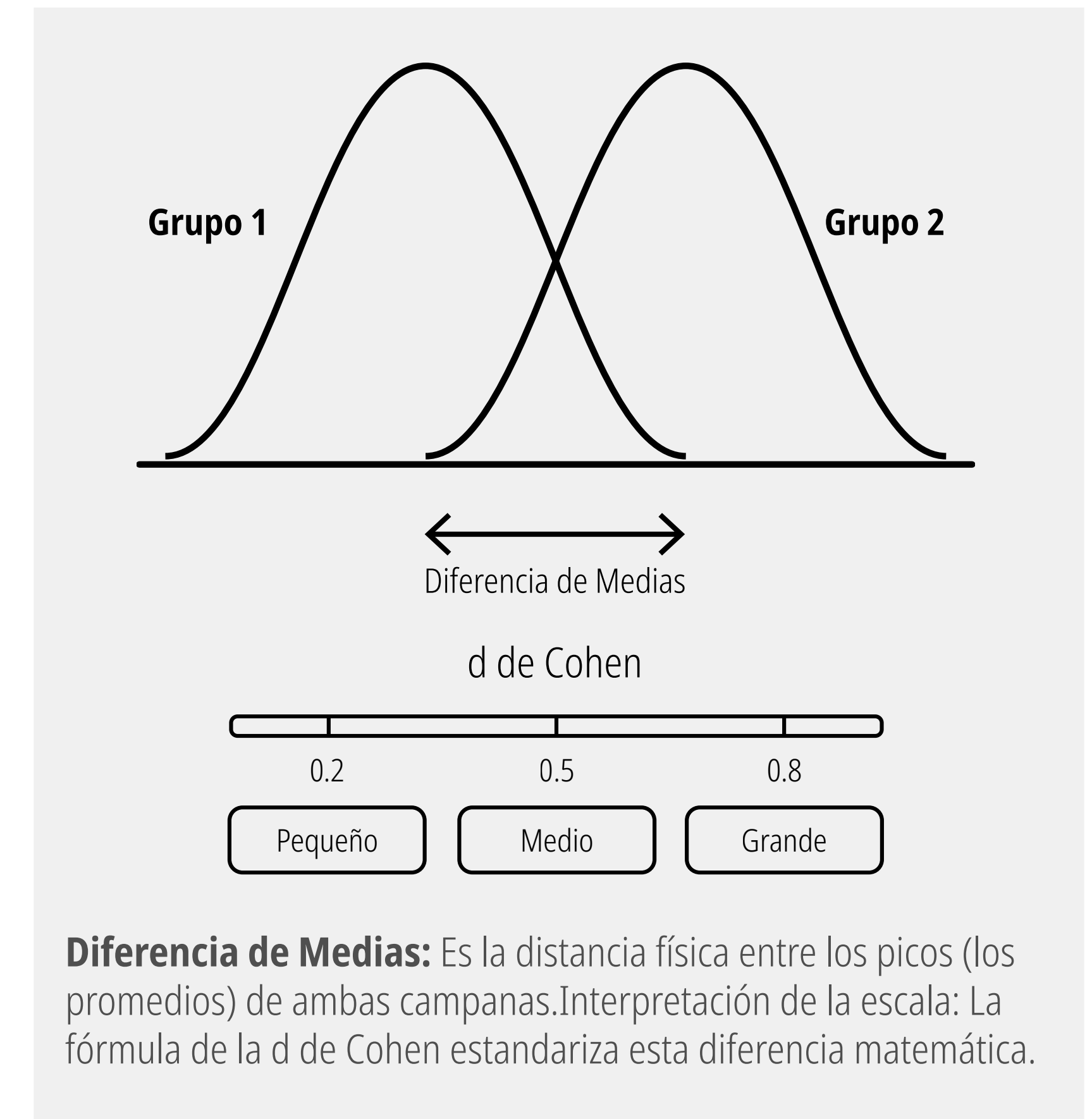


Multicolinearidad

Este concepto se aplica principalmente en modelos de regresión múltiple. La multicolinealidad ocurre cuando dos o más variables independientes (las que usas para predecir) están altamente correlacionadas entre sí.



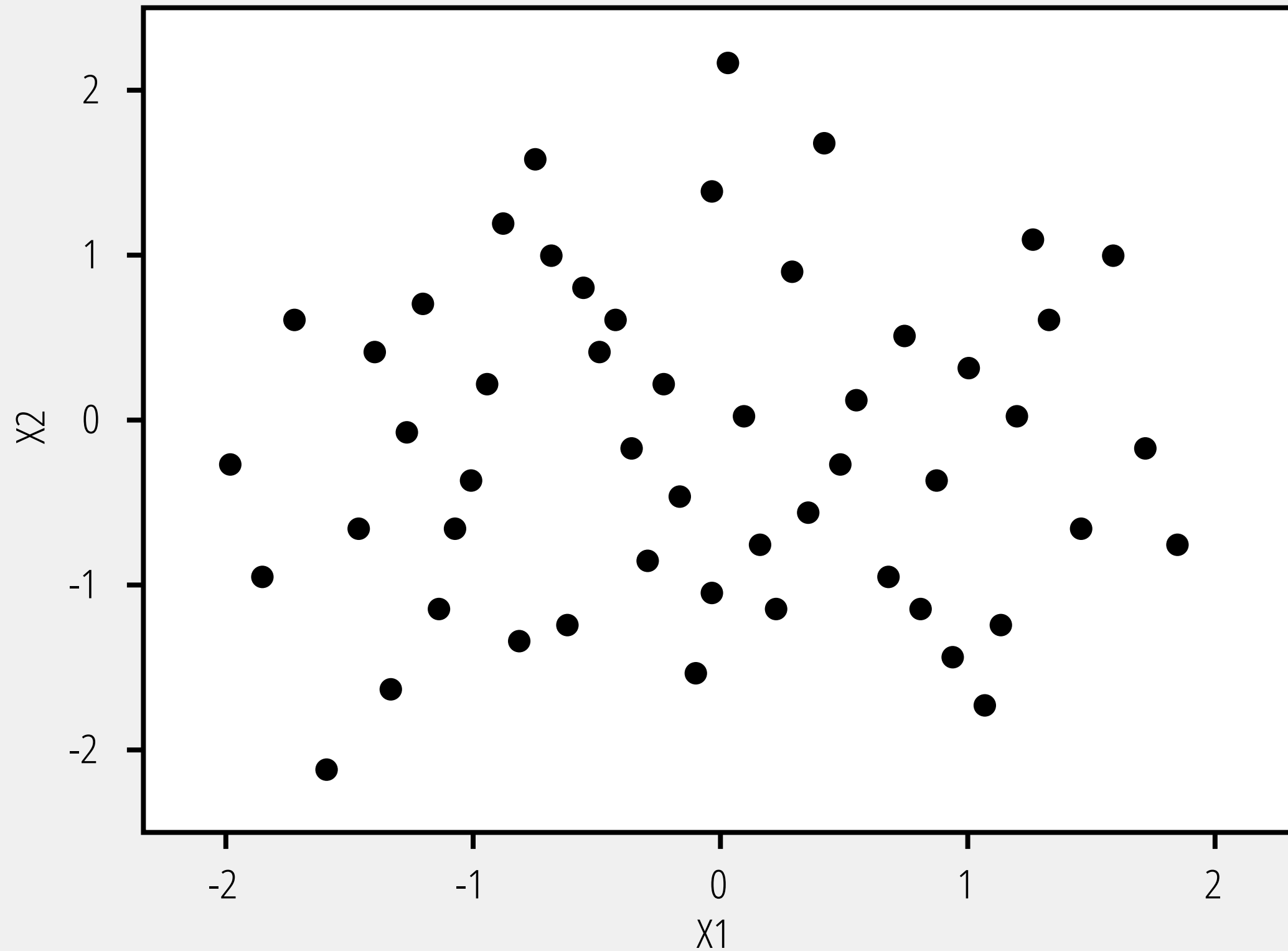
Curva Azul (Menor Multicolinealidad): Es alta y estrecha. Esto es lo ideal. Significa que la varianza es baja y el modelo está muy seguro del valor real del coeficiente. Tu estimación es precisa. **Curva Naranja (Mayor Multicolinealidad):** Es baja y ancha. Al estar las variables altamente correlacionadas, el modelo se "confunde" y no puede distinguir qué variable está aportando realmente la información. Como resultado, la varianza del coeficiente se dispara (la curva se aplana), haciendo que la estimación sea imprecisa, inestable y poco confiable.



Multicolinealidad

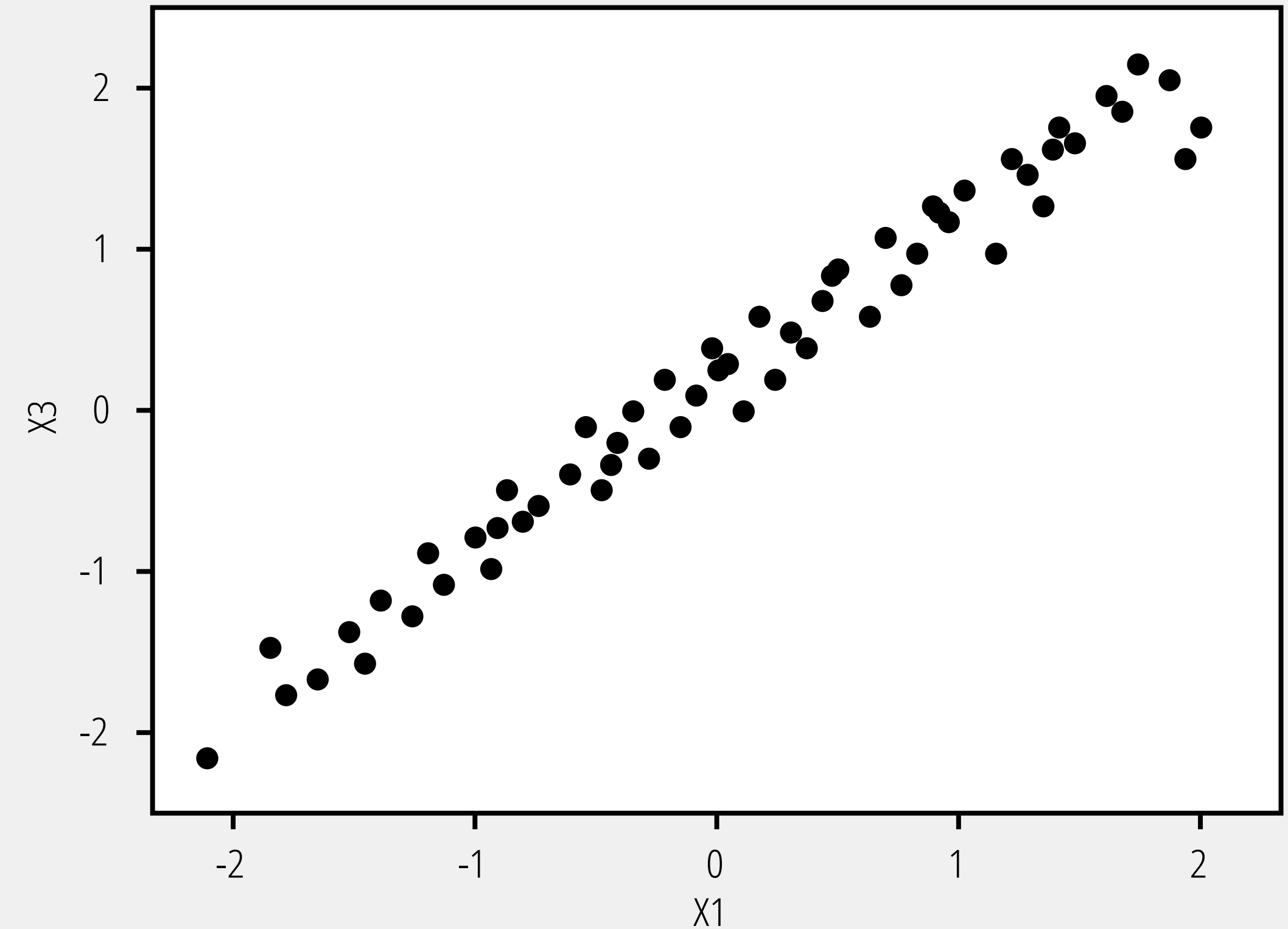
La multicolinealidad causa redundancia, lo que conduce a resultados inestables y dificulta la interpretación.

Baja Correlación: X1 vs X2



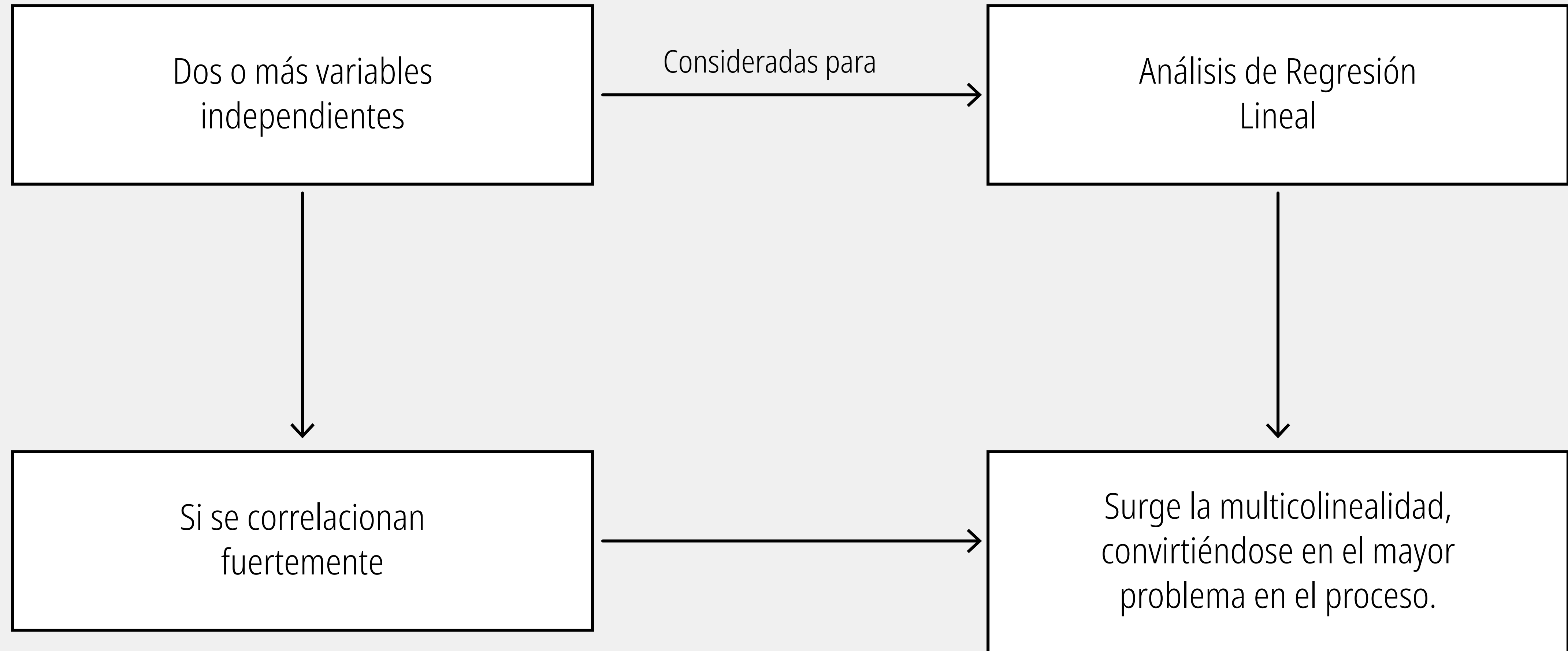
Baja Correlación - Sin Multicolinealidad

Alta Correlación: X1 vs X3



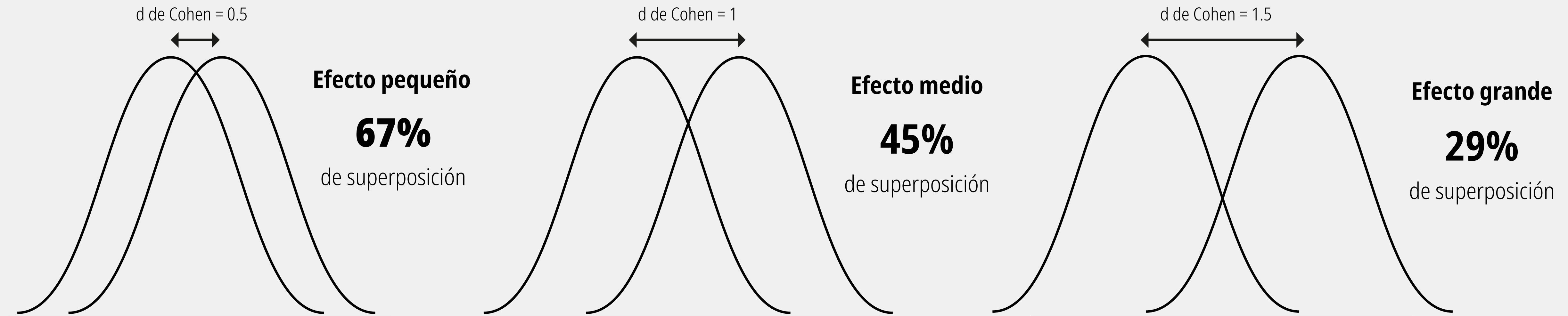
Alta Correlación - Multicolinealidad

Multicolinearidad



Tamaño del Efecto: d de Cohen

A medida que el valor de la d de Cohen aumenta, las medias de los grupos se alejan y la zona en la que se confunden (superposición) se reduce drásticamente, confirmando que el efecto analizado es de gran magnitud.



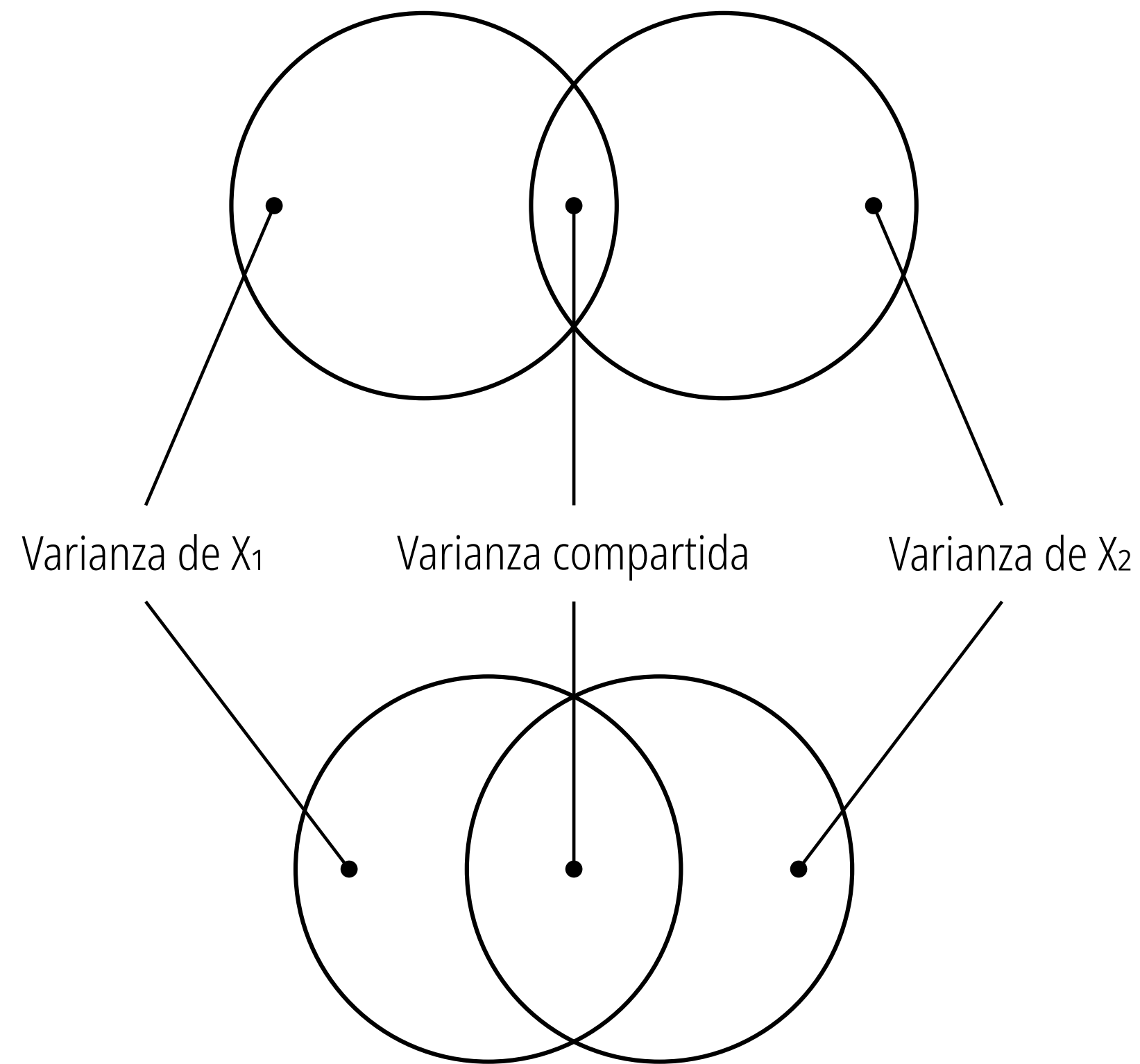
Los promedios de ambos grupos están muy cerca uno del otro. En la práctica, aunque estadísticamente haya una diferencia, es tan pequeña que sería difícil distinguir a un individuo (o dato) de un grupo frente al otro.

La distancia entre los promedios aumenta y las curvas comienzan a separarse. La diferencia entre los dos grupos es ahora mucho más clara y el efecto de lo que se esté evaluando empieza a ser evidente.

Los promedios están marcadamente separados. En este escenario, los grupos son claramente distintos. La intervención, característica o cambio que separa a un grupo del otro tiene un impacto sustancial, fuerte y fácilmente observable.

Multicolinealidad

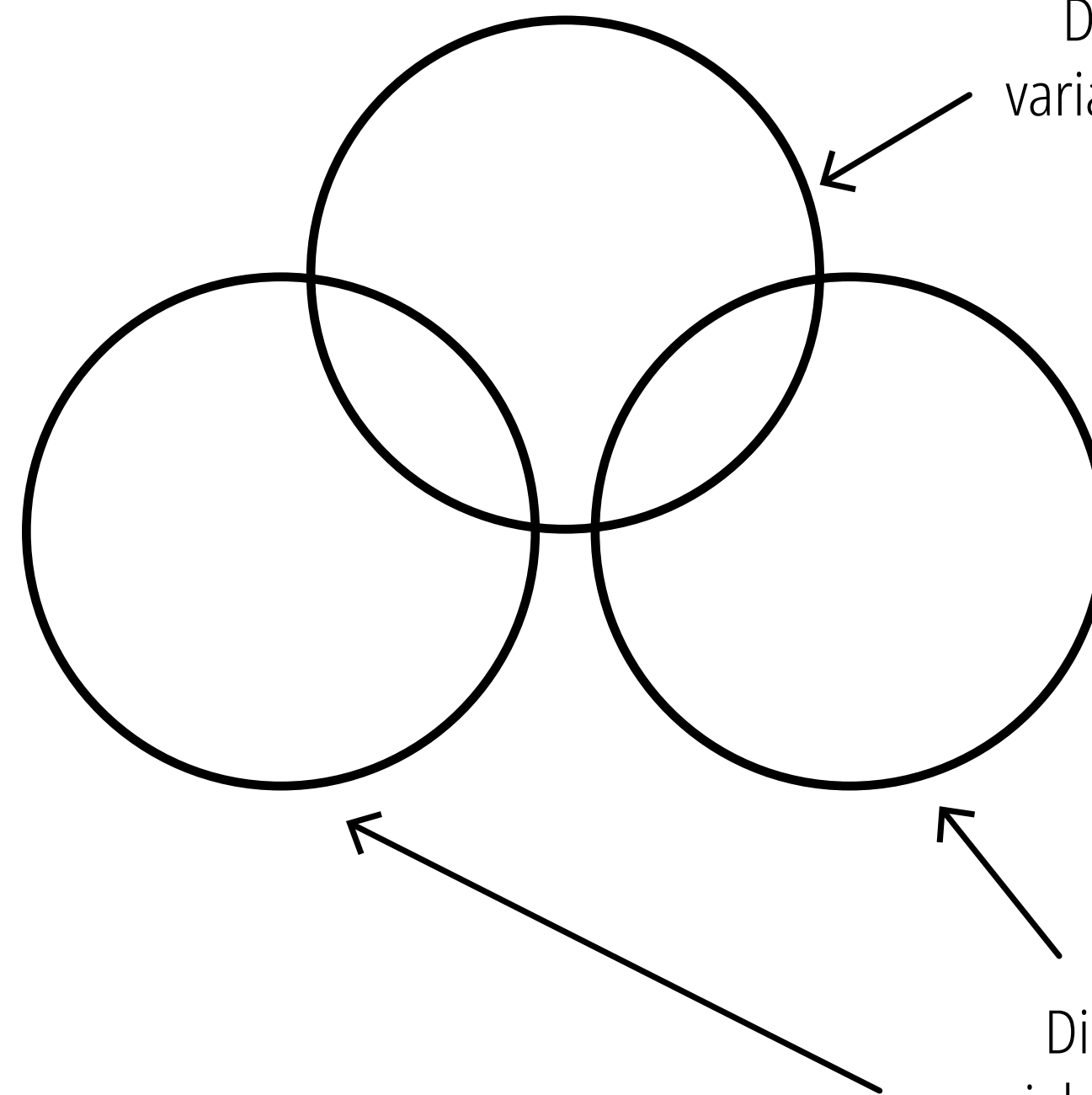
Variables ligeramente correlacionadas



Variables altamente correlacionadas

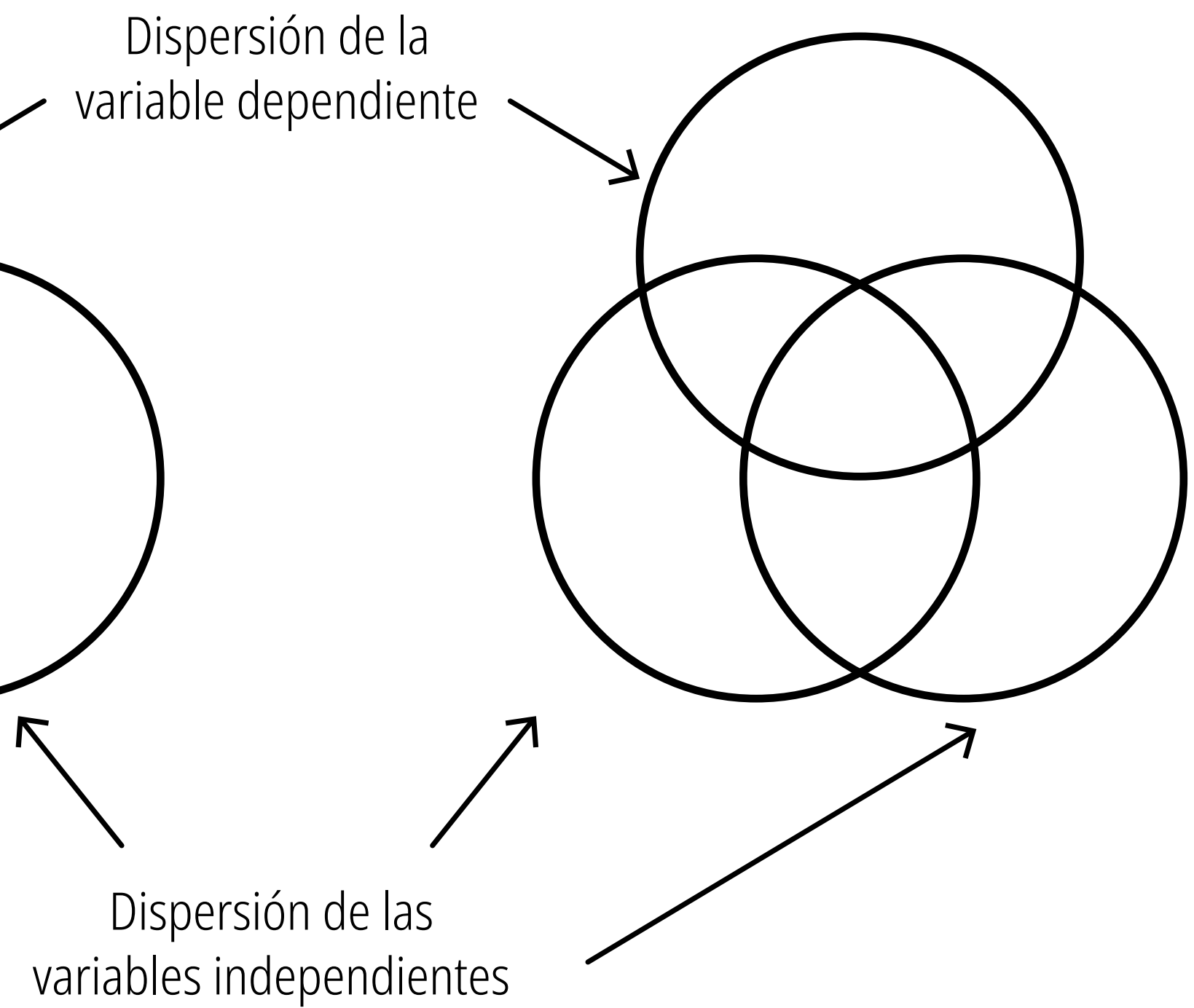
Sin multicolinealidad

Sin correlación de las variables independientes



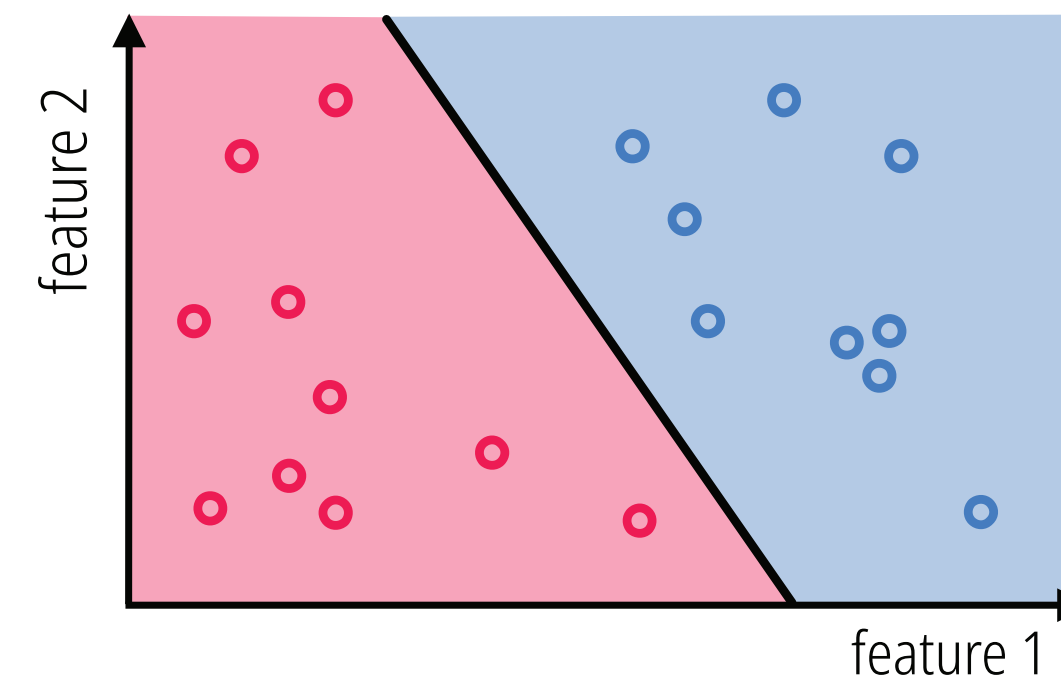
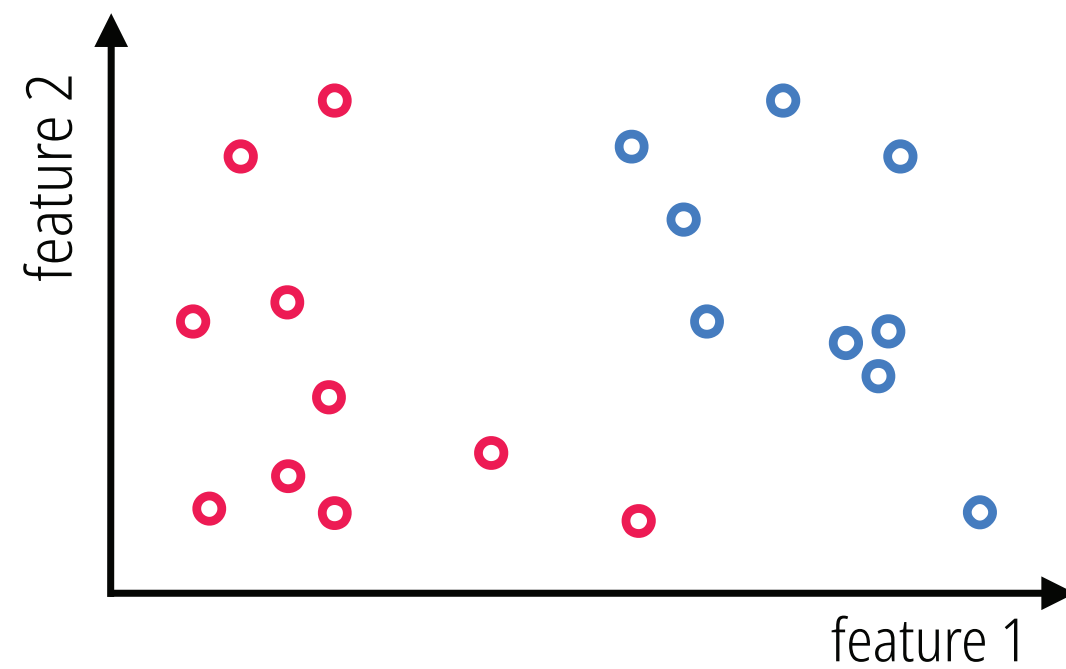
Multicolinealidad

Alta correlación de las variables independientes

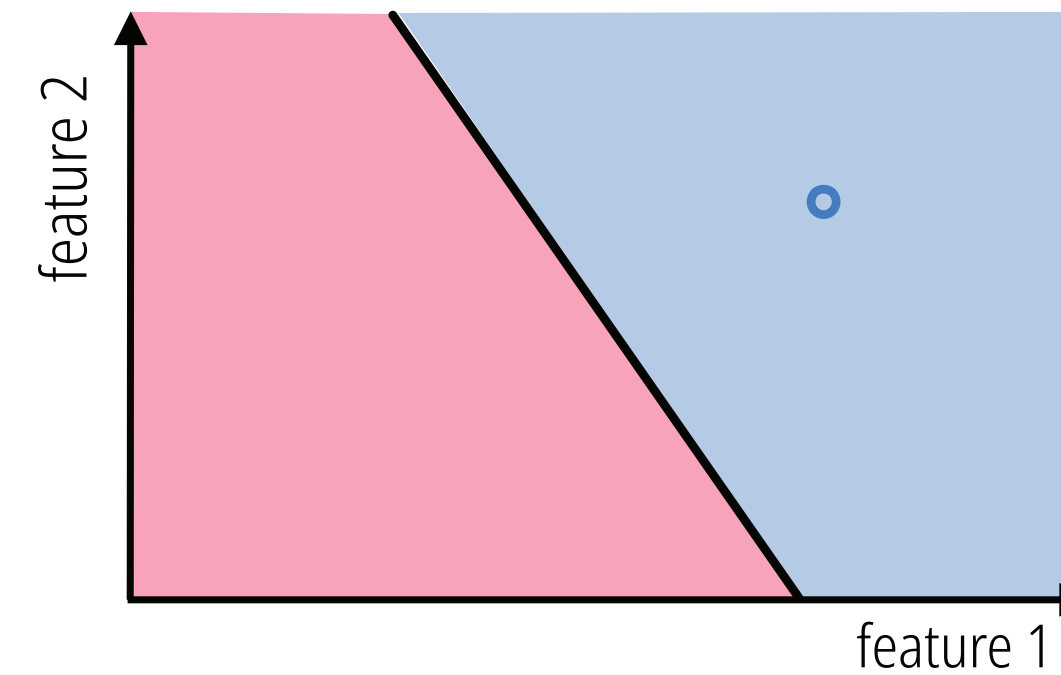
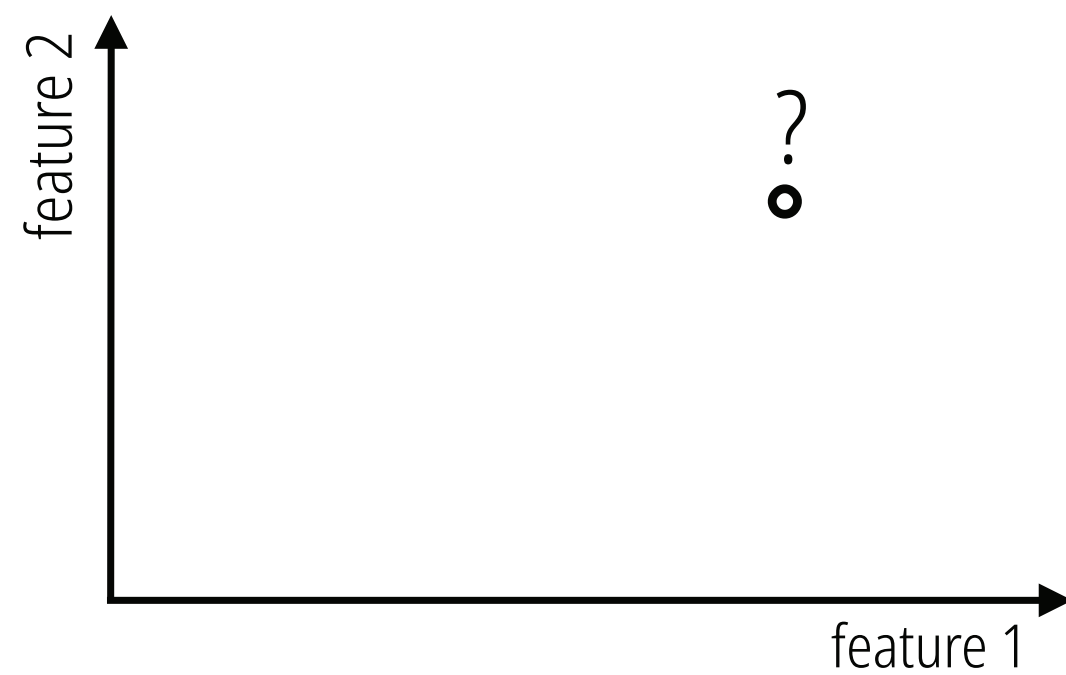


Tarea: Clasificación

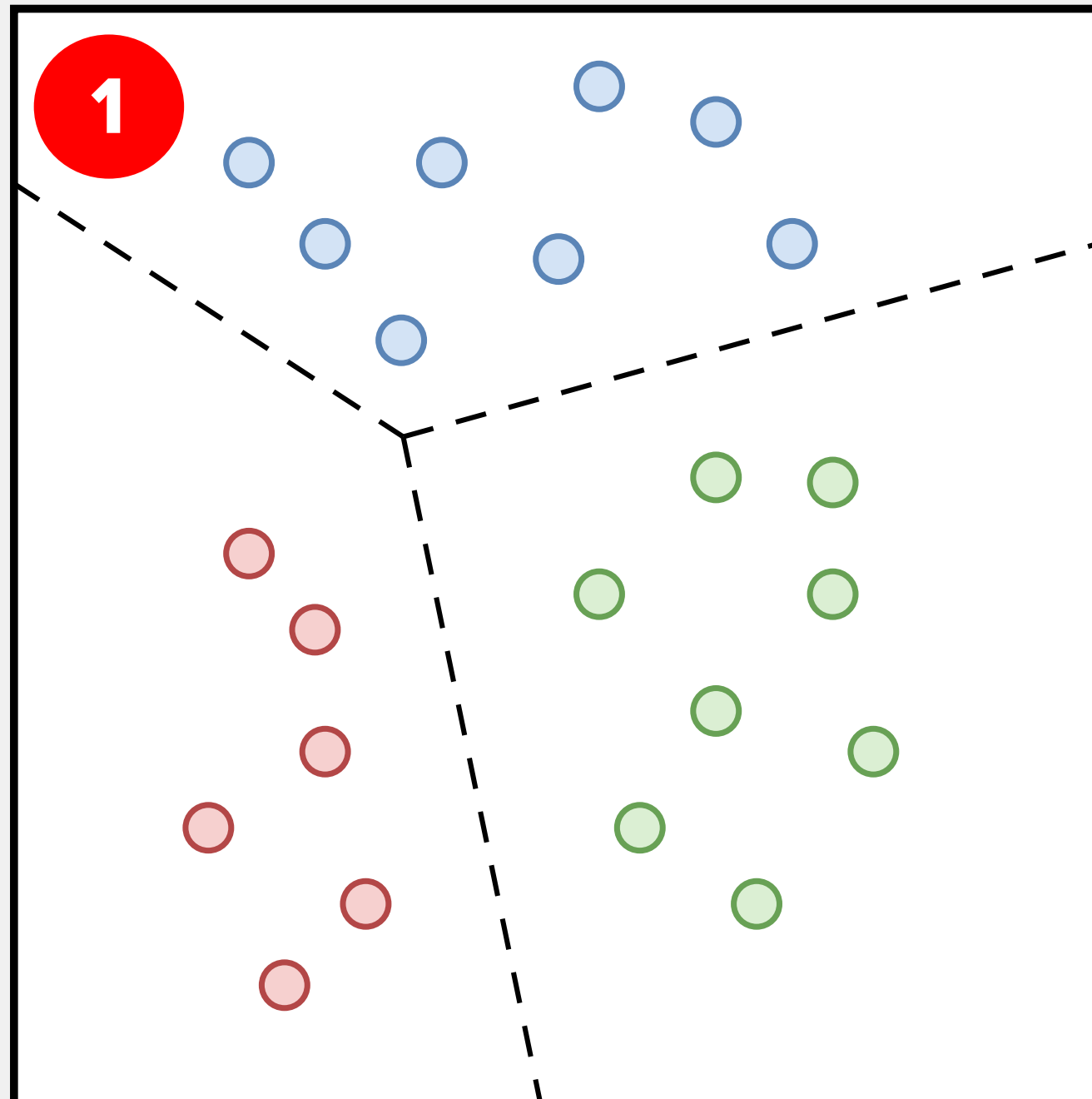
Geométricamente hablando, una forma común de ver la tarea de clasificación en dos dimensiones consiste en encontrar una línea de separación (o, de manera más general, una curva de separación) que separe con precisión dos tipos de datos.



La figura ilustra el concepto de un modelo lineal o clasificador utilizado para realizar la clasificación en un conjunto de datos de prueba bidimensional.

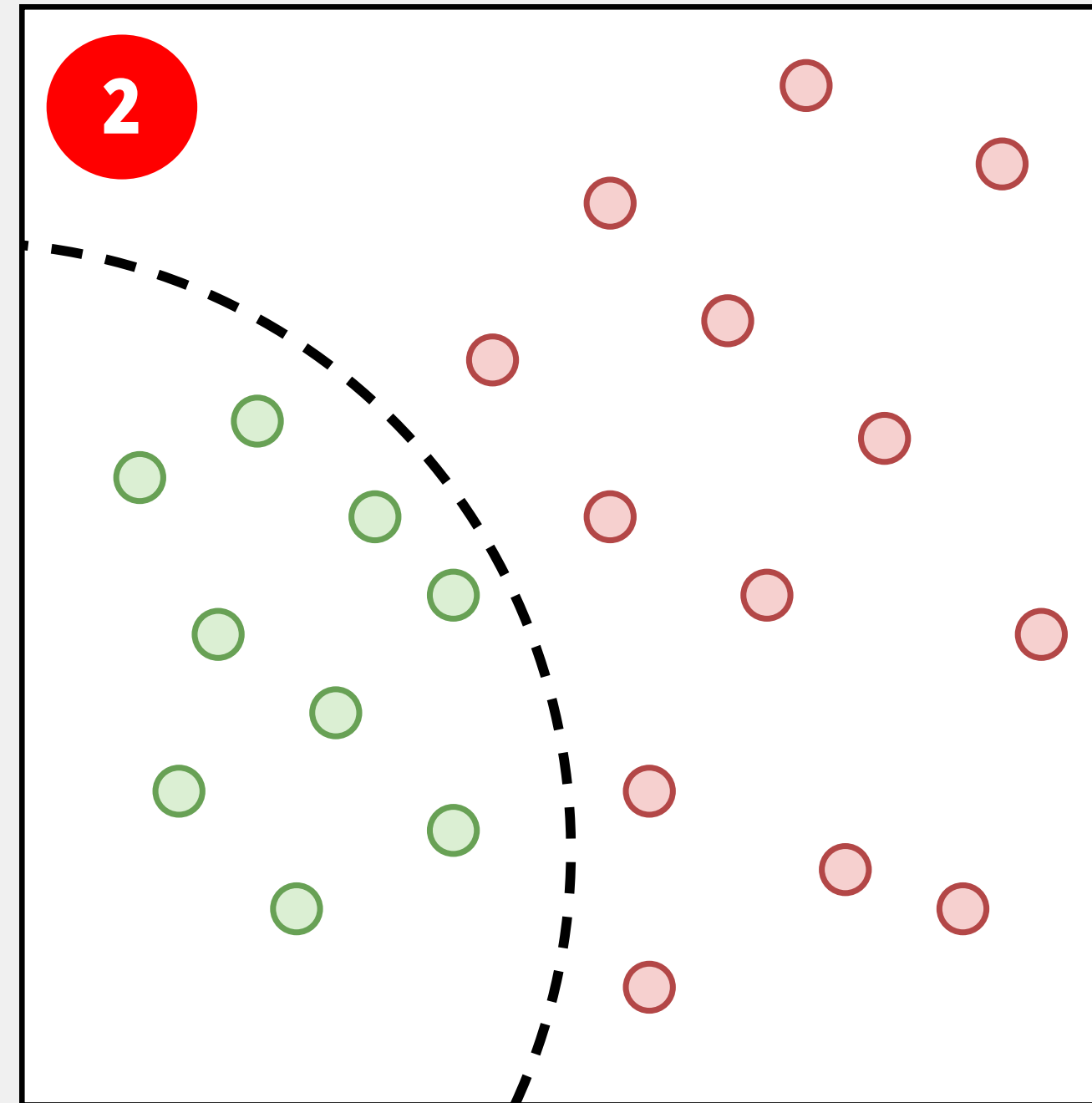


Tarea: Clasificación



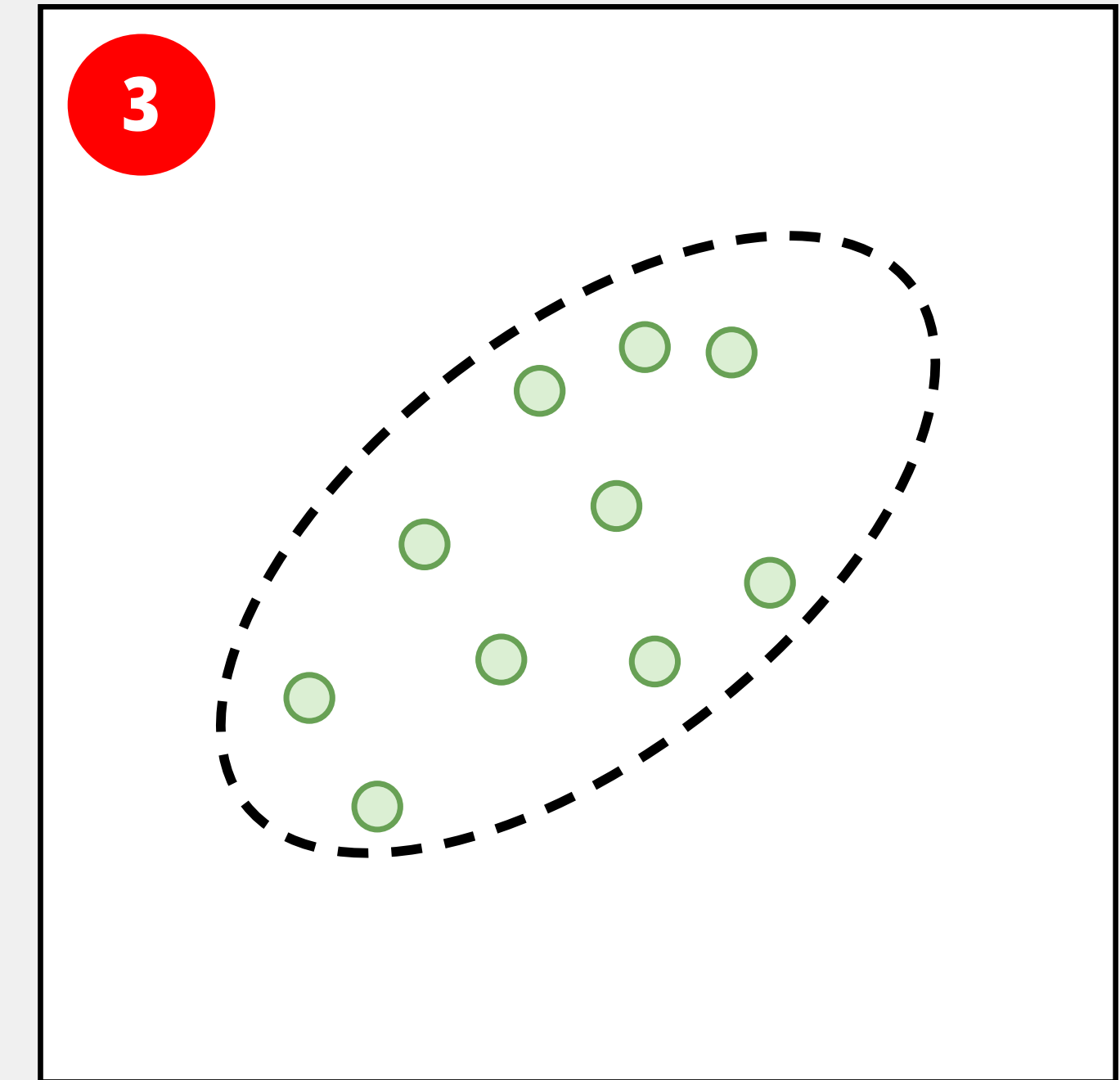
Clasificación multiclase

El objetivo de este modelo es asignar cada punto de datos a una de múltiples categorías mutuamente excluyentes (en este caso, tres: rojo, azul y verde).



One-vs-Rest

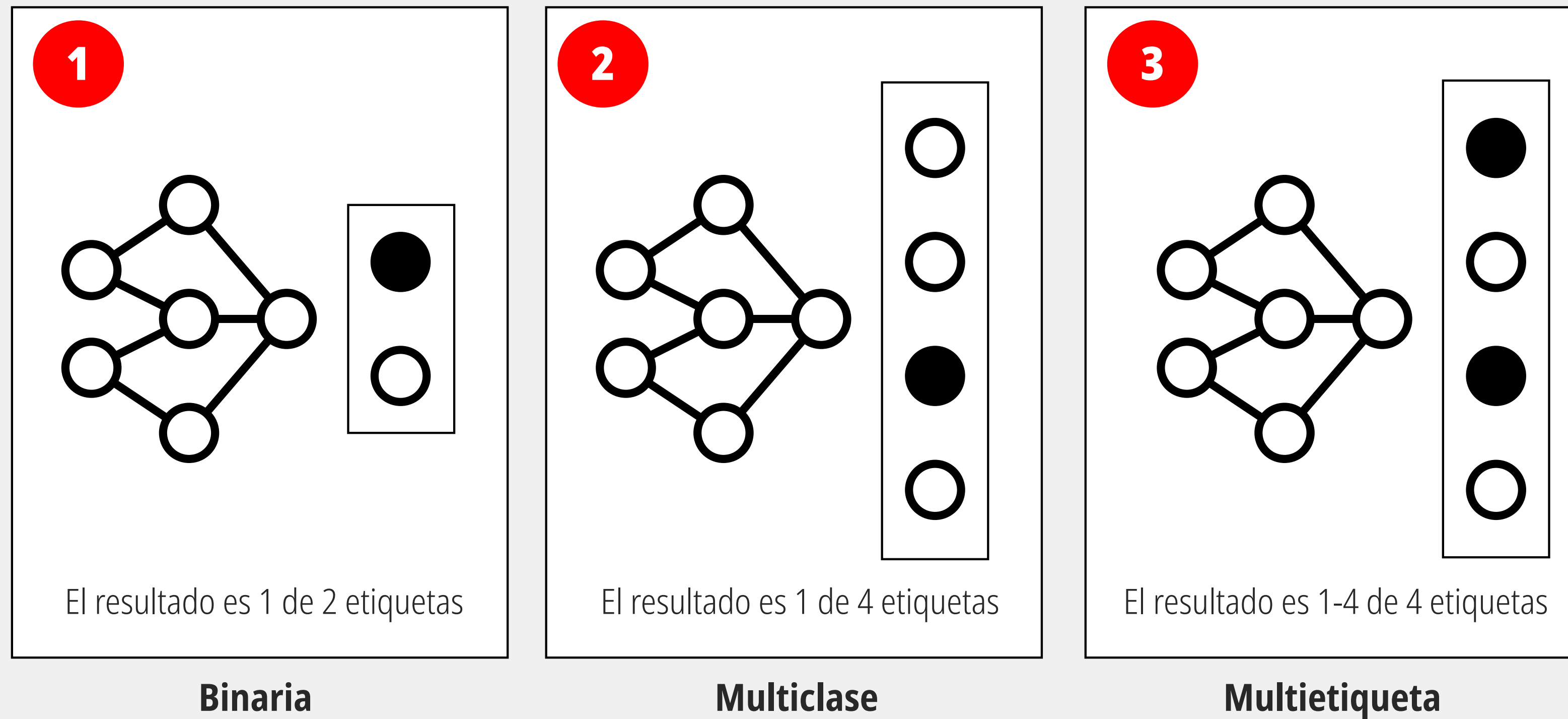
La meta principal es detectar un patrón específico frente a un conjunto de datos mixto.



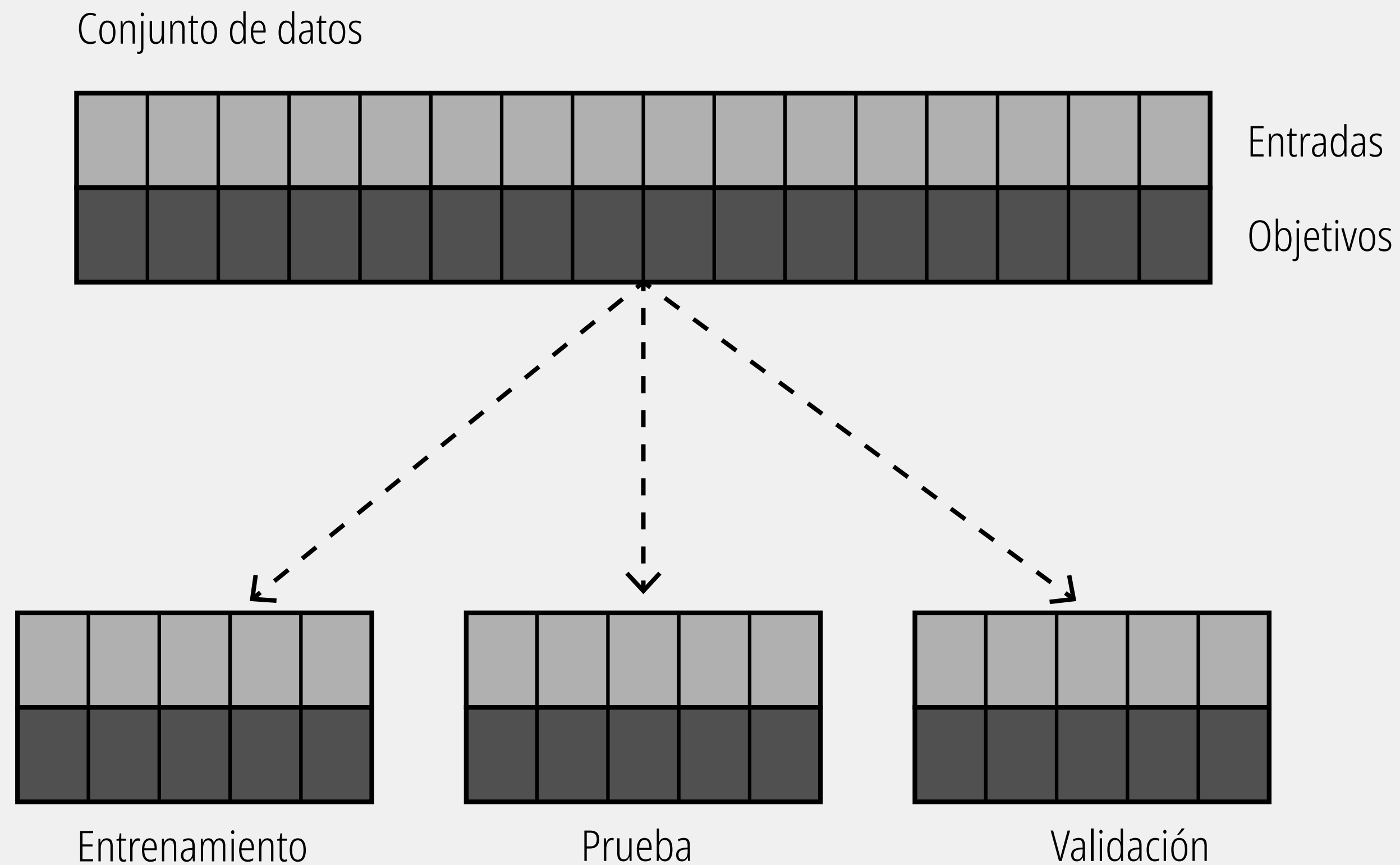
One-Class Classification

En este escenario, el algoritmo se entrena exclusivamente con ejemplos de la clase "normal" (los puntos verdes). El modelo aprende a trazar una frontera (la elipse punteada) que encapsula estas instancias.

Tarea: Clasificación



Partición estratégica de un conjunto de datos



El conjunto de datos para aprendizaje supervisado está dividido en **Entradas** (las variables o características que el sistema observa, como la frecuencia temporal o el monto de una operación) y los **Objetivos** (el resultado que queremos que el modelo aprenda a predecir, como identificar si el patrón corresponde a un comportamiento normal o a una anomalía estructurada). Si un algoritmo procesa toda esta información de golpe, corre el riesgo de simplemente "memorizar" el histórico, volviéndose inútil al enfrentarse a escenarios futuros. Para evitar esto, los datos se dividen en tres subconjuntos aislados.

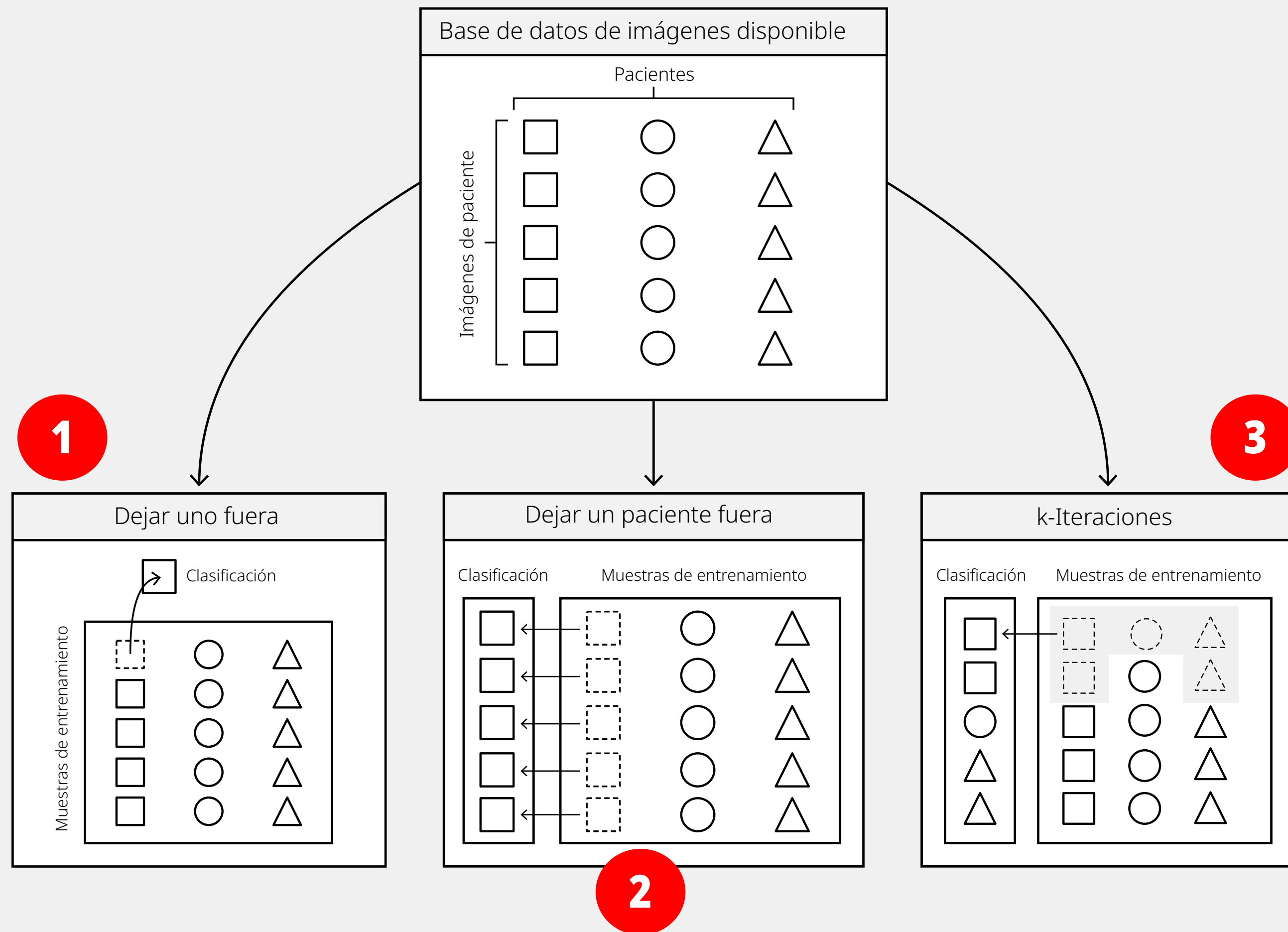
El **subconjunto de entrenamiento**, es la porción más grande de los datos (usualmente entre el 60% y el 80%). Es aquí donde el núcleo matemático del modelo analiza los ejemplos, ajusta sus pesos internos y construye su lógica predictiva base.

El **subconjunto de validación**; durante el proceso de entrenamiento, el modelo se evalúa periódicamente contra este segundo bloque para ajustar configuraciones internas delicadas (hiperparámetros). Esta validación cruzada actúa como un calibrador dinámico que previene el "sobreajuste" (overfitting), asegurando que el modelo esté aprendiendo a generalizar las reglas subyacentes del comportamiento en lugar de memorizar el "ruido" específico de los datos con los que fue entrenado.

El **subconjunto de prueba** es la prueba de fuego y debe mantenerse estrictamente oculto y aislado durante todas las fases previas de desarrollo. Una vez que la arquitectura del modelo está definida, calibrada y optimizada, se expone por primera y única vez a este bloque de datos. Evaluar el rendimiento en esta etapa es la única métrica verdaderamente honesta de cómo se comportará el sistema en producción ante eventos completamente inéditos.

Validación cruzada

Permite asegurar que el modelo sea capaz de generalizar y funcionar bien con datos nuevos y no vistos, en lugar de simplemente memorizar los datos con los que fue entrenado.

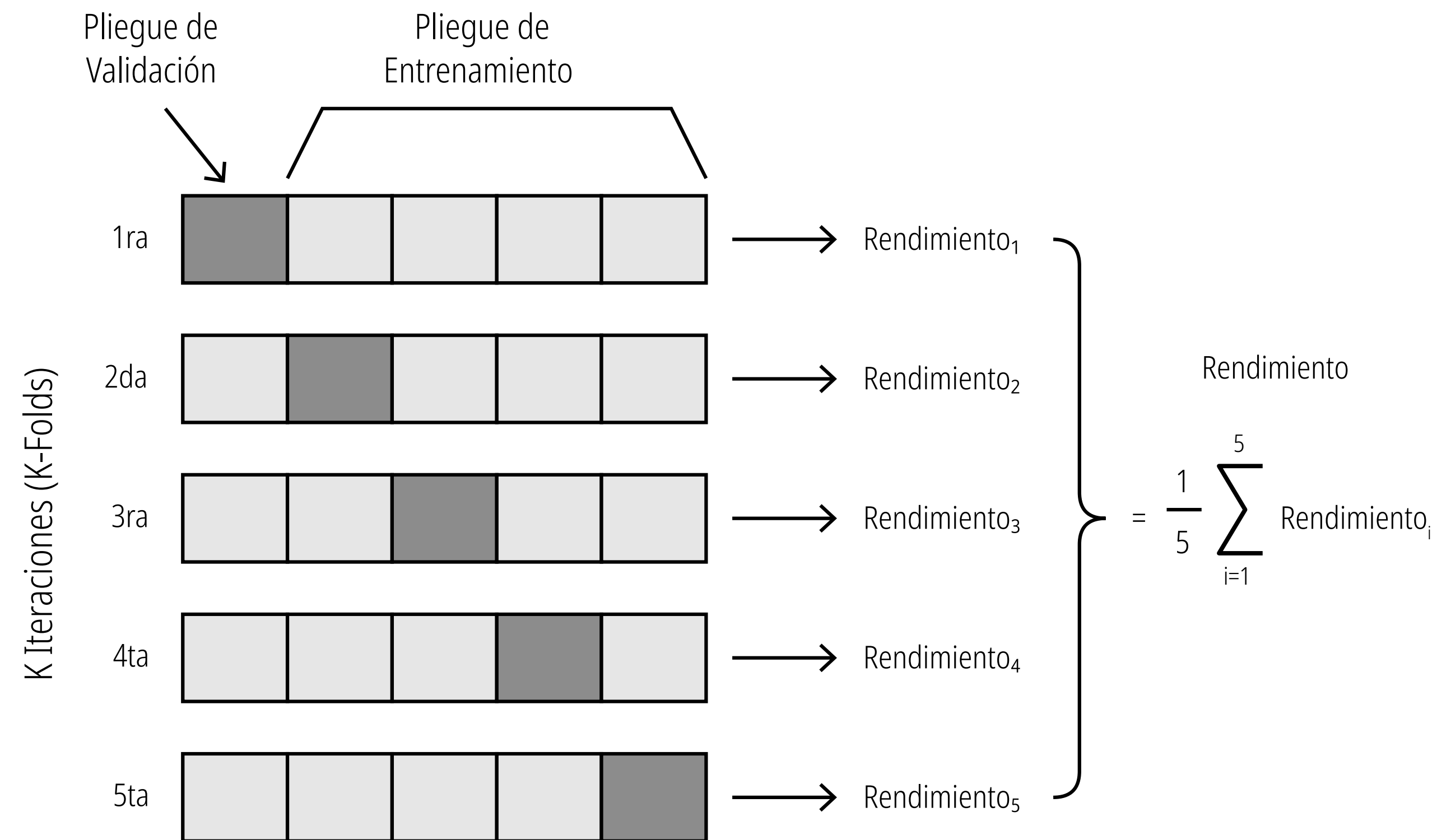


- 1 Validación de dejar uno fuera** (Leave-One-Out Cross-Validation), donde se aísla una sola imagen de todo el conjunto para evaluar la clasificación, utilizando absolutamente todo el resto de las muestras para entrenar el modelo.
- 2 Validación de dejar un paciente fuera** (Leave-One-Subject-Out Cross-Validation), un enfoque vital cuando se tienen múltiples registros por individuo; aquí se reservan en bloque todas las imágenes correspondientes a un mismo paciente para la fase de prueba, entrenando el algoritmo exclusivamente con los pacientes restantes para medir cómo generaliza el modelo ante sujetos completamente nuevos.
- 3 Validación cruzada de k-iteraciones** (K-Fold Cross-Validation), en la cual los datos se dividen aleatoriamente en varios subconjuntos o "pliegues", seleccionando iterativamente un grupo mixto de imágenes como conjunto de prueba mientras las demás conforman las muestras de entrenamiento. el algoritmo exclusivamente con los pacientes restantes para medir cómo generaliza el modelo ante sujetos completamente nuevos.

En lugar de dividir el conjunto de datos una sola vez en una parte para "entrenar" y otra para "probar" (lo cual puede dejar fuera información valiosa o dar resultados sesgados por casualidad), la validación cruzada repite este proceso de división y evaluación múltiples veces.

Validación cruzada

La validación cruzada es una técnica estadística robusta diseñada para evaluar la verdadera capacidad de generalización de un modelo predictivo. En lugar de realizar una única división estática de los datos históricos (donde una parte se usa para enseñar y otra para evaluar), esta metodología recicla el conjunto de información múltiples veces.



El proceso operativo sigue esta secuencia:

- 1) El bloque total de datos (que contiene tanto las entradas como los objetivos) se particiona en 5 segmentos o "pliegues" idénticos.
- 2) El modelo se entrena desde cero 5 veces distintas. En la 1ra iteración, el primer bloque (marcado en gris oscuro) se aísla como el Pliegue de Validación. El modelo se entrena exclusivamente con los 4 bloques restantes (el Pliegue de Entrenamiento) y luego se expone al bloque de validación para evaluar su capacidad de detectar patrones, generando una métrica de rendimiento. En la 2da iteración, el rol de validación se desplaza al segundo bloque.
- 4) El modelo se vuelve a entrenar con los otros 4 bloques y se evalúa nuevamente para obtener el rendimiento. Esta rotación metódica continúa hasta que los 5 bloques hayan servido exactamente una vez como prueba de fuego. K-Fold garantiza que cada línea de datos históricos haya sido evaluada.
- 5) Al final del proceso, agrupa calcula el promedio aritmético de las iteraciones. Si las métricas individuales oscilan violentamente entre iteraciones, indica que el filtro predictivo es inestable; si el promedio es alto y las variaciones son mínimas, el algoritmo está listo para enfrentarse a flujos de datos en producción.

El objetivo es asegurar que la precisión del modelo sea consistente y no el resultado de haber memorizado un fragmento de datos "fácil" o particular.

Validación cruzada de K iteraciones (K-fold)

Es un método para determinar modelos robustos validados de forma cruzada a través de un procedimiento similar al ensamblaje que restringe la complejidad del modelo final para que sea más interpretable por los humanos.

1

En lugar de promediar un grupo de modelos validados de forma cruzada, cada uno de los cuales logra un error de validación mínimo en una partición aleatoria de entrenamiento-validación de los datos, con la validación cruzada de K iteraciones elegimos un único modelo final que alcanza el error de validación promedio más bajo sobre todas las particiones en conjunto. Al seleccionar un único modelo para representar todo el conjunto de datos, en contraposición a un promedio de diferentes modelos (como se hace con el ensamblaje), facilitamos la interpretación del modelo seleccionado.

2

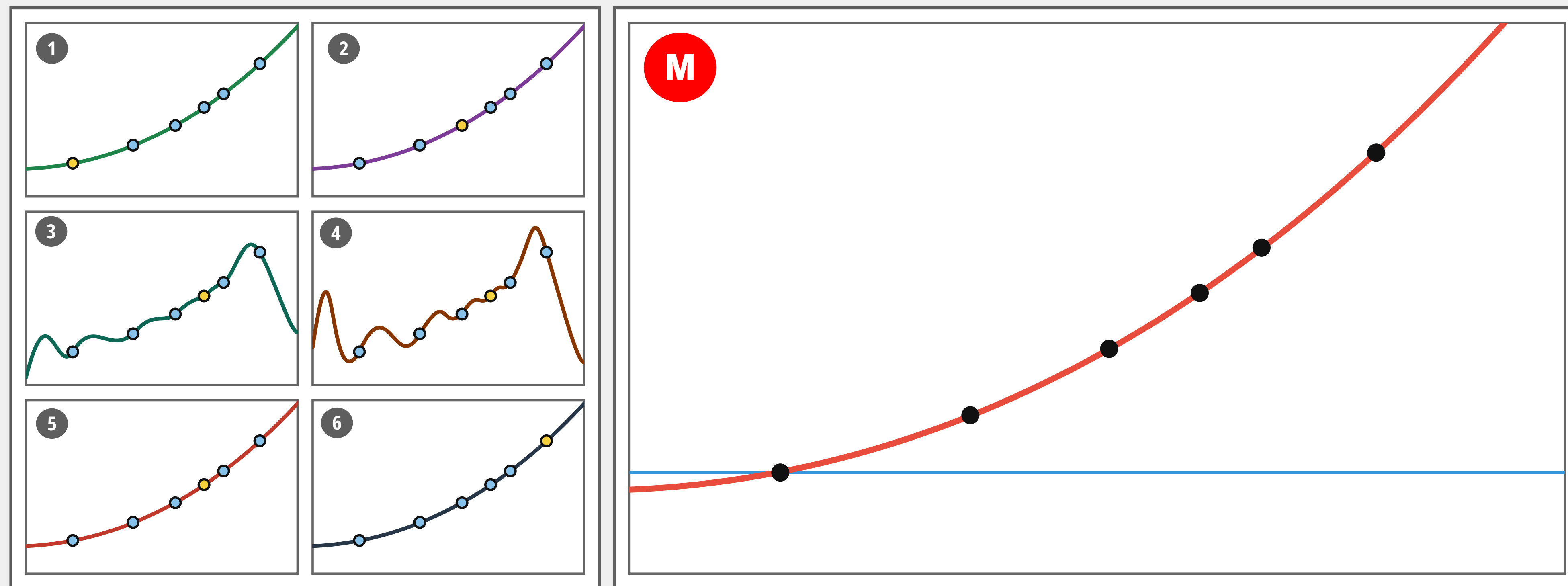
Por supuesto, el deseo de que cualquier modelo no lineal sea interpretable significa que sus bloques de construcción fundamentales (aproximadores universales de un cierto tipo) también deben ser interpretables. Las redes neuronales, por ejemplo, **casi nunca son interpretables** por los humanos, mientras que los aproximadores de forma fija (más comúnmente polinomios) y los basados en árboles (comúnmente tocones o stumps) pueden interpretarse dependiendo del problema en cuestión.

3

Para simplificar aún más el resultado final de este procedimiento, en lugar de utilizar particiones de entrenamiento-validación completamente aleatorias (como se hace con el ensamblaje), dividimos los datos aleatoriamente en un conjunto de K fragmentos no superpuestos.

Validación cruzada de K iteraciones (K-fold)

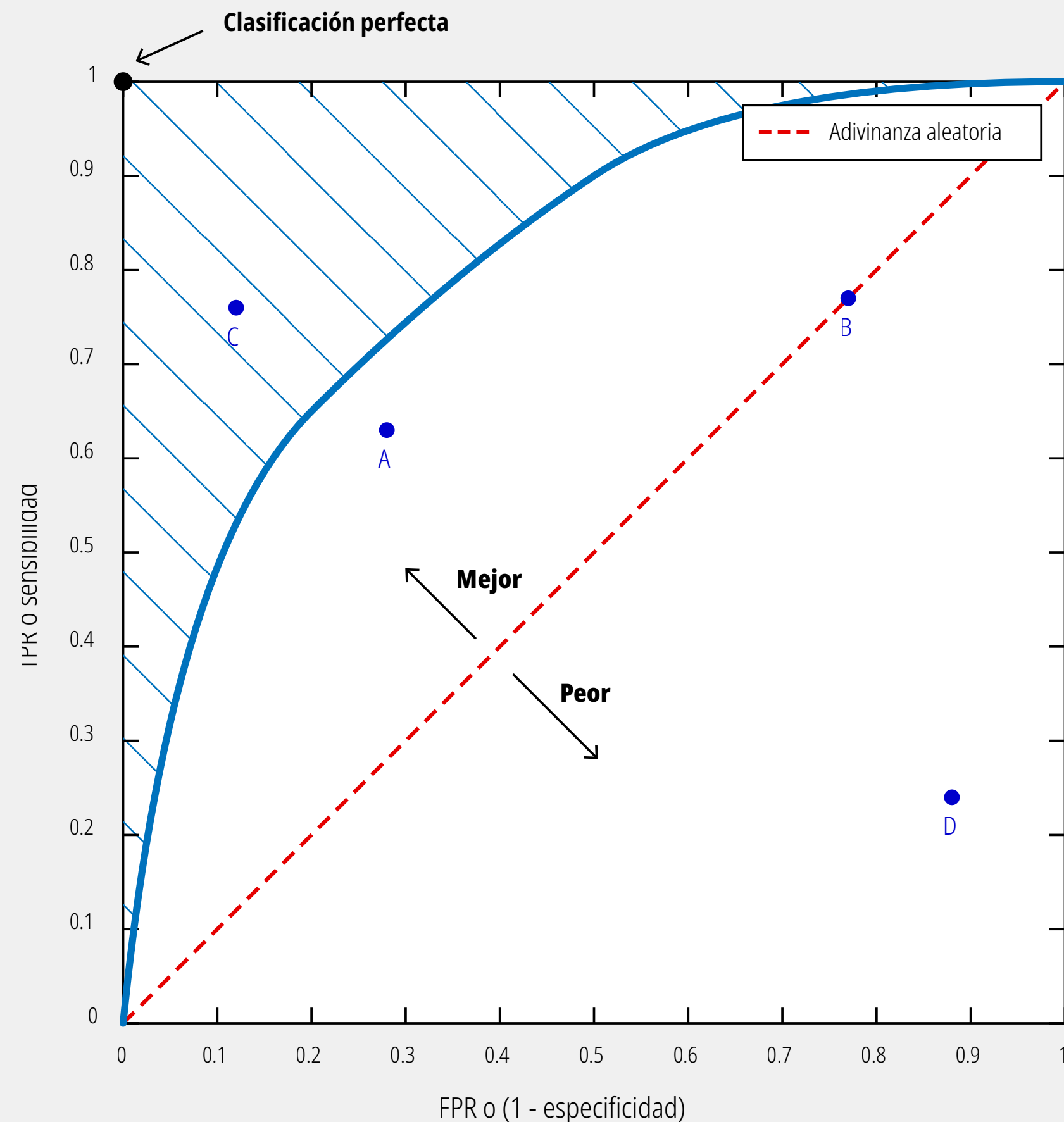
Iteramos a través de K particiones de entrenamiento-validación de los datos que consisten en K - 1 de estas partes como entrenamiento, con la porción restante como validación, lo que permite que cada punto del conjunto de datos pertenezca a un conjunto de validación exactamente una vez. Cada una de estas particiones se conoce como iteración o pliegue (del inglés fold), de las cuales hay K en total



En cada iteración, validamos de forma cruzada el mismo conjunto de modelos y registramos la puntuación de validación de cada uno. Posteriormente, elegimos el mejor modelo individual que haya producido el menor error de validación promedio. Una vez hecho esto, el modelo elegido se vuelve a entrenar sobre todo el conjunto de datos para proporcionar un predictor final ajustado a los datos. Dado que en este procedimiento no se combinan ni promedian modelos, puede producir muy fácilmente modelos menos precisos (en términos del error de prueba) para problemas generales de aprendizaje en comparación con el ensamblaje. Sin embargo, cuando la interpretabilidad humana de un modelo tiene mayor prioridad que la necesidad de un rendimiento excepcional, la validación cruzada de K iteraciones produce un modelo con un rendimiento superior al de un modelo individual validado de forma cruzada que aún puede ser comprendido por los seres humanos.

Evaluación: Curva ROC

Curva ROC (del inglés Receiver Operating Characteristic o Característica Operativa del Receptor). Es una representación gráfica fundamental utilizada en el aprendizaje automático para evaluar y visualizar el rendimiento de los modelos de clasificación binaria a través de diferentes umbrales de decisión.



Eje Y (TPR o Sensibilidad): Representa la Tasa de Verdaderos Positivos (True Positive Rate). Nos indica: de todos los casos que realmente son positivos, ¿qué porcentaje logró identificar el modelo de forma correcta? Lo ideal es que este valor llegue a 1 (100%).

Eje X (FPR o 1 - especificidad): Representa la Tasa de Falsos Positivos (False Positive Rate). Nos indica: de todos los casos que realmente son negativos, ¿qué porcentaje el modelo clasificó erróneamente como positivos (falsas alarmas)? Lo ideal es que este valor se mantenga en 0.

Líneas y Zonas Clave

Línea diagonal roja discontinua ("Adivinanza aleatoria"): Representa una base de referencia (baseline). Un modelo cuya curva o puntos caigan sobre esta línea es equivalente a lanzar una moneda al aire; no tiene ninguna capacidad predictiva real. Esquina superior izquierda ("Clasificación perfecta"): Es el "santo grial" del modelo (coordenada 0, 1). Significa que detecta absolutamente todos los casos positivos (Sensibilidad = 1) sin cometer un solo error de falsa alarma (FPR = 0). La Curva Azul y el Área Rayada: La línea azul sólida muestra cómo se comporta un modelo específico. El área rayada debajo de ella representa una métrica crucial llamada AUC (Área Bajo la Curva). Como indica la flecha "Mejor", cuanto más se abombe la curva hacia la esquina superior izquierda, mayor será el área y mejor será la capacidad del modelo para distinguir entre clases.

Los puntos dispersos representan el rendimiento del modelo utilizando diferentes configuraciones o umbrales estrictos: Punto C (arriba a la izquierda): Es un rendimiento excelente. Tiene una alta sensibilidad (detecta alrededor del 75% de los positivos) con una tasa muy baja de falsos positivos (cerca del 12%). Punto A: Es un rendimiento decente, pero inferior a C. Detecta poco más del 60% de los positivos, pero genera casi un 30% de falsas alarmas. Punto D (abajo a la derecha): Se encuentra en la zona "Peor". Su rendimiento es inferior a la adivinanza aleatoria. Irónicamente, si un modelo está constantemente por debajo de la línea diagonal (se equivoca sistemáticamente), a menudo puedes simplemente invertir sus predicciones (cambiar los "sí" por "no") para convertirlo en un modelo con buen rendimiento.

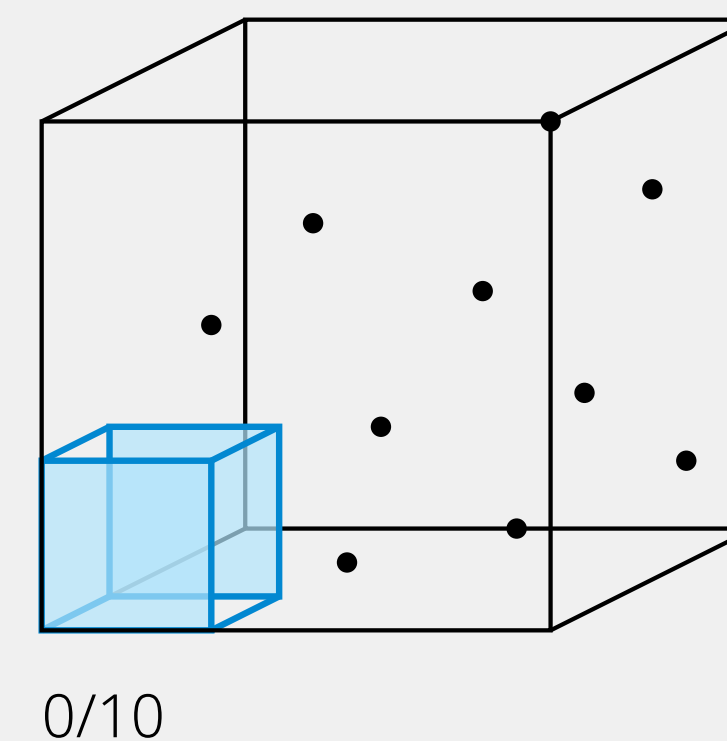
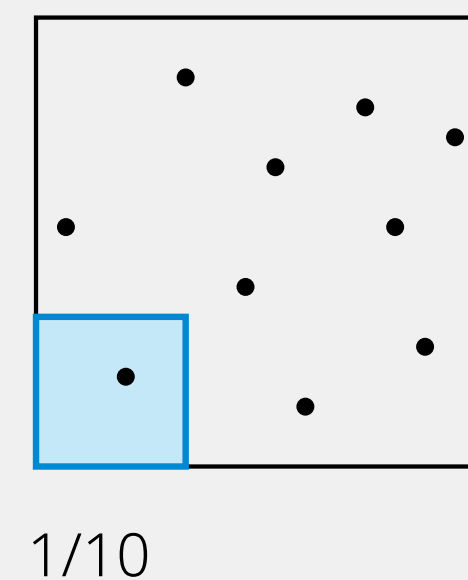
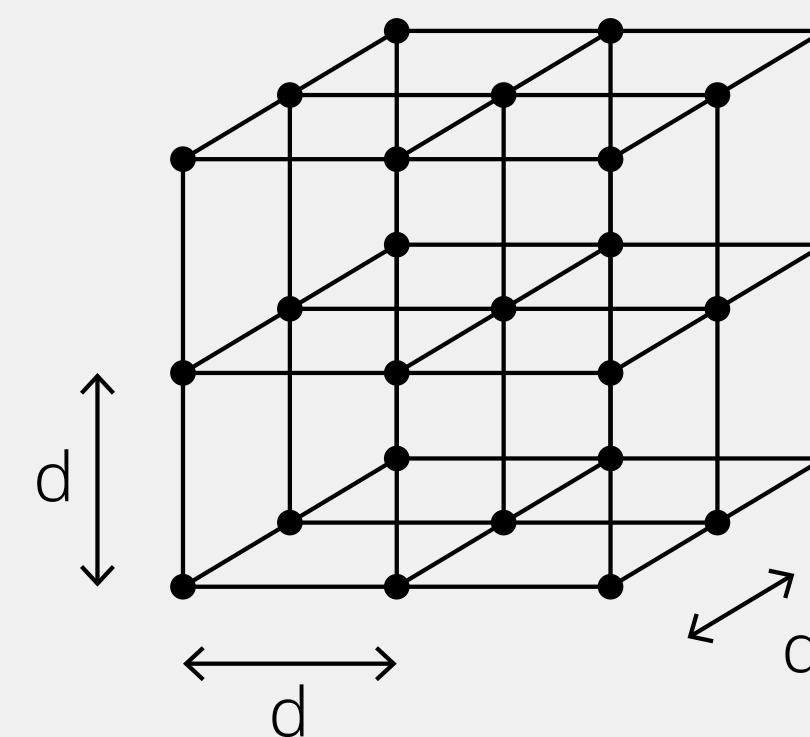
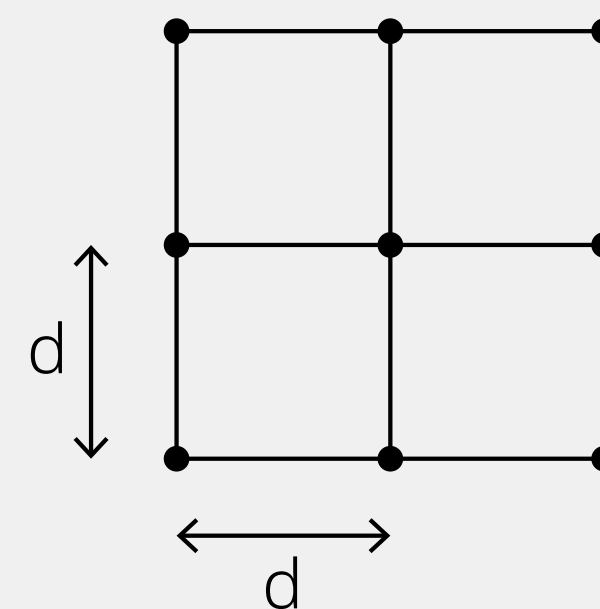
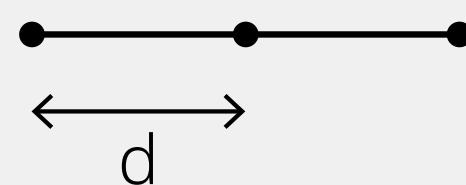
Maldición de la dimensionalidad

Describe la dificultad exponencial que uno encuentra al intentar tratar con funciones de dimensión de entrada creciente. La maldición de la dimensionalidad sigue siendo un problema incluso si decidimos tomar muestras de forma aleatoria.

El número de puntos de entrada que debemos muestrear de manera uniforme si deseamos que cada uno esté a una distancia de d de sus vecinos crece exponencialmente a medida que aumenta la dimensión de entrada de una función.

Si se utilizan tres puntos para cubrir un espacio de una sola entrada de esta manera (panel izquierdo), se requieren $3^2 = 9$ puntos en dos dimensiones, y $3^3 = 27$ puntos en tres dimensiones (y esta tendencia continúa).

El muestreo aleatorio (fila inferior) tampoco resuelve el problema.

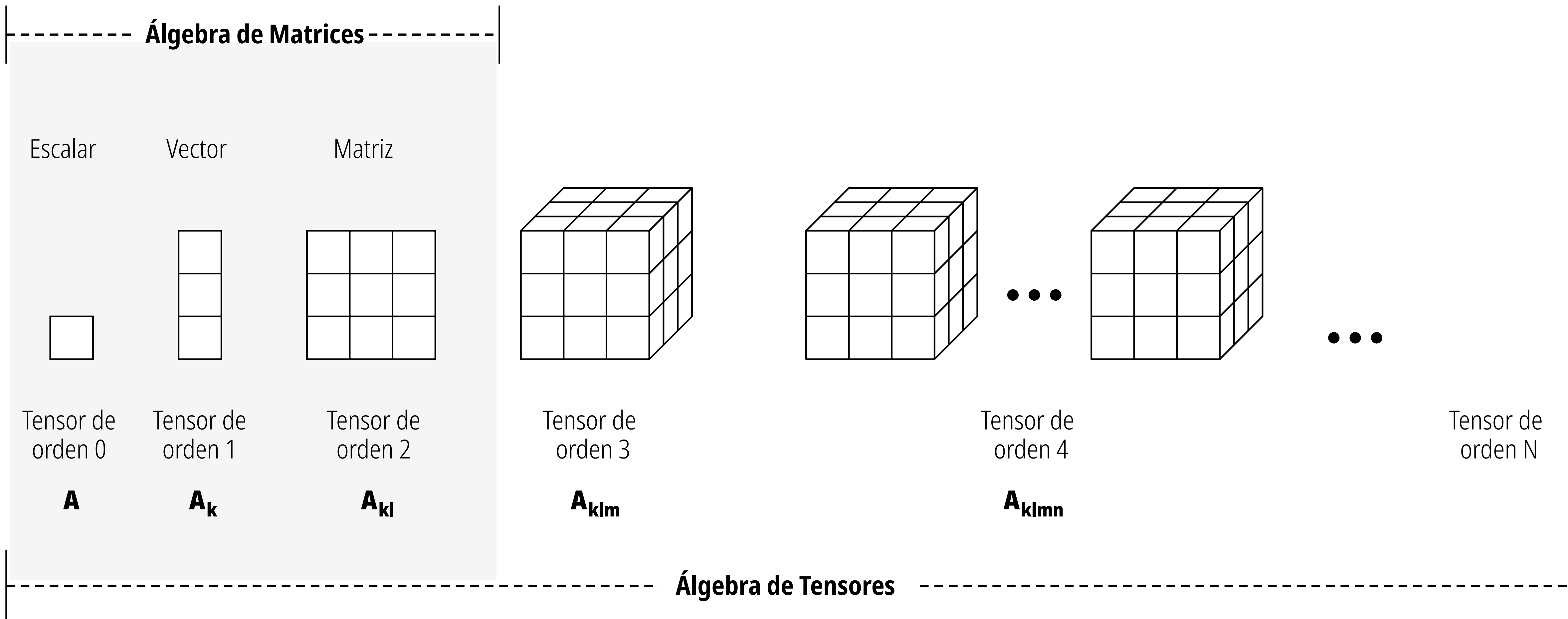


Regresión y Clasificación: Perceptron multicapa (MLP)

El perceptrón multicapa clásico, tal como fue introducido por Rumelhart, **Hinton** y Williams, se puede describir mediante:

- Una función lineal que agrega los valores de entrada.
- Una función sigmoide, también conocida como función de activación.
- Una función de umbral para el proceso de clasificación, y una función identidad para problemas de regresión.
- Una función de pérdida o costo que calcula el error global de la red.
- Un procedimiento de aprendizaje para ajustar los pesos de la red, es decir, el llamado algoritmo de retropropagación.

Desde escalares a tensores



Desde escalares a tensores

Escalar

7

Vector

$$\begin{bmatrix} 7 \\ 4 \end{bmatrix} \quad 0 \quad \begin{bmatrix} 7 & 4 \end{bmatrix}$$

Matriz

$$\begin{bmatrix} 7 & 10 \\ 4 & 3 \\ 5 & 1 \end{bmatrix}$$

Tensor

$$\begin{bmatrix} \begin{bmatrix} 7 & 4 \end{bmatrix} & \begin{bmatrix} 0 & 1 \end{bmatrix} \\ \begin{bmatrix} 1 & 9 \end{bmatrix} & \begin{bmatrix} 2 & 3 \end{bmatrix} \\ \begin{bmatrix} 5 & 6 \end{bmatrix} & \begin{bmatrix} 8 & 8 \end{bmatrix} \end{bmatrix}$$

Vector

Un vector es una colección de números ordenados o escalares. Si estás familiarizado con el análisis de datos, un vector es como una columna o fila en un dataframe. Un vector es como un arreglo (array) o una lista.

Convencionalmente los vectores se representan como vectores columna

Minúscula en negrita indica un vector

Índice para el caso n

Todos los elementos de x hasta n

Espacio de coordenadas reales de n dimensiones

Pertenece a

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \dots \\ x_n \end{bmatrix} = \begin{pmatrix} x_n \end{pmatrix} \in \mathbb{R}^n$$

Matriz

Una matriz es una colección de vectores o listas de números. En el análisis de datos, esto es equivalente a un dataframe de 2 dimensiones. En programación es equivalente a un arreglo multidimensional (array) o una lista de listas.

Mayúscula indica una matriz $\longrightarrow$

Convencionalmente el primer índice representa filas y el segundo índice representa columna

n columnas $\downarrow$

$$\mathbf{W} = \begin{bmatrix} W_{11} & W_{12} & \dots & W_{1n} \\ W_{21} & W_{22} & \dots & W_{2n} \\ \dots & \dots & \dots & \dots \\ W_{m1} & W_{m2} & \dots & W_{mn} \end{bmatrix}$$

m filas $\uparrow$

Espacio de coordenadas reales de $m \times n$ dimensiones $\downarrow$

$$= (W_{mn}) \in \mathbb{R}^{m \times n}$$

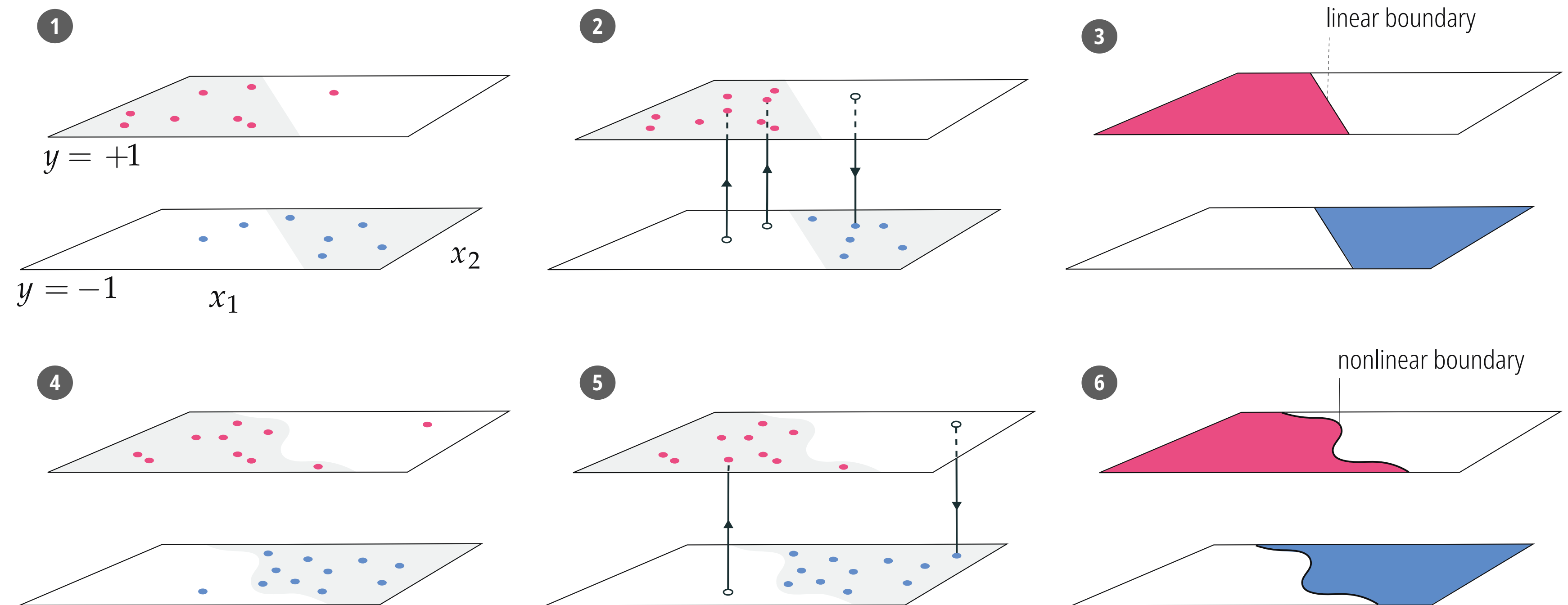
Todos los elementos de W hasta mn $\uparrow$

Teoría: Aproximadores Universales

Históricamente, los ingenieros tenían que mirar los datos e inventar fórmulas matemáticas para transformar los datos y ayudar al modelo a trazar fronteras no lineales (por ejemplo, elevar al cuadrado una variable). Esto es casi imposible en problemas del mundo real complejos.

En lugar de que el humano adivine las transformaciones, se utiliza un algoritmo capaz de aprender cualquier forma matemática por sí mismo. Se basan en la teoría matemática de que, con suficientes componentes, ciertas arquitecturas pueden aproximar cualquier función continua (como la frontera no lineal roja/azul perfecta de la imagen anterior).

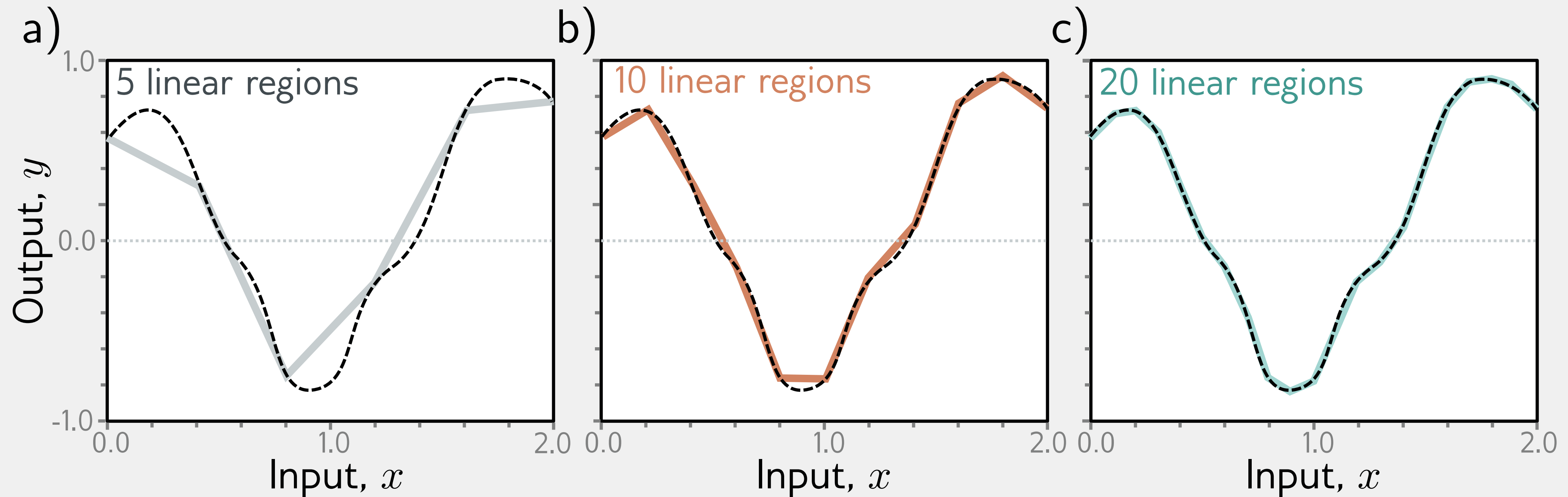
Aproximadores universales: Redes Neuronales (Neural Networks), los Árboles de Decisión (Trees) y los aproximadores de forma fija (como los kernels o funciones de base radial).



(1) Un conjunto de datos prototípico y realista de clasificación de dos clases es un conjunto ruidoso y (relativamente) pequeño de puntos que puede ser modelado aproximadamente por una función escalón con una frontera lineal. **(2)** Progresamos para eliminar todo el ruido de los datos devolviendo los valores reales de las etiquetas a nuestros puntos ruidosos. **(3)** El conjunto de datos de clasificación lineal de dos clases perfecto, donde tenemos infinitos puntos que yacen con precisión sobre una función escalón con una frontera lineal. **(4)** Un conjunto de datos prototípico y realista de clasificación no lineal de dos clases es un conjunto ruidoso y (relativamente) pequeño de puntos que puede ser modelado aproximadamente por una función escalón con una frontera no lineal. **(5)** Progresamos eliminando todo el ruido de los datos, creando un conjunto de datos sin ruido. **(6)** El conjunto de datos de clasificación no lineal de dos clases perfecto, donde tenemos infinitos puntos que yacen con precisión sobre una función escalón con una frontera no lineal.

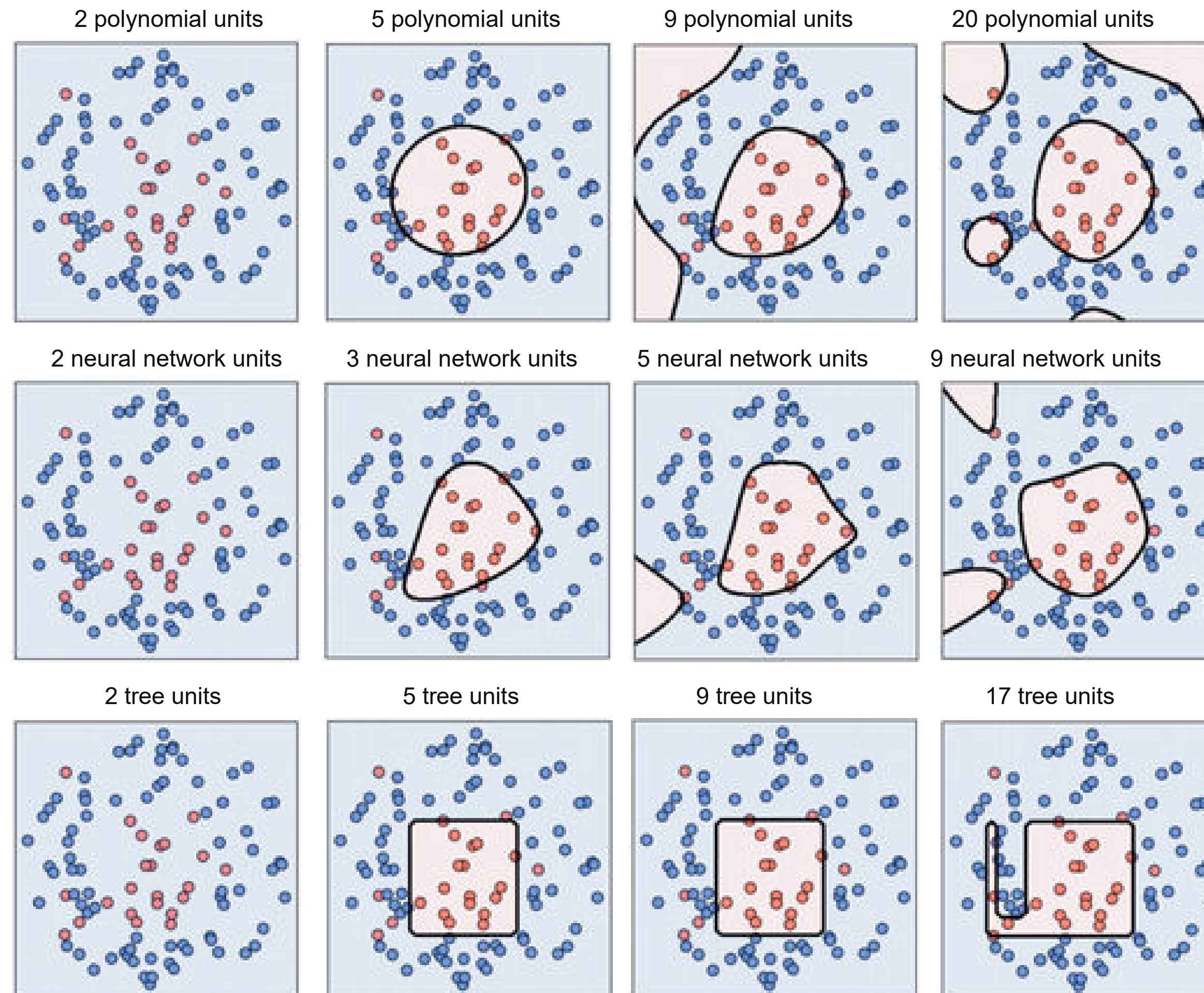
Teorema: Aproximación Universal

Con suficiente capacidad (unidades ocultas), una red superficial (Shallow Neural Network, SNN) puede describir cualquier función continua 1D definida en un subconjunto compacto de la recta real con precisión arbitraria.



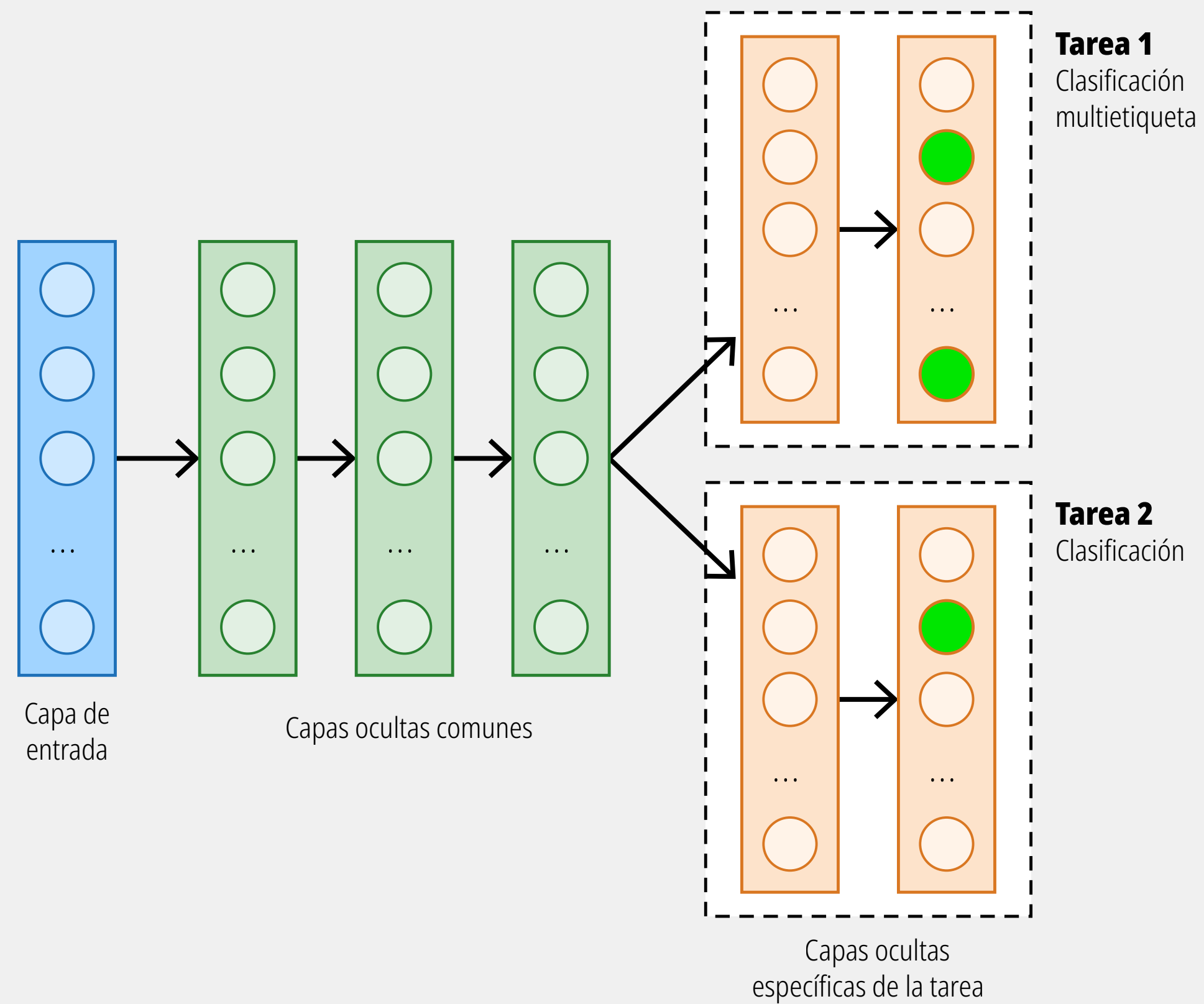
Para visualizar esto, considere que cada vez que agregamos una unidad oculta, añadimos otra región lineal a la función. A medida que estas regiones se vuelven más numerosas, representan secciones más pequeñas de la función, las cuales son cada vez mejor aproximadas por una recta. El teorema de aproximación universal demuestra que, para cualquier función continua, existe una red superficial que puede aproximar esta función a cualquier precisión especificada.

Teoría: Aproximadores Universales

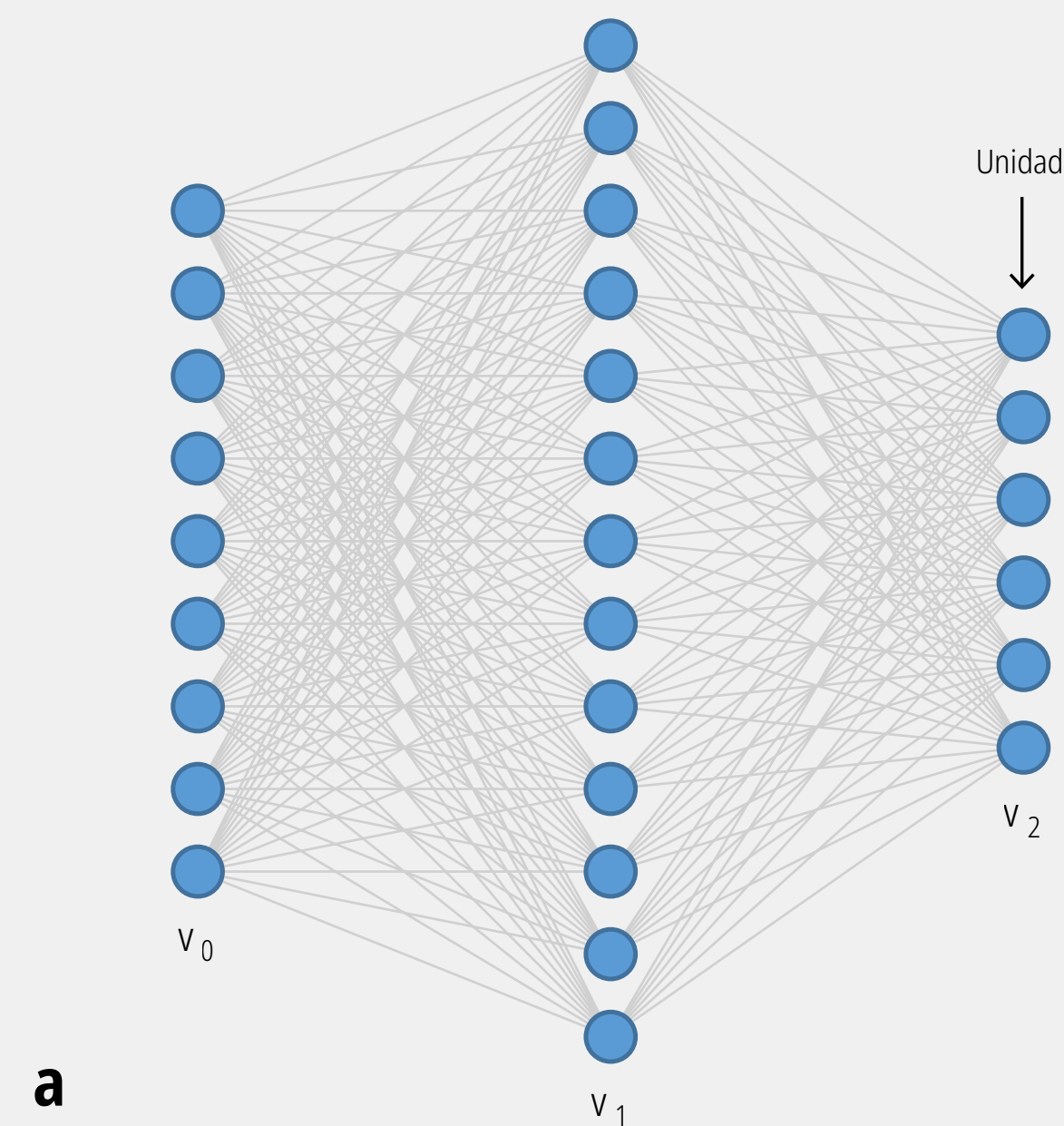


Notación: Redes neuronales

No existe una notación estándar.

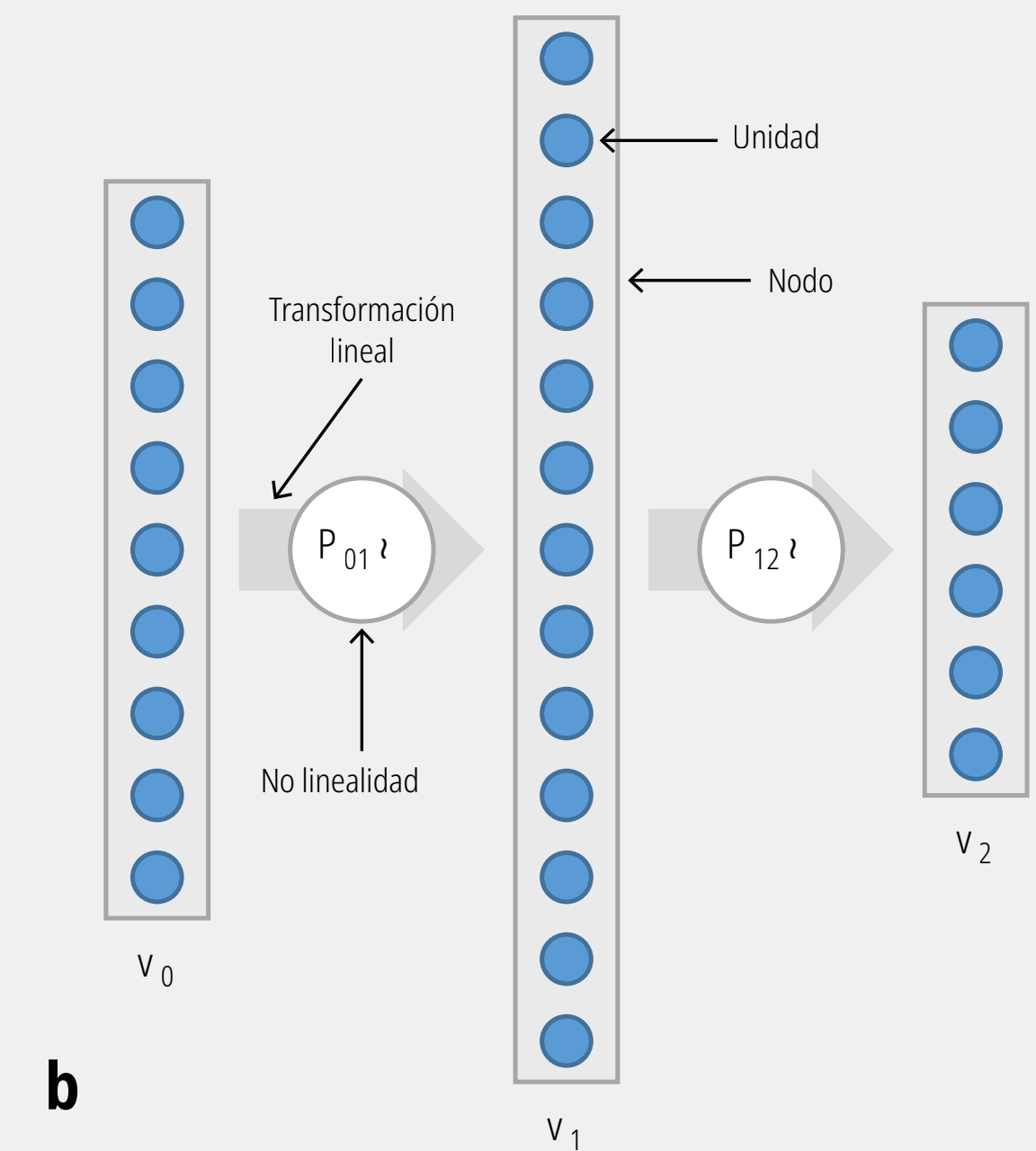


Notación antigua para un perceptrón de 2 capas



a

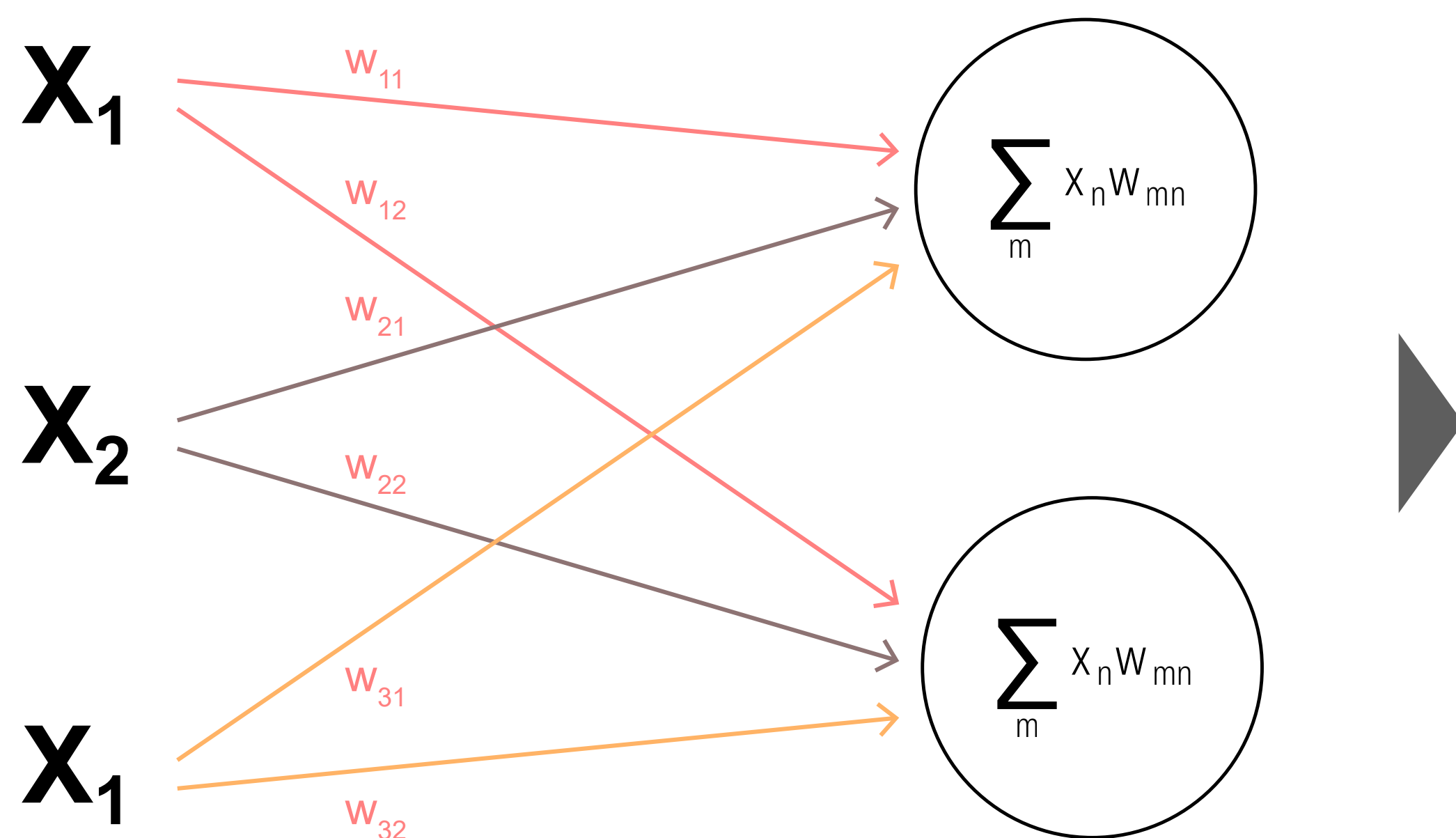
Nueva notación



b

Perceptrón multicapa

Una matriz es una colección de vectores o listas de números. En el análisis de datos, esto es equivalente a un dataframe de 2 dimensiones. En programación es equivalente a un arreglo multidimensional (array) o una lista de listas.



$$\mathbf{x} = \begin{bmatrix} X_1 \\ X_2 \\ X_3 \end{bmatrix}$$

El vector de entrada para el entrenamiento.

$$\mathbf{W} = \begin{bmatrix} W_{11} & W_{12} \\ W_{21} & W_{22} \\ W_{31} & W_{32} \end{bmatrix}$$

Dado que tenemos 3 unidades de entrada conectadas a 2 unidades ocultas, tenemos 3x2 pesos.

$$\mathbf{z} = \mathbf{W}^T \times \mathbf{x} = \begin{bmatrix} W_{11} & W_{12} & W_{13} \\ W_{21} & W_{22} & W_{23} \end{bmatrix} \begin{bmatrix} X_1 \\ X_2 \\ X_3 \end{bmatrix} = \begin{bmatrix} Z_1 \\ Z_2 \end{bmatrix}$$

$$\mathbf{z} = \mathbf{W}^T \times \mathbf{x} = \begin{bmatrix} X_1 W_{11} + X_2 W_{12} + X_3 W_{13} \\ X_1 W_{21} + X_2 W_{22} + X_3 W_{23} \end{bmatrix} = \begin{bmatrix} Z_1 \\ Z_2 \end{bmatrix}$$

La salida de la función lineal es igual a la multiplicación del vector $\mathbf{x}$ y la matriz $\mathbf{W}$. Para realizar la multiplicación en este caso, necesitamos transponer la matriz $\mathbf{W}$ para hacer coincidir el número de columnas en $\mathbf{W}$ con el número de filas en $\mathbf{x}$. Transponer significa "voltear" las columnas de $\mathbf{W}$ de tal manera que la primera columna se convierta en la primera fila, la segunda columna se convierta en la segunda fila, y así sucesivamente.

Perceptrón multicapa

Salida de la función

Entradas de la función

Término de sesgo (bias)

Elemento n del vector de entradas

Elemento de la matriz de parámetros

Nombre de la función

Índice para cada neurona y fila de la matriz

$$z_m = \mathbf{f}(x_n, w_{mn}) = b + \sum_m x_n w_{mn}$$

The diagram illustrates the equation for the output of a perceptron layer. It includes labels for each part of the equation: 'Salida de la función' (Output of the function) points to z_m ; 'Entradas de la función' (Inputs of the function) points to the inputs of the activation function $\mathbf{f}$; 'Término de sesgo (bias)' (Bias term) points to b ; 'Elemento n del vector de entradas' (Element n of the input vector) points to x_n ; 'Elemento de la matriz de parámetros' (Element of the parameter matrix) points to w_{mn} ; 'Nombre de la función' (Name of the function) points to $\mathbf{f}$; and 'Índice para cada neurona y fila de la matriz' (Index for each neuron and row of the matrix) points to the summation index m .

Aquí, f es una función de cada elemento del vector x y de cada elemento de la matriz W . El índice m identifica las filas en W^T y las filas en z . El índice n indica las columnas en W^T y las filas en x . Nota que agregamos un término de sesgo b , que tiene la función de simplificar el aprendizaje de un umbral adecuado para la función. Si tienes curiosidad al respecto, lee la sección "Función de agregación lineal" aquí. En resumen, la función lineal es una suma ponderada de las entradas más un sesgo.

Función de activación: Sigmoide

Cada elemento del vector z se convierte en una entrada para la función sigmoide.

Salida de la función $\rightarrow a_m = \sigma(z_m) = \frac{1}{1 + e^{-z_m}}$

Salida de la función lineal se convierte en la entrada de sigmoide

Símbolo de función sigmoide (sigma)

Símbolo de número de Euler (~ 2.71828)

La salida de sigmoide es otro vector a de m dimensiones, una entrada por cada unidad en la capa oculta, como sigue:

$$\mathbf{a} = \begin{bmatrix} a_1 \\ a_2 \end{bmatrix}$$

Aquí, $\mathbf{a}$ significa "activación", que es una forma común de referirse a la salida de las unidades ocultas. Esta función sigmoide que "envuelve" el resultado de la función lineal se denomina comúnmente función de activación. La idea es que una unidad se "activa" más o menos de la misma manera que una neurona se activa cuando recibe una señal de entrada lo suficientemente fuerte. La elección de una sigmoide es arbitraria. Se podrían seleccionar muchas funciones no lineales diferentes en esta etapa de la red, como una **Tanh** o una **ReLU**. Desafortunadamente, no existe un método estricto y fundamentado para elegir las funciones de activación de las capas ocultas. Es principalmente una cuestión de ensayo y error.

Mini-Lab

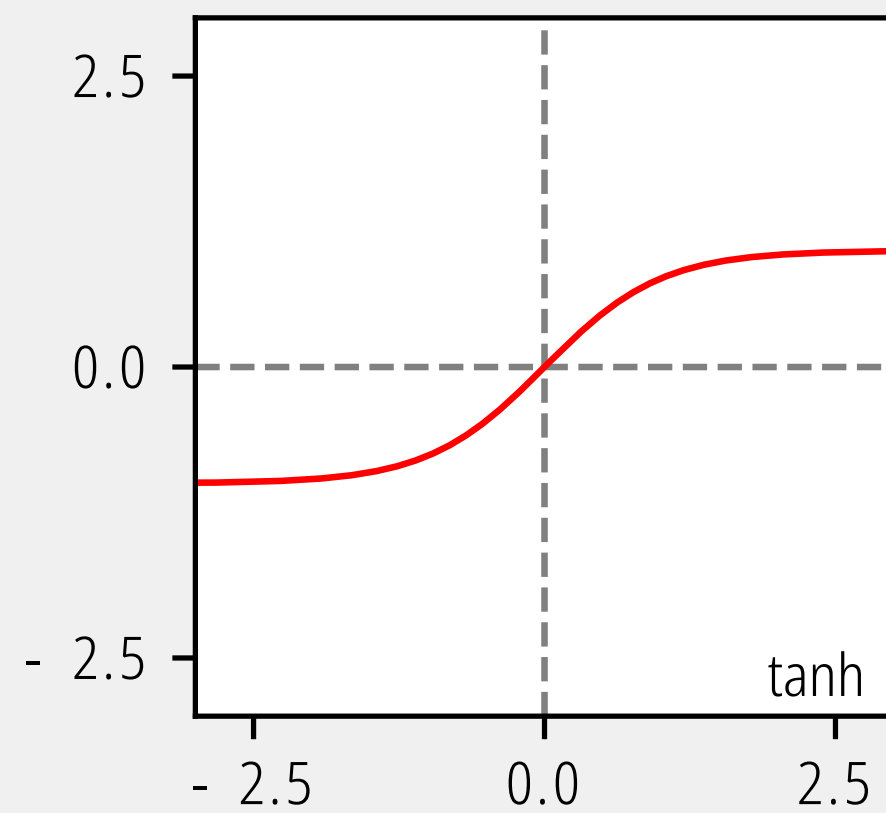
Veamos la función sigmoide.



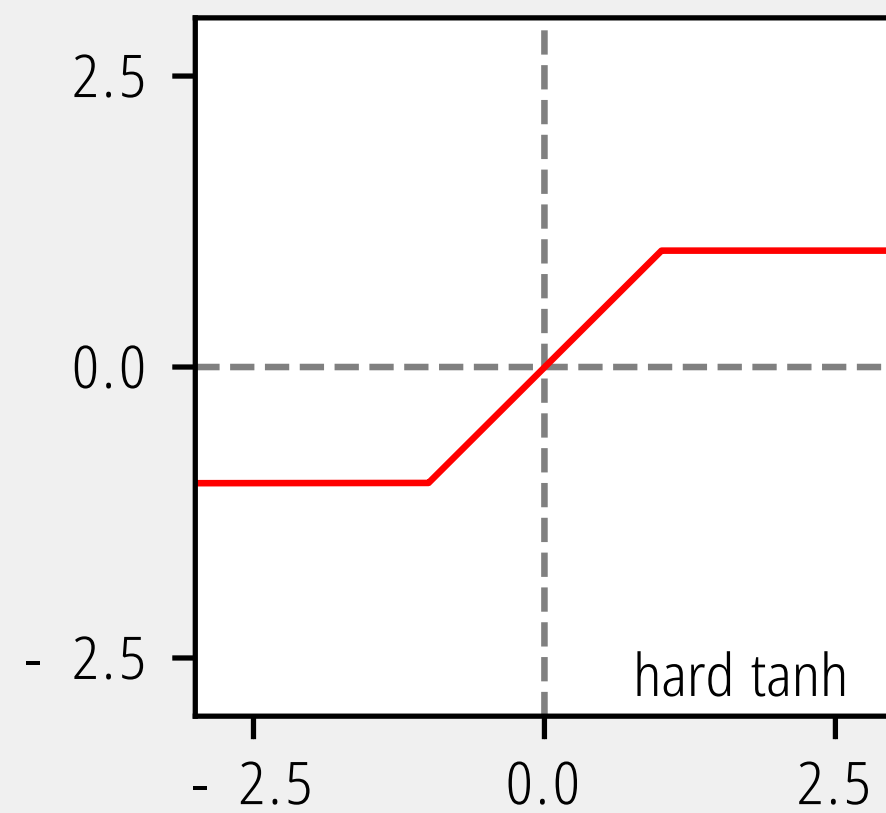
```
from scipy.special import expit
import numpy as np
import altair as alt
import pandas as pd
z = np.arange(-5.0, 5.0, 0.1)
a = expit(z)
df = pd.DataFrame({"a":a, "z":z})
df["z1"] = 0
df["a1"] = 0.5
sigmoid = alt.Chart(df).mark_line().encode(x="z", y="a")
threshold = alt.Chart(df).mark_rule(color="red").encode(x="z1", y="a1")
(sigmoid + threshold).properties(title='Chart 1')
```

Funciones de activación: Capas ocultas

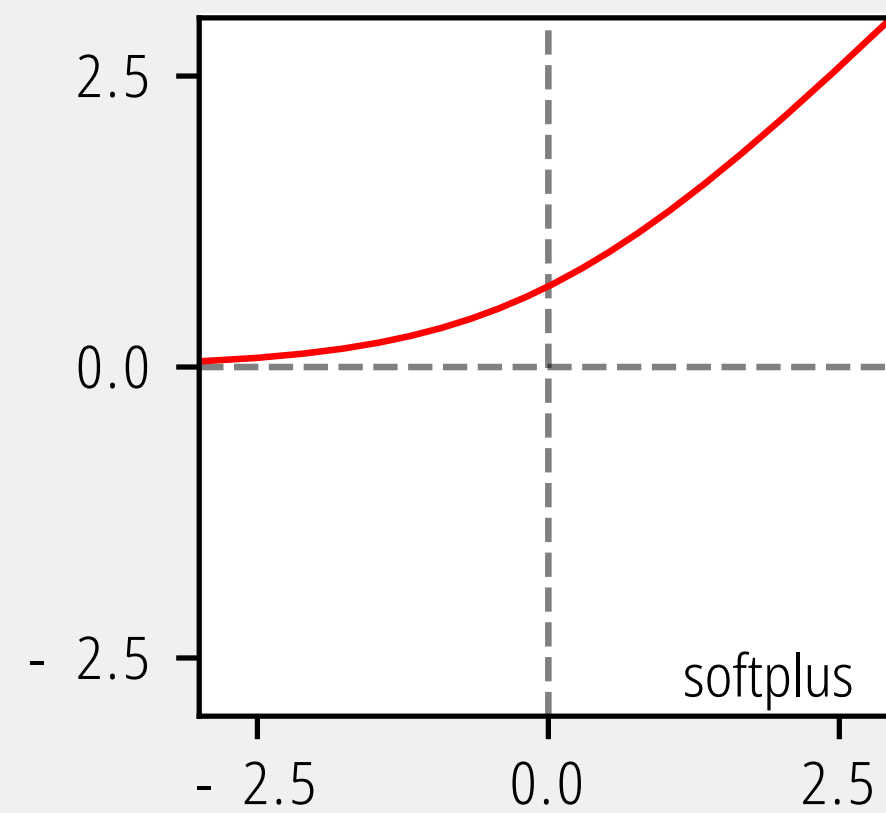
En general, las redes multicapa lineales tienen un uso limitado precisamente porque solo pueden calcular funciones lineales. Para resolver problemas más complejos, es necesario introducir funciones no lineales y diferenciables, como la sigmoide logística.



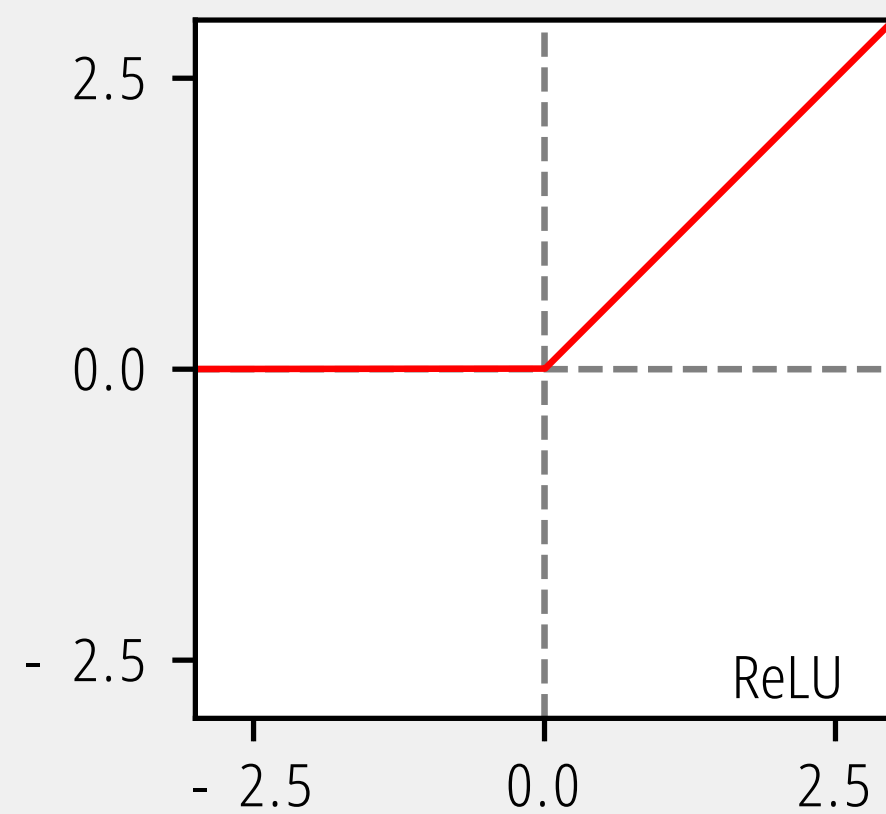
(a)



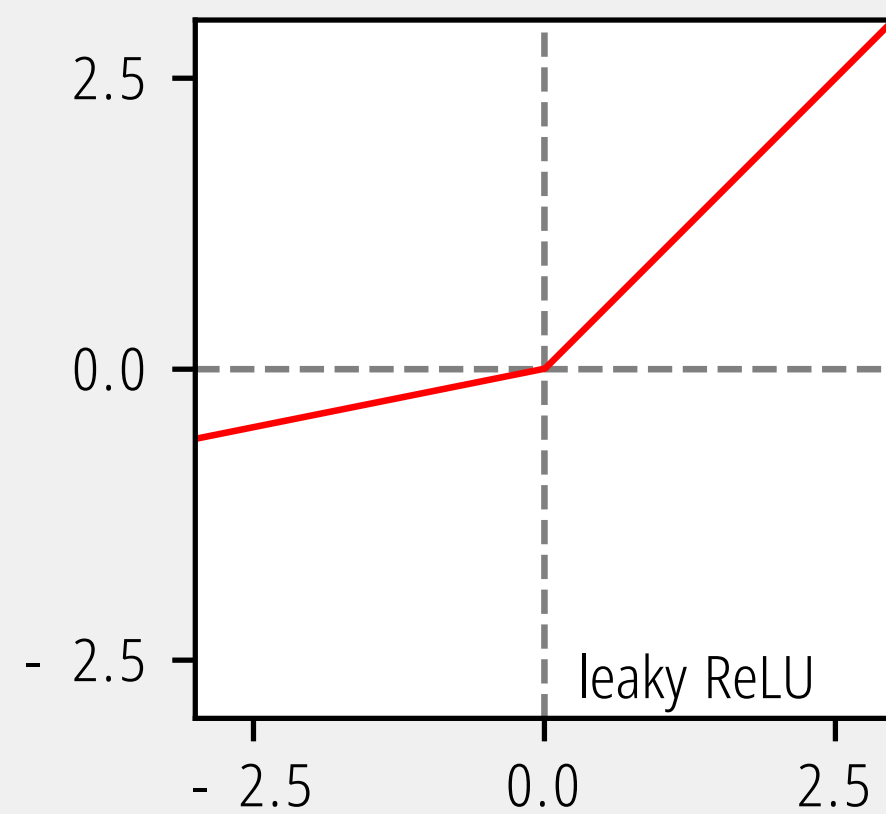
(b)



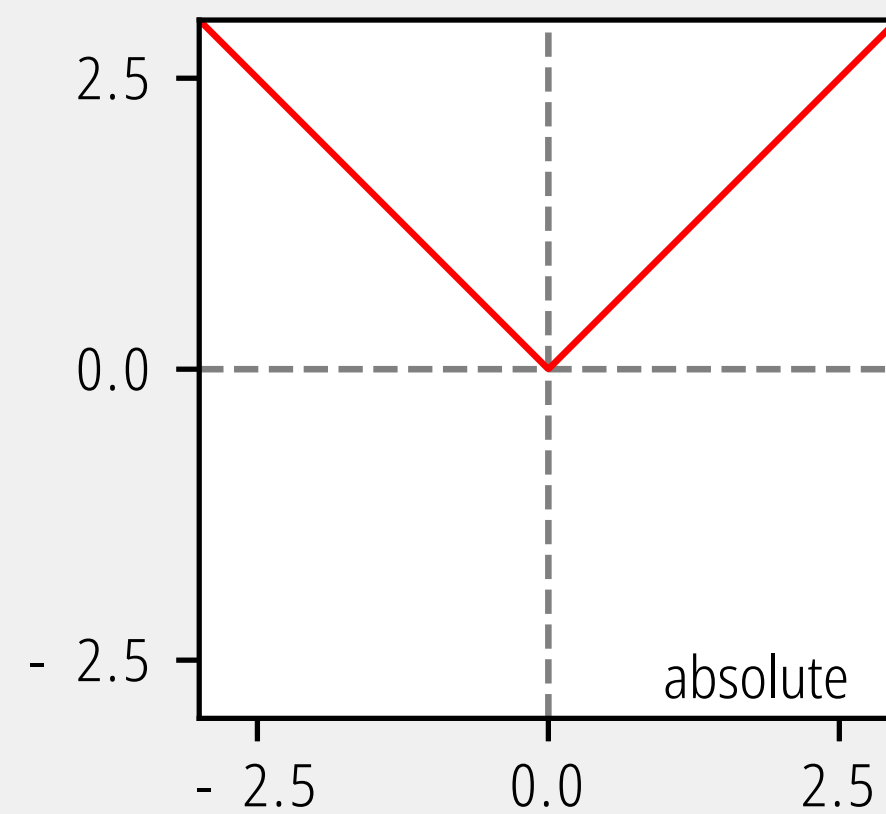
(c)



(d)



(e)



(f)

$$(a) \tanh(a) = \frac{e^a - e^{-a}}{e^a + e^{-a}}$$

$$(b) h(a) = \max(-1, \min(1, a))$$

$$(c) h(a) = \ln(1 + \exp(a))$$

$$(d) h(a) = \max(0, a)$$

$$(e) h(a) = \max(0, a) + \alpha \min(0, a)$$

$$(f) h(a) = |a|$$

Usar una función de activación lineal (identidad) en las unidades ocultas hace que la red neuronal sea matemáticamente equivalente a una red sin capas ocultas. Esto se debe a que la composición de múltiples transformaciones lineales siempre da como resultado otra transformación lineal. La única excepción ocurre cuando la capa oculta actúa como un "cuello de botella" (tiene menos unidades, M , que el número de entradas, N , o salidas, K). En este caso, la red no puede representar una transformación lineal general debido a la pérdida de información. Esta arquitectura lineal con cuello de botella realiza la misma función que el Análisis de Componentes Principales (PCA), una técnica estándar de análisis de datos.

Función de salida

Para los problemas de regresión (problemas que requieren un valor de salida real, como la predicción de ingresos o calificaciones de exámenes), cada unidad de salida implementa una función identidad como:

$$\hat{\mathbf{y}} = \mathbf{f}(a_m) = a_m$$

En términos simples, una función identidad devuelve el mismo valor que la entrada. No hace nada más. El punto es que la variable a ya es la salida de una función lineal, por lo tanto, es el valor que necesitamos para este tipo de problema.

$$\hat{\mathbf{y}} = a_m = \sigma(z_m) = \frac{1}{1 + e^{-z_m}}$$

Para problemas de clasificación binaria, cada unidad de salida implementa una función de umbral como:

$$\hat{\mathbf{y}} = \mathbf{f}(a_m) \begin{cases} +1 & \text{si } a > 0.5 \\ -1 & \text{de lo contrario} \end{cases}$$

Para los problemas de clasificación multiclase, podemos usar una función **softmax** como:

$$\hat{\mathbf{y}} = \sigma(\mathbf{a})_i = \frac{e^{\beta a_i}}{\sum_{j=1}^k e^{\beta z_j}}$$

Pérdida

La función de costo es la medida de la “bondad” o “maldad” (dependiendo de cómo te guste ver las cosas) del rendimiento de la red. Este puede ser un término confuso. La gente a veces la llama función objetivo, función de pérdida o función de error.

- **Función de pérdida (Loss function):** Generalmente se refiere a la medida del error para un solo caso de entrenamiento.
- **Función de costo (Cost function):** Se refiere al error agregado para todo el conjunto de datos.
- **Función objetivo (Objective function):** Es un término más genérico que se refiere a cualquier medida del error global en una red.

En su trabajo original, Rumelhart, Hinton y Williams utilizaron la suma de errores al cuadrado definida como:

$$E = \frac{1}{2} \sum_k (a_k - y_k)^2$$

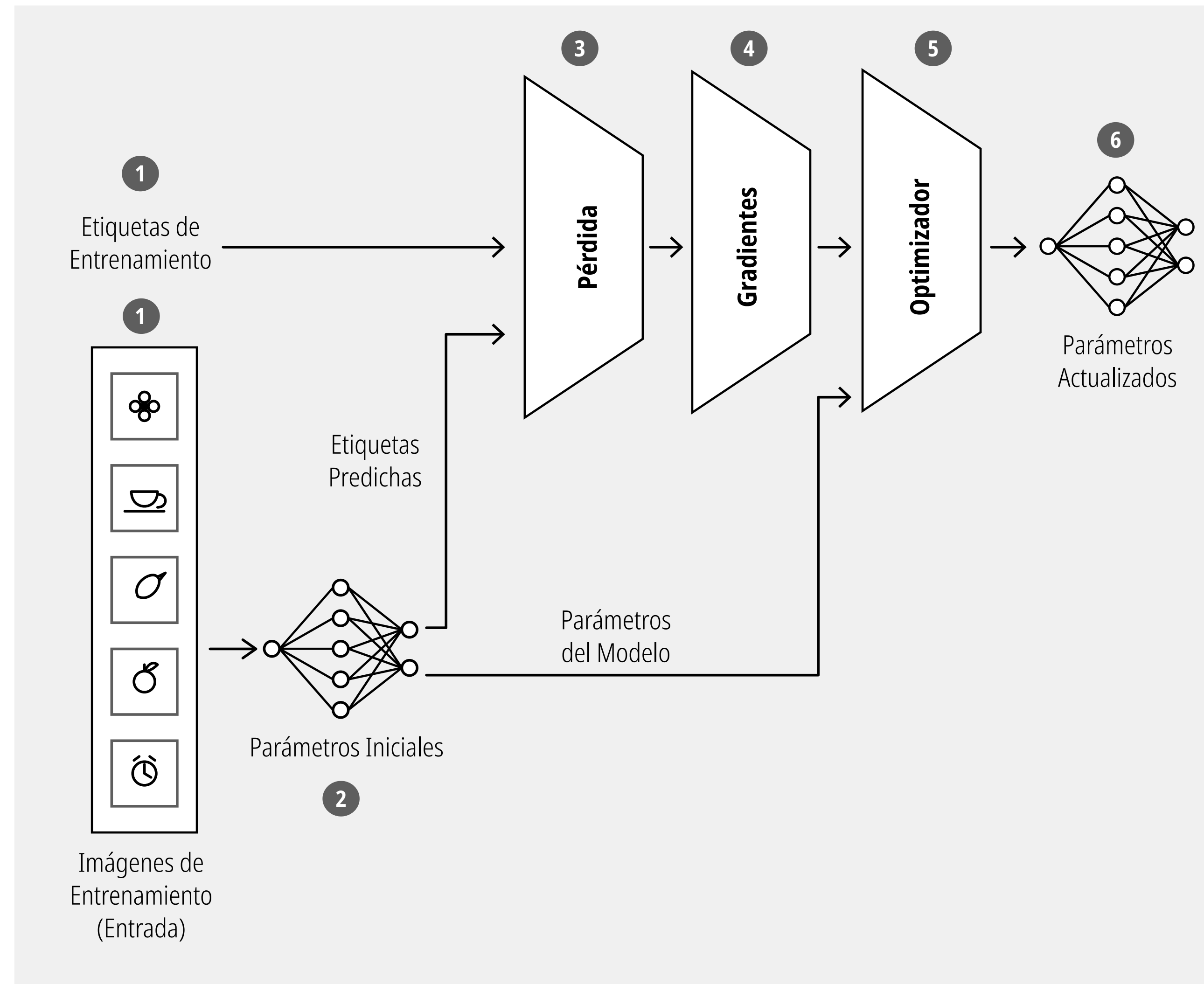
Diagrama de la fórmula:

- Suma de errores al cuadrado para todo el conjunto de datos (apunta al símbolo $\sum$)
- Añadido para simplificar el cálculo del gradiente (apunta al $\frac{1}{2}$)
- Valor predicho para el ejemplo de entrenamiento k (apunta a a_k)
- Índice para el par entrada / salida (apunta a k)
- Valor esperado para el ejemplo de entrenamiento k (ground truth) (apunta a y_k)

Por ejemplo, el “error cuadrático medio”, la “suma de errores al cuadrado” y la “entropía cruzada binaria” son todas funciones objetivo. Para nuestros propósitos, usaré todos estos términos indistintamente: todos se refieren a la medida del rendimiento de la red. Hoy en día, probablemente desearías usar diferentes funciones de costo para diferentes tipos de problemas.

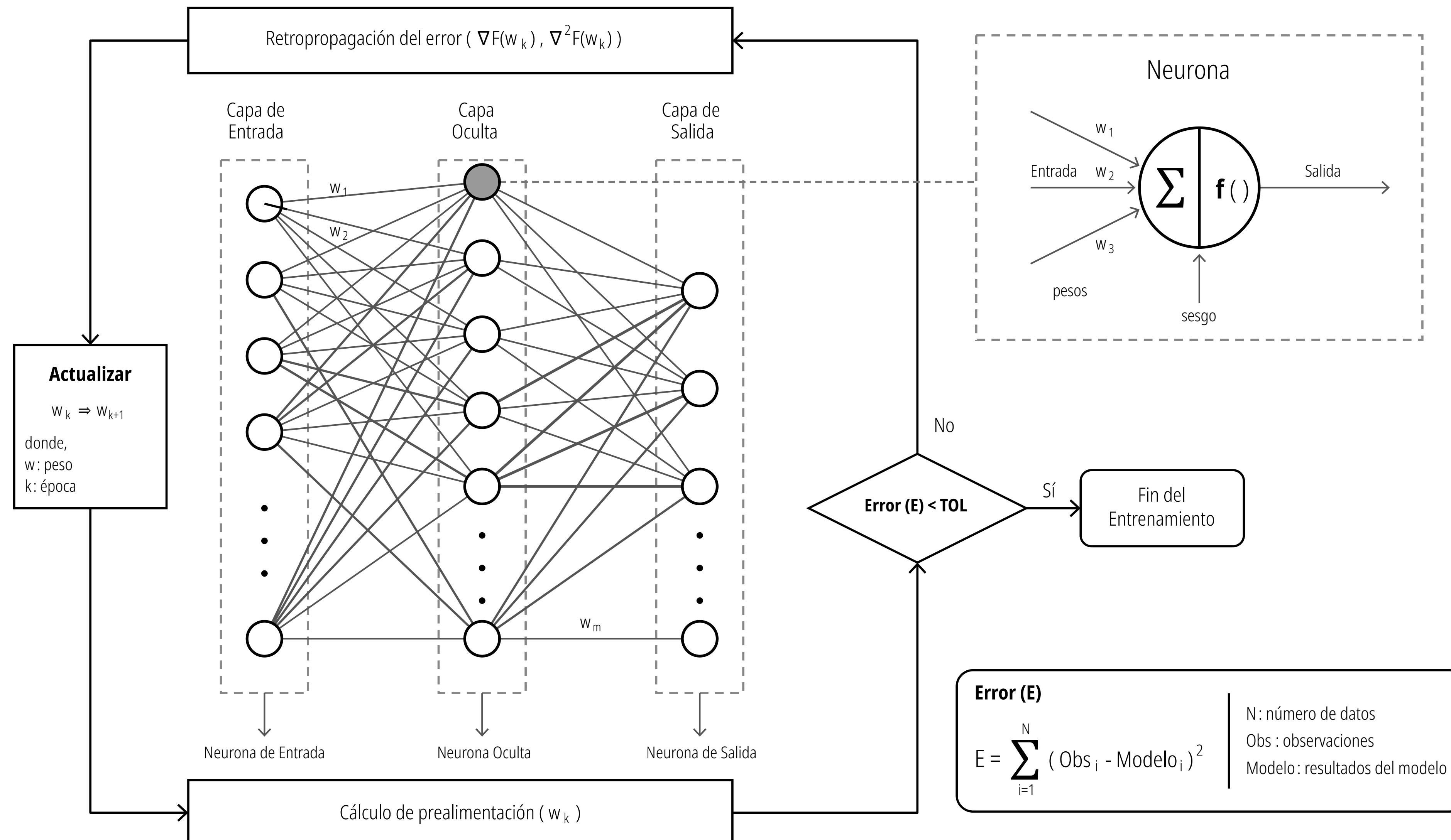
Redes Neuronales Artificiales: Componentes

Un ciclo se repite miles o millones de veces con diferentes lotes de datos, reduciendo el error progresivamente hasta que el modelo alcanza un nivel de precisión aceptable.



- 1 Datos (entradas y etiquetas).** El proceso comienza a la izquierda con las Imágenes de Entrenamiento (los datos reales, como las fotos del dron, la taza, etc.) y las Etiquetas de Entrenamiento (la respuesta correcta que indica qué es cada imagen).
- 2 Predicción (Parámetros Iniciales).** Las imágenes se introducen en la red neuronal, la cual opera con una configuración inicial de pesos matemáticos. Al procesar las imágenes, la red emite sus propias conclusiones, llamadas Etiquetas Predichas. A esta fase se le conoce como propagación hacia adelante (forward pass).
- 3 Función de Pérdida. Aquí se cruza la realidad con la predicción.** El sistema toma las Etiquetas Predichas (lo que la red creyó ver) y las compara con las Etiquetas de Entrenamiento (lo que realmente era). Esta diferencia matemática es la Pérdida (Loss), una métrica que define qué tan equivocado estuvo el modelo.
- 4 Cálculo de Gradientes.** Utilizando el valor de la pérdida, el sistema aplica una técnica llamada retropropagación (backpropagation). Aquí se calculan los Gradientes, que, recordando la figura anterior del valle en 3D, representan la dirección matemática y la inclinación exacta que indican cómo cada parámetro contribuyó al error.
- 5 Optimizador.** El algoritmo de optimización (como el Gradiente Descendente) entra en acción. Toma los Gradientes calculados en el paso 4 y los Parámetros del Modelo actuales. Su trabajo es calcular el ajuste necesario para mover los parámetros "cuesta abajo" hacia un menor margen de error.
- 6 Parámetros Actualizados.** El optimizador aplica los cambios, resultando en una nueva red neuronal con Parámetros Actualizados. El modelo ahora es ligeramente más preciso que en la ronda anterior.

Todas las redes neuronales se pueden dividir en dos partes: una fase de propagación hacia adelante, donde la información “fluye” hacia adelante para calcular las predicciones y el error; y la fase de propagación hacia atrás, donde el algoritmo de retropropagación calcula las derivadas del error y actualiza los pesos de la red.



Propagación hacia adelante (forward propagation)

Multiplicación de matrices

De la capa de entrada hacia la capa oculta

$$\begin{bmatrix} W_{11} & W_{12} & W_{13} \\ W_{21} & W_{22} & W_{23} \end{bmatrix}^T \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

Función lineal

$$z_m = b + \sum_m x_n W_{mn}$$

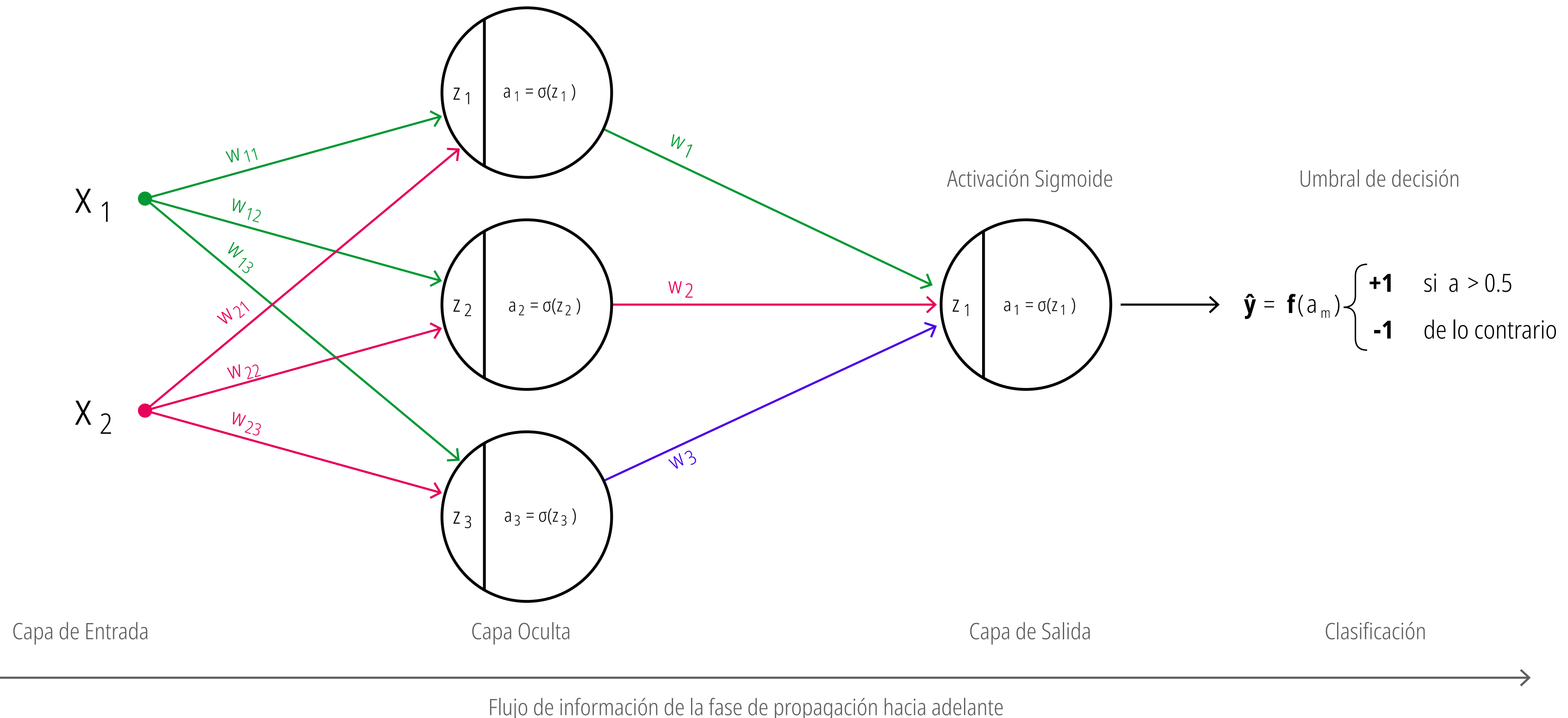
Activación sigmoide

$$a_m = \sigma(z_m) = \frac{1}{1 + e^{-z_m}}$$

Multiplicación de matrices

De la capa oculta hacia la capa de salida

$$\begin{bmatrix} w_1 & w_2 & w_3 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix}$$



Propagación hacia adelante (forward propagation)

La fase de propagación hacia adelante implica "encadenar" todos los pasos que hemos definido hasta ahora: la función lineal, la función sigmoide y la función de umbral.

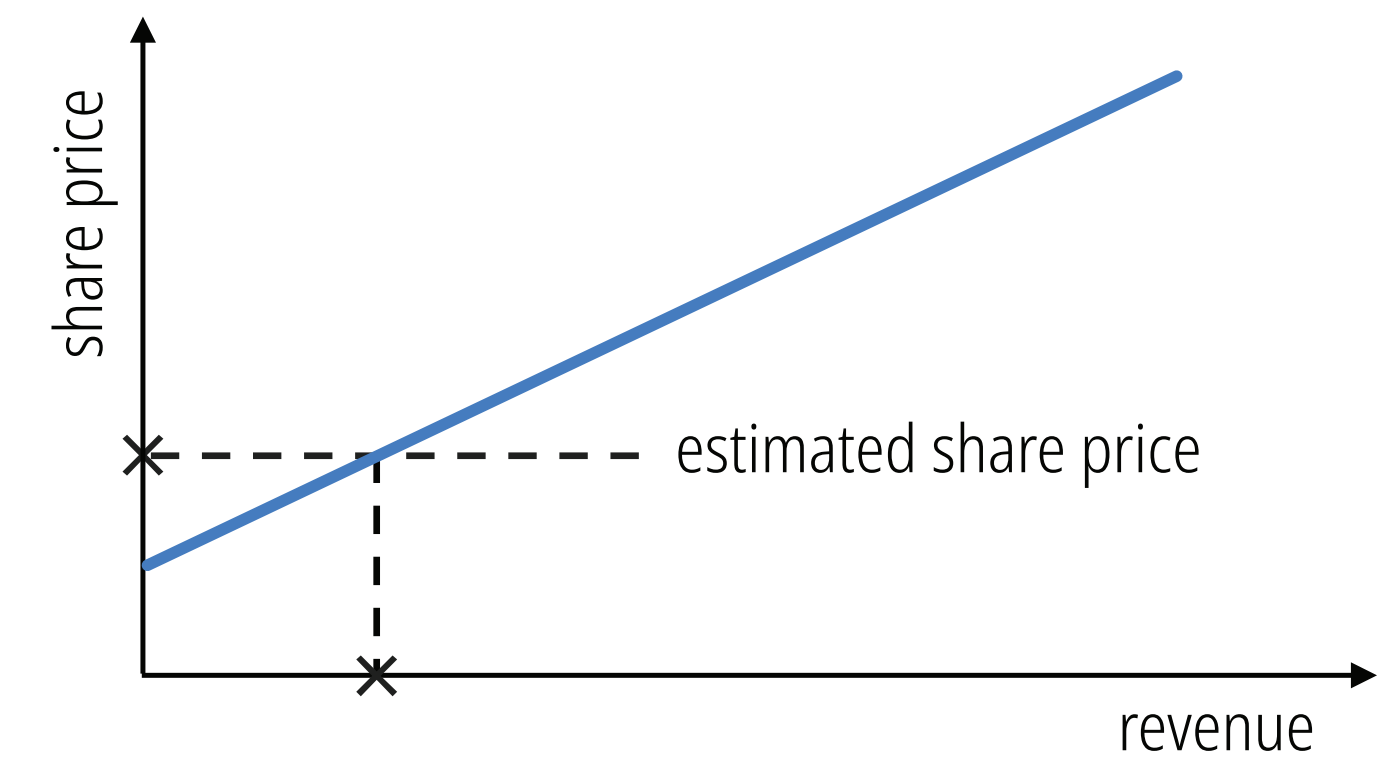
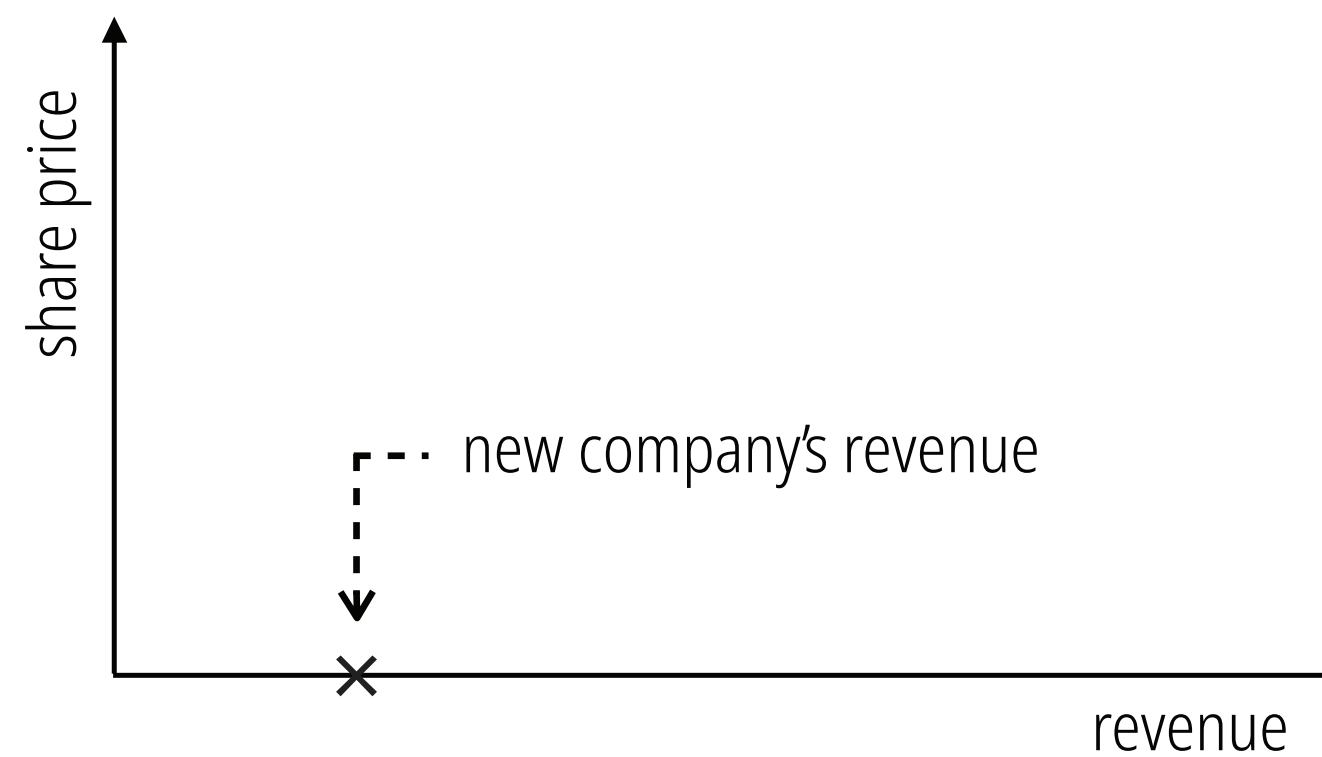
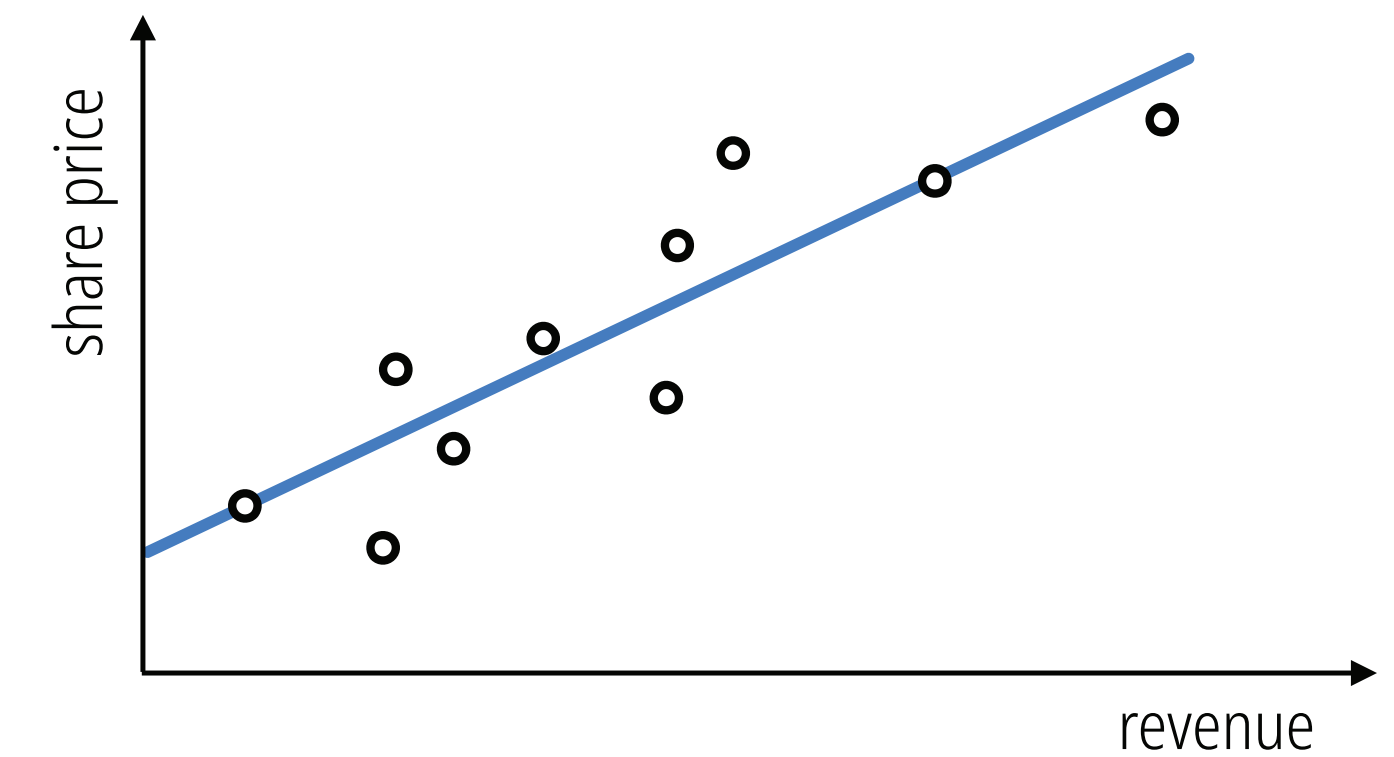
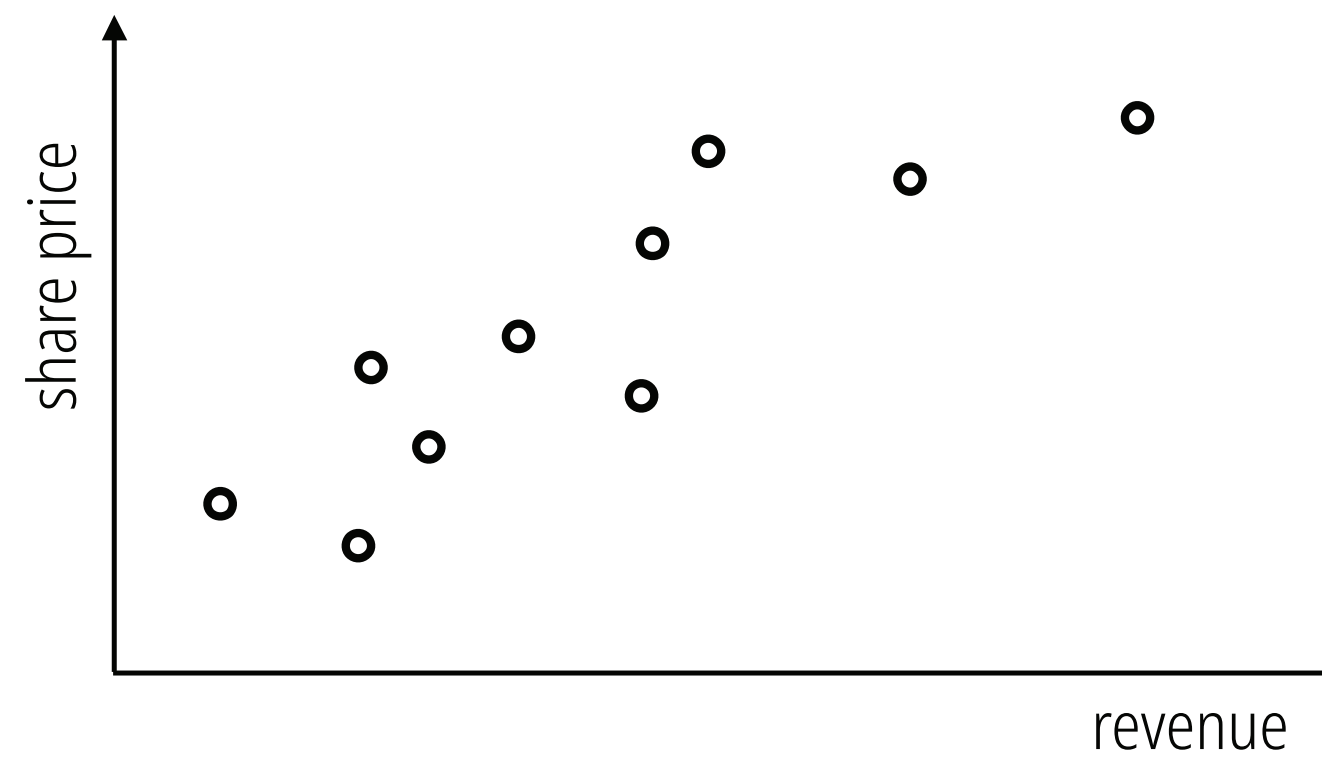
$$\hat{y} = \tau \left(\sigma^{(3)} \left(\lambda^{(3)} \left(\sigma^{(2)} \left(\lambda^{(2)} \left(\sigma^{(1)} \left(\lambda^{(1)} \left(x_n, W_{mn} \right) \right) \right) \right) \right) \right) \right)$$

Optimización: Mínimos y máximos de una función

Supongamos que quisiéramos predecir el precio de las acciones de una empresa que está a punto de salir a bolsa.

Reunimos un conjunto de datos de entrenamiento que consiste en varias corporaciones (preferiblemente activas en el mismo sector) con precios de acciones conocidos.

Necesitamos diseñar la(s) característica(s) que se consideren relevantes para la tarea en cuestión.



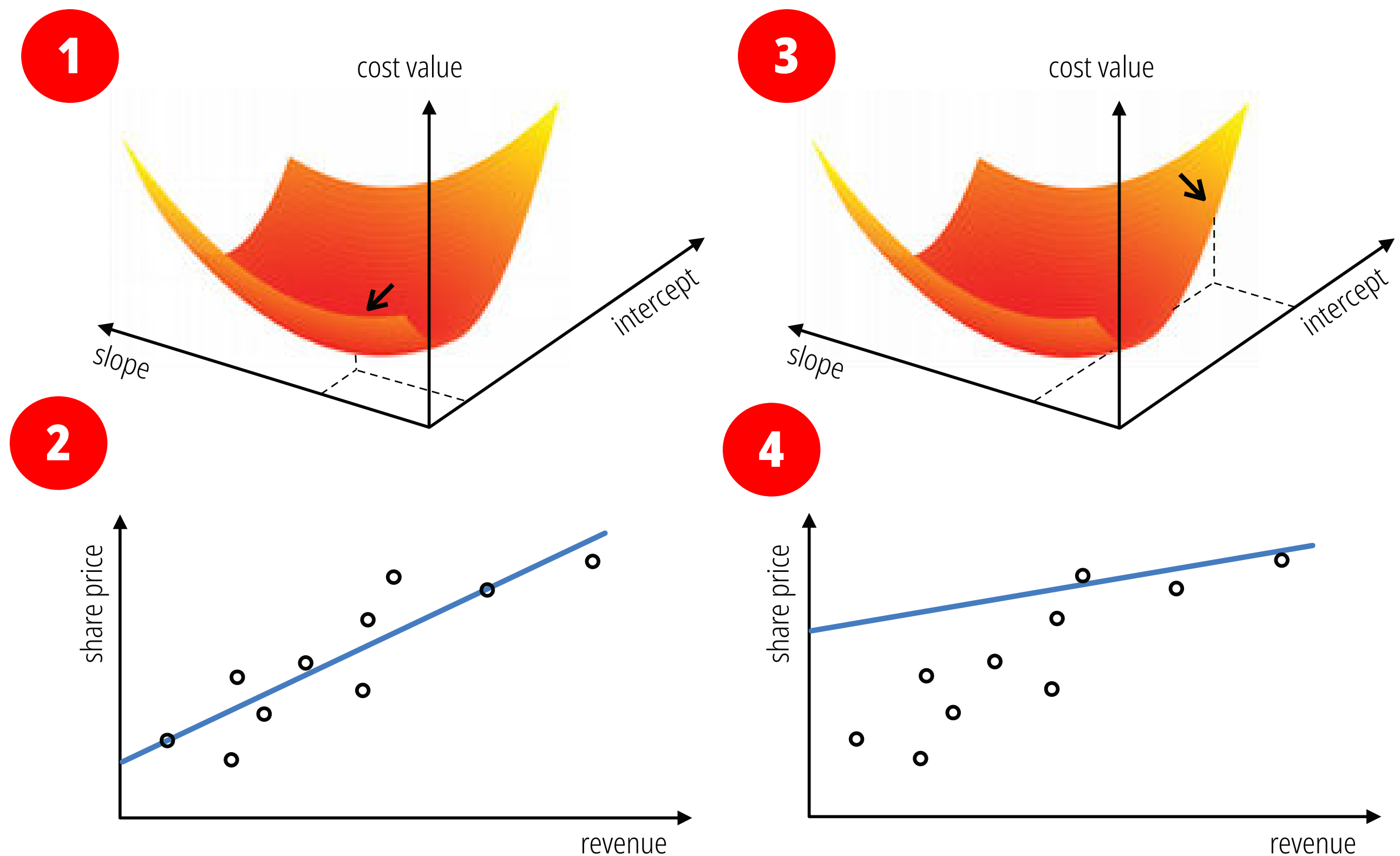
Los ingresos de la empresa son una de esas posibles características, ya que podemos esperar que cuanto mayores sean los ingresos, más cara debería ser una acción. Para conectar el precio de la acción (salida) con los ingresos (entrada), podemos entrenar un modelo lineal simple o una línea de regresión utilizando nuestros datos de entrenamiento.

Optimización: Mínimos y máximos de una función

La función de costo bidimensional asociada al aprendizaje de los parámetros de pendiente e intersección de un modelo lineal para el problema de regresión del precio de las acciones.

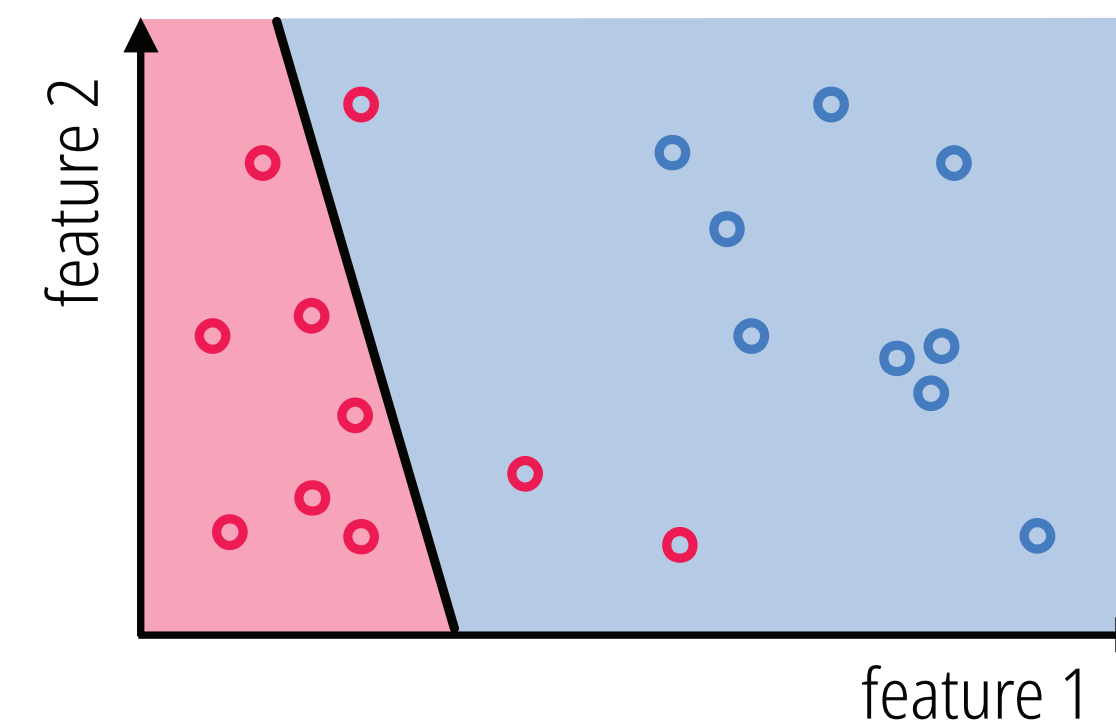
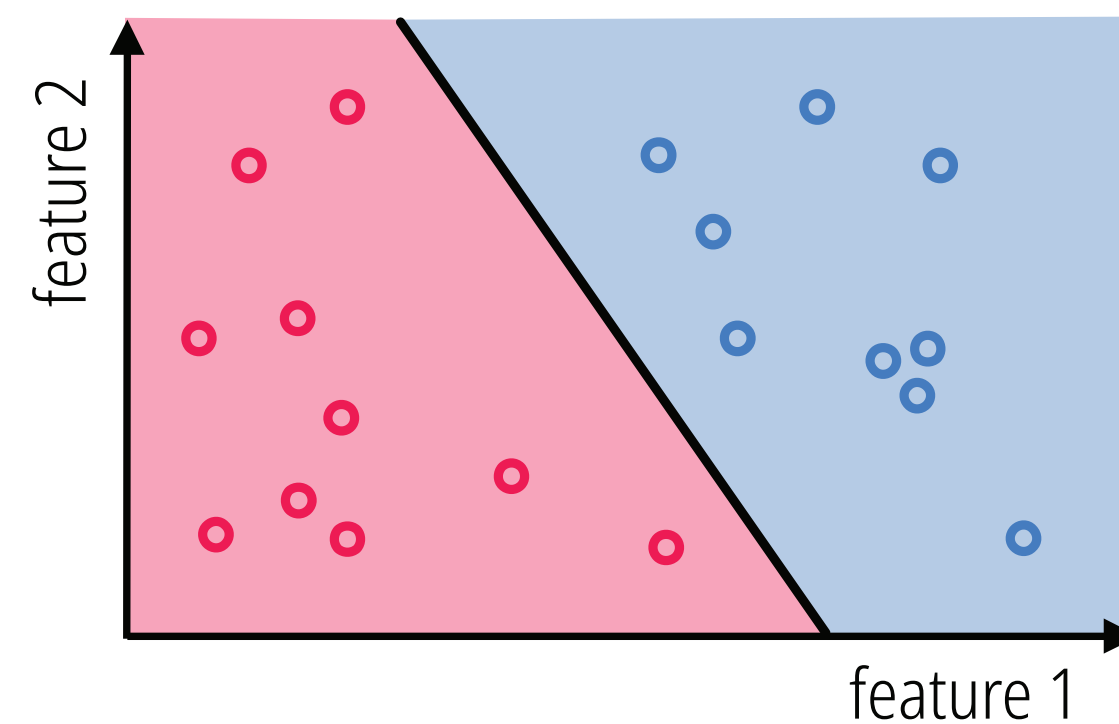
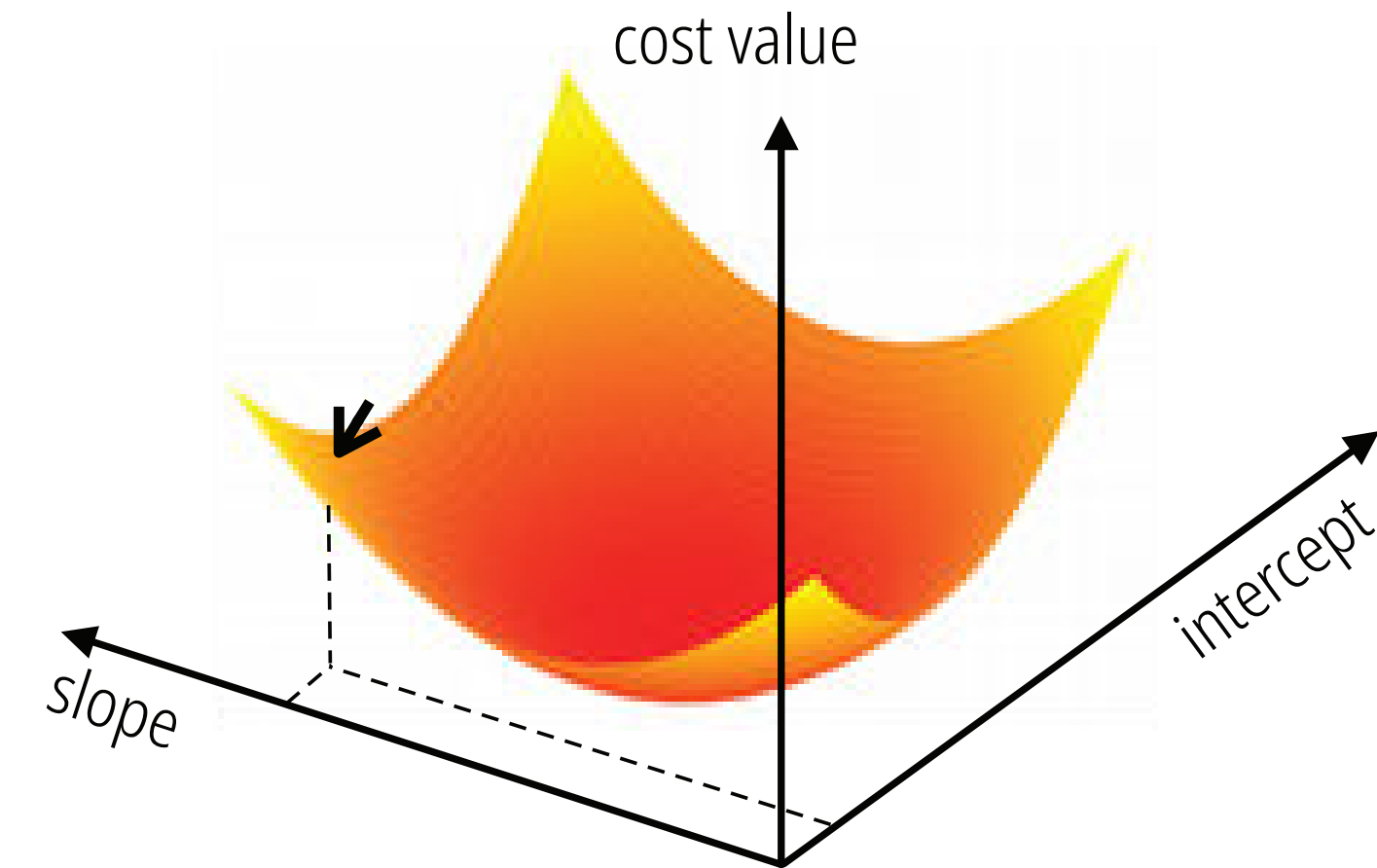
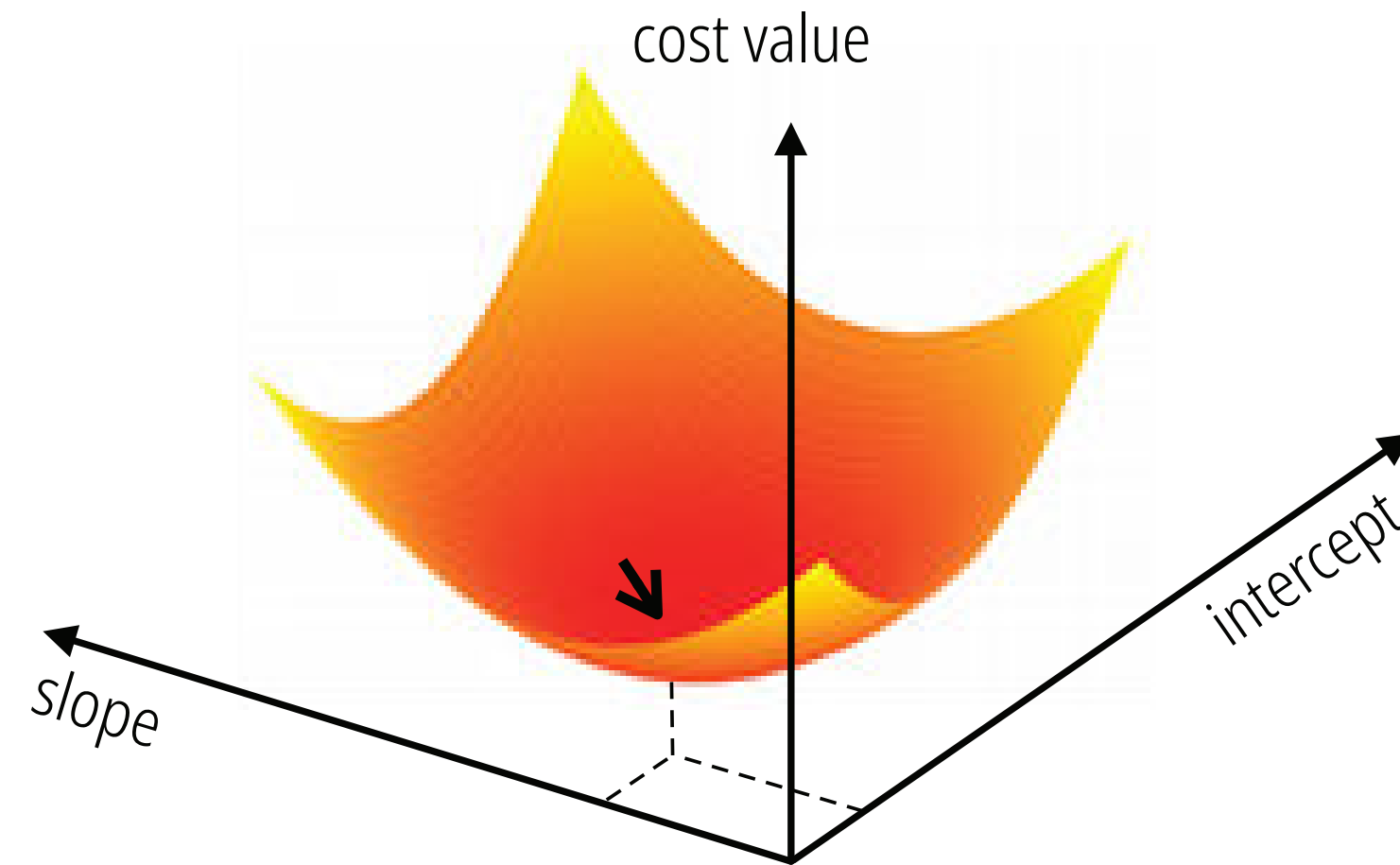
La función de costo bidimensional asociada al aprendizaje de los parámetros de pendiente e intersección de un modelo lineal para el problema de regresión del precio de las acciones.

También se muestran aquí dos conjuntos diferentes de valores de parámetros, uno (1,2) en el mínimo de la función de costo y el otro (3,4) en un punto con un valor mayor de la función de costo. (2,4) El modelo lineal correspondiente a cada conjunto de parámetros del panel superior. El conjunto de parámetros que da como resultado el mejor ajuste se encuentra en el mínimo de la superficie de costo.



Optimización: Mínimos y máximos de una función

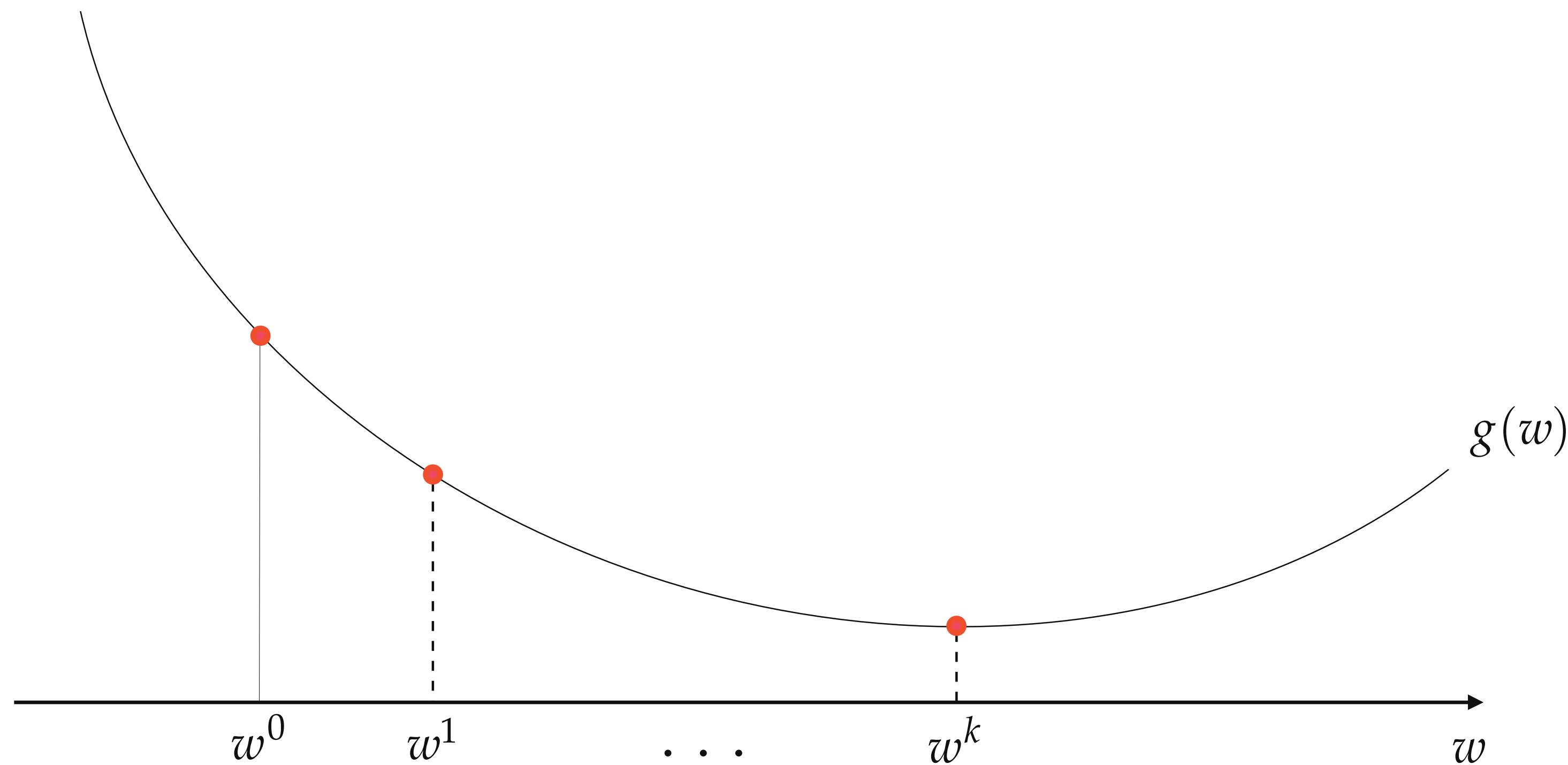
Un dibujo figurativo de la función de costo bidimensional asociada al aprendizaje de los parámetros de pendiente e intersección de un modelo lineal que separa dos clases.



Optimización: Mínimos y máximos de una función

Los métodos de optimización local funcionan minimizando una función objetivo en una secuencia de pasos (downhill).

$$g(\mathbf{w}^0) > g(\mathbf{w}^1) > \dots > g(\mathbf{w}^K).$$

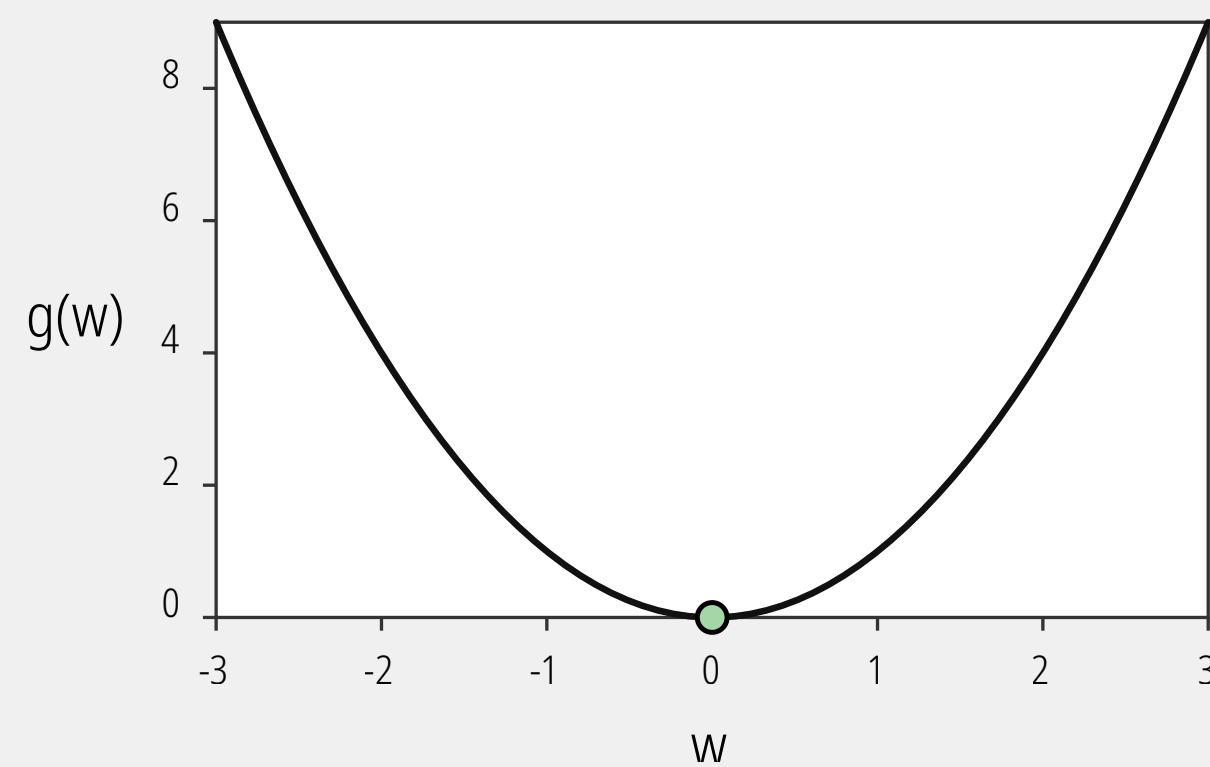


Aquí se muestra un método de optimización local genérico aplicado para minimizar una función de una sola entrada. Comenzando con el punto inicial $\mathbf{w}_0$, nos movemos hacia puntos más bajos en la función de costo como una pelota rodando cuesta abajo (downhill).

Optimización: Mínimos y máximos de una función

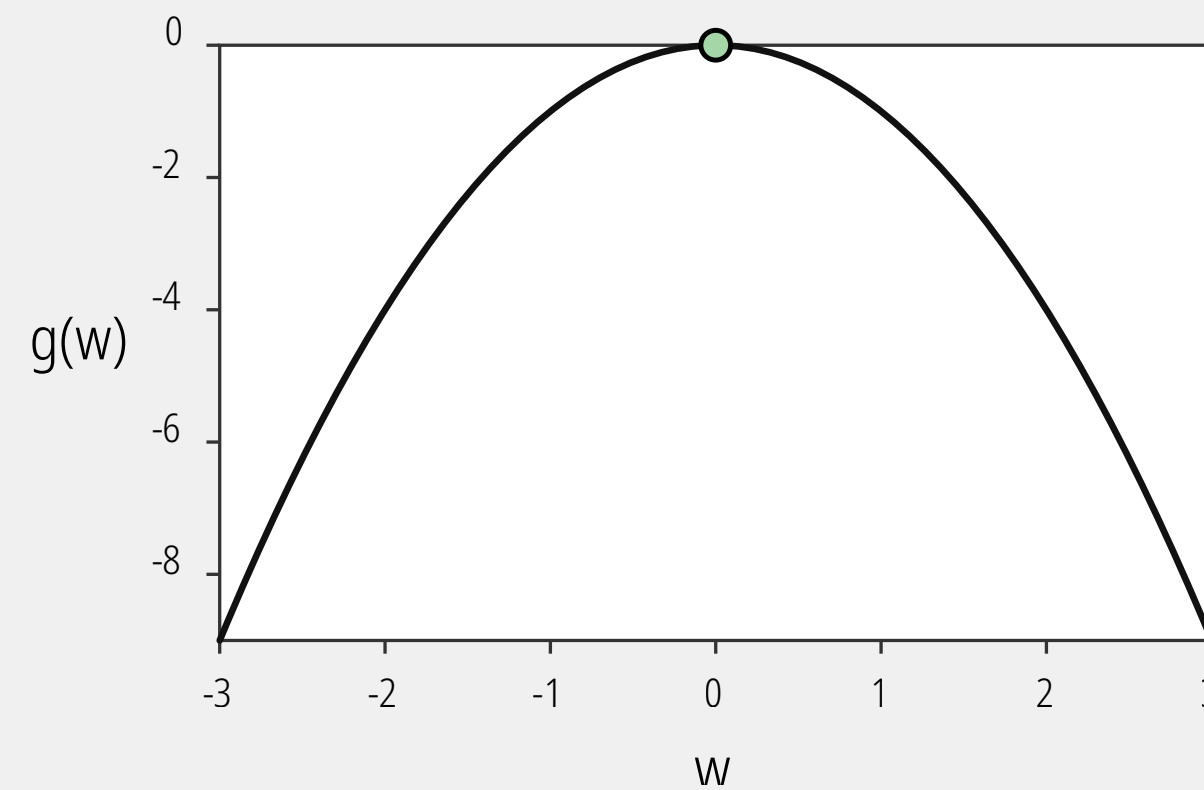
Conceptos fundamentales de la optimización unidimensional, clasificando los diferentes tipos de puntos críticos que se pueden encontrar al evaluar una función matemática $g(w)$.

1



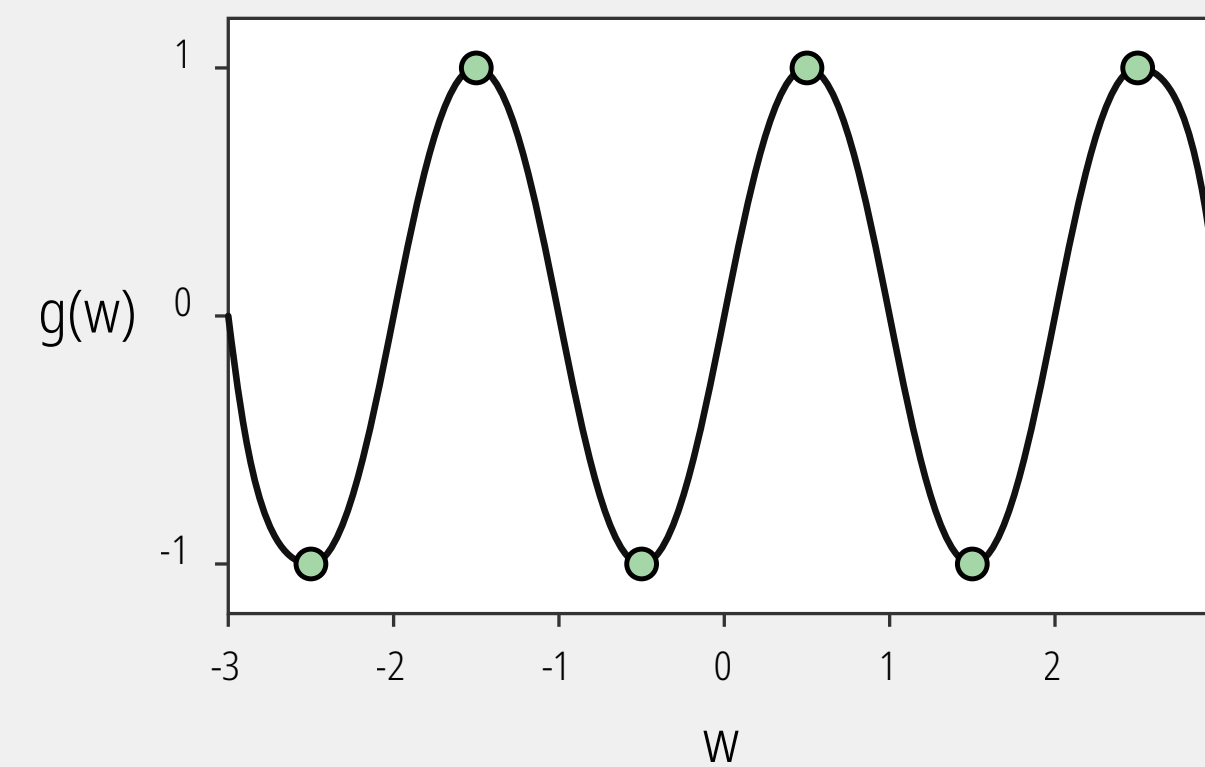
Mínimo global único: Muestra una función estrictamente convexa (forma de U) con un solo punto inferior. Es el escenario ideal al entrenar modelos, ya que cualquier algoritmo de optimización (como el descenso del gradiente) garantiza encontrar este fondo absoluto sin perderse.

2



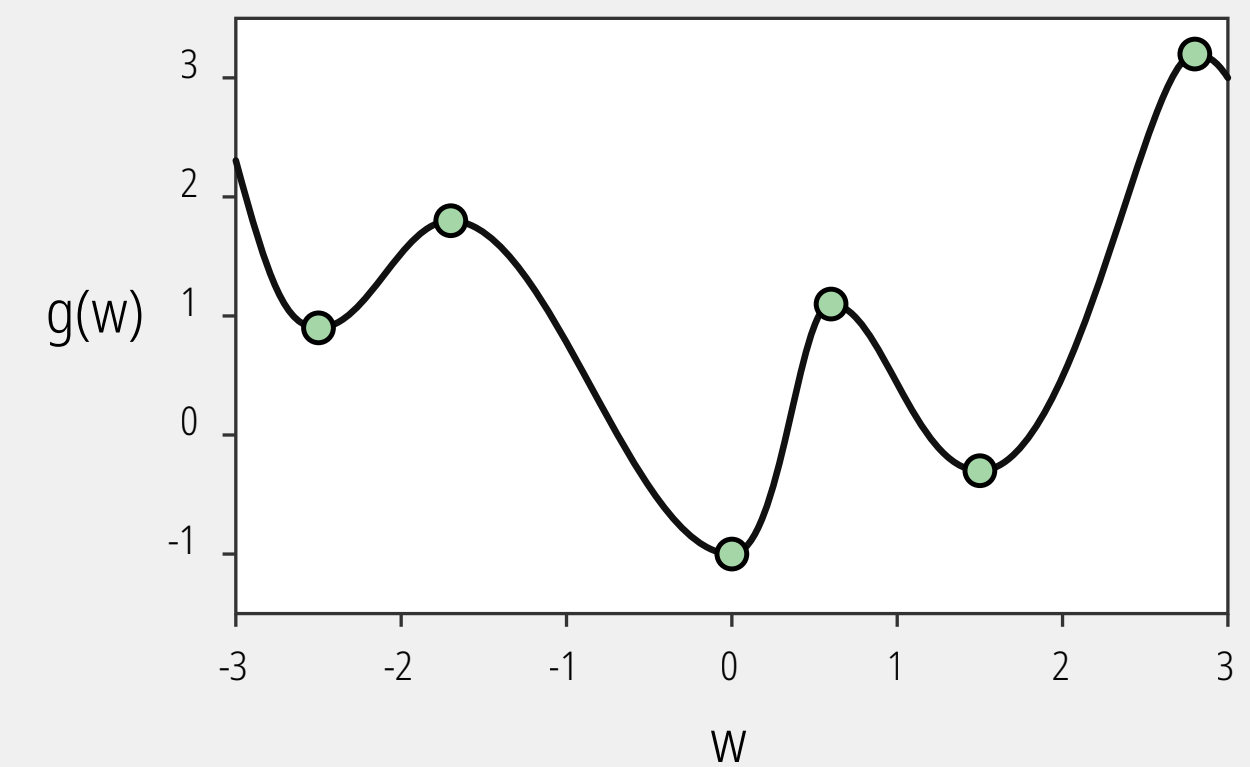
Máximo global único: Representa una función cóncava con un solo pico absoluto. Ilustra el escenario inverso al anterior, utilizado cuando el objetivo matemático es maximizar un valor (como una función de recompensa o una probabilidad).

3



Múltiples óptimos globales equivalentes: Exhibe una función periódica (similar a un seno o coseno). Posee varios mínimos y máximos, pero todos los valles tienen exactamente la misma profundidad y todas las crestas la misma altura; cualquier valle es una solución globalmente válida.

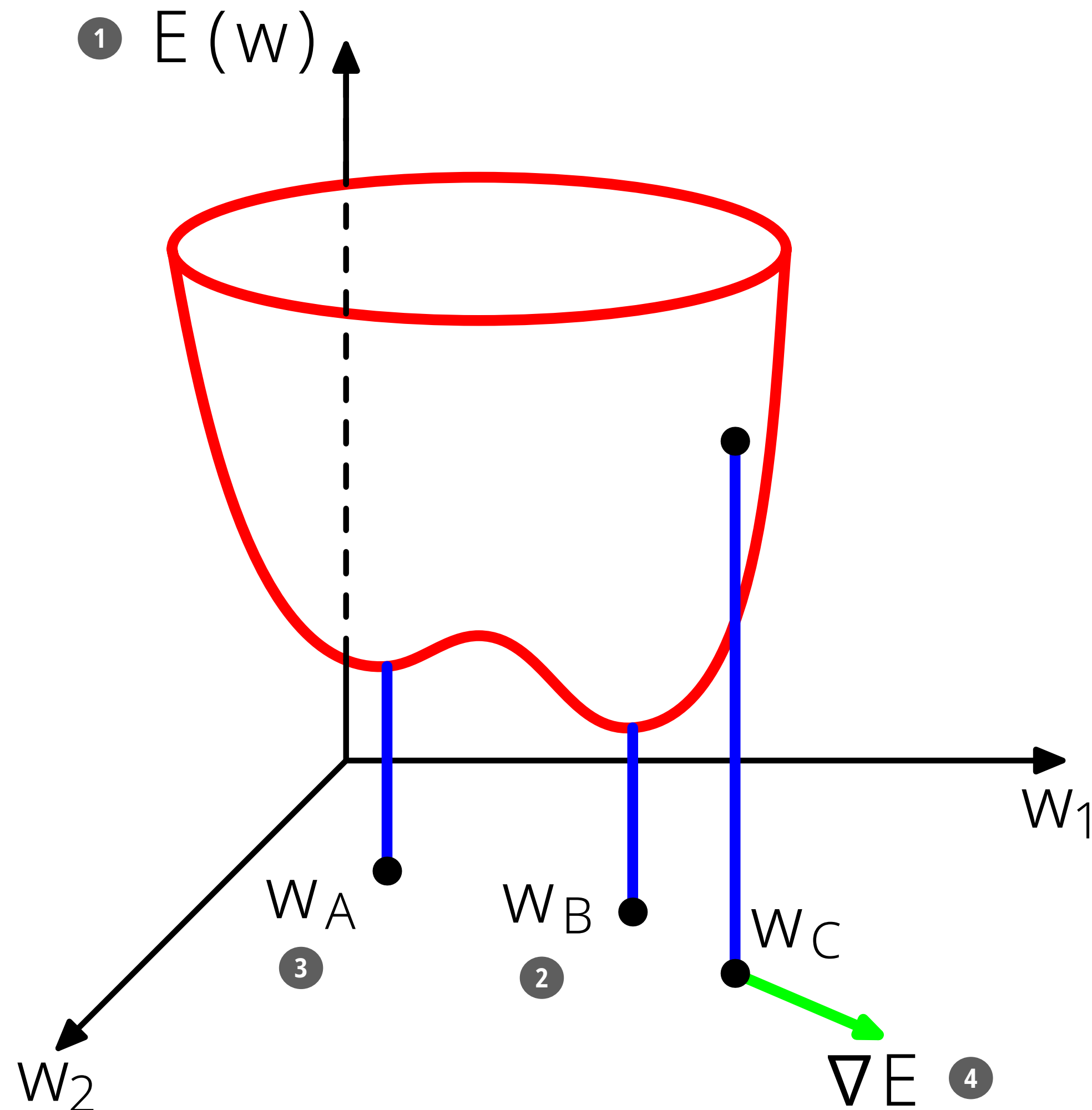
4



Óptimos locales y globales: Ilustra un paisaje no convexo, el caso más realista y desafiante en problemas complejos. Presenta mínimos y máximos locales (valles o picos intermedios donde los algoritmos pueden quedar atascados, creyendo erróneamente que llegaron al final) junto con los verdaderos óptimos globales (los puntos absolutos más extremos de toda la curva).

Optimización: Función de error (revisitado)

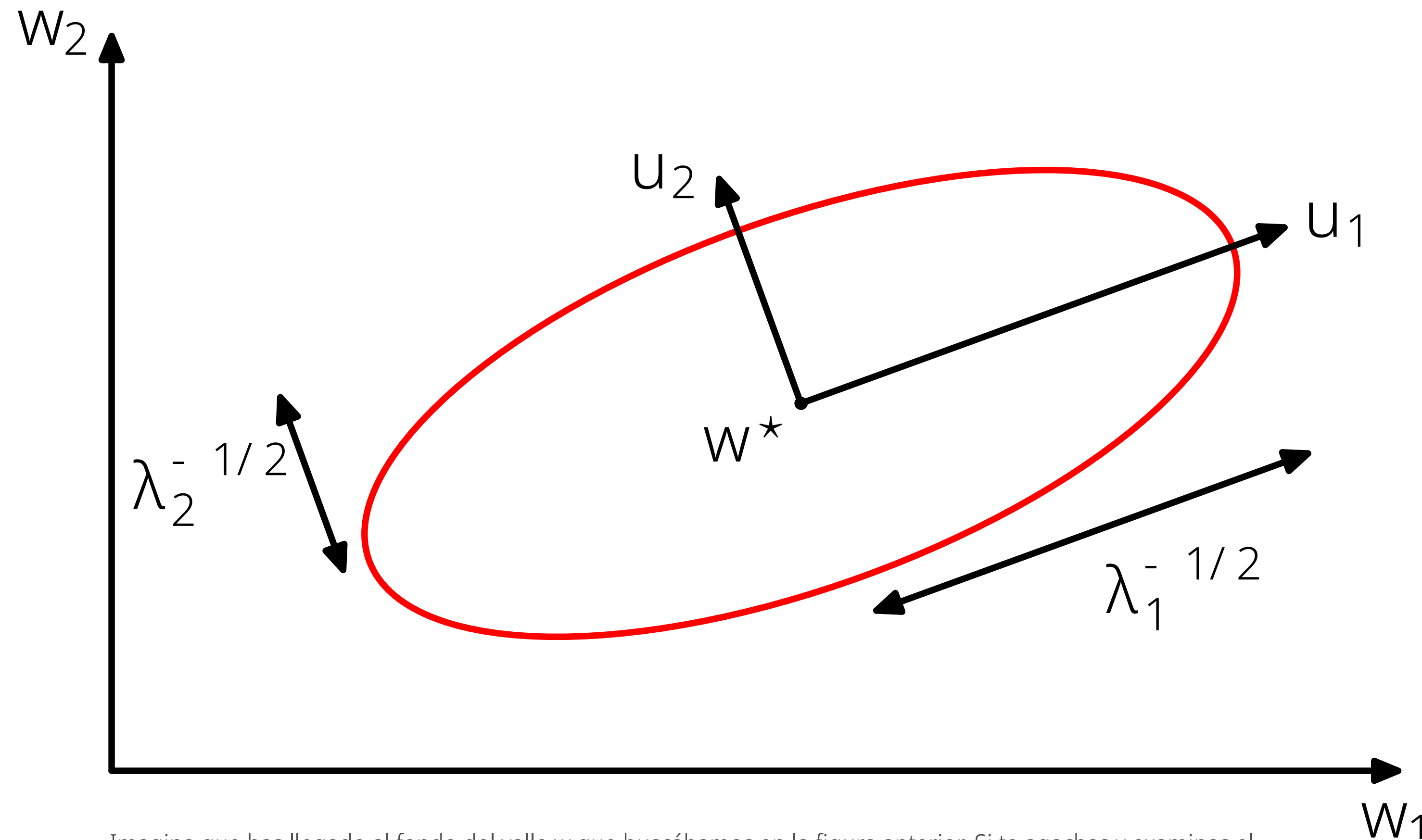
Imagina que estás caminando con los ojos vendados por un terreno montañoso donde tu altitud representa la cantidad de equivocaciones de tu modelo: tu único objetivo es descender hasta el punto más bajo posible.



- 1 **Superficie de error $E(w)$:** Es el “paisaje” tridimensional formado por todas las combinaciones posibles de los parámetros o pesos del modelo (ejes w_1 y w_2). La altura de este paisaje dicta el nivel de error: a mayor altitud, peores son las predicciones.
- 2 **Mínimo global (w_B):** Es el destino ideal. Representa el punto más bajo de toda la superficie y contiene la combinación exacta de parámetros que produce el menor error posible.
- 3 **Mínimo local (w_A):** Es un fondo de valle secundario donde el modelo puede quedarse atascado. Al estar ahí, cualquier paso a los lados te hace subir, engañando al algoritmo para que crea que llegó a la solución óptima, cuando en realidad existe un valle más profundo al que no llegó.
- 4 **Vector gradiente:** Actúa como la “brújula” matemática del modelo (ilustrado en el punto w_C). Este vector evalúa la pendiente exacta en tu posición actual y tiene la característica de apuntar siempre hacia la subida más pronunciada del paisaje. (Para descender, el algoritmo debe dar pasos en la dirección opuesta a este vector).

Optimización: Función de error

Para entrenar el modelo, algoritmos clásicos como el Descenso del Gradiente simplemente calculan este vector y obligan a los parámetros a dar un paso en la dirección exactamente opuesta (cuesta abajo), repitiendo el proceso iterativamente para deslizarse por la ladera hasta encontrar el fondo del valle.



Imagina que has llegado al fondo del valle w que buscábamos en la figura anterior. Si te agachas y examinas el terreno justo a tu alrededor, notarás que, de cerca, la superficie pierde sus irregularidades y se asemeja a un cuenco suave y simétrico.

- **El “cuenco” del mínimo:** Cerca del punto de mínimo error, el paisaje se comporta como una función cuadrática con forma de cuenco. Las líneas de nivel (como la elipse roja) representan puntos donde el error es constante.
- **La Matriz Hessiana:** Como el cuenco no es un círculo perfecto (es más empinado en unas direcciones que en otras), se utiliza la matriz Hessiana para medir y describir la curvatura exacta del paisaje en todas las direcciones.
- **Dirección (Vectores Propios / u_1, u_2):** Estos vectores actúan como una brújula que define los ejes principales de la elipse, alineándose naturalmente con las direcciones de máxima y mínima curvatura del terreno.
- **Aplastamiento (Valores Propios / λ s):** Determinan qué tan empinada es la pendiente en cada dirección. Valor grande: Curvatura extrema (pendiente pronunciada), lo que hace que la elipse sea estrecha o corta en ese eje.
- **Valor pequeño:** Curvatura suave (fondo más plano), permitiendo que la elipse se alargue sin que el error suba rápido.
- **Confirmación del mínimo:** Para asegurar que realmente has llegado al fondo de un “cuenco” (un mínimo local) y no a un “punto de silla”, el terreno debe curvarse hacia arriba en todas las direcciones. Esto significa que la matriz Hessiana debe ser “definida positiva” (todos sus valores propios deben ser estrictamente mayores a cero).

Optimización: Gradiente Descendente

Mecánica del descenso del gradiente, ilustrando cómo el algoritmo interactúa con la "topología" o forma matemática de la función de costo.

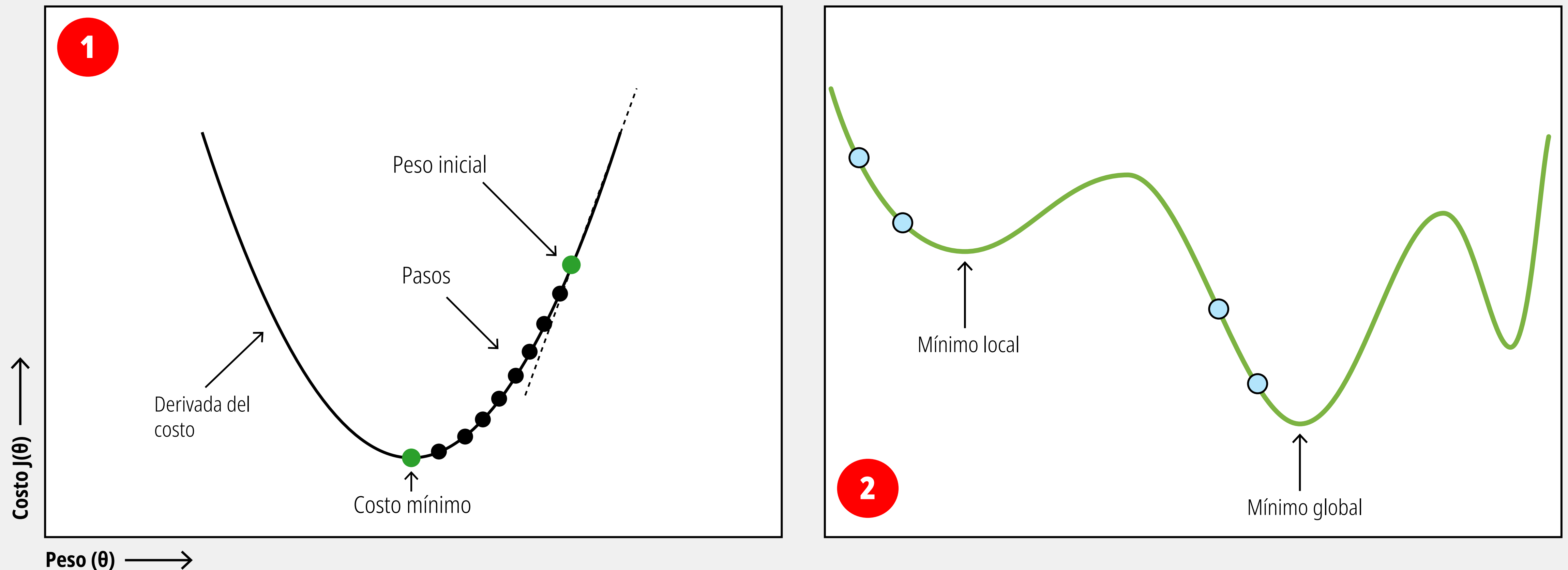


Figura No. 1: Este gráfico representa una función de error sencilla, con forma de tazón o parábola. Es el comportamiento típico en algoritmos más simples (como una regresión lineal). Figura No. 2: Este panel muestra un paisaje de optimización ondulado y complejo. Esta es la realidad al optimizar arquitecturas avanzadas, como redes neuronales profundas o modelos de lenguaje (LLMs), donde se ajustan miles de millones de parámetros simultáneamente. **Mínimo local:** Es un valle superficial. Si el algoritmo llega aquí, la derivada se vuelve cero (plana), y el modelo puede "creer" que ha terminado de optimizar. Es una trampa matemática; el algoritmo se estanca en una solución subóptima porque cualquier paso inmediato parece empeorar el error. **Mínimo global:** Es el verdadero fondo de la curva, el punto que representa la configuración matemáticamente perfecta (o de menor error absoluto) para los parámetros del modelo. La trayectoria de puntos azules muestra un escenario exitoso donde el algoritmo logra descender hacia este punto óptimo.

Optimización: Gradiente Descendente

Esta técnica consiste en ajustar iterativamente los parámetros del modelo evaluando la pendiente matemática del error.

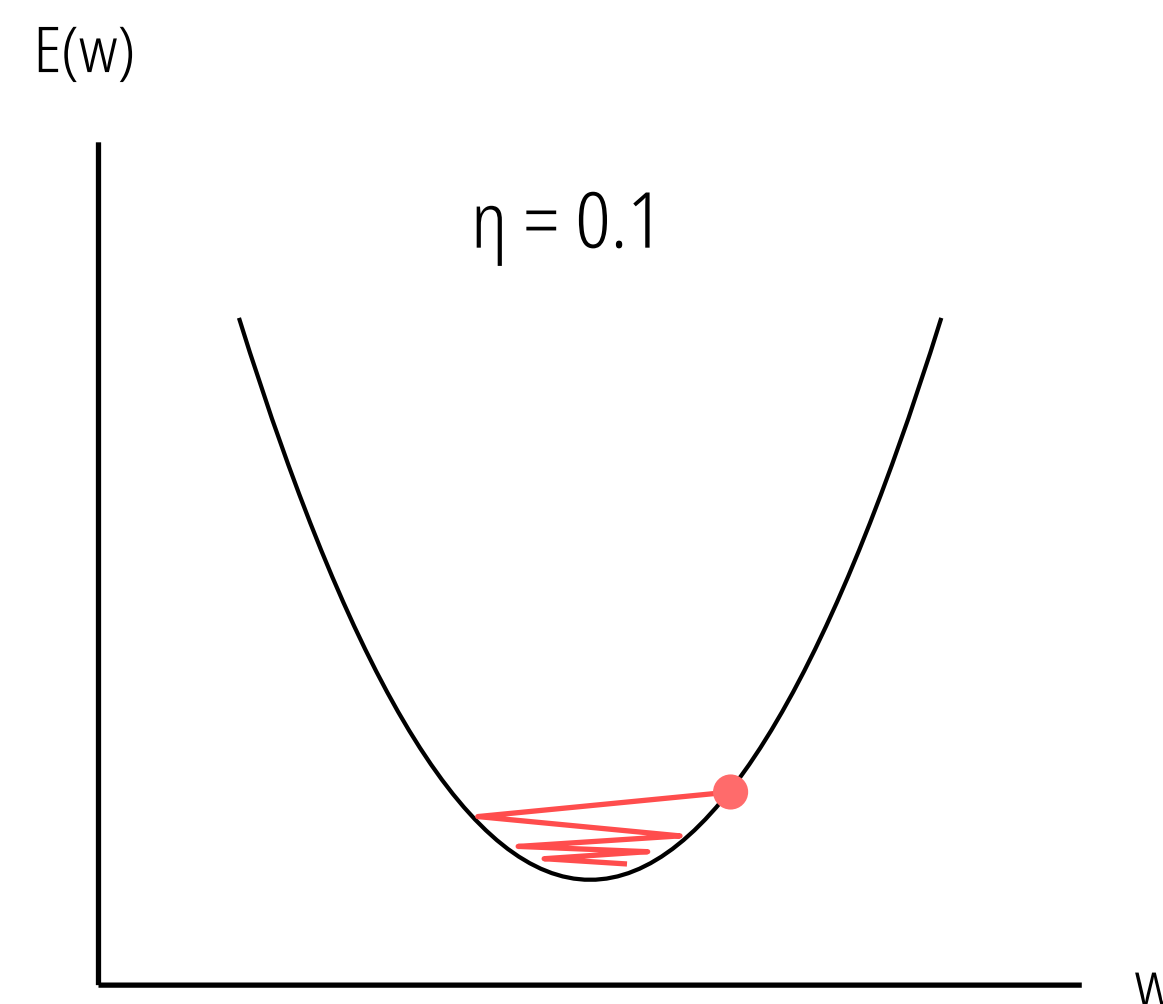
$$w_{j+1} = w_j + \eta (-\Delta_j)$$

Diagrama de la ecuación de actualización de pesos:

- w_{j+1} : peso j en el siguiente paso de tiempo (índice para cada fila de la matriz X)
- w_j : peso j en el paso de tiempo actual
- η : tasa de aprendizaje o tamaño de paso (letra griega eta)
- $-\Delta_j$: gradiente j (con signo negativo)

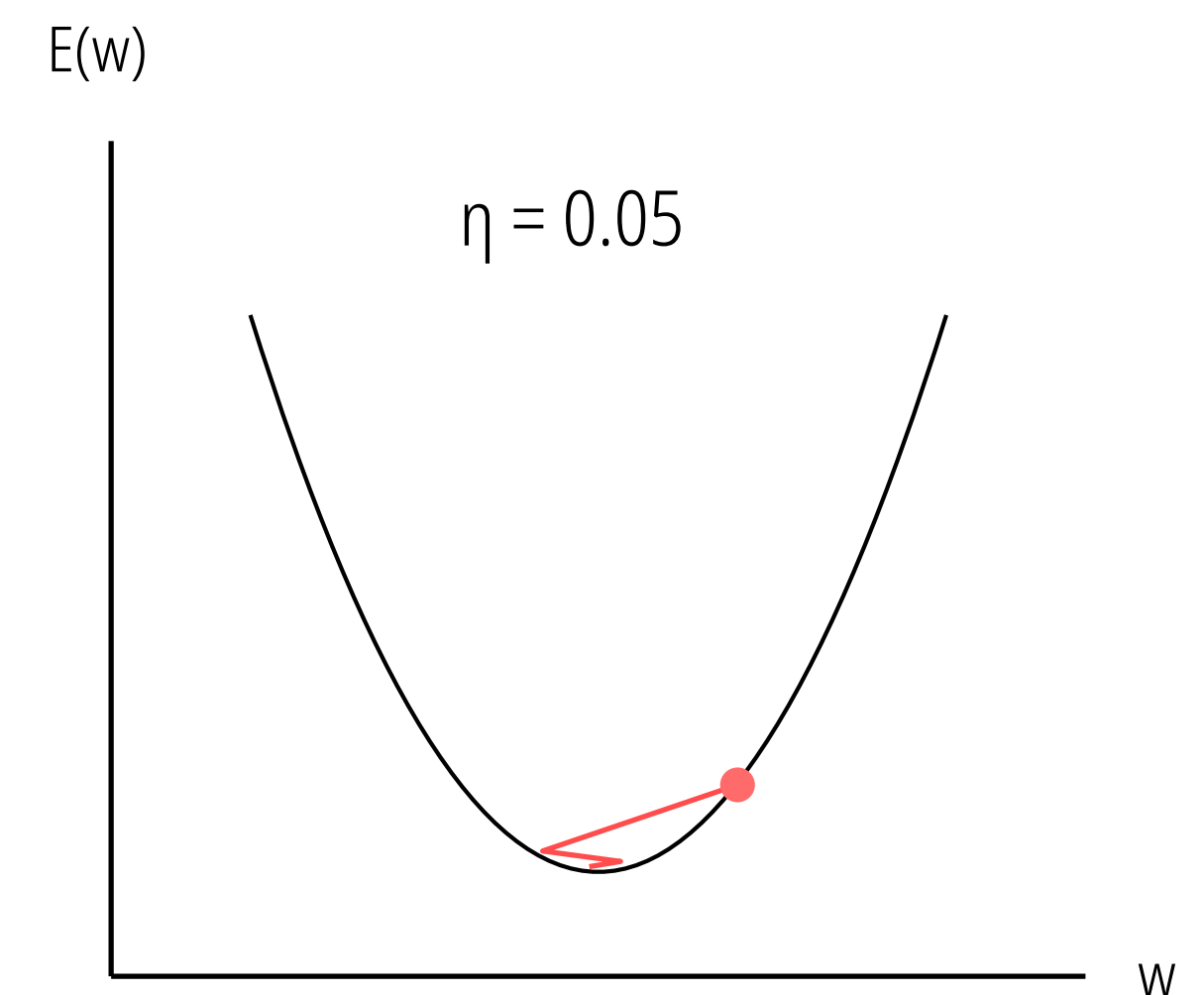
La ecuación describe la evolución de un peso específico. Para calcular su nuevo valor en el siguiente paso de tiempo, el algoritmo toma el peso actual y le suma la dirección del gradiente negativo. Este signo negativo es crucial, ya que obliga matemáticamente al modelo a moverse en la dirección contraria a la pendiente, asegurando un descenso hacia el punto donde el error es menor. La letra griega eta representa la **tasa de aprendizaje (o tamaño de paso)**. Actúa como un multiplicador que define qué tan grande será el ajuste aplicado al peso. Es el mecanismo de seguridad del algoritmo: sin él, la magnitud pura del gradiente podría sugerir saltos demasiado abruptos que desestabilizarían todo el proceso de entrenamiento.

Tamaño de paso mayor



Sobreajuste por exceso de velocidad (eta = 0.1): Un tamaño de paso mayor hace que el modelo salte violentamente de un lado a otro del valle de la función de error E . Al intentar descender con demasiado impulso, rebasa el punto mínimo repetidas veces ("overshooting"), provocando un patrón de zig-zag o efecto rebote que retrasa significativamente la convergencia.

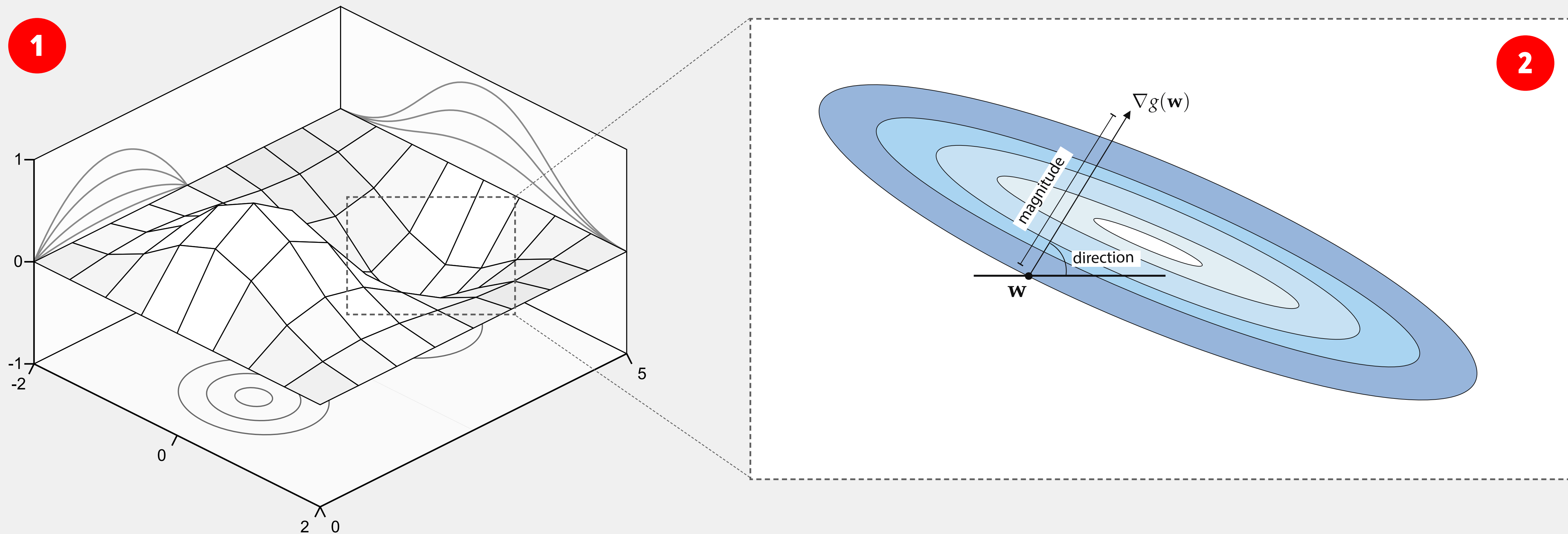
Tamaño de paso menor



Descenso controlado y eficiente (eta = 0.05): Un tamaño de paso menor y bien calibrado permite un descenso mucho más suave y directo. El modelo se desliza hacia el fondo de la curva sin rebotar drásticamente en las paredes, logrando estabilizarse en el punto de error mínimo de forma estable y segura.

Optimización: Gradiente Descendente

Como el objetivo es minimizar el error (llegar al fondo del valle), el algoritmo toma la dirección que le da el gradiente y da un paso en la dirección exactamente opuesta. Al repetir este paso iterativamente, el punto w irá bajando por la pendiente hasta llegar al centro (el error mínimo).



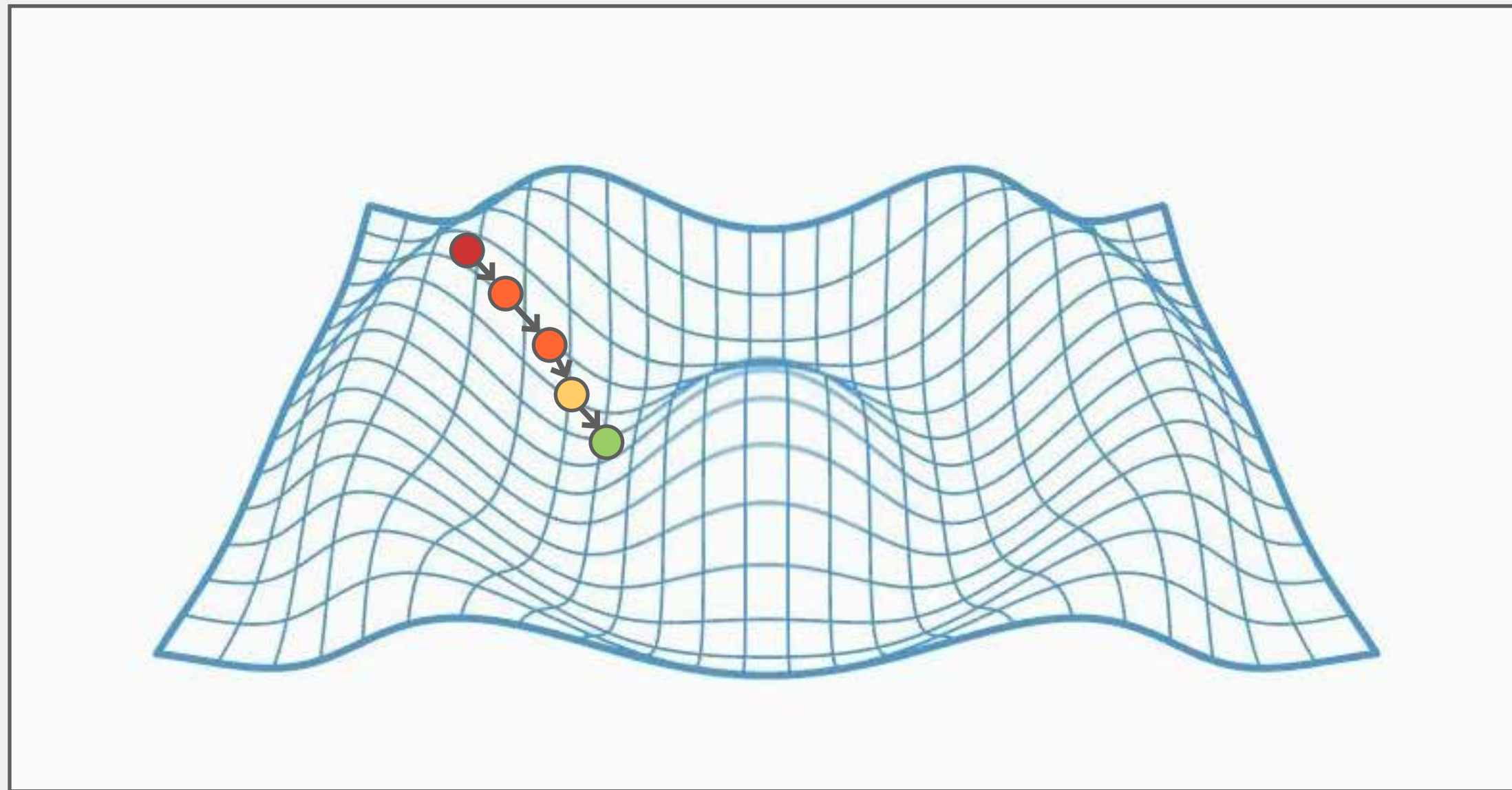
Representa el nivel de error. Los picos altos son configuraciones donde el modelo se equivoca mucho; los valles profundos son donde el modelo es altamente preciso. El recuadro punteado se enfoca en uno de estos valles, que es hacia donde queremos que nuestro modelo "descienda" iterativamente. Debajo de la superficie 3D, puedes ver unos anillos grises: son la proyección en 2D de esos picos y valles, llamados curvas de nivel.

Cada anillo representa una altura constante; mientras más al centro vayas, más profundo estás en el valle. Punto w : Representa la configuración actual de los parámetros o pesos del modelo en un momento dado. Gradiente: Esta flecha es el cálculo matemático de la derivada en ese punto exacto. El gradiente tiene una propiedad geométrica crucial: siempre apunta en la dirección de mayor ascenso (la subida más empinada hacia el pico) y siempre es perpendicular (ortogonal) a la curva de nivel. Magnitud y Dirección: El gradiente nos dice hacia dónde está la subida (dirección) y qué tan empinada es (magnitud).

Optimización: Gradiente Descendente

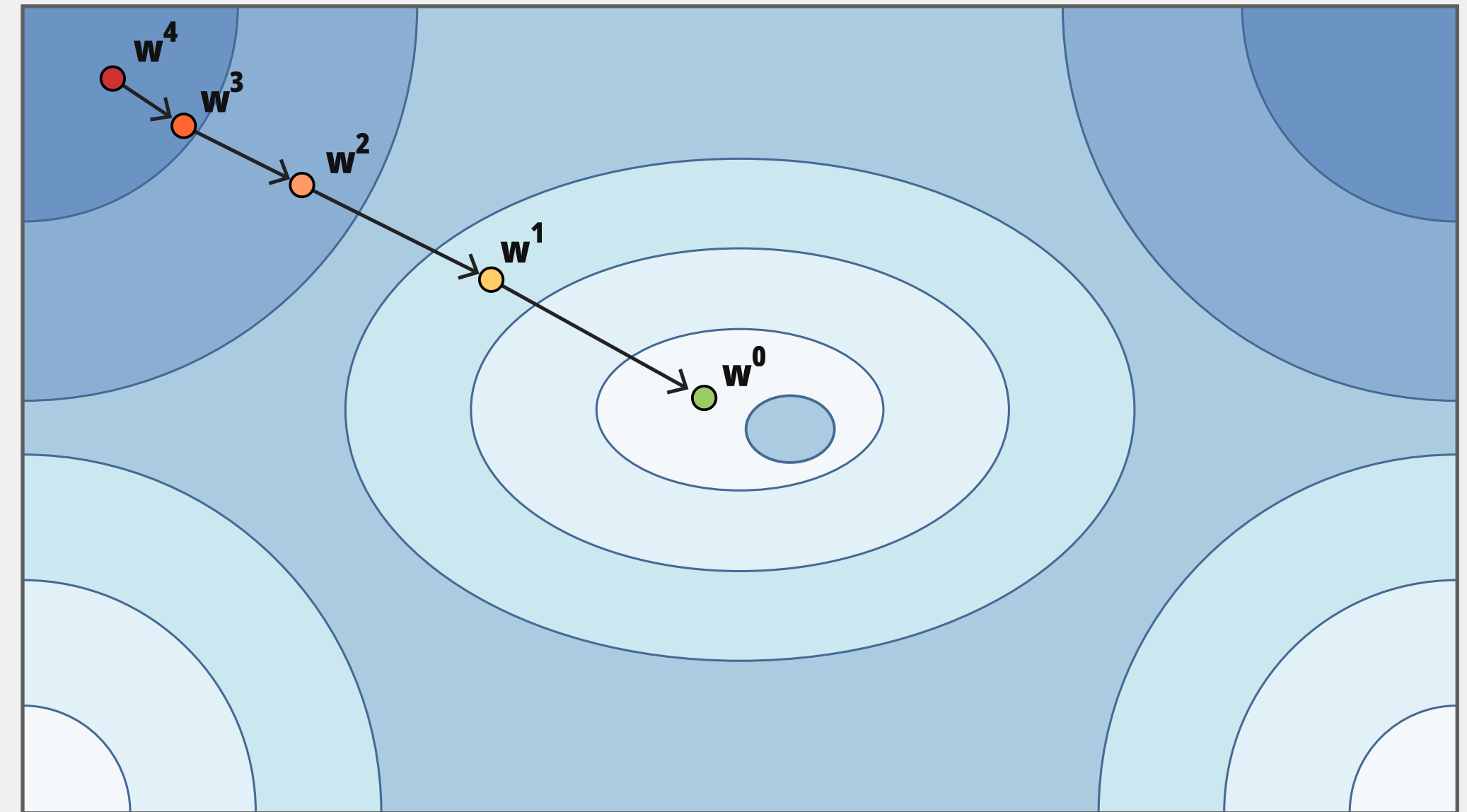
La trayectoria iterativa que sigue el algoritmo, paso a paso, para minimizar el error de un modelo. El “aprendizaje” no ocurre en un solo cálculo, sino que es una secuencia de pasos guiados que navegan cuesta abajo a través del paisaje de errores hasta encontrar la mejor solución posible.

1



Superficie 3D: Representa las sucesivas actualizaciones que hace el algoritmo. Comienza en una posición elevada en la ladera (donde el nivel de error es alto) y, en cada iteración, calcula la pendiente para descender un poco más, hasta detenerse en el fondo del pozo o valle (el punto verde), que representa el error mínimo.

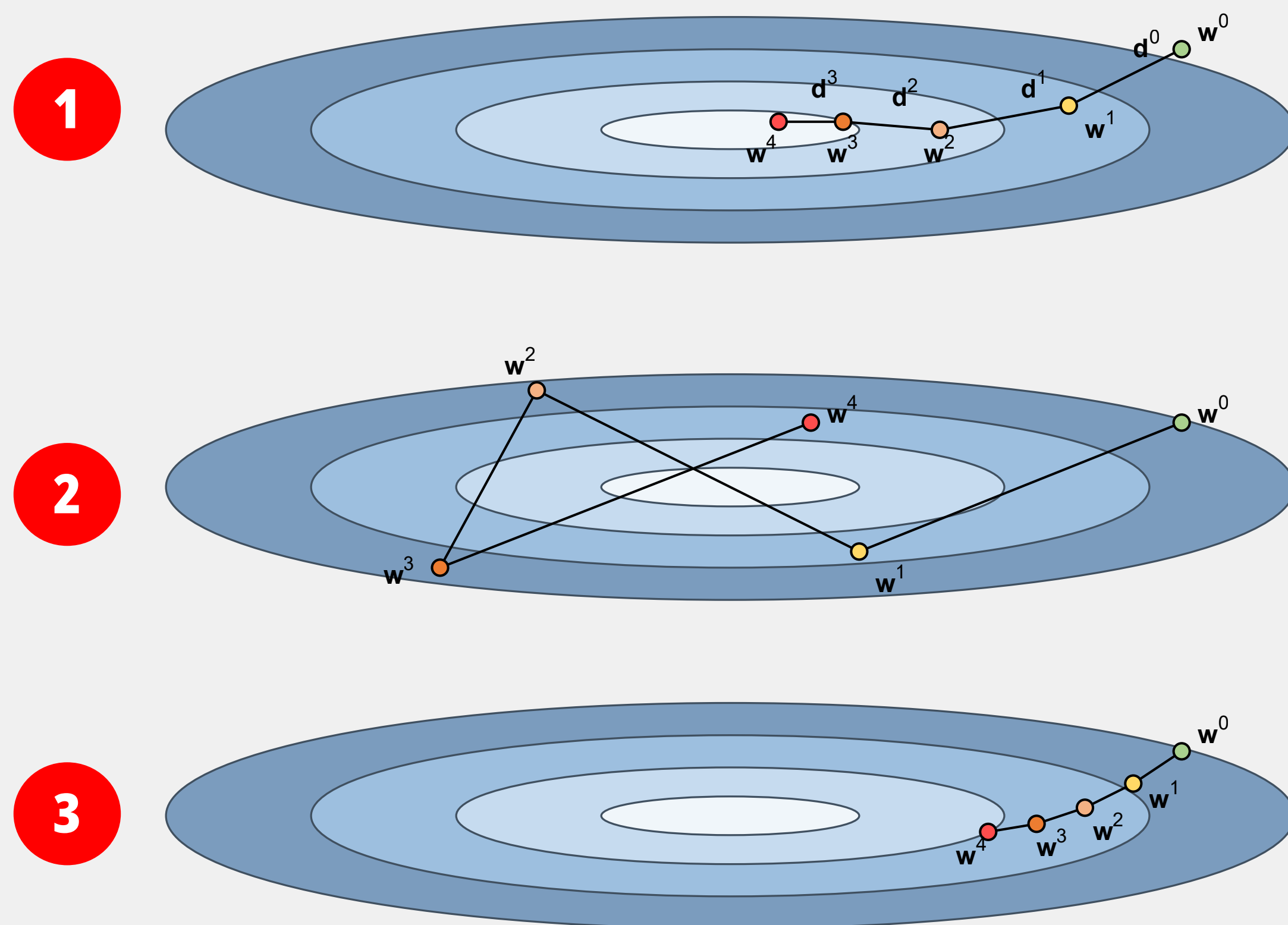
2



Proyección 2D (Mapa de contorno): Las zonas más oscuras en los bordes indican un costo alto, y el centro claro es la zona de error mínimo. Aquí se etiquetan explícitamente las iteraciones de los parámetros (pesos) del modelo. El proceso arranca en el estado 4 y, con cada iteración matemática, el algoritmo da un paso direccional cruzando las curvas de nivel, hasta converger finalmente en el punto centra, logrando la configuración óptima.

Optimización: Gradiente Descendente

Optimización bajo diferentes configuraciones de su "tamaño de paso". En el ámbito del machine learning y la arquitectura de DNNs, a este tamaño de paso se le conoce como la tasa de aprendizaje (learning rate).



1. Optimización equilibrada: Muestra el escenario ideal. Los vectores de dirección tienen una magnitud matemáticamente adecuada. El algoritmo desciende de manera suave y eficiente desde el punto inicial, tomando rutas directas hacia el mínimo central sin desviarse. Logra una convergencia rápida y estable.

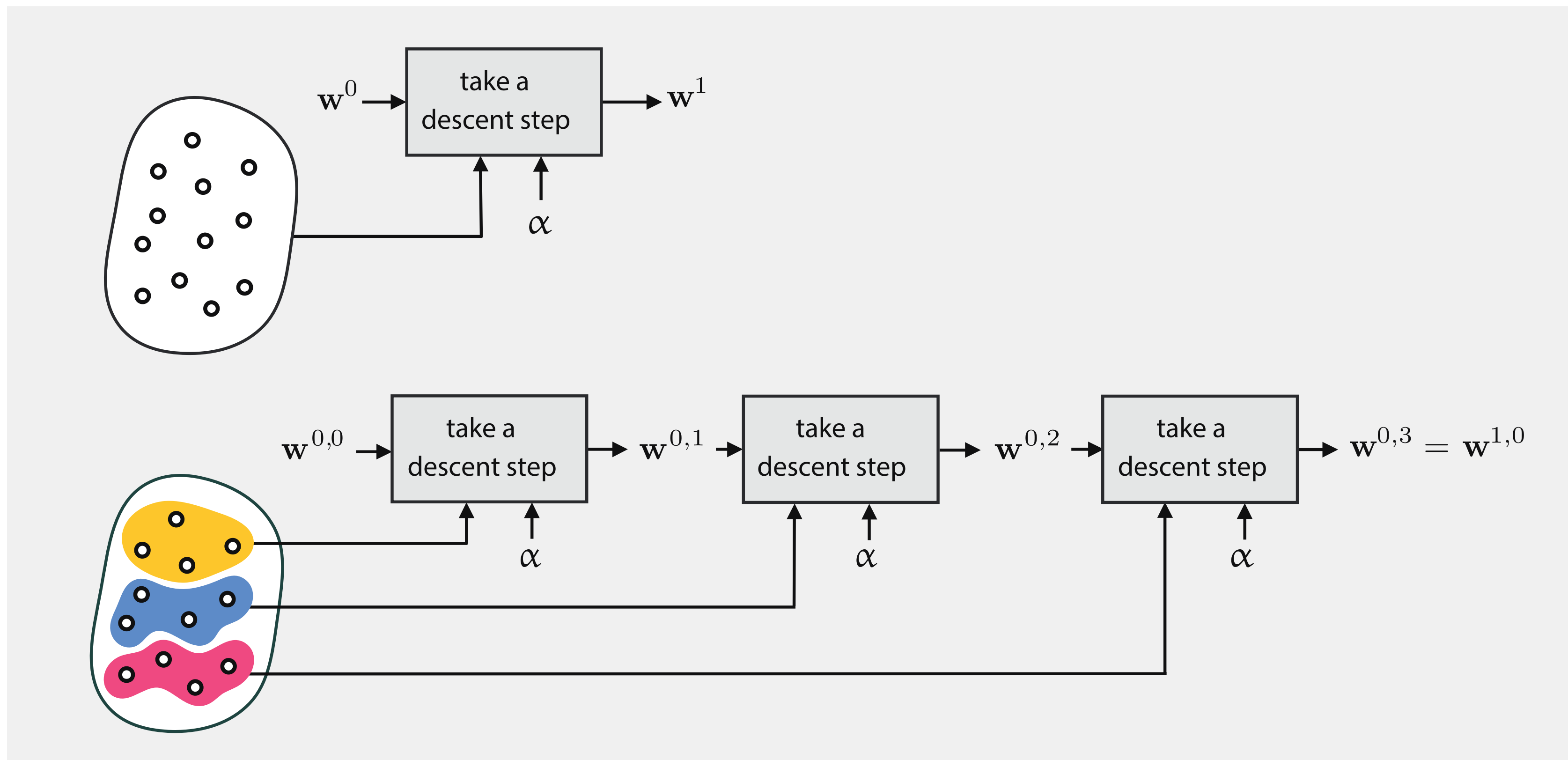
2. Tasa de aprendizaje excesiva (overshooting): Aquí, los multiplicadores escalares de los vectores de dirección son demasiado grandes. En lugar de descender hacia el fondo de la curva, el algoritmo "salta" por encima del mínimo, rebotando violentamente de un lado a otro. Este comportamiento oscilatorio impide que el modelo converja. En la práctica, un salto así de errático puede hacer que el error se dispare, provocando que el entrenamiento diverja por completo.

3. Tasa de aprendizaje deficiente: Los vectores de dirección son minúsculos. El modelo evalúa correctamente el gradiente y avanza de forma segura, pero los pasos son tan pequeños que el algoritmo requerirá un número gigantesco de iteraciones para llegar al centro. A nivel de ingeniería, esto se traduce en una grave ineficiencia computacional. El proceso consumirá recursos de procesamiento de manera excesiva y tardará demasiado tiempo en completar la tarea de optimización.

El gráfico utiliza curvas de nivel (las elipses concéntricas) para representar la topología de una función de pérdida o costo bidimensional. El área central más clara es el mínimo global de la función (el punto de menor error). La secuencia de puntos representa los valores de los parámetros en cada iteración del algoritmo, mientras que los vectores indican la dirección y magnitud de cada actualización.

Redes neuronales: Mini-Lotes y Aprendizaje en Línea

A diferencia de la optimización tradicional de lote completo (full batch), que evalúa todo el conjunto de datos de entrenamiento simultáneamente, la optimización por mini-lotes divide los datos en subconjuntos más pequeños. El modelo realiza los pasos de optimización (minimización de costos) de forma secuencial por cada mini-lote. Un recorrido completo a través de todos los datos bajo este método se conoce como una época (epoch).



Ventajas Clave:

Mayor Velocidad: Acelera significativamente el proceso de aprendizaje, especialmente cuando se combina con el descenso de gradiente en conjuntos de datos muy grandes.

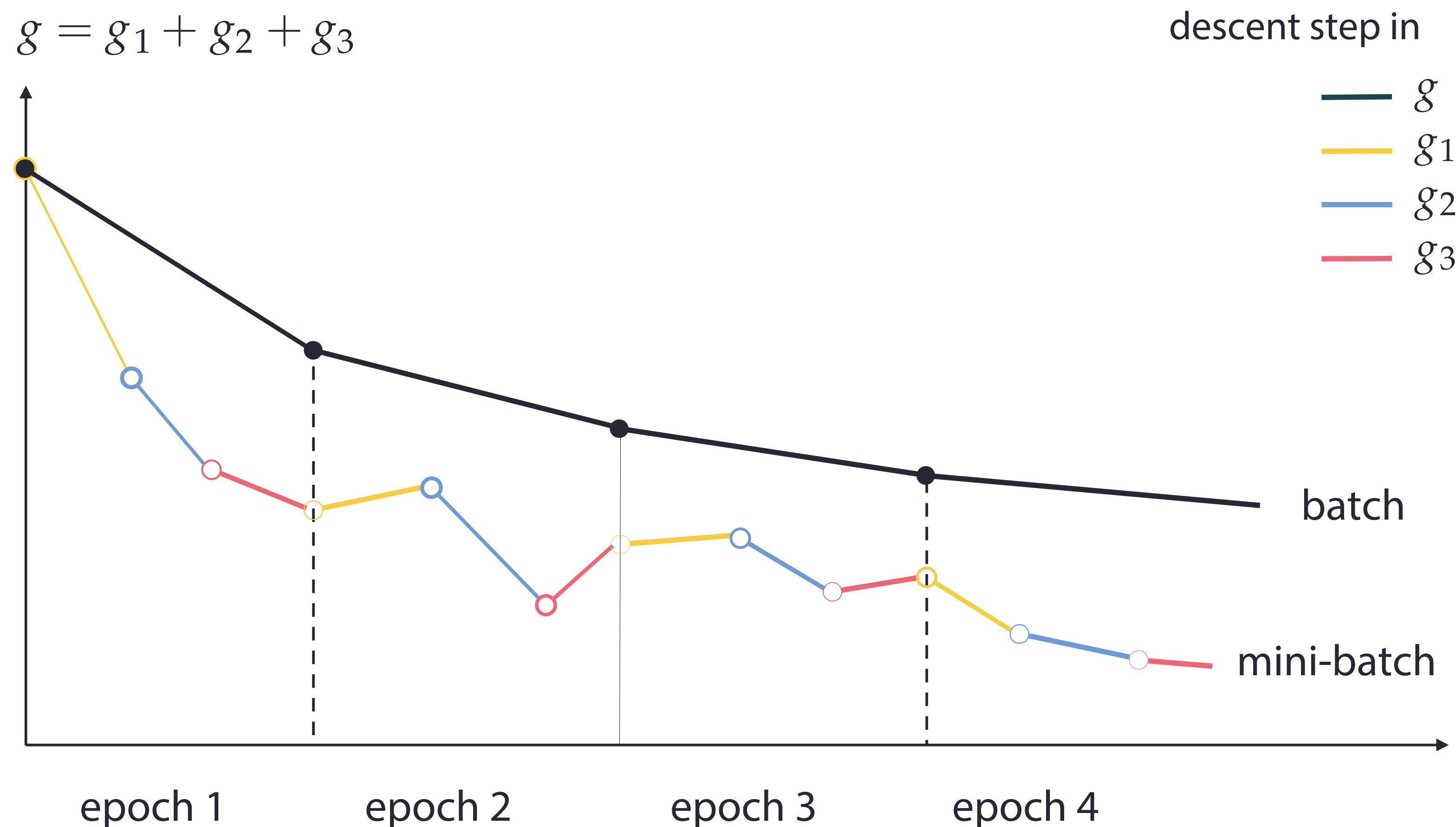
Eficiencia de Memoria: Limita drásticamente el consumo de memoria activa (RAM), ya que el sistema solo necesita cargar los datos del mini-lote actual en cada iteración, en lugar de la base de datos completa.

Conexión con Online Learning: Esta estructura de mini-lotes puede interpretarse o utilizarse como una técnica de aprendizaje en línea. Esto es útil en escenarios donde los datos no están estáticos, sino que se generan continuamente en pequeños bloques, permitiendo actualizar los parámetros del modelo sobre la marcha a medida que llega nueva información.

Comparación esquemática de la primera iteración del descenso de gradiente de lote completo (panel superior) y estocástico (panel inferior), a través de la lente del aprendizaje automático utilizando (en aras de la simplicidad) un pequeño conjunto de datos de puntos. En el barrido de lote completo damos un paso en todos los puntos simultáneamente, mientras que (panel inferior) en el enfoque de mini-lotes realizamos un barrido por estos puntos de forma secuencial, como si recibiéramos los datos en línea (online).

Optimización: Batch vs Mini-Batches

El conjunto de datos de entrenamiento se divide en fracciones más pequeñas (mini-lotes). El algoritmo calcula el gradiente y actualiza los parámetros para cada fracción individualmente.



Múltiples pasos (actualizaciones) dentro de una misma época. La trayectoria fluctúa un poco más (va en zigzag), pero en la práctica suele hacer que el modelo converja más rápido, ya que aprende y se ajusta continuamente durante la época.

Optimización: Gradiente Descendente

El recuadro muestra el pseudocódigo formal. El corazón de todo el proceso radica en la ecuación de actualización, que se ejecuta repetidamente hasta que el modelo converge.

1

Algorithm 1: Stochastic gradient descent

Input: Training set of data points indexed by $n \in \{1, \dots, N\}$
 Error function per data point $E_n(w)$
 Learning rate parameter η
 Initial weight vector w

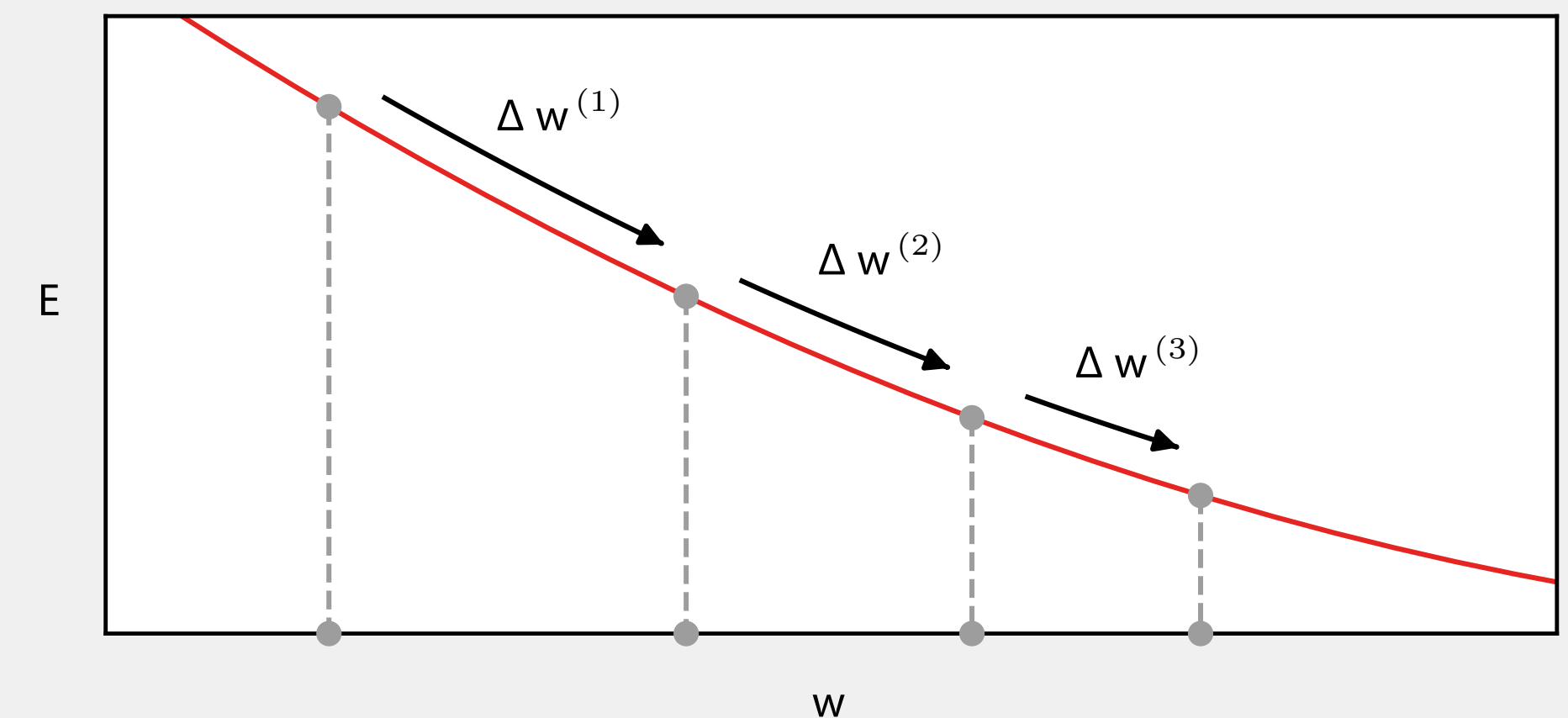
Output: Final weight vector w

```

 $n \leftarrow 1$ 
repeat
   $w \leftarrow w - \eta \nabla E_n(w)$  // update weight vector
   $n \leftarrow n + 1 \pmod{N}$  // iterate over data
until convergence
return  $w$ 
  
```

2

Un mecánico brillante permite que el algoritmo dé saltos grandes al principio (cuando está muy lejos de la solución) y haga ajustes finos y milimétricos al final, evitando rebotar y pasarse de largo del punto óptimo.



Se representa el nivel de Error frente al valor del parámetro o peso (w). La curva roja es la topografía del error que queremos minimizar. Observa detenidamente las flechas negras que representan la magnitud del ajuste en cada iteración (longitud de pasos). A medida que el punto desciende por la curva roja, la pendiente se va haciendo cada vez menos empinada (se aplanan). Como el tamaño de nuestro paso depende directamente del valor de esa pendiente, los pasos se vuelven progresivamente más pequeños conforme el modelo se acerca al fondo (frenado automático).

Optimización: Gradiente Descendente

1

Algorithm 2: Stochastic gradient descent with momentum

Input: Training set of data points indexed by $n \in \{1, \dots, N\}$

Batch size B

Error function per mini-batch $E_{n:n+B-1}(\mathbf{w})$

Learning rate parameter η

Momentum parameter μ

Initial weight vector $\mathbf{w}$

Output: Final weight vector $\mathbf{w}$

$n \leftarrow 1$

$\Delta \mathbf{w} \leftarrow \mathbf{0}$

repeat

$\Delta \mathbf{w} \leftarrow -\eta \nabla E_{n:n+B-1}(\mathbf{w}) + \mu \Delta \mathbf{w}$ // calculate update term

$\mathbf{w} \leftarrow \mathbf{w} + \Delta \mathbf{w}$ // weight vector update

$n \leftarrow n + B$

if $n > N$ then

shuffle data

$n \leftarrow 1$

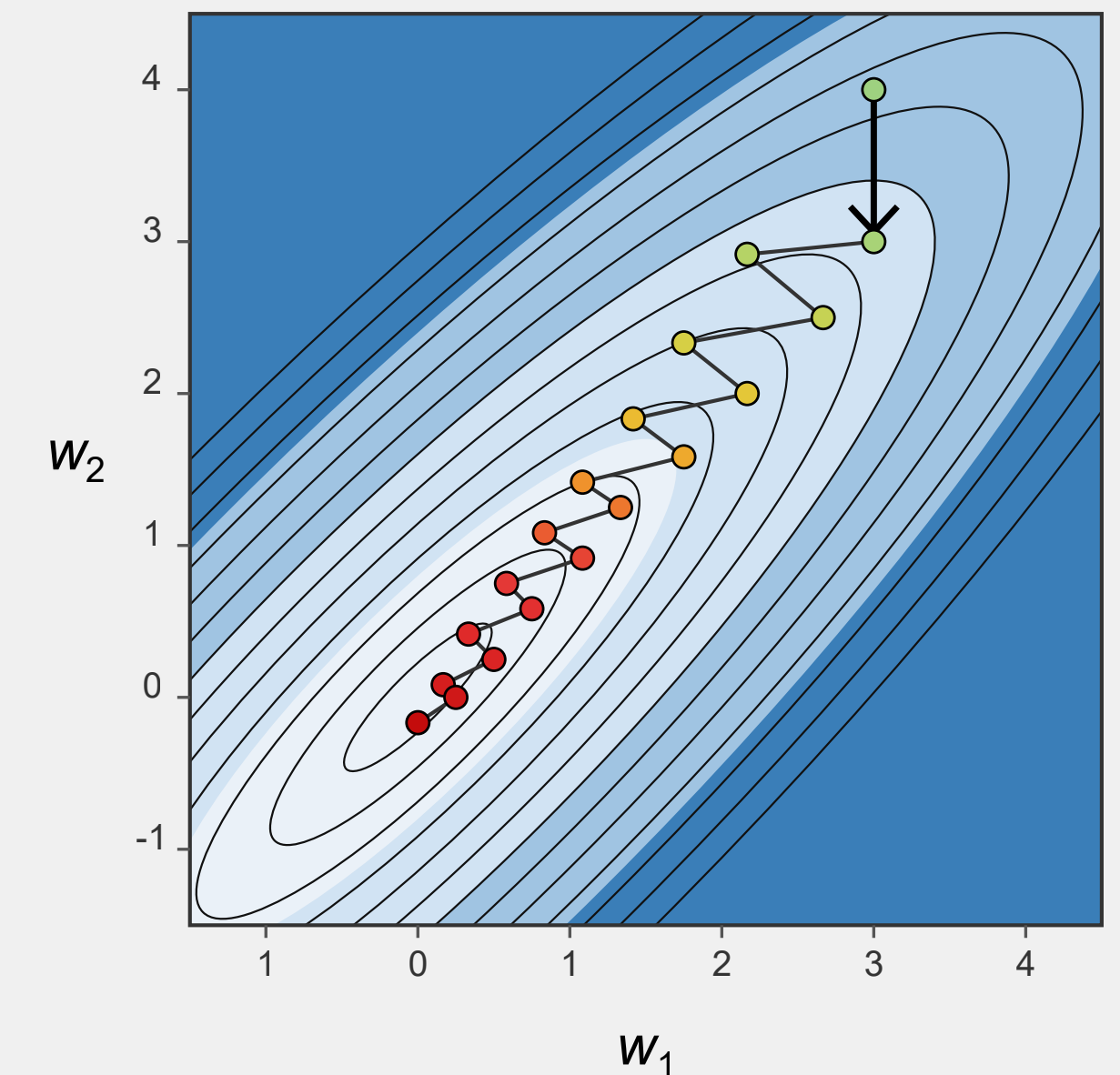
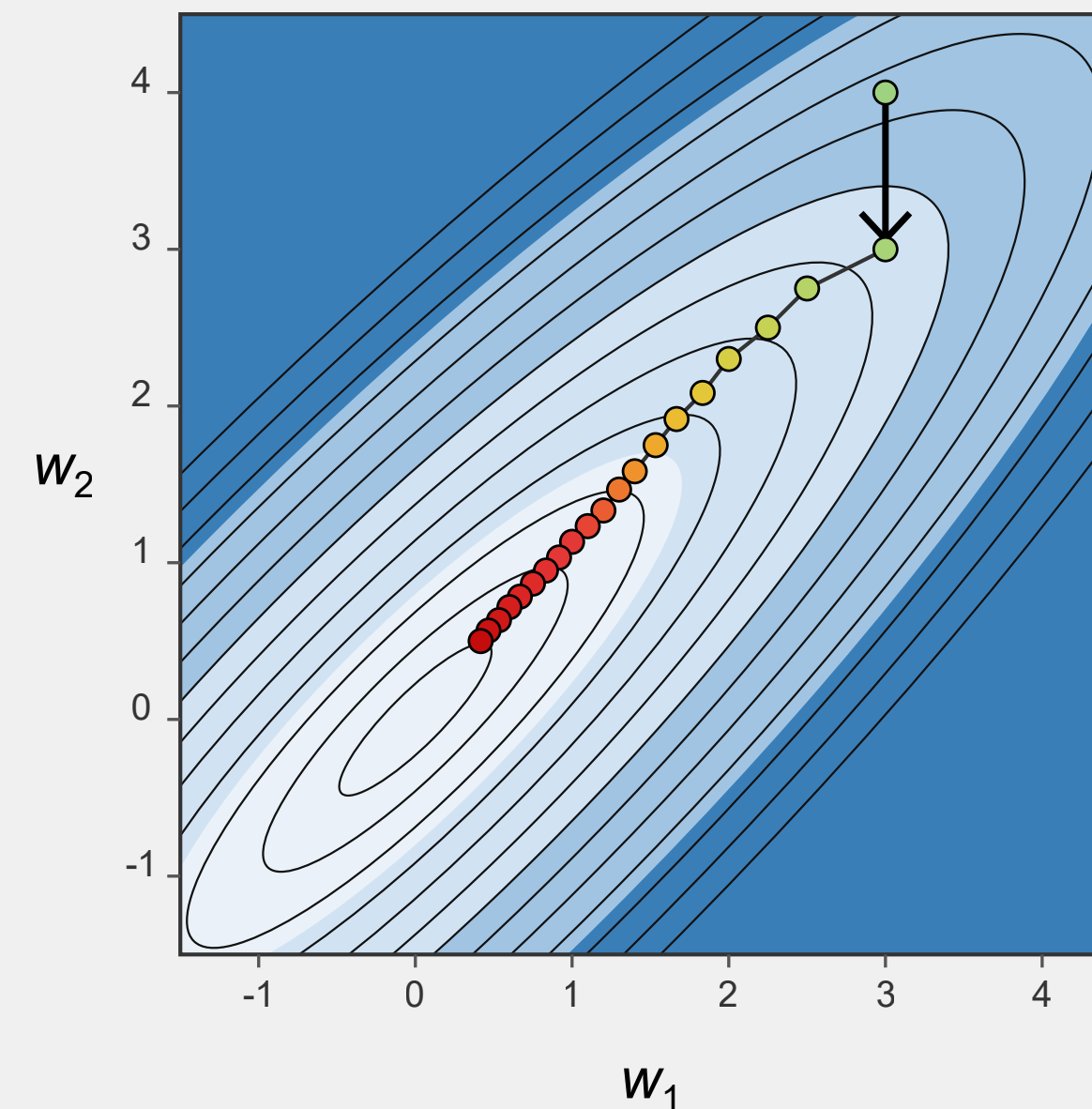
end if

until convergence

return $\mathbf{w}$

2

El pseudocódigo introduce dos conceptos nuevos muy importantes para el rendimiento y la estabilidad: Mini-batches (Lotes): En lugar de usar un solo dato (muy ruidoso) o todos los datos (muy costoso en memoria), el algoritmo procesa un pequeño bloque de datos en cada paso. Esto equilibra la velocidad de cálculo con la estabilidad matemática. El Parámetro de Momento: Esta es la verdadera magia. En la ecuación de actualización, nota cómo se suma un término adicional. Esto significa que el algoritmo ya no solo se mueve basándose en la pendiente actual, sino que recuerda la dirección y velocidad de su paso anterior.



Al aplicar la inercia del paso anterior, ocurre un fenómeno físico: los continuos rebotes hacia la izquierda y derecha se terminan cancelando entre sí, mientras que el avance constante hacia adelante se acumula. El resultado es una trayectoria fluida, directa y mucho más rápida hacia el error mínimo. En síntesis, el momento le da al algoritmo "peso", permitiéndole atravesar zonas de alta fricción matemática sin perder su dirección principal.

Optimización: Gradiente Descendente

Algorithm 3: Adam optimization

Input: Training set of data points indexed by $n \in \{1, \dots, N\}$
 Batch size B
 Error function per mini-batch $E_{n:n+B-1}(\mathbf{w})$
 Learning rate parameter η
 Decay parameters β_1 and β_2
 Stabilization parameter δ

Output: Final weight vector $\mathbf{w}$

```

 $n \leftarrow 1$ 
 $\mathbf{s} \leftarrow \mathbf{0}$ 
 $\mathbf{r} \leftarrow \mathbf{0}$ 
repeat
  Choose a mini-batch at random from  $\mathcal{D}$ 
   $\mathbf{g} = -\nabla E_{n:n+B-1}(\mathbf{w})$  // evaluate gradient vector
   $\mathbf{s} \leftarrow \beta_1 \mathbf{s} + (1 - \beta_1) \mathbf{g}$ 
   $\mathbf{r} \leftarrow \beta_2 \mathbf{r} + (1 - \beta_2) \mathbf{g} \odot \mathbf{g}$  // element-wise multiply
   $\hat{\mathbf{s}} \leftarrow \mathbf{s} / (1 - \beta_1^n)$  // bias correction
   $\hat{\mathbf{r}} \leftarrow \mathbf{r} / (1 - \beta_2^n)$  // bias correction
   $\Delta \mathbf{w} \leftarrow -\eta \frac{\hat{\mathbf{s}}}{\sqrt{\hat{\mathbf{r}} + \delta}}$  // element-wise operations
   $\mathbf{w} \leftarrow \mathbf{w} + \Delta \mathbf{w}$  // weight vector update
   $n \leftarrow n + B$ 
  if  $n + B > N$  then
    shuffle data
     $n \leftarrow 1$ 
  end if
until convergence
return  $\mathbf{w}$ 

```

Optimización: Normalización

1

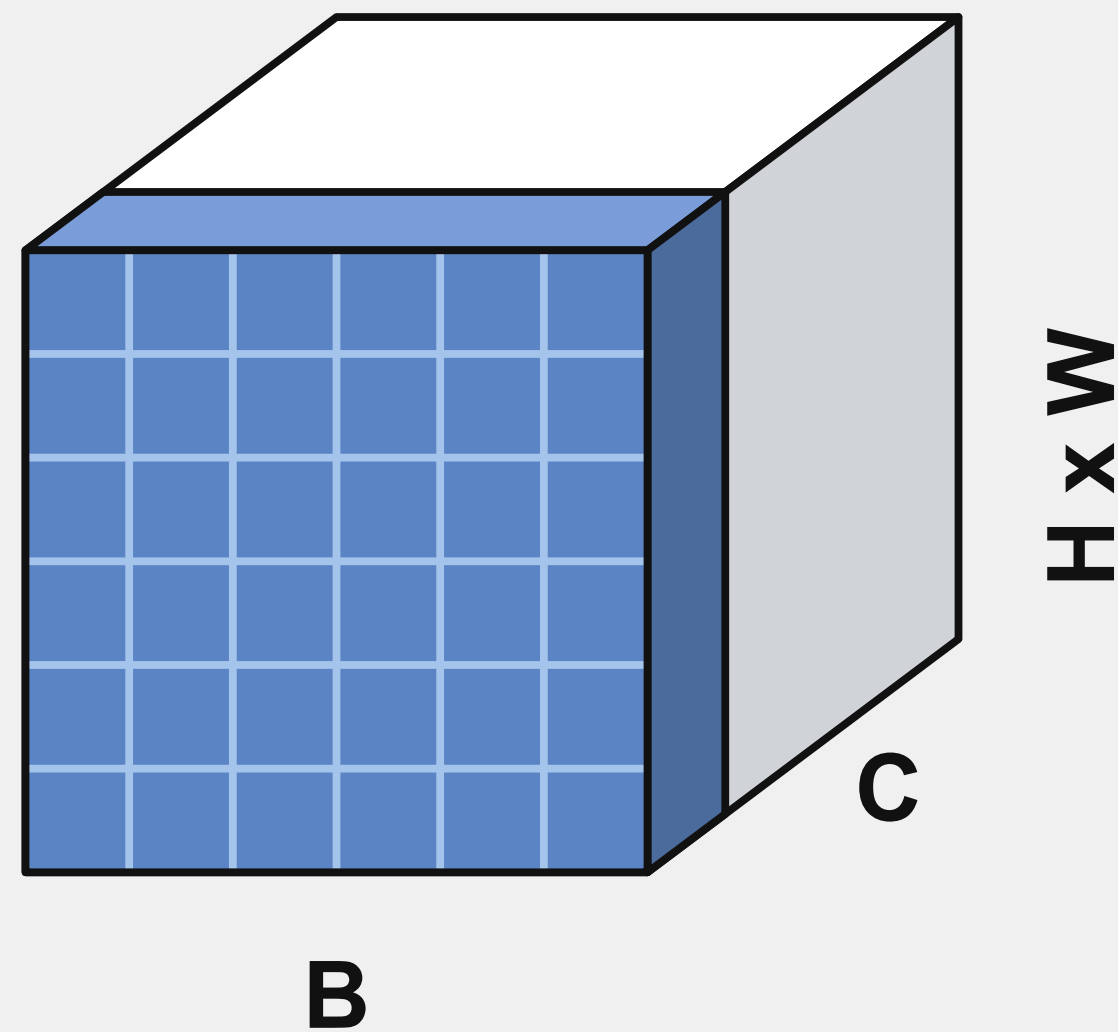
$$\tilde{x}_{ni} = \frac{x_{ni} - \mu_i}{\sigma_i}$$

$$\mu_i = \frac{1}{N} \sum_{n=1}^N x_{ni}$$
$$\sigma_i^2 = \frac{1}{N} \sum_{n=1}^N (x_{ni} - \mu_i)^2$$

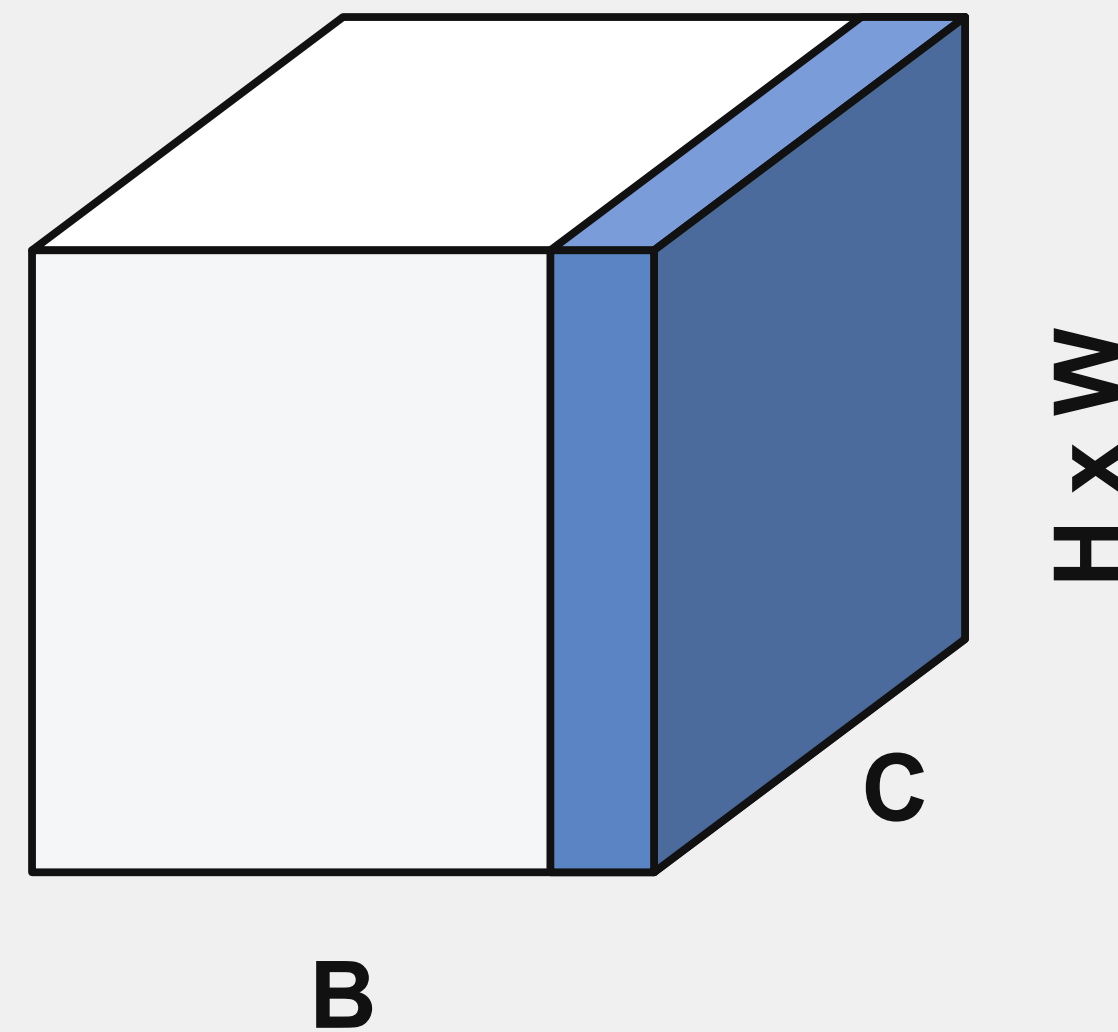
La normalización de las variables calculadas durante el paso hacia adelante (forward pass) a través de una red neuronal elimina la necesidad de que la red tenga que lidiar con valores extremadamente grandes o extremadamente pequeños.

Optimización: Normalización

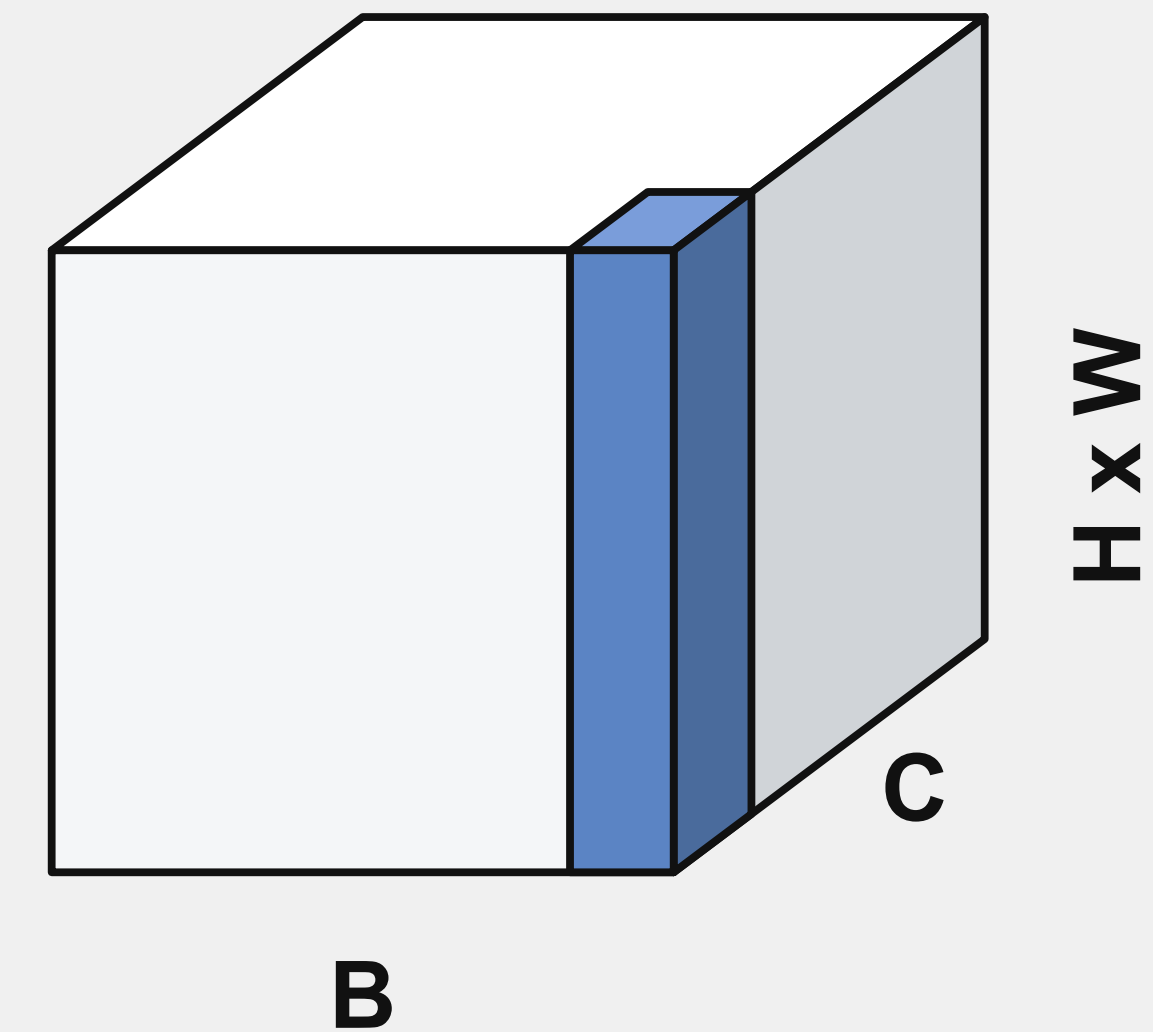
Normalización por Lotes



Normalización de Capa



Normalización de Instancia



Aunque en principio los pesos y sesgos de una red neuronal pueden adaptarse a cualquier valor que adopten las variables de entrada y ocultas, en la práctica la normalización puede ser crucial para asegurar un entrenamiento efectivo. Se consideran tres tipos de normalización dependiendo de si normalizamos a lo largo de los datos de entrada, a través de mini-lotes (mini-batches), o a través de las capas.

Normalización: Estandarización

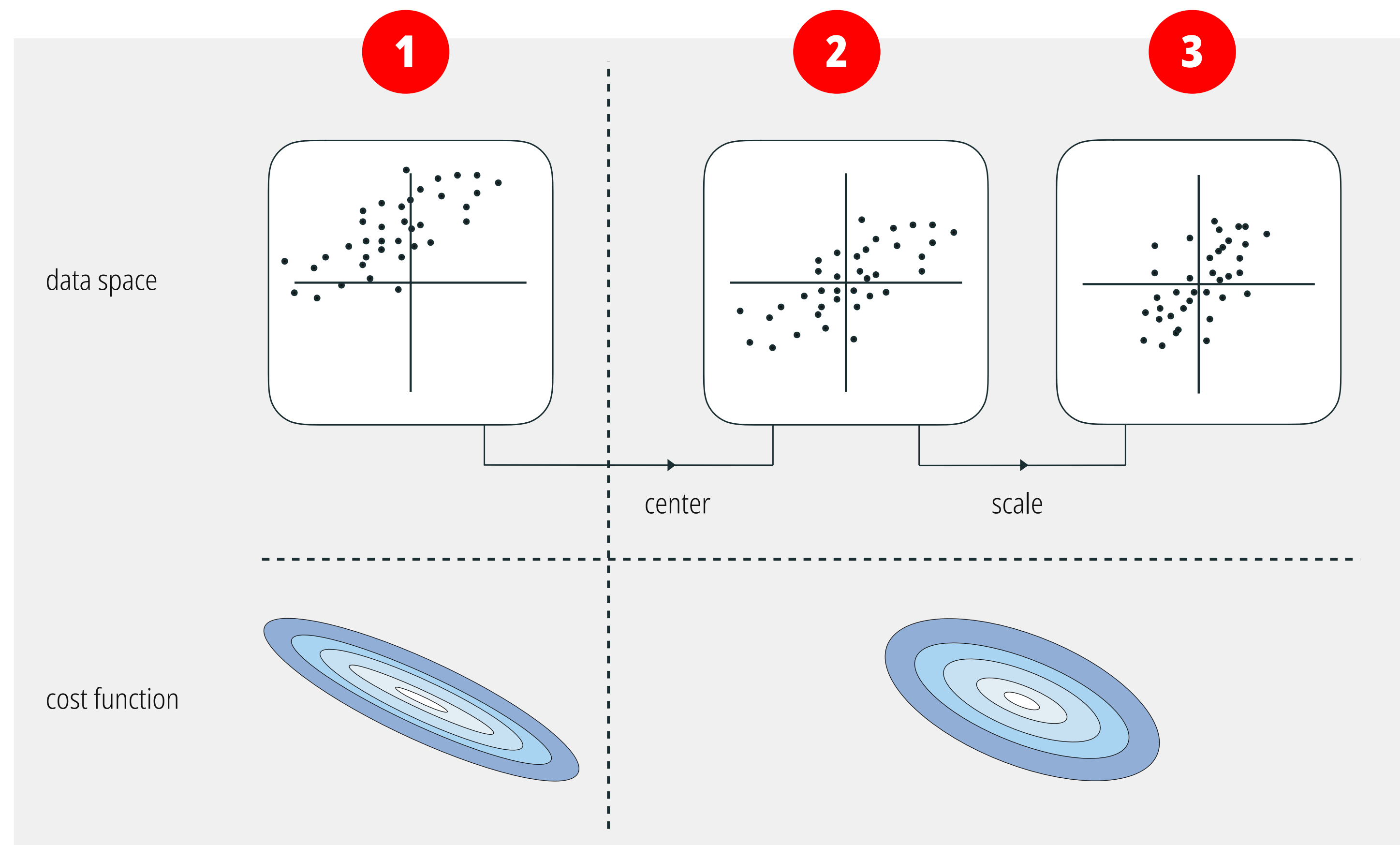
Se ilustra gráficamente el proceso de normalización estándar (cc. estandarización o cálculo del Z-score) y por qué es fundamental en el aprendizaje automático, especialmente para algoritmos de optimización.

Estado 1: Datos Originales. Muestra los datos crudos. Tienen una escala arbitraria, no están centrados en el origen (0,0) y tienen una dispersión diferente a lo largo del eje X y el eje Y.

Estado 2: Centrado. Se calcula la media de cada característica (eje) y se le resta a cada punto de datos. Esto traslada la "nube" de puntos para que su centro de gravedad exacto esté en el origen del plano cartesiano (0,0).

Estado 3: Escalado. Cada punto se divide por la desviación estándar de su respectiva característica. Esto "comprime" o "estira" los ejes para que ambas variables tengan la misma escala (una varianza de 1). La nube de puntos ahora se ve más redonda y simétrica, sin que un eje domine sobre el otro.

Cuando un algoritmo de optimización (como el Descenso de Gradiente) intenta encontrar el punto más bajo (el mínimo) en esta superficie (no normalizada), rebotará de lado a lado por las paredes del valle, avanzando muy lentamente hacia el centro. Al normalizar los datos, la función de costo se transforma en una forma de "tazón" mucho más simétrica y circular. En esta superficie, el gradiente apunta de manera mucho más directa hacia el centro (el mínimo global).

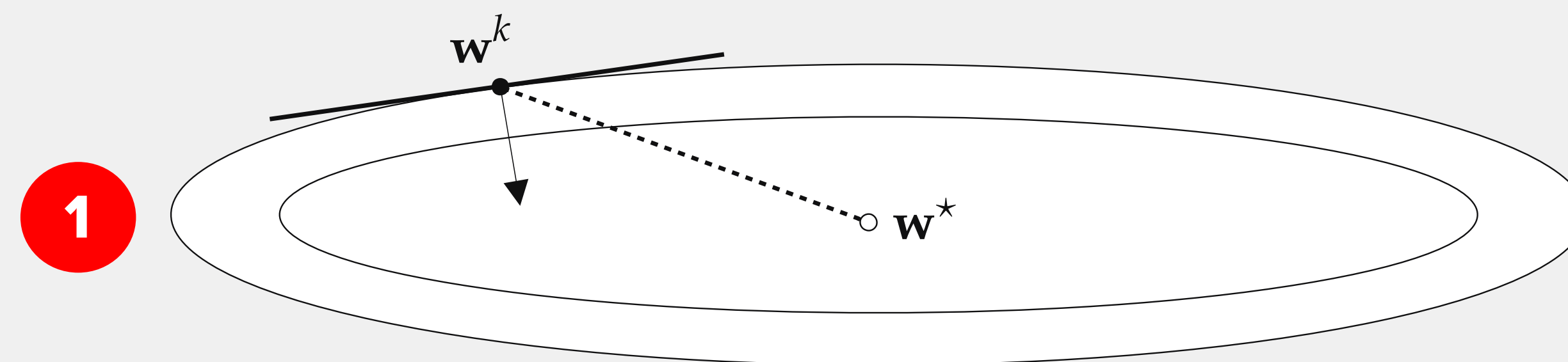


El espacio de entrada de un conjunto de datos genérico (panel superior izquierdo), así como su versión centrada en la media (panel superior central) y escalada (panel superior derecho). Como se ilustra en la fila inferior, donde se muestra una función de costo prototípica correspondiente a estos datos, la normalización estándar da como resultado una función de costo con contornos menos elípticos y más circulares en comparación con la función de costo original.

Normalización (cont.): Estandarización

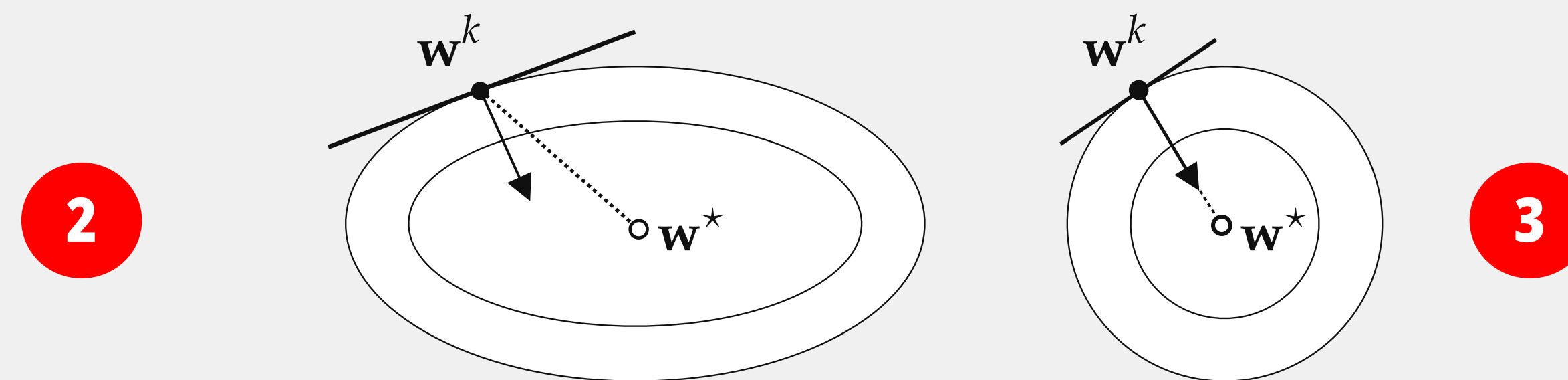
Hacer que los contornos de una función de costo de aprendizaje automático sean más circulares ayuda a acelerar la convergencia de los esquemas de optimización local, particularmente los métodos de primer orden como el descenso de gradiente.

1. Datos sin normalizar: La función de costo es una elipse muy alargada. Al trazar la dirección perpendicular al contorno (la flecha sólida), vemos que apunta en una dirección muy diferente a la ruta ideal (la flecha punteada). Esto obligará al algoritmo a rebotar de arriba a abajo por el "valle", avanzando hacia w^* de manera muy ineficiente (el efecto zigzag).



2. Normalización parcial o mejor escalado: Los contornos son elípticos pero menos extremos. Aquí, la flecha sólida (el paso del gradiente) y la flecha punteada (la ruta ideal) comienzan a alinearse mejor. El zigzag será menor.

3. Normalización estándar óptima: Los datos han sido completamente estandarizados, creando contornos perfectamente circulares. Debido a la geometría del círculo, la línea perpendicular a la tangente en cualquier punto siempre apuntará directamente hacia el centro. Aquí, la flecha sólida y la punteada son exactamente la misma. El algoritmo irá directo hacia w^* sin desviarse.



Al aplicar la normalización estándar a los datos de entrada, moderamos los contornos a menudo elípticos de su función de costo asociada, como los que se muestran en el panel superior, transformándolos en formas más circulares como se muestra en los paneles inferior izquierdo e inferior derecho. Esto significa que la dirección del descenso de gradiente, que apunta lejos del minimizador de una función de costo cuando sus contornos son elípticos (lo que conduce al problema común de zigzag con el descenso de gradiente), apunta más hacia el minimizador de la función a medida que sus contornos se vuelven más circulares. Esto hace que cada paso del descenso de gradiente sea mucho más efectivo, lo que típicamente permite el uso de valores mucho más grandes para el parámetro de longitud de paso (tasa de aprendizaje), lo que significa que se requieren considerablemente menos pasos para minimizar adecuadamente la función de costo.

NOTA: Las técnicas de escalado de características también pueden ayudar a acondicionar mejor un conjunto de datos para su uso con métodos de segundo orden, ayudando potencialmente a evitar problemas de inestabilidad numérica

Escalamiento: PCA-Scaling (whitening)

El Whitening (o esferización mediante PCA) es una técnica avanzada de preprocesamiento de datos en Machine Learning que va un paso más allá de la normalización estándar. Su objetivo principal es transformar un conjunto de datos para que sus variables (características) cumplan dos condiciones matemáticas exactas

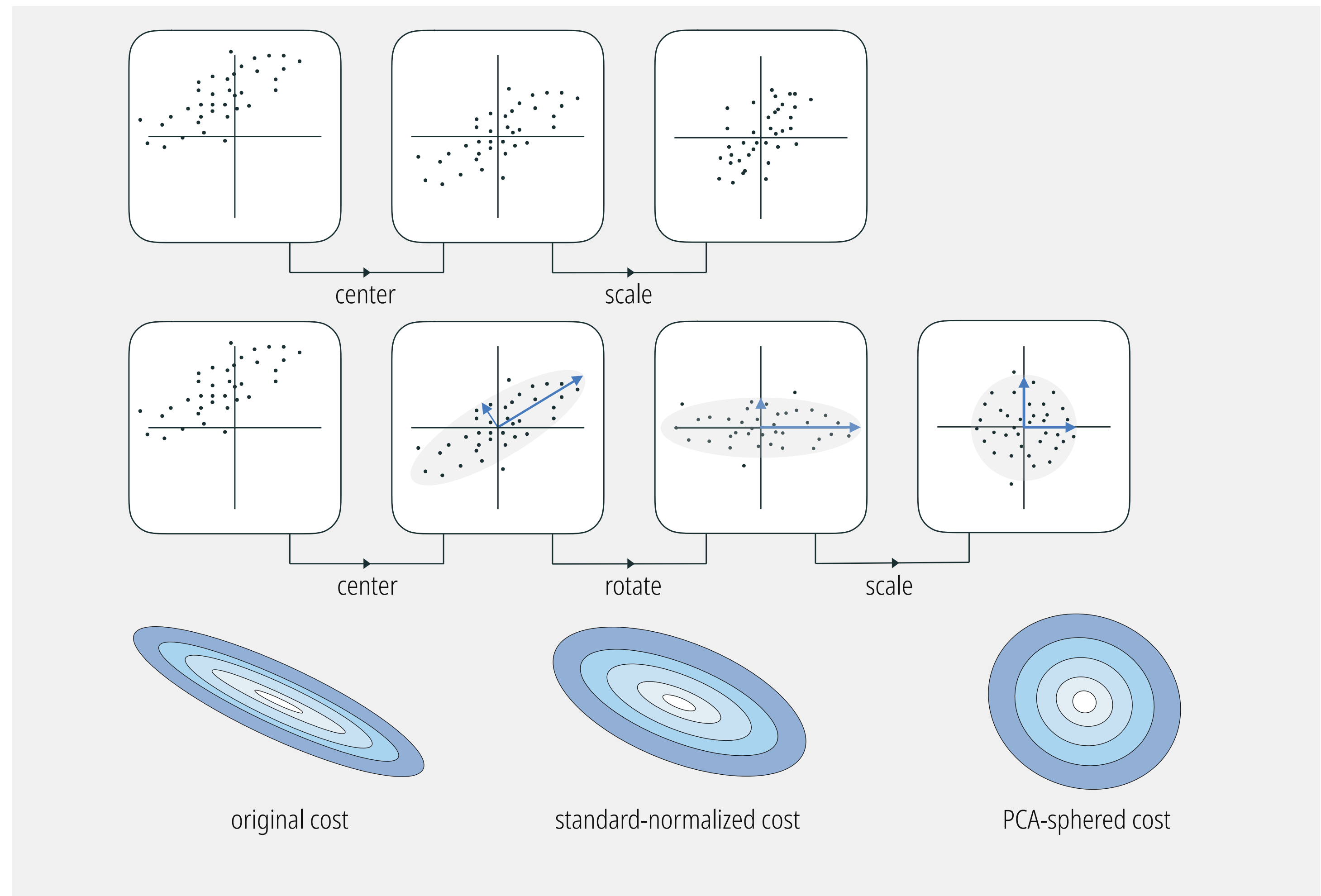
Condiciones:

Varianza unitaria: Todas las variables tienen una varianza de 1 (igual que en la normalización estándar).

Cero correlación: Ninguna de las variables está correlacionada con las demás (la covarianza entre cualquier par de variables distintas es 0).

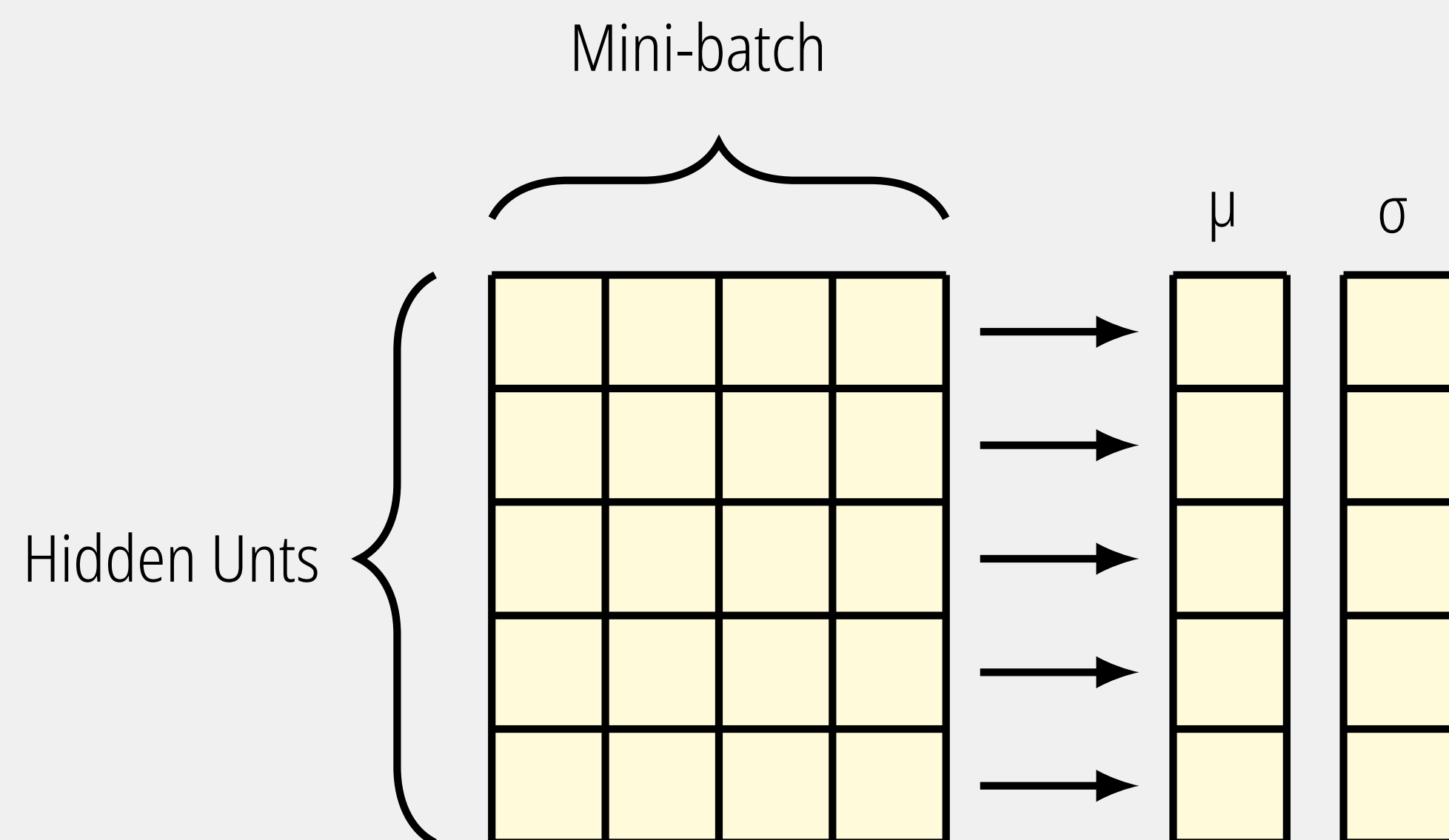
El procedimiento de normalización estándar (fila superior) comparado con la esferización mediante PCA (fila central) en un conjunto genérico de datos de entrada.

Con la esferización mediante PCA, insertamos un único paso adicional en el proceso de normalización estándar, entre el centrado respecto a la media y el escalado por desviaciones estándar, donde rotamos los datos utilizando PCA. Esto no solo reduce el espacio consumido por los datos más que la normalización estándar (compare los paneles superior derecho y central derecho), sino que también tiende a hacer que cualquier función de costo asociada sea considerablemente más fácil de minimizar al moderar mejor sus contornos, haciéndolos más circulares.



Optimización: Normalización por lotes

Calcula la media y la varianza a través del lote.



Toma una sola neurona (una fila) y analiza los valores de activación de esa neurona específica para todos los ejemplos presentes en el lote actual.

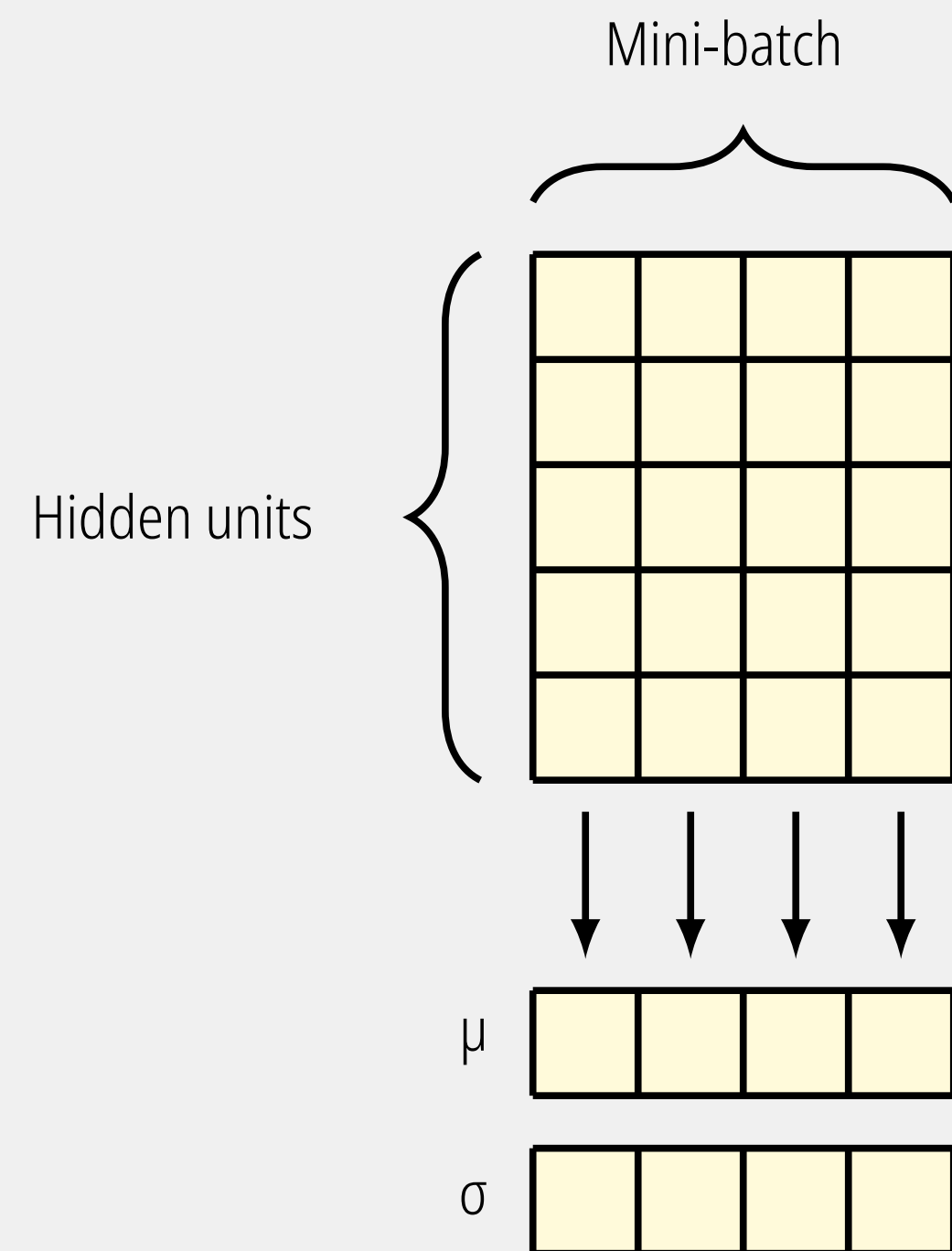
Obtiene un valor de la media y la varianza independiente para cada neurona de la capa.

Es el estándar histórico para redes neuronales convolucionales (visión por computadora). Su principal debilidad es que depende fuertemente de que el tamaño del lote (mini-batch) sea lo suficientemente grande; si el lote es muy pequeño (ej. 2 o 4), las estadísticas calculadas son inexactas y el modelo se vuelve inestable.

Batch Norm asegura que una característica particular se comporte de manera estable frente a distintos ejemplos.

Optimización: Normalización por capa

Calcula la media y la varianza a través de las neuronas.



Toma un solo ejemplo de datos (una columna) y promedia los valores de todas las neuronas de esa capa para ese ejemplo en particular.

Obtiene un valor de media y varianza independiente para cada ejemplo en el lote.

Al no depender del tamaño del lote, funciona perfectamente incluso si procesas los datos de uno en uno.

Por esto, es el estándar absoluto en arquitecturas de Procesamiento de Lenguaje Natural (como la arquitectura Transformer que usan los LLMs) y redes recurrentes.

Layer Norm asegura que la representación interna de un ejemplo único sea estable a través de todas sus características.

Regularización (General)

La elección entre estas geometrías (especialmente $q = 1$ vs $q = 2$) define si el modelo resultante será una matriz densa continua o una estructura esparcida donde los nodos irrelevantes se eliminan por completo.

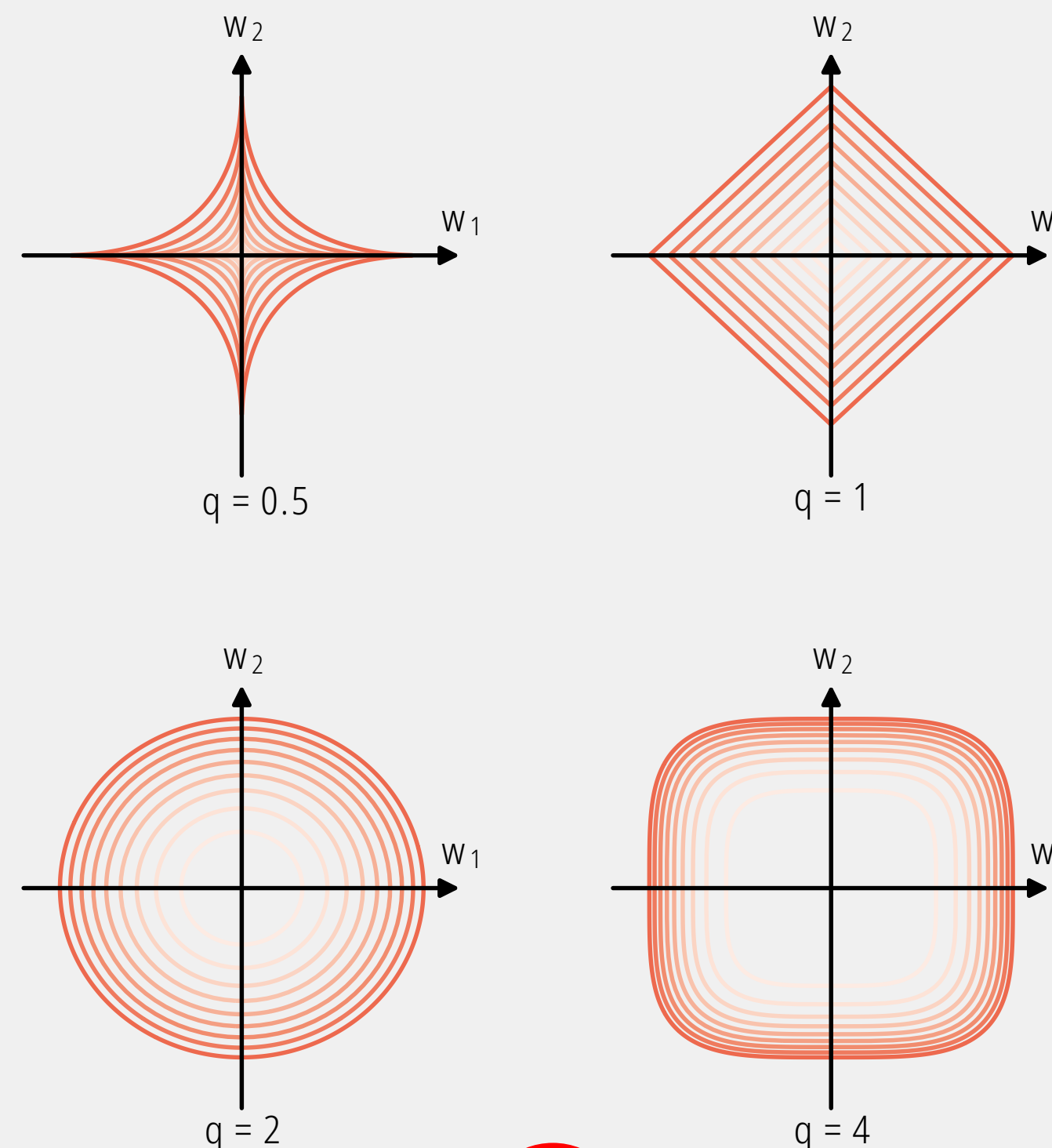
1

$$E(\mathbf{w}) + \frac{\lambda}{2} \sum_{j=1}^M |w_j|^q$$

Lambda es el hiperparámetro que dicta la "fuerza" de la regularización y q es el valor que define la forma geométrica de la penalización (la norma).

2

$$\sum_{j=1}^M |w_j|^q \leq \eta$$



3

$q = 2$ (Regularización L2 / Ridge): La optimización encuentra el punto donde el error toca este círculo. Reduce la magnitud de todos los pesos proporcionalmente, pero casi nunca los lleva a cero exacto. Es computacionalmente muy estable y diferenciable en todos sus puntos, ideal para cálculos vectoriales rápidos.

$q = 1$ (Regularización L1 / Lasso): Debido a las esquinas afiladas que descansan exactamente sobre los ejes, el punto óptimo de error tiene una alta probabilidad de golpear una de estas esquinas. Cuando esto sucede, uno de los pesos se vuelve exactamente 0. Esto introduce "esparcidad" (sparsity), actuando como un selector de características natural.

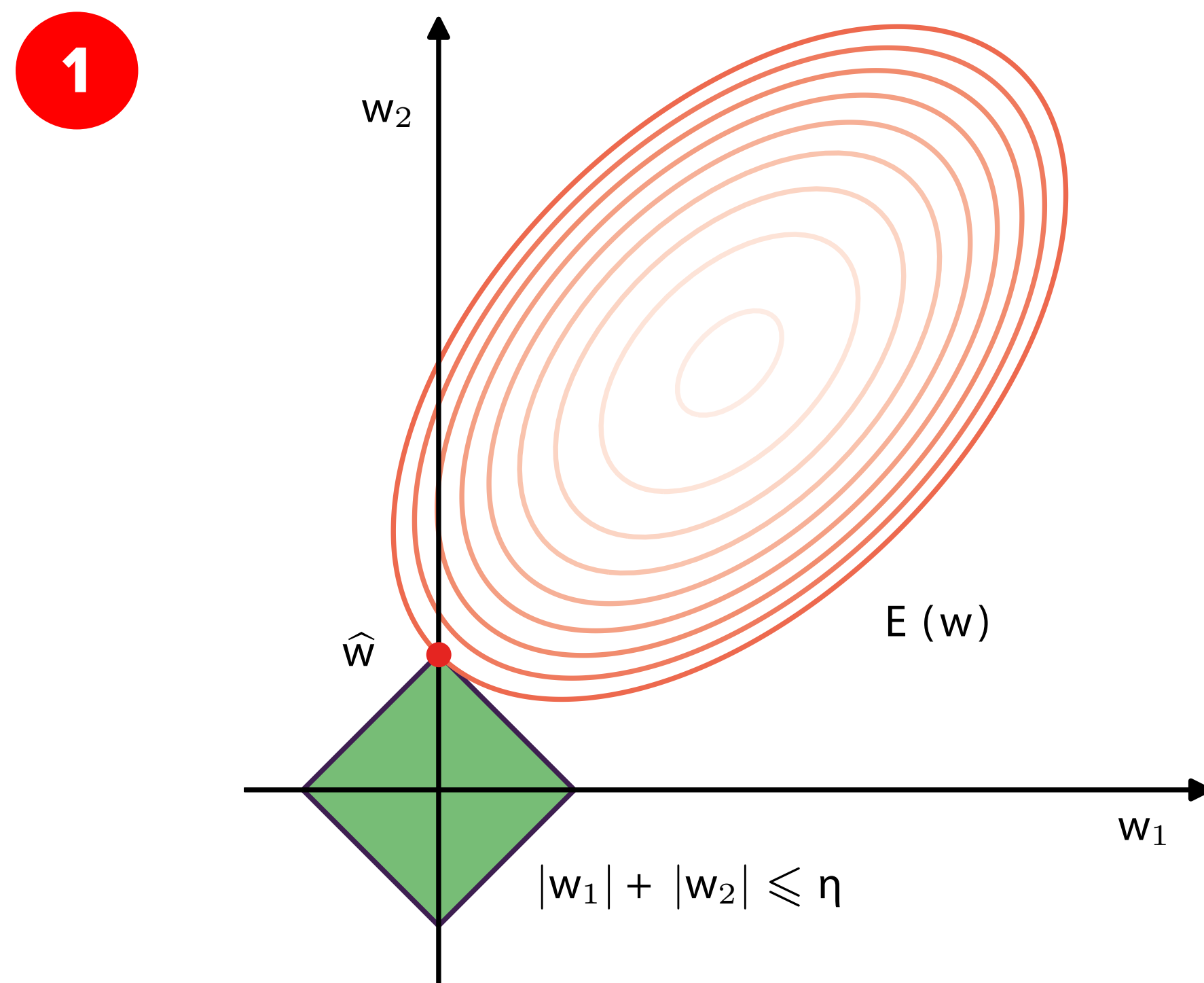
$q = 0.5$ (Normas fraccionarias): Fuerza una esparcidad extrema, "apagando" agresivamente la mayoría de los pesos y dejando solo los absolutamente críticos. Sin embargo, al ser un espacio no convexo y no diferenciable en el origen, su implementación algorítmica es compleja y propensa a quedar atrapada en mínimos locales.

$q = 4$: Tiende hacia un cuadrado con esquinas redondeadas. En la práctica, penaliza fuertemente cualquier peso que intente superar un umbral específico, manteniendo los pesos agrupados, pero rara vez se usa en comparación con L1 o L2.

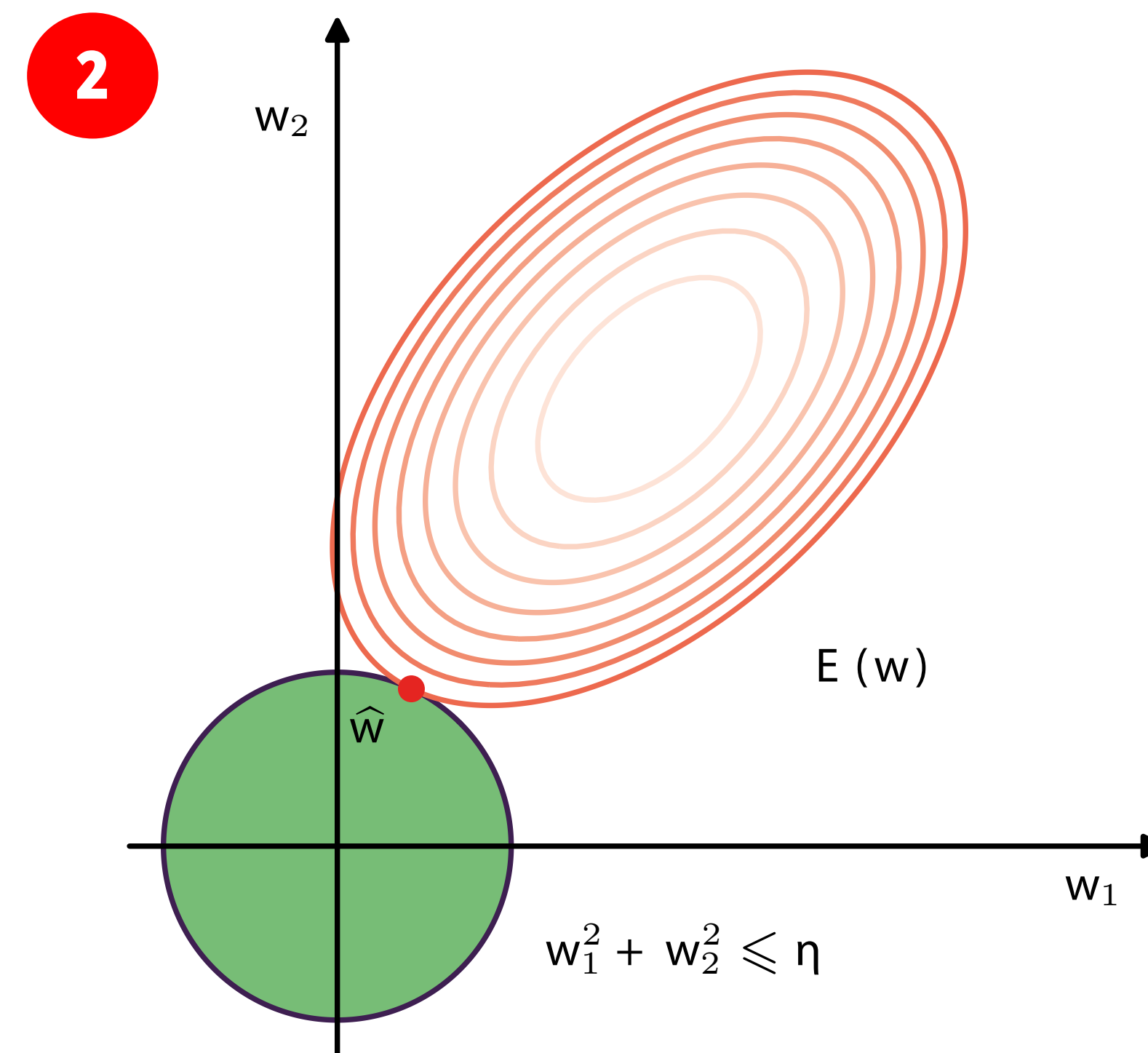
Figura 1: Es la función de error empírico (como el Error Cuadrático Medio o la Entropía Cruzada). Representa qué tan bien se ajusta el modelo a los datos. Figura 2: Lo mismo, pero expresado como un problema de optimización con restricciones. Matemáticamente, minimizar la ecuación del paso 1 es equivalente a decir: "Encuentra los pesos w que minimicen el error $E(w)$, pero con la estricta condición de que el vector de pesos no puede salir de un volumen definido por la tasa de aprendizaje"

Regularización (General)

El mecanismo por el cual el optimizador (como el Descenso de Gradiente) encuentra el equilibrio final entre reducir el error del modelo y cumplir con la penalización de la regularización.



A medida que los óvalos de error se expanden desde su centro buscando tocar el área verde, tienen una altísima probabilidad estadística de golpear primero una de esas esquinas prominentes.



Cuando los óvalos de error se expanden, tocarán el círculo en un punto de tangencia arbitrario. La probabilidad de que este punto exacto caiga matemáticamente sobre un eje (donde un peso es cero) es extremadamente baja.

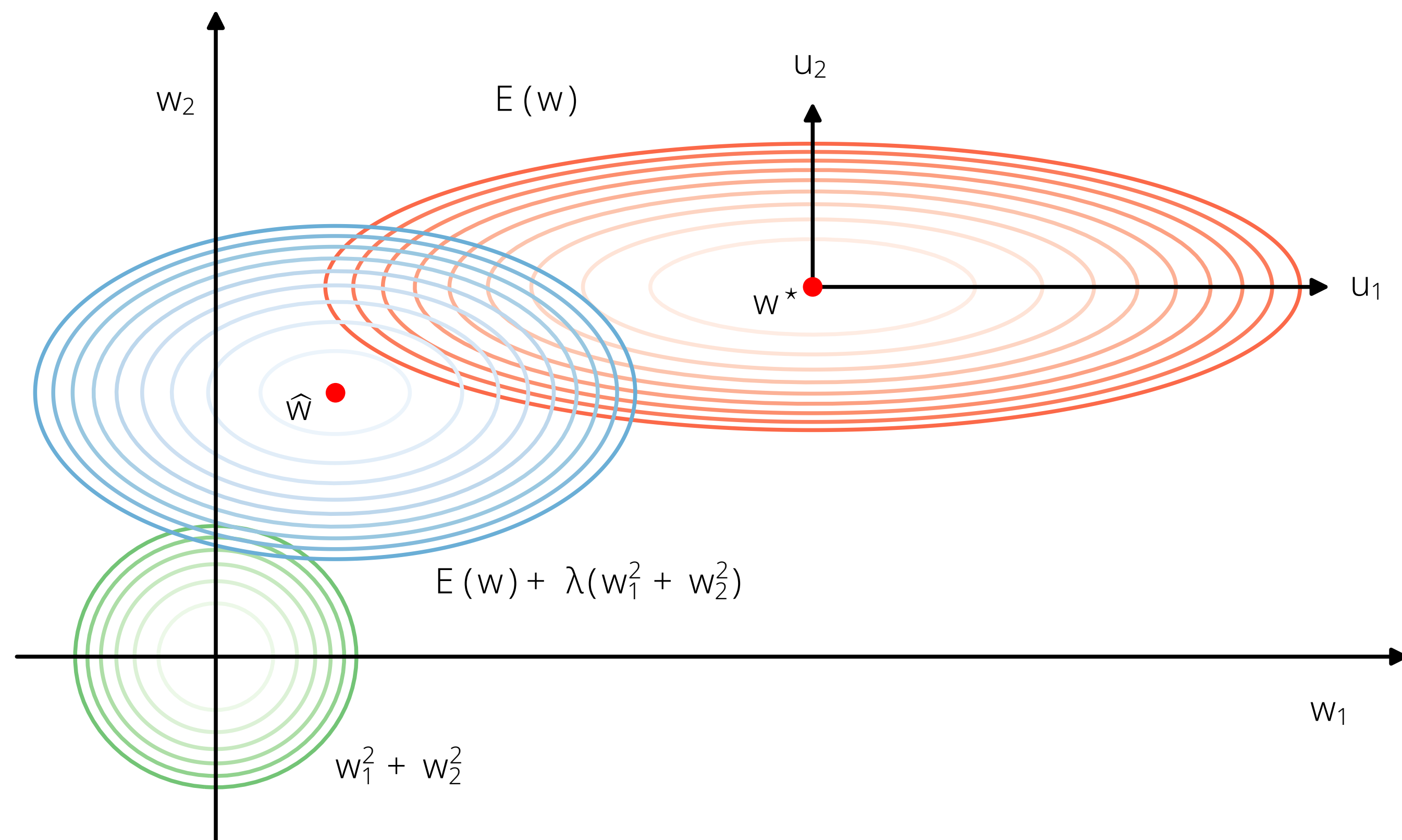
Weight Decay (Regularización L2)

Para entenderlo de forma intuitiva, piensa en el entrenamiento de un modelo como un juego de tira y afloja entre dos objetivos opuestos: ajustarse a los datos lo mejor posible y mantener el modelo lo más simple posible (evitando pesos muy grandes que causen sobreajuste).

Las Curvas Rojas (El objetivo original): Es la función de error sin regularizar. Imagina que es un valle. El punto rojo central, es el fondo del valle: los valores exactos para los parámetros que te dan el menor error en tus datos de entrenamiento. Nota que estas curvas rojas son elípticas y alargadas horizontalmente. Esto significa que si cambias el valor de w_1 (moviéndote a izquierda o derecha), el error casi no sube; el modelo es poco sensible a w_1 . Pero si cambias w_2 (moviéndote arriba o abajo), cruzas muchas líneas de contorno rápidamente; el modelo es muy sensible a w_2 .

Las Curvas Verdes (La penalización): Es el término del "Weight Decay" o Regularización L2, matemáticamente expresado como $w_1^2 + w_2^2$. La forma: Son círculos perfectos centrados en el origen (0,0). Este término "tira" de todos los parámetros hacia el cero. Su filosofía es: "Un parámetro solo debe alejarse de cero si realmente justifica su existencia mejorando mucho el error".

Las Curvas Azules (El compromiso final): Es la suma de las dos fuerzas anteriores y el parámetro lambda decide qué tan fuerte tira la cuerda verde.



En la creación de sistemas robustos —como cuando se desarrollan modelos predictivos o perfiles de comportamiento que necesitan ignorar el "ruido" y enfocarse en la señal verdadera—, el Weight Decay actúa como un filtro natural. Silencia los parámetros inútiles o ruidosos (w_1) y preserva la fuerza de las variables que realmente contienen información crítica (w_2).

Optimización: Backpropagation

Algorithm 4: Backpropagation

Input: Input vector $\mathbf{x}_n$
 Network parameters $\mathbf{w}$
 Error function $E_n(\mathbf{w})$ for input x_n
 Activation function $h(a)$

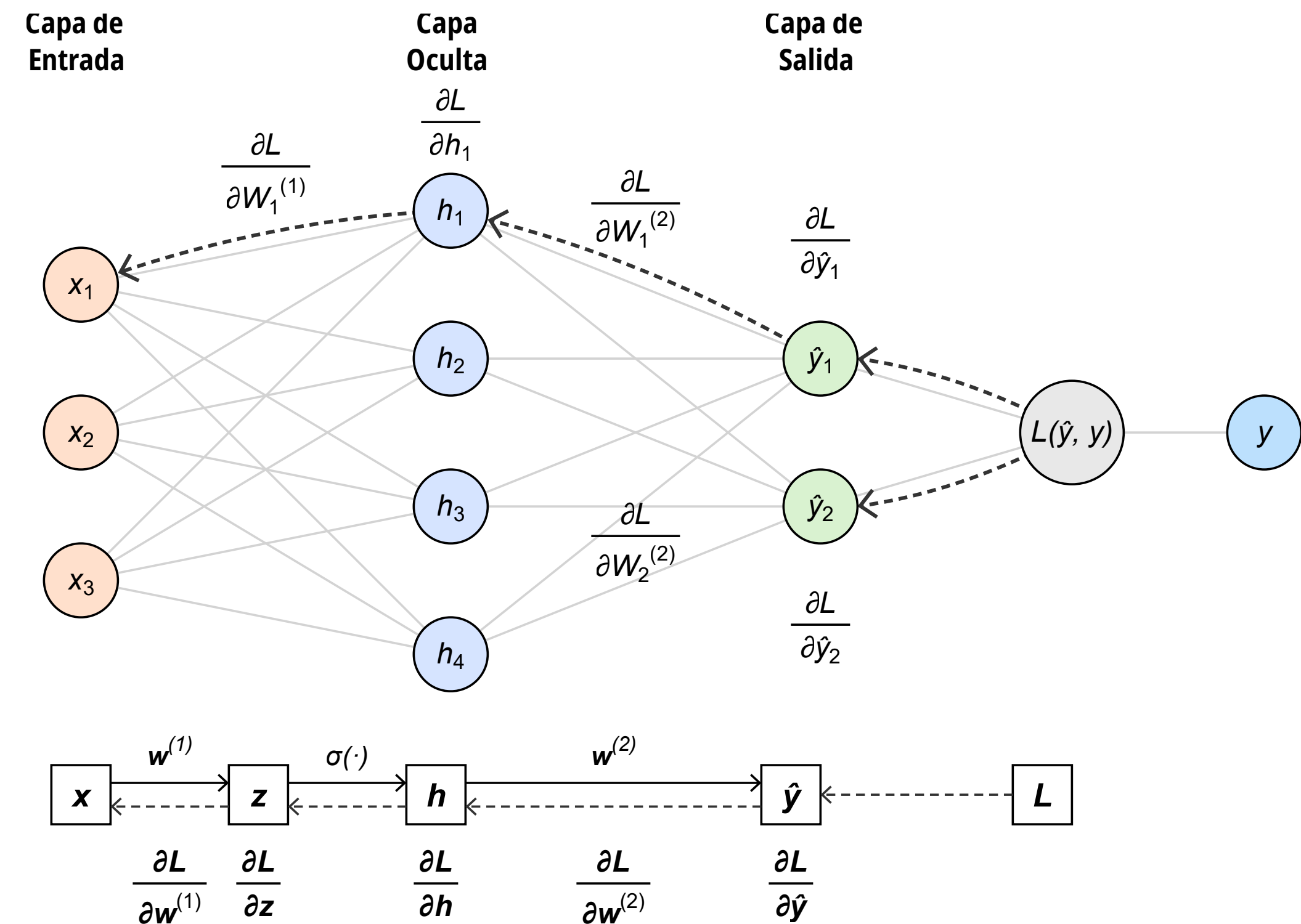
Output: Error function derivatives $\{\partial E_n / \partial w_{ji}\}$

// Forward propagation
 for $j \in$ all hidden and output units do
 $a_j \leftarrow \sum_i w_{ji} z_i$ // $\{z_i\}$ includes inputs $\{x_i\}$
 $z_j \leftarrow h(a_j)$ // activation function
 end for

// Error evaluation
 for $k \in$ all output units do
 $\delta_k \leftarrow \frac{\partial E_n}{\partial a_k}$ // compute errors
 end for

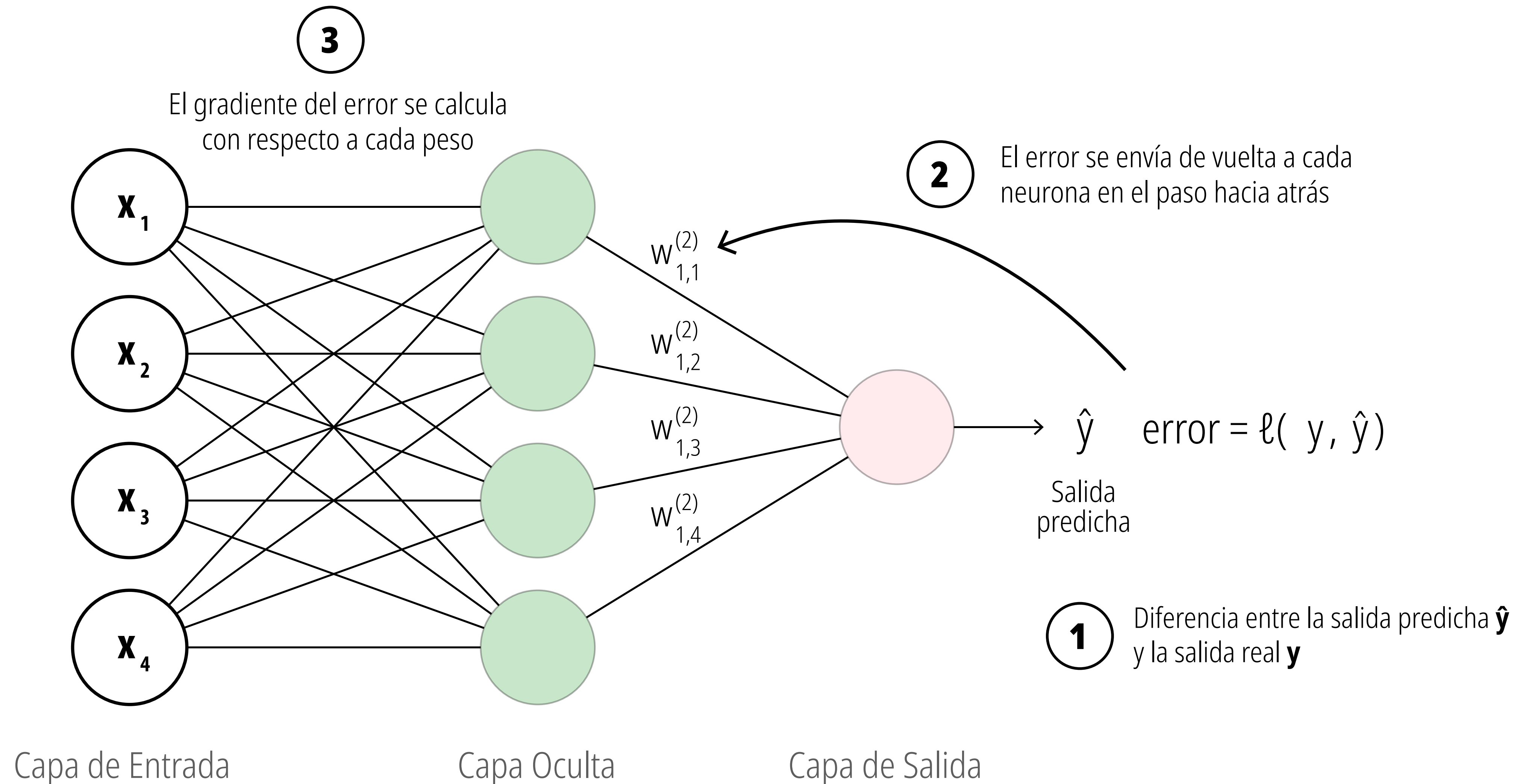
// Backward propagation, in reverse order
 for $j \in$ all hidden units do
 $\delta_j \leftarrow h'(a_j) \sum_k w_{kj} \delta_k$ // recursive backward evaluation
 $\frac{\partial E_n}{\partial w_{ji}} \leftarrow \delta_j z_i$ // evaluate derivatives
 end for

return $\left\{ \frac{\partial E_n}{\partial w_{ji}} \right\}$



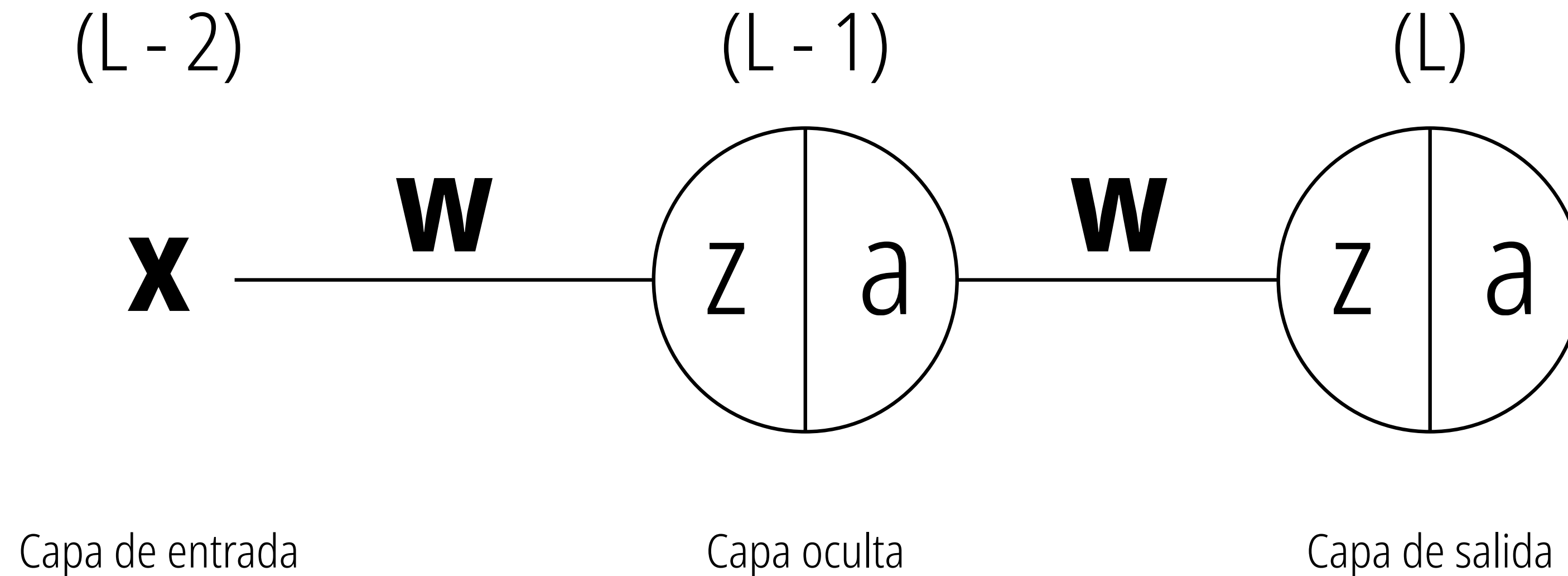
Propagación hacia atrás (backpropagation)

La fase de propagación hacia adelante implica “encadenar” todos los pasos que hemos definido hasta ahora: la función lineal, la función sigmoide y la función de umbral.



Propagación hacia atrás (backpropagation)

Se puede pensar en esto como tener una red con una sola unidad de entrada, una sola unidad oculta y una sola unidad de salida. Primero deduciremos la retropropagación para esta red simplificada y luego la generalizaremos para el caso de múltiples neuronas.



El propósito principal de la retropropagación es responder a la siguiente pregunta: “¿Cómo cambia el error cuando cambiamos los pesos en una cantidad diminuta?” (tenga en cuenta que usaré las palabras “derivadas” y “gradientes” indistintamente).

Propagación hacia atrás (backpropagation)

Para lograr esto, es necesario comprender lo siguiente:

- El error E depende del valor de la función de activación sigmoide a .
- El valor de la función de activación sigmoide a depende del valor de la función lineal z .
- El valor de la función lineal z depende del valor de los pesos w .

$$\left\{ \frac{\partial E}{\partial w^{(L)}} \right\} = \overbrace{\frac{\partial E}{\partial a^{(L)}}}^{\text{Derivada parcial del error con respecto a la activación sigmoide}} \times \underbrace{\frac{\partial a^{(L)}}{\partial z^{(L)}}}_{\text{Derivada parcial de la activación sigmoide con respecto a la función lineal}} \times \overbrace{\frac{\partial z^{(L)}}{\partial w^{(L)}}}^{\text{Derivada parcial de la función lineal con respecto al peso}}$$

↑
Índice de la capa actual

Propagación hacia atrás (backpropagation)

$$\frac{\partial E}{\partial w_{jk}^{(L)}} = \frac{\partial E}{\partial a_j^{(L)}} \times \frac{\partial a_j^{(L)}}{\partial z_j^{(L)}} \times \frac{\partial z_j^{(L)}}{\partial w_{jk}^{(L)}}$$



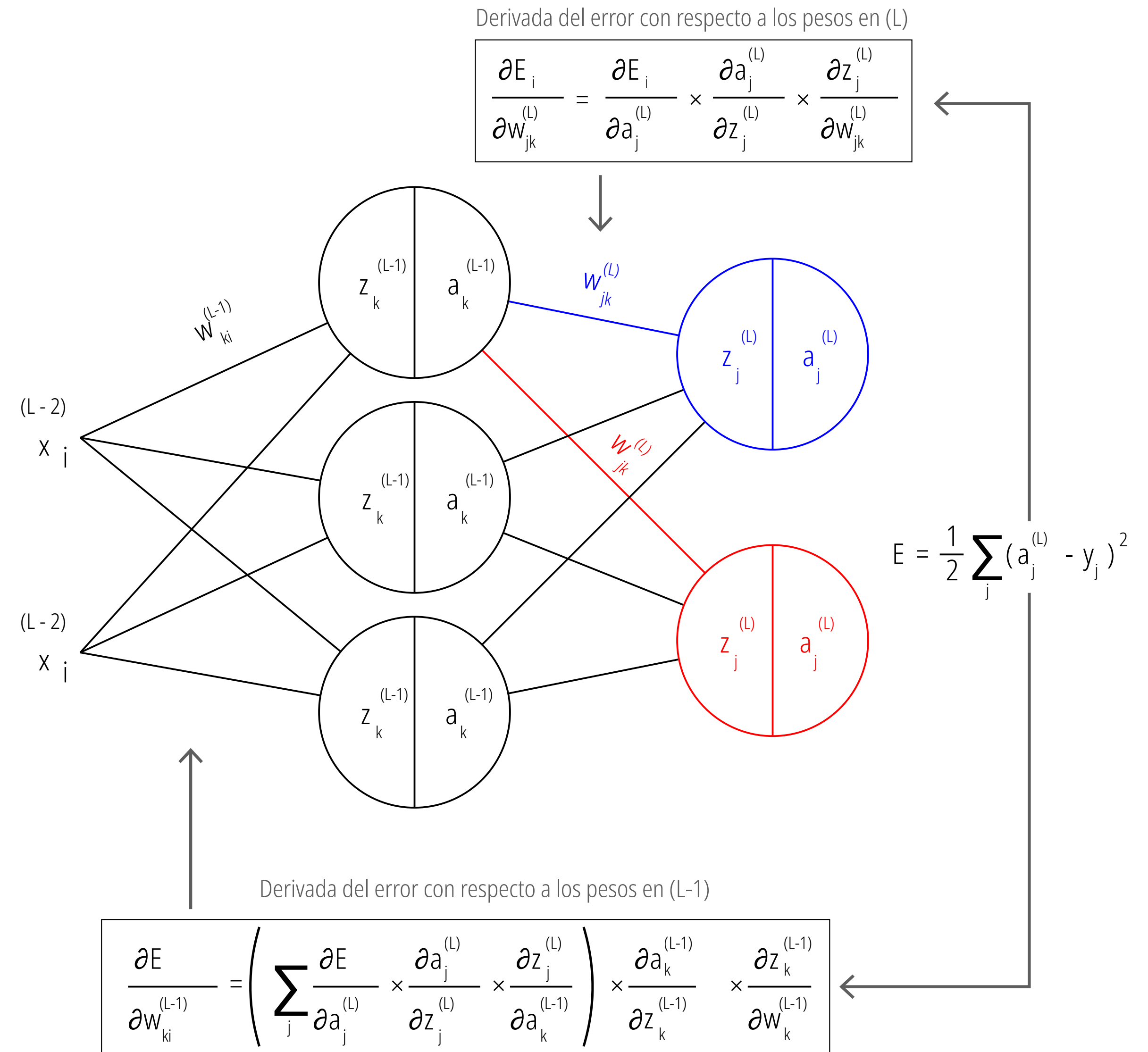
$$\frac{\partial E}{\partial w_{jk}^{(L)}} = a_j^{(L)} - y \times a_j^{(L)} (1 - a_j^{(L)}) \times a_k^{(L-1)}$$



$$\frac{\partial E}{\partial a_k^{(L-1)}} = \sum_j \frac{\partial E}{\partial a_j^{(L)}} \times \frac{\partial a_j^{(L)}}{\partial z_j^{(L)}} \times \frac{\partial z_j^{(L)}}{\partial a_k^{(L-1)}}$$

Utilizando la regla de la cadena del cálculo diferencial, el algoritmo determina exactamente qué fracción de culpa tiene cada peso y sesgo en el error total, permitiendo actualizar hasta la capa más profunda de la red.

- **Cálculo de responsabilidades (Capa de salida L):** La caja superior muestra cómo se calcula el gradiente para un peso en la última capa. La red descompone el problema multiplicando tres derivadas más simples: cómo cambia el error respecto a la activación final, cómo cambia esa activación respecto a la suma lineal (z), y cómo cambia esa suma respecto al peso específico. Es una cadena matemática de causa y efecto.
- **El efecto multiplicador en capas ocultas (Capa L-1):** La caja inferior revela la complejidad real de las redes profundas. Para saber cómo ajustar un peso en una capa oculta, el modelo ya no observa una sola salida. Como ese peso alimenta a múltiples neuronas en la capa siguiente, la matemática exige calcular y sumar los gradientes que provienen de todas las rutas a las que ese peso afectó.



Actualización de Parámetros

- **Aplicando el Descenso de Gradiente en cascada:** Las ecuaciones de la derecha son la ejecución práctica de todo el cálculo anterior. Muestran la regla de actualización estándar aplicada a diferentes profundidades de la red: restarle al valor actual del parámetro la dirección de su gradiente correspondiente.
- **Unidad de aprendizaje en toda la red:** Observa cómo la tasa de aprendizaje o tamaño de paso (η) se mantiene constante y multiplica al gradiente sin importar si estamos ajustando un peso (w) de la última capa, un peso de la capa oculta, o un término de sesgo (b). Esto asegura que toda la red avance hacia la minimización del error de forma sincronizada y controlada.

Para los pesos en la capa (L):

$$w_{jk}^L = w_{jk}^L - \eta \times \frac{\partial E}{\partial w_{jk}^L}$$

Para los pesos en la capa (L-1):

$$w_{ki}^{L-1} = w_{ki}^{L-1} - \eta \times \frac{\partial E}{\partial w_{ki}^{L-1}}$$

Para el sesgo en la capa (L):

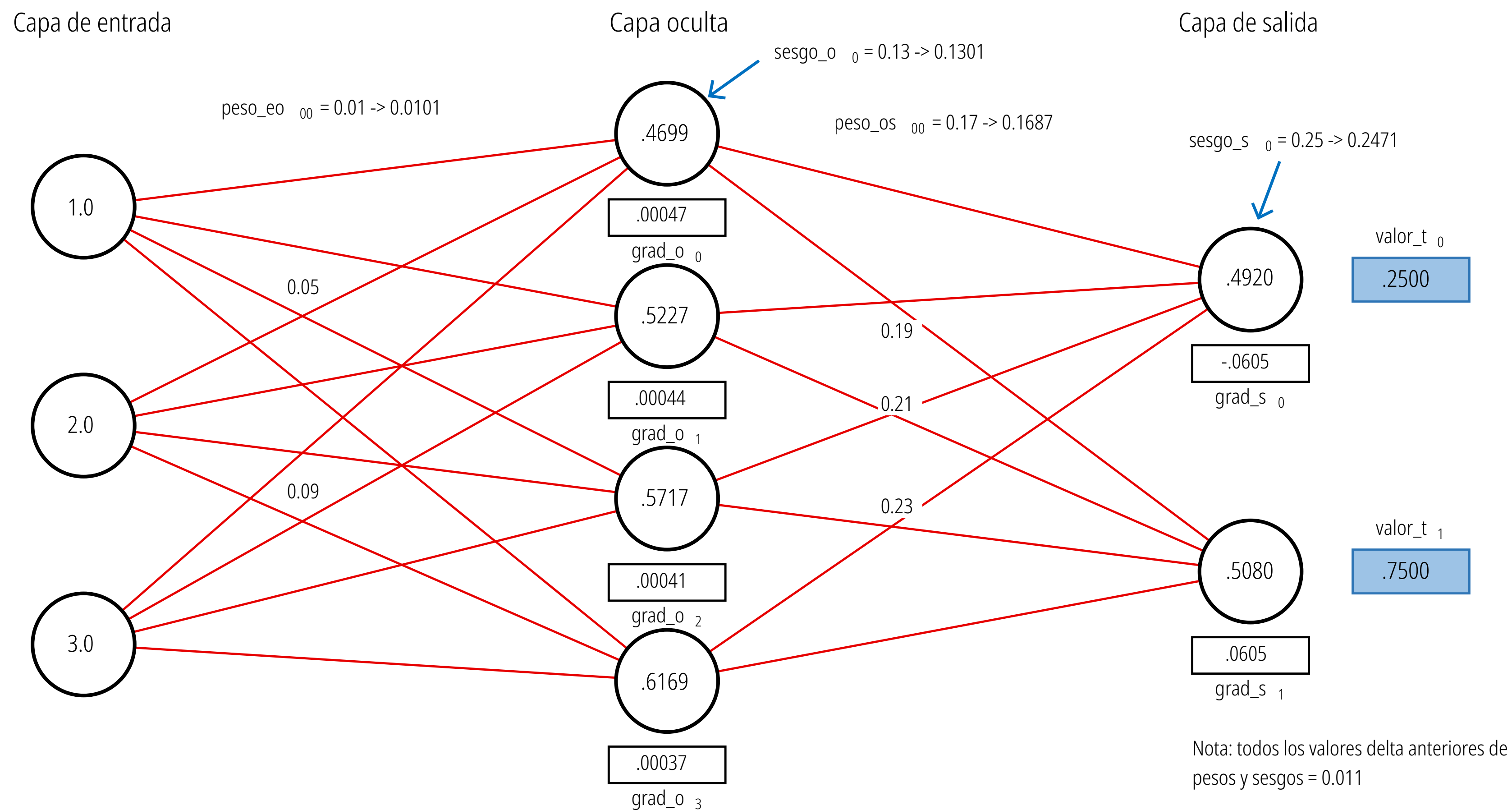
$$b^{(L)} = b^{(L)} - \eta \times \frac{\partial E}{\partial b^{(L)}}$$

Para el sesgo en la capa (L-1):

$$b^{(L-1)} = b^{(L-1)} - \eta \times \frac{\partial E}{\partial b^{(L-1)}}$$

Actualización de Parámetros

Micro-ajuste de un solo paso temporal



El Flujo Hacia Adelante (Predicción)

Arquitectura de la red: El modelo toma 3 valores de entrada (1.0, 2.0, 3.0), los procesa a través de una capa oculta de 4 neuronas, y emite 2 predicciones finales en la capa de salida.

Predicción vs. Realidad: En la extrema derecha, vemos la raíz del aprendizaje. La primera neurona de salida arrojó un valor de 0.4920, pero su objetivo real (valor_t_0) era 0.2500. La segunda arrojó 0.5080, pero debía llegar a 0.7500. Esta discrepancia matemática es lo que activa el algoritmo de corrección.

El Flujo Hacia Atrás (Retropropagación)

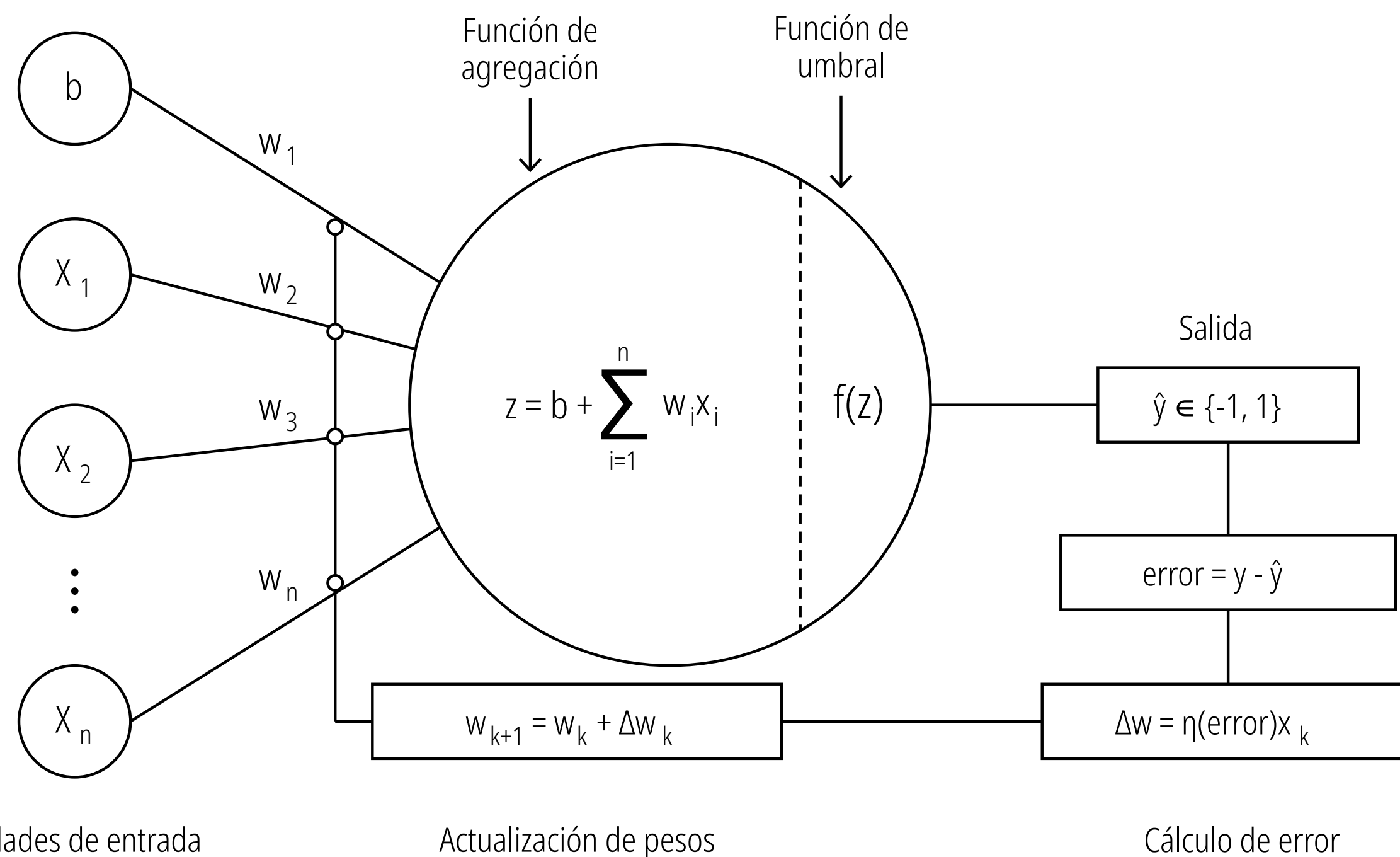
La cascada de gradientes: Los recuadros debajo de las neuronas (ej. grad_s_0 y grad_o_0) muestran la "culpa" matemática asignada a cada nodo. Observa cómo los gradientes en la capa de salida (ej. -0.0605) son más grandes, mientras que los gradientes que retroceden hacia la capa oculta (ej. 0.00047) se vuelven mucho más pequeños al diluirse por la regla de la cadena.

La actualización en acción: Las anotaciones con flechas (->) ilustran el resultado final del Descenso de Gradiente. La red toma los pesos y sesgos originales y les aplica una ligera corrección basada en esos gradientes.

Por ejemplo, un peso de salida (peso_os_{00}) se reduce de 0.17 a 0.1687. Un sesgo de salida (sesgo_s_0) baja de 0.25 a 0.2471. Un peso de entrada (peso_eo_{00}) aumenta pequeñísimamente de 0.01 a 0.0101.

El desarrollo de las redes neuronales artificiales experimentó un salto evolutivo clave en la transición del Perceptrón clásico a ADALINE. Aunque ambos modelos comparten una arquitectura idéntica de una sola capa, su diferencia radical radica en el "cuándo" y el "cómo" aprenden.

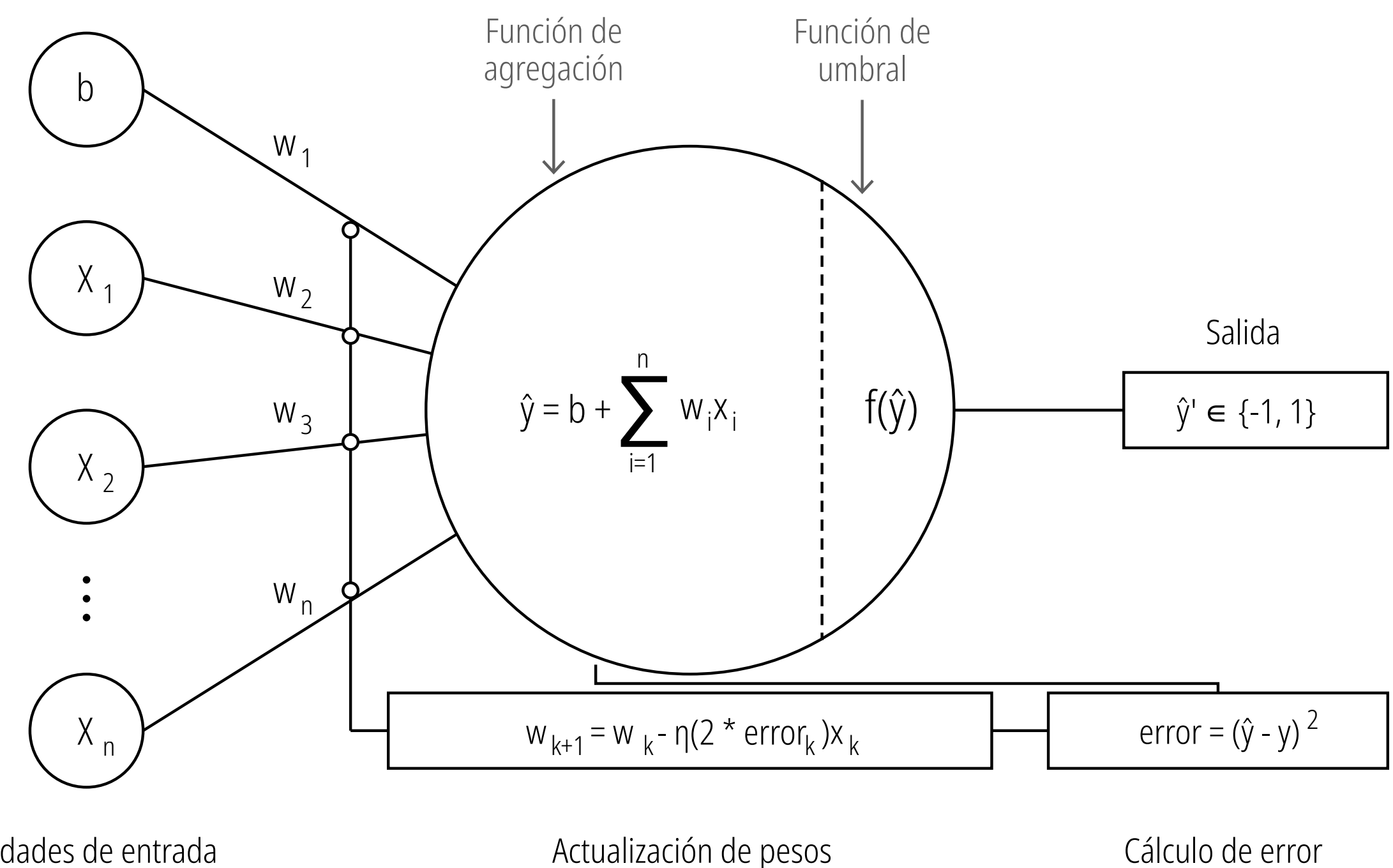
Bucle de entrenamiento del Perceptrón



Evaluación discreta (post-umbral): El modelo calcula su error comparando la etiqueta real con la salida final ya clasificada (después de pasar por la función de umbral). Es una evaluación en "blanco y negro": el algoritmo solo sabe si acertó o se equivocó, sin percibir qué tan cerca estuvo de la respuesta correcta.

Actualización reactiva y rígida: Como consecuencia de este cálculo binario, la actualización de los pesos solo ocurre cuando hay un error de clasificación. Si el modelo clasifica correctamente el dato, asume que el trabajo está terminado y no realiza ninguna mejora en sus parámetros, ignorando cualquier margen de optimización.

Bucle de entrenamiento ADALINE

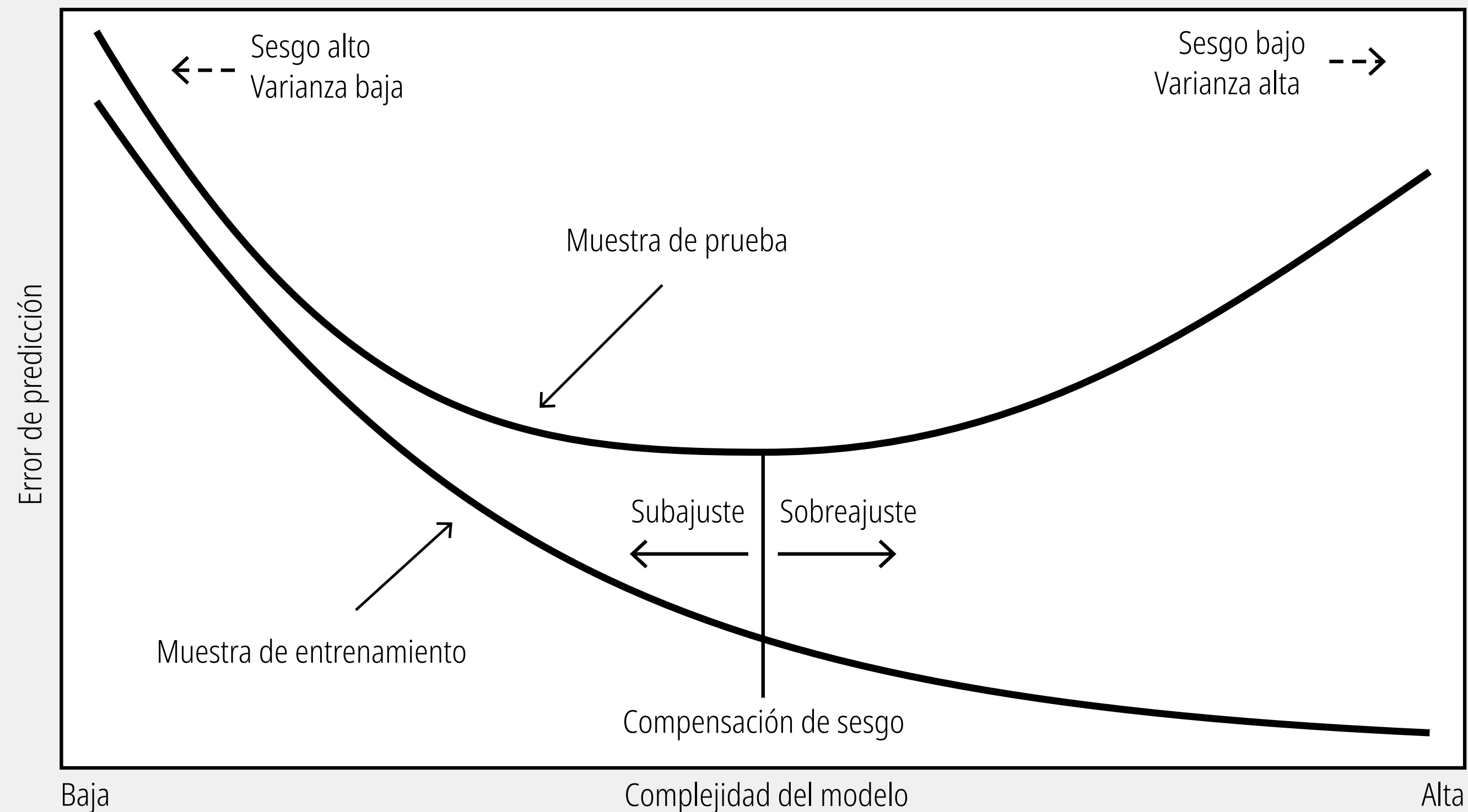


Evaluación continua (pre-umbral): La gran innovación de ADALINE es que calcula el error utilizando el valor numérico puro de la función de agregación (antes de la función de umbral). Esto le permite usar una función de coste cuadrática para medir la "distancia" matemática exacta entre su predicción lineal y el resultado ideal.

Optimización proporcional (Descenso de Gradiente): Al contar con un error continuo y derivable, el modelo puede ajustar sus pesos de manera suave y proporcional a la magnitud de la falla. Esto significa que ADALINE sigue optimizando y ajustando sus parámetros para acercarse lo más posible a la perfección matemática, incluso si la clasificación final ya era correcta.

Régimen clásico: Concesión Sesgo-Varianza

La regla de oro tradicional del Machine Learning. Define cómo se comporta el error de un modelo a medida que aumentas su complejidad (por ejemplo, agregando más variables o capas).



Subajuste (Sesgo Alto / Varianza Baja): A la izquierda, el modelo es demasiado simple. No tiene la capacidad matemática para capturar los patrones en los datos, por lo que comete muchos errores tanto en los datos de entrenamiento como en la prueba.

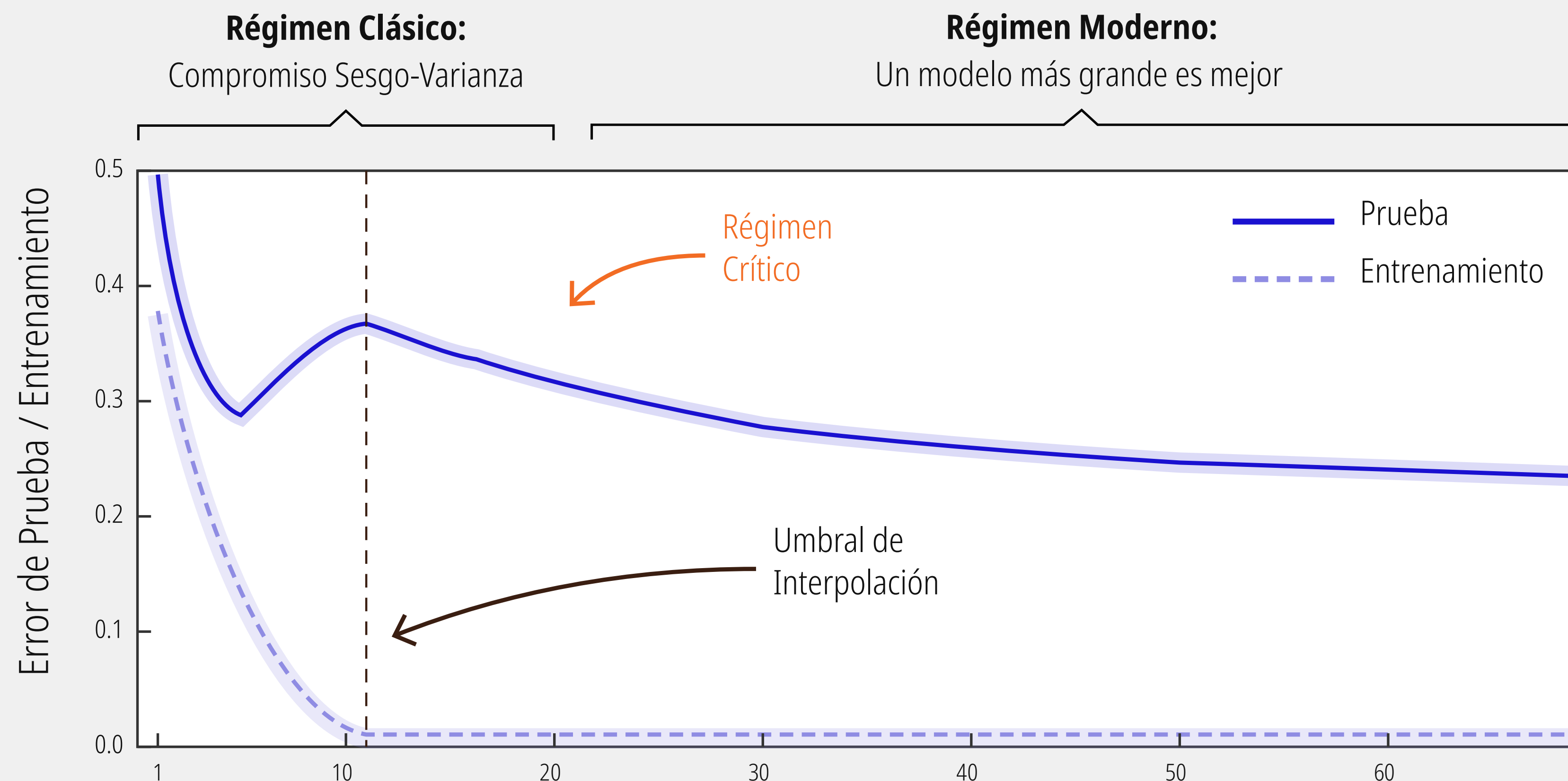
Sobreajuste (Sesgo Bajo / Varianza Alta): A la derecha, el modelo es demasiado complejo. Actúa como un estudiante que memoriza las respuestas del examen de práctica (el error de entrenamiento cae a cero), pero fracasa rotundamente ante preguntas nuevas (el error de prueba se dispara).

El "Punto Óptimo": El valle o fondo de la curva en forma de U. Clásicamente, el objetivo del ingeniero es detener el entrenamiento o ajustar la complejidad exactamente en este punto, logrando el equilibrio perfecto antes de que el error de prueba empiece a subir.

La estadística clásica dictaba que "un modelo demasiado grande es perjudicial", mientras que el régimen moderno (Deep Learning) ha demostrado que, si cruzas el valle de la muerte del sobreajuste, "un modelo gigante es mejor".

Régimen moderno: Doble descenso

Ocurre cuando ignoramos la advertencia del régimen clásico y seguimos aumentando la complejidad del modelo a niveles masivos.



Este fenómeno es el principio matemático subyacente que fundamenta por qué arquitecturas enormes, como los Grandes Modelos de Lenguaje (LLMs) que requieren gestionar complejas cargas de contexto en VRAM, logran generalizar tan bien en la práctica.

Multilayer Perceptron

Tanto **MLPRegressor** como **MLPClassifier** pertenecen al módulo `sklearn.neural_network` de Scikit-Learn y se basan en la misma arquitectura matemática: el Perceptrón Multicapa (Multi-Layer Perceptron). Por esta razón, comparten casi todos sus parámetros.

¿Qué los diferencia?

Su objetivo matemático o función de pérdida. **MLPClassifier** minimiza log-loss (Entropía cruzada) para clasificación, mientras que **MLPRegressor** minimiza MSE (Error Cuadrático Medio) para regresión.

Configuración de la Arquitectura

Define la capacidad de aprendizaje de tu modelo.

Parámetro	Propósito
<code>hidden_layer_sizes</code>	Define profundidad y ancho (ej. (100, 50)).
<code>activation</code>	Función de activación (relu, tanh, logistic).

Motor de Optimización

Cómo aprende el modelo durante el entrenamiento.

Parámetro	Propósito
<code>solver</code>	El algoritmo utilizado para la optimización de los pesos (adam y sgd).
<code>learning_rate</code>	Controla el tamaño del “paso” en el ajuste de pesos.
<code>batch_size</code>	Tamaño del subconjunto de datos por iteración.

Multilayer Perceptron

Control y Regularización

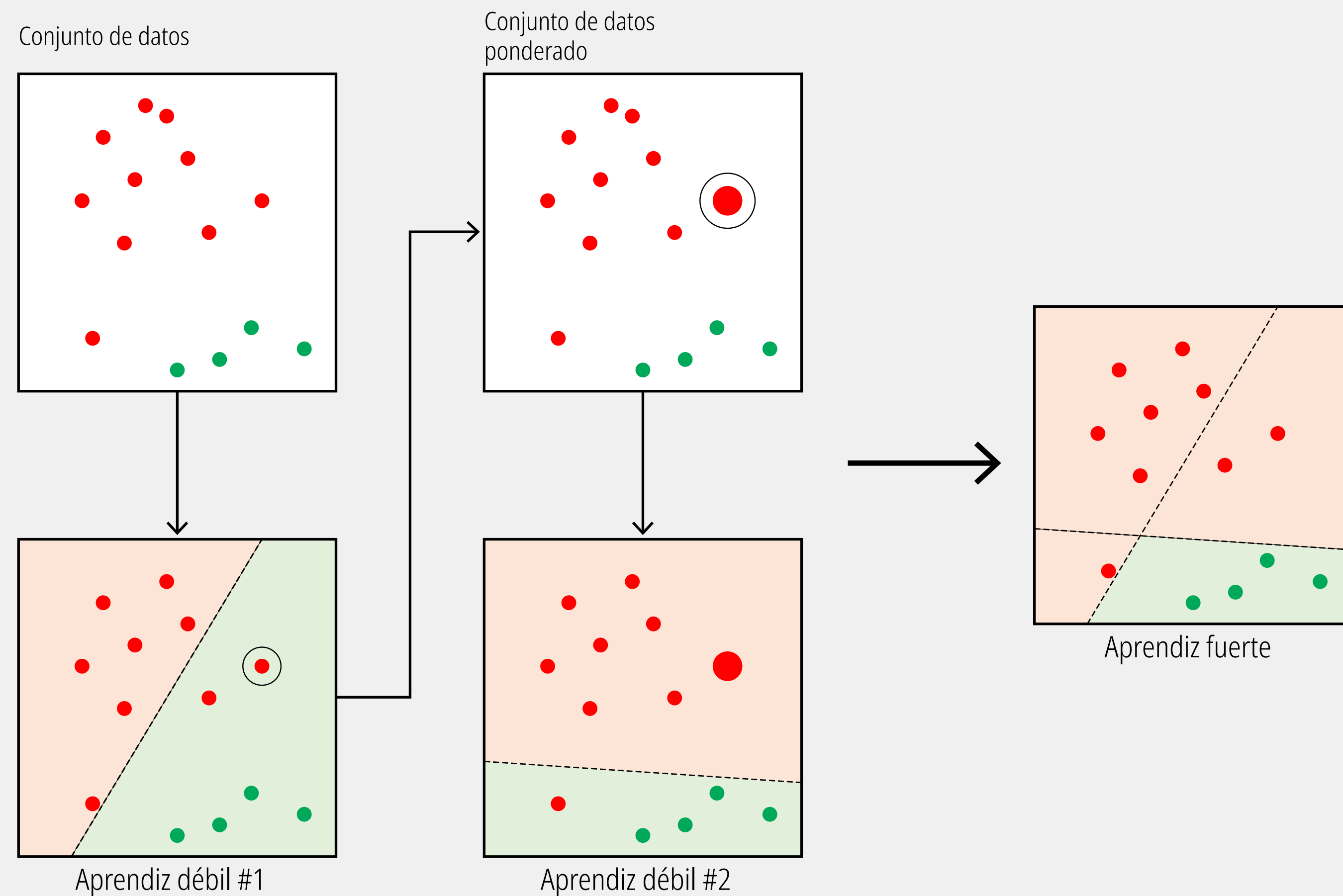
Evita el sobreajuste (overfitting) y optimiza el tiempo.

Parámetro	Propósito
alpha	Penalización L2 (evita pesos excesivamente altos).
early_stopping	Si es True, el modelo se detiene si la pérdida no mejora.
validation_fraction	Tamaño del subconjunto de datos por iteración.
tol	Tolerancia mínima para considerar que hubo una mejora.

Buenas prácticas (Tips)

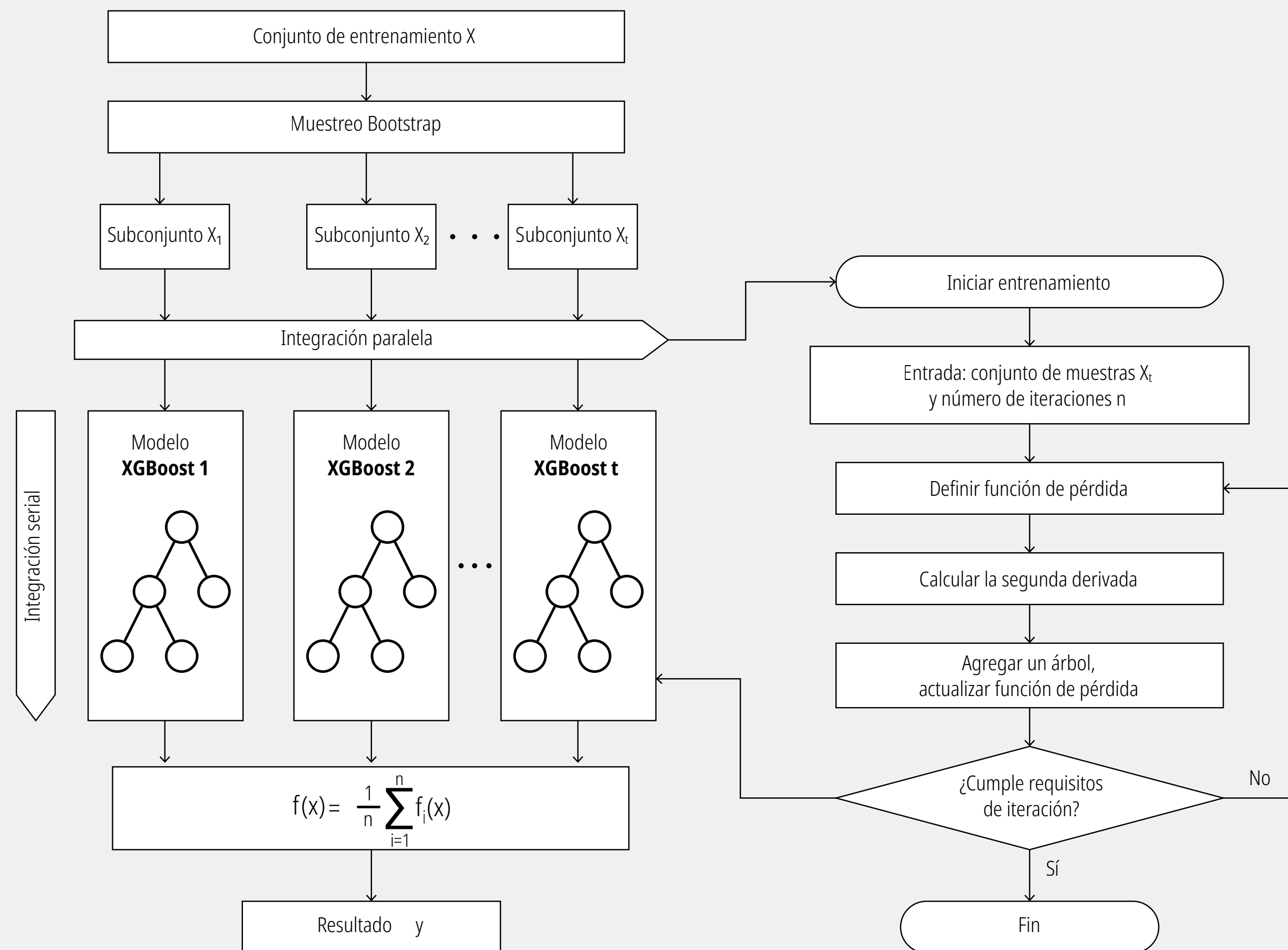
- **Reproducibilidad:** Siempre fija un random_state.
- **Escalado:** Los MLP son sensibles a la escala de los datos. ¡Usa siempre StandardScaler en tu pipeline!
- **Iteraciones:** Aumenta max_iter si recibes advertencias de "ConvergenceWarning"
- **Debug:** Usa verbose=True para visualizar el progreso de la función de pérdida durante el entrenamiento.

XGBoost:



Aunque la estructura base sigue siendo de árboles de decisión, XGBoost cambia radicalmente la filosofía: en lugar de construir cientos de árboles independientes y promediarlos, construye los árboles de forma secuencial, donde cada nuevo árbol intenta corregir los errores matemáticos (residuos) del árbol anterior.

XGBoost:



Parámetros del Bosque (Ensamble y Secuencia)

Controlan la relación entre los árboles y la dinámica del aprendizaje continuo.

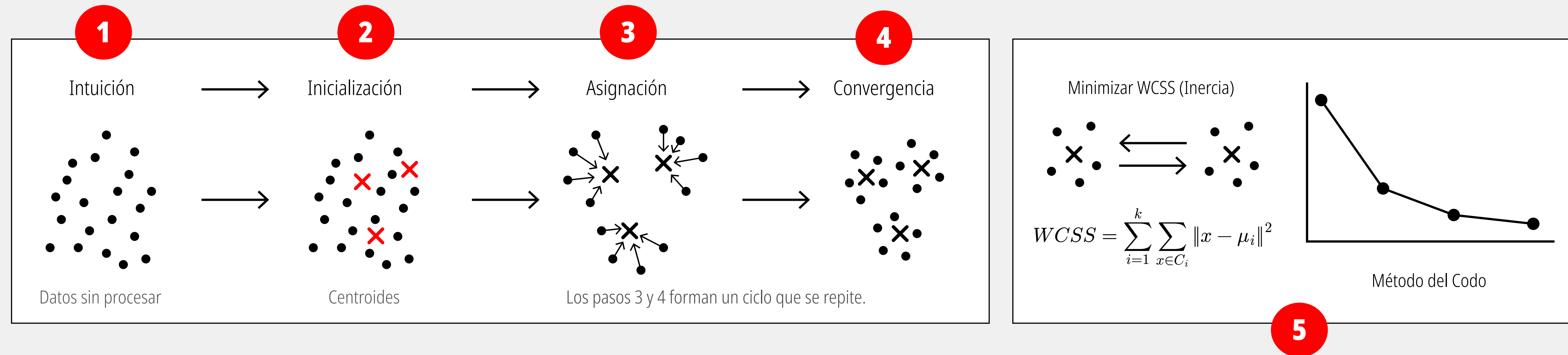
n_estimators (Por defecto: 100): La cantidad máxima de árboles secuenciales a construir. Atención: A diferencia de Random Forest (donde más árboles rara vez dañan), en XGBoost demasiados árboles sí causan sobreajuste porque el modelo memorizará los errores.

learning_rate (o eta, Por defecto: 0.3): El parámetro más importante de XGBoost. Es un freno matemático. Reduce la contribución de cada nuevo árbol al modelo final. Un valor más bajo (ej. 0.05 o 0.1) hace que el modelo aprenda más lento, requiriendo un n_estimators más alto, pero genera resultados mucho más robustos y generalizables.

subsample (Por defecto: 1.0): Equivalente conceptual al bootstrap. Controla el porcentaje de filas de datos que se usan para entrenar cada árbol individual. Bajar este valor (ej. a 0.8, que usa el 80% de los datos) añade aleatoriedad y ayuda a prevenir el sobreajuste.

Clustering: K-Means

Uno de los métodos más populares de aprendizaje automático no supervisado utilizado para agrupamiento (clustering). El objetivo de K-Means es particionar un conjunto de datos en k grupos distintos basándose en la similitud de sus características o distancias.



1) El proceso comienza con un diagrama de dispersión de datos sin etiquetas. Cada punto negro representa una observación cruda en el espacio que el algoritmo intentará clasificar o agrupar sin conocimiento previo.

2) Se debe definir de antemano el número de clústeres deseados, representado por la variable k. En este ejemplo, k=3. El algoritmo inicializa el proceso colocando aleatoriamente tres puntos de referencia iniciales, los cuales se conocen como "centroides".

3) Calcula la distancia (generalmente la distancia euclidiana) entre cada punto de datos y los centroides disponibles. Las pequeñas flechas muestran cómo cada punto de datos se asigna matemáticamente al centroide que tiene más cerca, formando los primeros agrupamientos preliminares.

4) Una vez que todos los puntos están asignados, se recalcula la posición de cada centroide. Las X se mueven al centro espacial exacto (la media aritmética) de todos los puntos que conforman su grupo actual.

WCSS significa Within-Cluster Sum of Squares (Suma de Cuadrados Dentro del Clúster) y también se le llama Inercia. Es una métrica que evalúa qué tan compactos son los clústeres. El objetivo del algoritmo es minimizar esta suma, la cual se calcula midiendo la distancia al cuadrado entre cada punto de datos (x) y el centroide de su clúster. A medida que se aumenta el número de clústeres, la inercia lógicamente disminuye. Para evitar el sobreajuste (donde cada punto es su propio clúster), se busca el punto de inflexión en la gráfica que se asemeja a un "codo". Este punto marca donde la reducción de inercia deja de ser drástica, indicando el número óptimo de clústeres para el conjunto de datos.